Contents

Foreword

1 ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work.

2 The procedures used to develop this document and those intended for its further maintenance are described in the ISO/IEC Directives, Part 1. In particular, the different approval criteria needed for the different types of document should be noted. This document was drafted in accordance with the editorial rules of the ISO/IEC Directives, Part 2 (see www.iso.org/directives or www.iec.ch/members_experts/refdocs).

3 ISO and IEC draw attention to the possibility that the implementation of this document may involve the use of (a) patent(s). ISO and IEC take no position concerning the evidence, validity or applicability of any claimed patent rights in respect thereof. As of the date of publication of this document, ISO and IEC had not received notice of (a) patent(s) which may be required to implement this document. However, implementers are cautioned that this may not represent the latest information, which may be obtained from the patent database available at www.iso.org/patents and patents.iec.ch. ISO and IEC shall not be held responsible for identifying any or all such patent rights.

4 Any trade name used in this document is information given for the convenience of users and does not constitute an endorsement.

5 For an explanation of the voluntary nature of standards, the meaning of ISO specific terms and expressions related to conformity assessment, as well as information about ISO's adherence to the World Trade Organization (WTO) principles in the Technical Barriers to Trade (TBT) see www.iso.org/iso/foreword.html. In the IEC, see www.iec.ch/understanding-standards.

6 This document was prepared by Joint Technical Committee ISO/IEC JTC1, Information technology, Subcommittee SC22, Programming languages, their environments and system software interfaces.

7 This fifth edition cancels and replaces the fourth edition (ISO/IEC 9899:2018), which has been technically revised.
The main changes are contained in Annex M.

8 Any feedback or questions on this document should be directed to the user's national standards body. A complete listing of these bodies can be found at www.iso.org/members.html and www.iec.ch/national-committees.

Introduction

1 With the introduction of new devices and extended character sets, new features could be added to future editions of this document. Subclauses in the language and library clauses warn implementers and programmers of usages which, though valid in themselves, could conflict with future additions.

2 Certain features are *obsolescent*, which means that they could be considered for withdrawal in future revisions of this document. They are retained because of their widespread use, but their use in new implementations (for implementation features) or new programs (for language [6.11] or library features [7.33]) is discouraged.

3 This document is divided into four major subdivisions:

- preliminary elements (Clauses 1–4);
- the characteristics of environments that translate and execute C programs (Clause 5);
- the language syntax, constraints, and semantics (Clause 6);
- the library facilities (Clause 7).

4 Examples are provided to illustrate possible forms of the constructions described. Footnotes are provided to emphasize consequences of the rules described in that subclause or elsewhere in this document. References are used to refer to other related subclauses. Recommendations are provided to give advice or guidance to implementers. Annexes define optional features, provide additional information and summarize the information contained in this document. A bibliography lists documents that were referred to during the preparation of this document.

5 The language clause (Clause 6) is derived from "The C Reference Manual"[15].

6 The library clause (Clause 7) is based on the *1984 /usr/group Standard*[16].

INTERNATIONAL STANDARD **©ISO/IEC** **ISO/IEC 9899:2024**

Information technology — Programming languages — C

1. Scope

1 This document specifies the form and establishes the interpretation of programs written in the C programming language. It is designed to promote the portability of C programs among a variety of data-processing systems. It is intended for use by implementers and programmers. It specifies:

- the representation of C programs;
- the syntax and constraints of the C language;
- the semantic rules for interpreting C programs;
- the representation of input data to be processed by C programs;
- the representation of output data produced by C programs;
- the restrictions and limits imposed by a conforming implementation of C.

2 This document does not specify:

- the mechanism by which C programs are transformed for use by a data-processing system;
- the mechanism by which C programs are invoked for use by a data-processing system;
- the mechanism by which input data are transformed for use by a C program;
- the mechanism by which output data are transformed after being produced by a C program;
- the size or complexity of a program and its data that will exceed the capacity of any specific data-processing system or the capacity of a particular processor;
- all minimal requirements of a data-processing system that is capable of supporting a conforming implementation.

3 Annex J gives an overview of portability issues that a C program might encounter.

2. Normative references

1 The following documents are referred to in the text in such a way that some or all their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

2 ISO/IEC 2382:2015, *Information technology — Vocabulary*.

3 ISO 4217, *Codes for the representation of currencies*.

4 ISO 8601 series, *Data elements and interchange formats — Information interchange — Representation of dates and times*.

5 ISO/IEC 10646, *Information technology —Universal Coded Character Set (UCS)*.

6 ISO/IEC 60559:2020, *Information technology — Microprocessor Systems — Floating-Point arithmetic*.

7 ISO 80000–2, *Quantities and units — Part 2: Mathematics*.

8 The Unicode Consortium. *Unicode Standard Annex, UAX #44, Unicode Character Database [online]*. Edited by Ken Whistler. Available at `https://www.unicode.org/reports/tr44`.

9 The Unicode Consortium. *The Unicode Standard, Derived Core Properties*. Available at `https://www.unicode.org/Public/UCD/latest/ucd/DerivedCoreProperties.txt`.

3. Terms, definitions, and symbols

1 For the purposes of this document, the terms and definitions given in ISO/IEC 2382, ISO 80000–2, and the following apply.

2 ISO and IEC maintain terminology databases for use in standardization at the following addresses:

— ISO Online browsing platform: available at `https://www.iso.org/obp`

— IEC Electropedia: available at `https://www.electropedia.org/`

3 Additional terms are defined where they appear in *italic* type or on the left side of a syntax rule. Terms explicitly defined in this document are not to be presumed to refer implicitly to similar terms defined elsewhere.

3.1

1 **access (verb)**

⟨execution-time action⟩ read or modify the value of an object

2 Note 1 to entry: Where only one of these two actions is meant, "read" or "modify" is used.

3 Note 2 to entry: "Modify" includes the case where the new value being stored is the same as the previous value.

4 Note 3 to entry: Expressions that are not evaluated do not access objects.

3.2

1 **alignment**

requirement that objects of a particular type be located on storage boundaries with addresses that are particular multiples of a byte (3.7) address

3.3

1 **argument**

actual argument

DEPRECATED: actual parameter

expression in the comma-separated list bounded by the parentheses in a function call expression, or a sequence of preprocessing tokens in the comma-separated list bounded by the parentheses in a function-like macro invocation

3.4

1 **arithmetically negate**

produce the negative of a given number

2 Note 1 to entry: For a floating-point number (5.2.5.3.3), this changes the sign; for an integer, this is equivalent to subtracting from zero.

3.5

1 **behavior**

external appearance or action

3.5.1

1 **implementation-defined behavior**

unspecified behavior where each implementation documents how the choice is made

2 Note 1 to entry: J.3 gives an overview over properties of C programs that lead to implementation-defined behavior.

3 EXAMPLE An example of implementation-defined behavior is the propagation of the high-order bit when a signed integer is shifted right.

3.5.2

1 **locale-specific behavior**

behavior that depends on local conventions of nationality, culture, and language that each implementation documents

2 Note 1 to entry: J.4 gives an overview over properties of C programs that lead to locale-specific behavior.

3 EXAMPLE An example of locale-specific behavior is whether the **`islower`** function returns true for characters other than the 26 lowercase Latin letters.

3.5.3

1 **undefined behavior**

behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this document imposes no requirements

2 Note 1 to entry: Possible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message).

3 Note 2 to entry: J.2 gives an overview over properties of C programs that lead to undefined behavior.

4 Note 3 to entry: Any other behavior during execution of a program is only affected as a direct consequence of the concrete behavior that occurs when encountering the erroneous or non-portable program construct or data. In particular, all observable behavior (5.1.2.4) appears as specified in this document when it happens before an operation with undefined behavior in the execution of the program.

5 EXAMPLE An example of undefined behavior is the behavior on dereferencing a null pointer.

3.5.4

1 **unspecified behavior**

behavior, that results from the use of an unspecified value, or other behavior upon which this document provides two or more possibilities and imposes no further requirements on which is chosen in any instance

2 Note 1 to entry: J.1 gives an overview over properties of C programs that lead to unspecified behavior.

3 EXAMPLE An example of unspecified behavior is the order in which the arguments to a function are evaluated.

3.6

1 **bit**

unit of data storage in the execution environment large enough to hold an object that can have one of two values. It may not be possible to express the address of each individual bit of an object.

3.7

1 **byte**

addressable unit of data storage large enough to hold any member of the basic character set of the execution environment

2 Note 1 to entry: It is possible to express the address of each individual byte of an object uniquely.

3.8

1 **low-order bit**

the least significant bit within a byte

3.9

1 **high-order bit**

the most significant bit within a byte

3.10

1 **character**

⟨abstract⟩ member of a set of elements used for the organization, control, or representation of data

3.10.1

1 **character**

single-byte character

⟨C⟩ bit representation that fits in a byte

3.10.2

1 **multibyte character**

sequence of one or more bytes representing a member of the extended character set of either the source or the execution environment

2 Note 1 to entry: The extended character set is a superset of the basic character set.

3.10.3

1 **wide character**

value representable by an object of type **wchar_t**, capable of representing any character in the current locale

3.11

1 **constraint**

restriction, either syntactic or semantic, by which the exposition of language elements is interpreted

3.12

1 **correctly rounded result**

representation in the result format that is nearest in value, subject to the current rounding mode, to what the result would be given unlimited range and precision

2 Note 1 to entry: In this document, the words "correctly rounded" may apply to an operation that produces a correctly rounded result, or to input for such an operation.

3 Note 2 to entry: ISO/IEC 60559 or implementation-defined rules apply for extreme magnitude results if the result format contains infinity.

3.13

1 **diagnostic message**

message belonging to an implementation-defined subset of the implementation's message output

3.14

1 **forward reference**

reference to a subsequent subclause in this document that contains additional information relevant to the subclause containing the reference

3.15

1 **implementation**

particular set of software, running in a particular translation environment under particular control options, that performs translation of programs for, and supports execution of functions in, a

particular execution environment

3.16

1 **implementation limit**

restriction imposed upon programs by the implementation

3.17

1 **memory location**

either an object of scalar type, or a maximal sequence of adjacent bit-fields all having nonzero width

2 Note 1 to entry: Two threads of execution can update and access separate memory locations without interfering with each other.

3 Note 2 to entry: A bit-field and an adjacent non-bit-field member are in separate memory locations. The same applies to two bit-fields, if one is declared inside a nested structure declaration and the other is not, or if the two are separated by a zero-length bit-field declaration, or if they are separated by a non-bit-field member declaration. It is not safe to concurrently update two non-atomic bit-fields in the same structure if all members declared between them are also (nonzero-length) bit-fields, no matter what the sizes of those intervening bit-fields happen to be.

4 EXAMPLE A structure declared as

```
struct {
      char a;
      int b:5, c:11,:0, d:8;
      struct { int ee:8; } e;
}
```

contains four separate memory locations: The member `a`, and bit-fields `d` and `e.ee` are each separate memory locations, and can be modified concurrently without interfering with each other. The bit-fields `b` and `c` together constitute the fourth memory location. The bit-fields `b` and `c` cannot be concurrently modified, but `b` and `a`, for example, can be.

3.18

1 **object**

region of data storage in the execution environment, the contents of which can represent values

2 Note 1 to entry: When referenced, an object can be interpreted as having a particular type; see 6.3.2.1.

3.19

1 **parameter**

formal parameter

DEPRECATED: formal argument

object declared as part of a function declaration or definition that acquires a value on entry to the function, or an identifier from the comma-separated list bounded by the parentheses immediately following the macro name in a function-like macro definition

3.20

1 **recommended practice**

specification that is strongly recommended as being in keeping with the intent of the standard, but that may be impractical for some implementations

3.21

1 **runtime-constraint**

requirement on a program when calling a library function

2 Note 1 to entry: Despite the similar terms, a runtime-constraint is not a kind of constraint as defined by 3.11,

and it is not necessary for it to be diagnosed at translation time.

3 Note 2 to entry: Implementations that support the extensions in Annex K are required to verify that the runtime-constraints for a library function are not violated by the program; see K.3.1.4.

4 Note 3 to entry: Implementations that support Annex L are permitted to invoke a runtime-constraint handler when they perform a trap.

3.22

1 **value**

precise meaning of the contents of an object when interpreted as having a specific type

3.22.1

1 **implementation-defined value**

unspecified value where each implementation documents how the choice is made

3.22.2

1 **unspecified value**

valid value of the relevant type where this document imposes no requirements on which value is chosen in any instance

3.23

1 **indeterminate representation**

object representation that either represents an unspecified value or is a non-value representation

3.24

1 **non-value representation**

an object representation that does not represent a value of the object type

3.25

1 **perform a trap**

interrupt execution of the program such that no further operations are performed

2 Note 1 to entry: Fetching a non-value representation permits an implementation to perform a trap but is not required to (see 6.2.6.1).

3 Note 2 to entry: Implementations that support Annex L are permitted to invoke a runtime-constraint handler when they perform a trap.

3.26

1 $\lceil x \rceil$

ceiling of x

the least integer greater than or equal to x

2 EXAMPLE $\lceil 2.4 \rceil$ is 3, $\lceil -2.4 \rceil$ is -2.

3.27

1 $\lfloor x \rfloor$

floor of x

the greatest integer less than or equal to x

2 EXAMPLE $\lfloor 2.4 \rfloor$ is 2, $\lfloor -2.4 \rfloor$ is -3.

3.28

1 **wraparound**

the process by which a value is reduced modulo 2^N, where N is the width of the resulting type

4. Conformance

1 In this document, "shall" is to be interpreted as a requirement on an implementation or on a program; conversely, "shall not" is to be interpreted as a prohibition.

2 If a "shall" or "shall not" requirement that appears outside of a constraint or runtime-constraint is violated, the behavior is undefined. Undefined behavior is otherwise indicated in this document by the words "undefined behavior" or by the omission of any explicit definition of behavior. There is no difference in emphasis among these three; they all describe "behavior that is undefined".

3 A program that is correct in all other aspects, operating on correct data, containing unspecified behavior shall be a correct program and act in accordance with 5.1.2.4.

4 The implementation shall not successfully translate a preprocessing translation unit containing a **`#error`** preprocessing directive unless it is part of a group skipped by conditional inclusion.

5 A *strictly conforming program* shall use only those features of the language and library specified in this document. It shall not produce output dependent on any unspecified, undefined, or implementation-defined behavior, and shall not exceed any minimum implementation limit.

6 EXAMPLE A strictly conforming program can use conditional features (see 6.10.10.4) provided the use is guarded by an appropriate conditional inclusion preprocessing directive using the related macro. For example:

```
#ifdef __STDC_IEC_60559_BFP__ /* FE_UPWARD defined */
      /* ... */
      fesetround(FE_UPWARD);
      /* ... */
#endif
```

7 The two forms of *conforming implementation* are hosted and freestanding. A *conforming hosted implementation* shall accept any strictly conforming program. A *conforming freestanding implementation* shall accept any strictly conforming program in which the use of the features specified in the library clause (Clause 7) is confined to the contents of the standard headers `<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbit.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, and `<stdnoreturn.h>`. Additionally, a *conforming freestanding implementation* shall accept any strictly conforming program where:

— the features specified in the header `<string.h>` are used, except the following functions: **`strcoll`**, **`strdup`**, **`strerror`**, **`strndup`**, **`strtok`**, **`strxfrm`**; and/or,

— the selected function **`memalignment`** from `<stdlib.h>` is used.

A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any strictly conforming program.[1)]

8 The strictly conforming programs that shall be accepted by a conforming freestanding implementation that defines **`__STDC_IEC_60559_BFP__`** or **`__STDC_IEC_60559_DFP__`** may also use features in the contents of the standard headers `<fenv.h>`, `<math.h>`, and the `strto`∗ floating-point numeric conversion functions (7.24.1) of the standard header `<stdlib.h>`, provided the program does not set the state of the **`FENV_ACCESS`** pragma to "on".

All identifiers that are reserved when `<stdlib.h>` is included in a hosted implementation are reserved when it is included in a freestanding implementation.

9 A *conforming program* is one that is acceptable to a conforming implementation.[2)]

[1)] This implies that a conforming implementation reserves no identifiers other than those explicitly reserved in this document.

[2)] Strictly conforming programs are intended to be maximally portable among conforming implementations. Conforming programs can depend upon nonportable features of a conforming implementation.

10 An implementation shall be accompanied by a document that defines all implementation-defined and locale-specific characteristics and all extensions.

Forward references: conditional inclusion (6.10.2), error directive (6.10.7), characteristics of floating types <float.h> (7.7), alternative spellings <iso646.h> (7.9), sizes of integer types <limits.h> (7.10), alignment <stdalign.h> (7.15), variable arguments <stdarg.h> (7.16), boolean type and values <stdbool.h> (7.19), common definitions <stddef.h> (7.21), integer types <stdint.h> (7.22), <stdnoreturn.h> (7.25).

5. Environment

1 An implementation translates C source files and executes C programs in two data-processing-system environments, which will be called the *translation environment* and the *execution environment* in this document. Their characteristics define and constrain the results of executing conforming C programs constructed according to the syntactic and semantic rules for conforming implementations.

Forward references: In this clause, only a few of many possible forward references have been noted.

5.1 Conceptual models

5.1.1 Translation environment

5.1.1.1 Program structure

1 A C program is not required to be translated in its entirety at the same time. The text of the program is kept in units called *source files*, (or *preprocessing files*) in this document. A source file together with all the headers and source files included via the preprocessing directive **`#include`** is known as a *preprocessing translation unit*. After preprocessing, a preprocessing translation unit is called a *translation unit*. Previously translated translation units may be preserved individually or in libraries. The separate translation units of a program communicate by (for example) calls to functions whose identifiers have external linkage, manipulation of objects whose identifiers have external linkage, or manipulation of data files. Translation units may be separately translated and then later linked to produce an executable program.

Forward references: linkages of identifiers (6.2.2), external definitions (6.9), preprocessing directives (6.10).

5.1.1.2 Translation phases

1 The precedence among the syntax rules of translation is specified by the following phases.[3)]

1. Physical source file multibyte characters are mapped, in an implementation-defined manner, to the source character set (introducing new-line characters for end-of-line indicators) if necessary.

2. Each instance of a backslash character (\) immediately followed by a new-line character is deleted, splicing physical source lines to form logical source lines. Only the last backslash on any physical source line shall be eligible for being part of such a splice. A source file that is not empty shall end in a new-line character, which shall not be immediately preceded by a backslash character before any such splicing takes place.

3. The source file is decomposed into preprocessing tokens[4)] and sequences of white-space characters (including comments). A source file shall not end in a partial preprocessing token or in a partial comment. Each comment is replaced by one space character. New-line characters are retained. Whether each nonempty sequence of white-space characters other than new-line is retained or replaced by one space character is implementation-defined.

4. Preprocessing directives are executed, macro invocations are expanded, and **`_Pragma`** unary operator expressions are executed. If a character sequence that matches the syntax of a universal character name is produced by token concatenation (6.10.5.3), the behavior is undefined. A **`#include`** preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.

[3)] This requires implementations to behave as if these separate phases occur, even though many are typically folded together in practice. Source files, translation units, and translated translation units necessarily can be stored as files or through/within any other implementation-defined medium. There does not have to be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation.

[4)] As described in 6.4, the process of dividing a source file's characters into preprocessing tokens is context-dependent. For example, see the handling of < within a **`#include`** preprocessing directive.

5. Each source character set member and escape sequence in character constants and string literals is converted to the corresponding member of the execution character set. Each instance of a source character or escape sequence for which there is no corresponding member is converted in an implementation-defined manner to some member of the execution character set other than the null (wide) character.[5)]
6. Adjacent string literal tokens are concatenated.
7. White-space characters separating tokens are no longer significant. Each preprocessing token is converted into a token. The resulting tokens are syntactically and semantically analyzed and translated as a translation unit.
8. All external object and function references are resolved. Library components are linked to satisfy external references to functions and objects not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

Forward references: universal character names (6.4.3), lexical elements (6.4), preprocessing directives (6.10), external definitions (6.9).

5.1.1.3 Diagnostics

1 A conforming implementation shall produce at least one diagnostic message (identified in an implementation-defined manner) if a preprocessing translation unit or translation unit contains a violation of any syntax rule or constraint, even if the behavior is also explicitly specified as undefined or implementation-defined. Diagnostic messages are not required to be produced in other circumstances.

2 EXAMPLE An implementation is required to issue a diagnostic for the translation unit:

```
char i;
int i;
```

because in those cases where wording in this document describes the behavior for a construct as being both a constraint error and resulting in undefined behavior, the constraint error is still required to be diagnosed.

Recommended practice

3 An implementation is encouraged to identify the nature of, and where possible localize, each violation. Of course, an implementation is free to produce any number of diagnostic messages, often referred to as warnings, as long as a valid program is still correctly translated. It can also successfully translate an invalid program. Annex I lists a few of the more common warnings.

5.1.2 Execution environments

5.1.2.1 General

1 Two execution environments are defined: *freestanding* and *hosted*. In both cases, *program startup* occurs when a designated C function is called by the execution environment. All objects with static storage duration shall be *initialized* (set to their initial values) before program startup. The manner and timing of such initialization are otherwise unspecified. *Program termination* returns control to the execution environment.

Forward references: storage durations of objects (6.2.4), initialization (6.7.11).

5.1.2.2 Freestanding environment

1 In a freestanding environment (in which C program execution may take place without any benefit of an operating system), the name and type of the function called at program startup are implementation-defined. Any library facilities available to a freestanding program, other than the minimal set required by Clause 4, are implementation-defined.

2 The effect of program termination in a freestanding environment is implementation-defined.

[5)] An implementation may convert each instance of the same non-corresponding source character to a different member of the execution character set.

5.1.2.3 Hosted environment

5.1.2.3.1 General

1 A hosted environment is not required to be provided, but shall conform to the following specifications if present.

5.1.2.3.2 Program startup

1 The function called at program startup is named **main**. The implementation declares no prototype for this function. It shall be defined with a return type of **int** and with no parameters:

```
int main(void) { /* ... */ }
```

or with two parameters (referred to here as `argc` and `argv`, though any names may be used, as they are local to the function in which they are declared):

```
int main(int argc, char *argv[]) { /* ... */ }
```

or equivalent;[6] or in some other implementation-defined manner.

2 If they are declared, the parameters to the **main** function shall obey the following constraints:

— The value of `argc` shall be nonnegative.

— `argv[argc]` shall be a null pointer.

— If the value of `argc` is greater than zero, the array members `argv[0]` through `argv[argc-1]` inclusive shall contain pointers to strings, which are given implementation-defined values by the host environment prior to program startup. The intent is to supply to the program information determined prior to program startup from elsewhere in the hosted environment. If the host environment is not capable of supplying strings with letters in both uppercase and lowercase, the implementation shall ensure that the strings are received in lowercase.

— If the value of `argc` is greater than zero, the string pointed to by `argv[0]` represents the *program name*; `argv[0][0]` shall be the null character if the program name is not available from the host environment. If the value of `argc` is greater than one, the strings pointed to by `argv[1]` through `argv[argc-1]` represent the *program parameters*.

— The parameters `argc` and `argv` and the strings pointed to by the `argv` array shall be modifiable by the program, and retain their last-stored values between program startup and program termination.

5.1.2.3.3 Program execution

1 In a hosted environment, a program may use all the functions, macros, type definitions, and objects described in the library clause (Clause 7).

5.1.2.3.4 Program termination

1 If the return type of the **main** function is a type compatible with **int**, a return from the initial call to the **main** function is equivalent to calling the **exit** function with the value returned by the **main** function as its argument;[7] reaching the `}` that terminates the **main** function returns a value of 0. If the return type is not compatible with **int**, the termination status returned to the host environment is unspecified.

Forward references: definition of terms (7.1.1), the **exit** function (7.24.4.4).

[6] Thus, **int** can be replaced by a typedef name defined as **int**, or the type of `argv` can be written as **char** `**` `argv`, or the return type may be specified by **typeof**`(1)`, and so on.

[7] In accordance with 6.2.4, the lifetimes of objects with automatic storage duration declared in **main** will have ended in the former case, even where they would not have in the latter.

5.1.2.4 Program semantics

1 The semantic descriptions in this document describe the behavior of an abstract machine in which issues of optimization are irrelevant.

2 An access to an object through the use of an lvalue of volatile-qualified type is a *volatile access*. A volatile access to an object, modifying an object, modifying a file, or calling a function that does any of those operations are all *side effects*,[8)] which are changes in the state of the execution environment.

Evaluation of an expression in general includes both value computations and initiation of side effects. Value computation for an lvalue expression includes determining the identity of the designated object.

3 *Sequenced before* is an asymmetric, transitive, pair-wise relation between evaluations executed by a single thread, which induces a partial order among those evaluations. Given any two evaluations A and B, if A is sequenced before B, then the execution of A shall precede the execution of B. (Conversely, if A is sequenced before B, then B is *sequenced after* A.) If A is not sequenced before or after B, then A and B are *unsequenced*. Evaluations A and B are *indeterminately sequenced* when A is sequenced either before or after B, but it is unspecified which.[9)] The presence of a *sequence point* between the evaluation of expressions A and B implies that every value computation and side effect associated with A is sequenced before every value computation and side effect associated with B. (A summary of the sequence points is given in Annex C.)

4 In the abstract machine, all expressions are evaluated as specified by the semantics. An actual implementation is not required to evaluate part of an expression if it can deduce that its value is not used and that no needed side effects are produced (including any caused by calling a function or through volatile access to an object).

5 When the processing of the abstract machine is interrupted by receipt of a signal, the values of objects that are neither lock-free atomic objects nor of type **`volatile`** **`sig_atomic_t`** are unspecified, as is the state of the dynamic floating-point environment. The representation of any object modified by the handler that is neither a lock-free atomic object nor of type **`volatile`** **`sig_atomic_t`** becomes indeterminate when the handler exits, as does the state of the dynamic floating-point environment if it is modified by the handler and not restored to its original state.

6 The least requirements on a conforming implementation are:

- Volatile accesses to objects are evaluated strictly according to the rules of the abstract machine.
- At program termination, all data written into files shall be identical to the result that execution of the program according to the abstract semantics would have produced.
- The input and output dynamics of interactive devices shall take place as specified in 7.23.3. The intent of these requirements is that unbuffered or line-buffered output appear as soon as possible, to ensure that prompting messages appear prior to a program waiting for input.

This is the *observable behavior* of the program.

7 What constitutes an interactive device is implementation-defined.

8 More stringent correspondences between abstract and actual semantics may be defined by each implementation.

9 EXAMPLE 1 An implementation can define a one-to-one correspondence between abstract and actual semantics: at every sequence point, the values of the actual objects would agree with those specified by the abstract semantics. The keyword **`volatile`** would then be redundant.

10 Alternatively, an implementation can perform various optimizations within each translation unit, such that the actual semantics would agree with the abstract semantics only when making function calls across translation

[8)] ISO/IEC 60559 requires certain user-accessible status flags and control modes. Floating-point operations implicitly set the status flags; modes affect result values of floating-point operations. Implementations that support such floating-point state are required to regard changes to it as side effects — see Annex F for details. The floating-point environment library `<fenv.h>` provides a programming facility for indicating when these side effects matter, freeing the implementations in other cases.

[9)] The executions of unsequenced evaluations can interleave. Indeterminately sequenced evaluations cannot interleave, but can be executed in any order.

unit boundaries. In such an implementation, at the time of each function entry and function return where the calling function and the called function are in different translation units, the values of all externally linked objects and of all objects accessible via pointers therein would agree with the abstract semantics. Furthermore, at the time of each such function entry the values of the parameters of the called function and of all objects accessible via pointers therein would agree with the abstract semantics. In this type of implementation, objects referred to by interrupt service routines activated by the **`signal`** function would require explicit specification of **`volatile`** storage, as well as other implementation-defined restrictions.

11 EXAMPLE 2 In executing the fragment

```
char c1, c2;
/* ... */
c1 = c1 + c2;
```

the "integer promotions" require that the abstract machine promote the value of each variable to **`int`** size and then add the two **`int`**s and truncate the sum. Provided the addition of two **`char`**s can be done without integer overflow, or with integer overflow wrapping silently to produce the correct result, the actual execution need only produce the same result, possibly omitting the promotions.

12 EXAMPLE 3 Similarly, in the fragment

```
float f1, f2;
double d;
/* ... */
f1 = f2 * d;
```

the multiplication can be executed using **`float`** arithmetic if the implementation can ascertain that the result would be the same as if it were executed using **`double`** arithmetic (for example, if `d` were replaced by the constant `2.0`, which has type **`double`**).

13 EXAMPLE 4 Implementations employing wide registers have to take care to honor appropriate semantics. Values are independent of whether they are represented in a register or in memory. For example, an implicit *spilling* of a register is not permitted to alter the value. Also, an explicit *store and load* is required to round to the precision of the storage type. In particular, casts and assignments are required to perform their specified conversion. For the fragment

```
double d1, d2;
float f;
d1 = f = expression;
d2 = (float) expression;
```

the values assigned to `d1` and `d2` are required to have been converted to **`float`**.

14 EXAMPLE 5 Rearrangement for floating-point expressions is often restricted because of limitations in precision as well as range. The implementation cannot generally apply the mathematical associative rules for addition or multiplication, nor the distributive rule, because of roundoff error, even in the absence of overflow and underflow. Likewise, implementations cannot generally replace decimal constants to rearrange expressions. In the following fragment, rearrangements suggested by mathematical rules for real numbers are often not valid (see F.9).

```
double x, y, z;
/* ... */
x = (x * y) * z;  // not equivalent to x *= y * z;
z = (x - y) + y;  // not equivalent to z = x;
z = x + x * y;    // not equivalent to z = x * (1.0 + y);
y = x / 5.0;      // not equivalent to y = x * 0.2;
```

15 EXAMPLE 6 To illustrate the grouping behavior of expressions, in the following fragment

```
int a, b;
/* ... */
a = a + 32760 + b + 5;
```

the expression statement behaves exactly the same as

```
a = (((a + 32760) + b) + 5);
```

due to the associativity and precedence of these operators. Thus, the result of the sum `(a + 32760)` is next added to `b`, and that result is then added to `5` which results in the value assigned to `a`. On a machine in which integer overflows produce an explicit trap and in which the range of values representable by an **`int`** is $[-32768, +32767]$, the implementation cannot rewrite this expression as

```
a = ((a + b) + 32765);
```

since if the values for `a` and `b` were, respectively, -32754 and -15, the sum `a + b` would produce a trap while the original expression would not; nor can the expression be rewritten either as

```
a = ((a + 32765) + b);
```

or

```
a = (a + (b + 32765));
```

since the values for `a` and `b` may have been, respectively, 4 and -8 or -17 and 12. However, on a machine in which integer overflow silently generates some value and where positive and negative integer overflows cancel, the preceding expression statement can be rewritten by the implementation in any of the previously specified ways because the same result will occur.

16 EXAMPLE 7 The grouping of an expression does not completely determine its evaluation. In the following fragment:

```
#include <stdio.h>
int sum;
char *p;
/* ... */
sum = sum * 10 - '0' + (*p++ = getchar());
```

the expression statement is grouped as if it were written as

```
sum = (((sum * 10) - '0') + ((*(p++)) = (getchar())));
```

but the actual increment of `p` can occur at any time between the previous sequence point and the next sequence point (the `;`), and the call to **`getchar`** can occur at any point prior to the need of its returned value.

Forward references: expressions (6.5.1), type qualifiers (6.7.4), statements (6.8), floating-point environment <`fenv.h`> (7.6), the **`signal`** function (7.14), files (7.23.3).

5.1.2.5 Multi-threaded executions and data races

1 Under a hosted implementation that does not define **`__STDC_NO_THREADS__`**, a program can have more than one *thread of execution* (or *thread*) running concurrently. The execution of each thread proceeds as defined by the remainder of this document. The execution of the entire program consists of an execution of all its threads.[10)] Under a freestanding implementation, it is implementation-defined whether a program can have more than one thread of execution.

2 The value of an object visible to a thread T at a particular point is the initial value of the object, a value stored in the object by T, or a value stored in the object by another thread, according to the rules in the rest of this subclause.

3 NOTE 1 In some cases, there could instead be undefined behavior. Much of this section is motivated by the desire to support atomic operations with explicit and detailed visibility constraints. However, it also implicitly supports a simpler view for more restricted programs.

4 Two expression evaluations *conflict* if one of them modifies a memory location and the other one reads or modifies the same memory location.

[10)] The execution can usually be viewed as an interleaving of all the threads. However, some kinds of atomic operations, for example, allow executions inconsistent with a simple interleaving as described in this subclause.

5 The library defines *atomic operations* (7.17) and operations on mutexes (7.28.4) that are specially identified as synchronization operations. These operations play a special role in making assignments in one thread visible to another. A *synchronization operation* on one or more memory locations is one of an *acquire operation*, a *release operation*, both an acquire and release operation, or a *consume operation*. A synchronization operation without an associated memory location is a *fence* and can be either an acquire fence, a release fence, or both an acquire and release fence. In addition, there are *relaxed atomic operations*, which are not synchronization operations, and atomic *read-modify-write operations*, which have special characteristics.

6 NOTE 2 For example, a call that acquires a mutex will perform an acquire operation on the locations composing the mutex. Correspondingly, a call that releases the same mutex will perform a release operation on those same locations. Informally, performing a release operation on A forces prior side effects on other memory locations to become visible to other threads that later perform an acquire or consume operation on A. Relaxed atomic operations are not included as synchronization operations although, like synchronization operations, they cannot contribute to data races.

7 All modifications to a particular atomic object M occur in some particular total order, called the *modification order* of M. If A and B are modifications of an atomic object M, and A happens before B, then A shall precede B in the modification order of M, which is defined later in this subclause.

8 NOTE 3 This states that the modification orders are expected to respect the "happens before" relation.

9 NOTE 4 There is a separate order for each atomic object. There is no requirement that these can be combined into a single total order for all objects. In general this will be impossible since different threads can observe modifications to different variables in inconsistent orders.

10 A *release sequence* headed by a release operation A on an atomic object M is a maximal contiguous sub-sequence of side effects in the modification order of M, where the first operation is A and every subsequent operation either is performed by the same thread that performed the release or is an atomic read-modify-write operation.

11 Certain library calls *synchronize with* other library calls performed by another thread. In particular, an atomic operation A that performs a release operation on an object M synchronizes with an atomic operation B that performs an acquire operation on M and reads a value written by any side effect in the release sequence headed by A.

12 NOTE 5 Except in the specified cases, reading a later value does not necessarily ensure visibility as described later in this subclause. Such a requirement would sometimes interfere with efficient implementation.

13 NOTE 6 The specifications of the synchronization operations define when one reads the value written by another. For atomic variables, the definition is clear. All operations on a given mutex occur in a single total order. Each mutex acquisition "reads the value written" by the last mutex release.

14 An evaluation A *carries a dependency*[11] to an evaluation B if:

— the value of A is used as an operand of B, unless:

- B is an invocation of the **`kill_dependency`** macro,
- A is the left operand of a `&&` or `||` operator,
- A is the left operand of a `?:` operator, or
- A is the left operand of a `,` operator;

or

— A writes a scalar object or bit-field M, B reads from M the value written by A, and A is sequenced before B, or

— for some evaluation X, A carries a dependency to X and X carries a dependency to B.

15 An evaluation A is *dependency-ordered before*[12] an evaluation B if:

[11] The "carries a dependency" relation is a subset of the "sequenced before" relation, and is similarly strictly intra-thread.

[12] The "dependency-ordered before" relation is analogous to the "synchronizes with" relation, but uses release/consume in place of release/acquire.

— A performs a release operation on an atomic object M, and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A, or

— for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.

16 An evaluation A *inter-thread happens before* an evaluation B if A synchronizes with B, A is dependency-ordered before B, or, for some evaluation X:

— A synchronizes with X and X is sequenced before B,

— A is sequenced before X and X inter-thread happens before B, or

— A inter-thread happens before X and X inter-thread happens before B.

17 NOTE 7 The "inter-thread happens before" relation describes arbitrary concatenations of "sequenced before", "synchronizes with", and "dependency-ordered before" relationships, with two exceptions. The first exception is that a concatenation is not permitted to end with "dependency-ordered before" followed by "sequenced before". The reason for this limitation is that a consume operation participating in a "dependency-ordered before" relationship provides ordering only with respect to operations to which this consume operation carries a dependency. The reason that this limitation applies only to the end of such a concatenation is that any subsequent release operation will provide the required ordering for a prior consume operation. The second exception is that a concatenation is not permitted to consist entirely of "sequenced before". The reasons for this limitation are (1) to permit "inter-thread happens before" to be transitively closed and (2) the "happens before" relation, defined subsequently in this subclause, provides for relationships consisting entirely of "sequenced before".

18 An evaluation A *happens before* an evaluation B if A is sequenced before B or A inter-thread happens before B. The implementation shall ensure that no program execution demonstrates a cycle in the "happens before" relation.

19 NOTE 8 This cycle would otherwise be possible only through the use of consume operations.

20 A *visible side effect* A on an object M with respect to a value computation B of M satisfies the conditions:

— A happens before B, and

— there is no other side effect X to M such that A happens before X and X happens before B.

The value of a non-atomic scalar object M, as determined by evaluation B, shall be the value stored by the visible side effect A.

21 NOTE 9 If there is ambiguity about which side effect to a non-atomic object is visible, then there is a data race and the behavior is undefined.

22 NOTE 10 This states that operations on ordinary variables are not visibly reordered. This is not detectable without data races, but ensures that data races, as defined here, and with suitable restrictions on the use of atomics, correspond to data races in a simple interleaved (sequentially consistent) execution.

23 The value of an atomic object M, as determined by evaluation B, shall be the value stored by some side effect A that modifies M, where B does not happen before A.

24 NOTE 11 The set of side effects from which a given evaluation may take its value is also restricted by the rest of the rules described here, and in particular, by the coherence requirements subsequently in this subclause.

25 If an operation A that modifies an atomic object M happens before an operation B that modifies M, then A shall be earlier than B in the modification order of M.

26 NOTE 12 Such a requirement is known as "write-write coherence".

27 If a value computation A of an atomic object M happens before a value computation B of M, and A takes its value from a side effect X on M, then the value computed by B shall either be the value stored by X or the value stored by a side effect Y on M, where Y follows X in the modification order of M.

28 NOTE 13 Such a requirement is known as "read-read coherence".

29 If a value computation A of an atomic object M happens before an operation B on M, then A shall take its value from a side effect X on M, where X precedes B in the modification order of M.

30 NOTE 14 Such a requirement is known as "read-write coherence".

31 If a side effect X on an atomic object M happens before a value computation B of M, then the evaluation B shall take its value from X or from a side effect Y that follows X in the modification order of M.

32 NOTE 15 Such a requirement is known as "write-read coherence".

33 NOTE 16 This effectively disallows compiler reordering of atomic operations to a single object, even if both operations are "relaxed" loads. By doing so, it effectively makes the "cache coherence" guarantee provided by most hardware available to C atomic operations.

34 NOTE 17 The value observed by a load of an atomic object depends on the "happens before" relation, which in turn depends on the values observed by loads of atomic objects. The intended reading is that there exists an association of atomic loads with modifications they observe that, together with suitably chosen modification orders and the "happens before" relation derived as described previously, satisfy the resulting constraints as imposed here.

35 The execution of a program contains a *data race* if it contains two conflicting actions in different threads, at least one of which is not atomic, and neither happens before the other. Any such data race results in undefined behavior.

36 NOTE 18 It can be shown that programs that correctly use simple mutexes and `memory_order_seq_cst` operations to prevent all data races, and use no other synchronization operations, behave as though the operations executed by their constituent threads were simply interleaved, with each value computation of an object being the last value stored in that interleaving. This is normally referred to as "sequential consistency". However, this applies only to data-race-free programs, and data-race-free programs cannot observe most program transformations that do not change single-threaded program semantics. In fact, most single-threaded program transformations continue to be allowed, since any program that behaves differently as a result of such transformations necessarily has undefined behavior even before such a transformation is applied.

37 NOTE 19 Compiler transformations that introduce assignments to a potentially shared memory location that would not be modified by the abstract machine are generally precluded by this document, since such an assignment can overwrite another assignment by a different thread in cases in which an abstract machine execution would not have encountered a data race. This includes implementations of data member assignment that overwrite adjacent members in separate memory locations. Reordering of atomic loads in cases in which the atomics in question can alias is also generally precluded, since this could violate the coherence requirements.

38 NOTE 20 Transformations that introduce a speculative read of a potentially shared memory location may not preserve the semantics of the program as defined in this document, since they potentially introduce a data race. However, they are typically valid in the context of an optimizing compiler that targets a specific machine with well-defined semantics for data races. They would be invalid for a hypothetical machine that is not tolerant of races or provides hardware race detection.

5.2 Environmental considerations

5.2.1 Character sets

1 Two sets of characters and their associated *collating sequences* shall be defined: the set in which source files are written (the *source character set*), and the set interpreted in the execution environment (the *execution character set*). Each set is further divided into a *basic character set*, whose contents are given by this subclause, and a set of zero or more locale-specific members (which are not members of the basic character set) called *extended characters*. The combined set is also called the *extended character set*. The values of the members of the execution character set are implementation-defined.

2 In a character constant or string literal, members of the execution character set shall be represented by corresponding members of the source character set or by *escape sequences* consisting of the backslash \ followed by one or more characters. A byte with all bits set to 0, called the *null character*, shall exist in the basic execution character set; it is used to terminate a character string.

3 Both the basic source and basic execution character sets shall have the following members: the 26 *uppercase letters* of the *Latin alphabet*


```
A  B  C  D  E  F  G  H  I  J  K  L  M
N  O  P  Q  R  S  T  U  V  W  X  Y  Z
```

the 26 *lowercase letters* of the Latin alphabet

```
a  b  c  d  e  f  g  h  i  j  k  l  m
n  o  p  q  r  s  t  u  v  w  x  y  z
```

the 10 decimal *digit*

```
0  1  2  3  4  5  6  7  8  9
```

the following 32 *graphic characters*

```
!  "  #  %  &  '  (  )  *  +  ,  -  .  /  :
;  <  =  >  ?  [  \  ]  ^  _  {  |  }  ~
@  $  `
```

the space character, and control characters representing horizontal tab, vertical tab, and form feed. The representation of each member of the source and execution basic character sets shall fit in a byte. In both the source and execution basic character sets, the value of each character after `0` in the preceding list of decimal digits shall be one greater than the value of the previous. In source files, there shall be some way of indicating the end of each line of text; this document treats such an end-of-line indicator as if it were a single new-line character. In the basic execution character set, there shall be control characters representing alert, backspace, carriage return, and new line. If any other characters are encountered in a source file (except in an identifier, a character constant, a string literal, a header name, a comment, or a preprocessing token that is never converted to a token), the behavior is undefined.

4 A *letter* is an uppercase letter or a lowercase letter as defined previously in this subclause; in this document the term does not include other characters that are letters in other alphabets.

5 The universal character name construct provides a way to name other characters.

Forward references: universal character names (6.4.3), character constants (6.4.4.5), preprocessing directives (6.10), string literals (6.4.5), comments (6.4.9), string (7.1.1).

5.2.2 Multibyte characters

1 The source character set may contain multibyte characters, used to represent members of the extended character set. The execution character set may also contain multibyte characters, which are not required to have the same encoding as for the source character set. For both character sets, the following shall hold:

- The basic character set shall be present and each character shall be encoded as a single byte.
- The presence, meaning, and representation of any additional members is locale-specific.
- A multibyte character set may have a *state-dependent encoding*, wherein each sequence of multibyte characters begins in an *initial shift state* and enters other locale-specific *shift states* when specific multibyte characters are encountered in the sequence. While in the initial shift state, all single-byte characters retain their usual interpretation and do not alter the shift state. The interpretation for subsequent bytes in the sequence is a function of the current shift state.
- A byte with all bits zero shall be interpreted as a null character independent of shift state. Such a byte shall not occur as part of any other multibyte character.

2 For source files, the following shall hold:

- An identifier, comment, string literal, character constant, or header name shall begin and end in the initial shift state.
- An identifier, comment, string literal, character constant, or header name shall consist of a sequence of valid multibyte characters.

5.2.3 Character display semantics

1 The *active position* is that location on a display device where the next character output by the **fputc** function would appear. The intent of writing a printing character (as defined by the **isprint** function) to a display device is to display a graphic representation of that character at the active position and then advance the active position to the next position on the current line. The direction of writing is locale-specific. If the active position is at the final position of a line (if there is one), the behavior of the display device is unspecified.

2 Alphabetic escape sequences representing non-graphic characters in the execution character set are intended to produce actions on display devices as follows:

`\a` (*alert*) Produces an audible or visible alert without changing the active position.

`\b` (*backspace*) Moves the active position to the previous position on the current line. If the active position is at the initial position of a line, the behavior of the display device is unspecified.

`\f` (*form feed*) Moves the active position to the initial position at the start of the next logical page.

`\n` (*new line*) Moves the active position to the initial position of the next line.

`\r` (*carriage return*) Moves the active position to the initial position of the current line.

`\t` (*horizontal tab*) Moves the active position to the next horizontal tabulation position on the current line. If the active position is at or past the last defined horizontal tabulation position, the behavior of the display device is unspecified.

`\v` (*vertical tab*) Moves the active position to the initial position of the next vertical tabulation position. If the active position is at or past the last defined vertical tabulation position, the behavior of the display device is unspecified.

3 Each of these escape sequences shall produce a unique implementation-defined value which can be stored in a single **char** object. The external representations in a text file are not necessarily identical to the internal representations, and are outside the scope of this document.

Forward references: the **isprint** function (7.4.2.8), the **fputc** function (7.23.7.3).

5.2.4 Signals and interrupts

1 Functions shall be implemented such that they may be interrupted at any time by a signal, or may be called by a signal handler, or both, with no alteration to earlier, but still active, invocations' control flow (after the interruption), function return values, or objects with automatic storage duration. All such objects shall be maintained outside the *function image* (the instructions that compose the executable representation of a function) on a per-invocation basis.

5.2.5 Environmental limits

5.2.5.1 General

1 Both the translation and execution environments constrain the implementation of language translators and libraries. The following summarizes the language-related environmental limits on a conforming implementation; the library-related limits are discussed in Clause 7.

5.2.5.2 Translation limits

1 The implementation shall be able to translate and execute a program that uses but does not exceed the following limitations for these constructs and entities:[13)]

— 127 nesting levels of blocks

— 63 nesting levels of conditional inclusion

— 12 pointer, array, and function declarators (in any combinations) modifying an arithmetic, structure, union, or **void** type in a declaration

[13)] Implementations are encouraged to avoid imposing fixed translation limits whenever possible.

— 63 nesting levels of parenthesized declarators within a full declarator

— 63 nesting levels of parenthesized expressions within a full expression

— 63 significant initial characters in an internal identifier or a macro name (each universal character name or extended source character is considered a single character)

— 31 significant initial characters in an external identifier (each universal character name specifying a short identifier of 00FFFF or less is considered 6 characters, each universal character name specifying a short identifier of 010000 or more is considered 10 characters, and each extended source character is considered the same number of characters as the corresponding universal character name, if any)[14)]

— 4095 external identifiers in one translation unit

— 511 identifiers with block scope declared in one block

— 4095 macro identifiers simultaneously defined in one preprocessing translation unit

— 127 parameters in one function definition

— 127 arguments in one function call

— 127 parameters in one macro definition

— 127 arguments in one macro invocation

— 4095 characters in a logical source line

— 4095 characters in a string literal (after concatenation)

— 32767 bytes in an object (in a hosted environment only)

— 15 nesting levels for **`#include`**d files

— 1023 **`case`** labels for a **`switch`** statement (excluding those for any nested **`switch`** statements)

— 1023 members in a single structure or union

— 1023 enumeration constants in a single enumeration

— 63 levels of nested structure or union definitions in a single member declaration list

5.2.5.3 Numerical limits

5.2.5.3.1 General

1 An implementation is required to document all the limits specified in this subclause, which are specified in the headers `<limits.h>` and `<float.h>`. Additional limits are specified in `<stdint.h>`.

Forward references: integer types `<stdint.h>` (7.22).

5.2.5.3.2 Characteristics of integer types `<limits.h>`

1 The values given subsequently shall be replaced by constant expressions suitable for use in conditional expression inclusion preprocessing directives. Their implementation-defined values shall be equal or greater to those shown.

— width for an object of type **`bool`**[15)]

```
BOOL_WIDTH                      1
```

— number of bits for smallest object that is not a bit-field (byte)

[14)] See "future language directions" (6.11.3).
[15)] This value is exact.

```
CHAR_BIT                        8
```

The macros **CHAR_WIDTH**, **SCHAR_WIDTH**, and **UCHAR_WIDTH** that represent the width of the types **char**, **signed char** and **unsigned char** shall expand to the same value as **CHAR_BIT**.

— width for an object of type **unsigned short int**

```
USHRT_WIDTH                     16
```

The macro **SHRT_WIDTH** represents the width of the type **short int** and shall expand to the same value as **USHRT_WIDTH**.

— width for an object of type **unsigned int**

```
UINT_WIDTH                      16
```

The macro **INT_WIDTH** represents the width of the type **int** and shall expand to the same value as **UINT_WIDTH**.

— width for an object of type **unsigned long int**

```
ULONG_WIDTH                     32
```

The macro **LONG_WIDTH** represents the width of the type **long int** and shall expand to the same value as **ULONG_WIDTH**.

— width for an object of type **unsigned long long int**

```
ULLONG_WIDTH                    64
```

The macro **LLONG_WIDTH** represents the width of the type **long long int** and shall expand to the same value as **ULLONG_WIDTH**.

— maximum width of a bit-precise integer type

```
BITINT_MAXWIDTH     /* see the following */
```

The macro **BITINT_MAXWIDTH** represents the maximum width N supported by the declaration of a bit-precise integer (6.2.5) in the type specifier **_BitInt**(N). The value **BITINT_MAXWIDTH** shall expand to a value that is greater than or equal to the value of **ULLONG_WIDTH**.

— maximum number of bytes in a multibyte character, for any supported locale

```
MB_LEN_MAX                      1
```

2 For all unsigned integer types for which <limits.h> or <stdint.h> define a macro with suffix **_WIDTH** holding its width N, there is a macro with suffix **_MAX** holding the maximal value $2^N - 1$ that is representable by the type and that has the same type as would an expression that is an object of the corresponding type converted according to the integer promotions. If the value is in the range of the type **uintmax_t** (7.22.1.5) the macro is suitable for use in conditional expression inclusion preprocessing directives.

3 For all signed integer types for which <limits.h> or <stdint.h> define a macro with suffix **_WIDTH** holding its width N, there are macros with suffix **_MIN** and **_MAX** holding the minimal and maximal values -2^{N-1} and $2^{N-1} - 1$ that are representable by the type and that have the same type as would an expression that is an object of the corresponding type converted according to the integer promotions. If the values are in the range of the type **intmax_t** (7.22.1.5) the macros are suitable for use in conditional expression inclusion preprocessing directives.

4 If an object of type **char** can hold negative values, the value of **CHAR_MIN** shall be the same as that of **SCHAR_MIN** and the value of **CHAR_MAX** shall be the same as that of **SCHAR_MAX**. Otherwise, the value of **CHAR_MIN** shall be 0 and the value of **CHAR_MAX** shall be the same as that of **UCHAR_MAX** (see 6.2.5.).

Forward references: representations of types (6.2.6), conditional inclusion (6.10.2), integer types <stdint.h> (7.22).

5.2.5.3.3 Characteristics of floating types <float.h>

1 The characteristics of floating types are defined in terms of a model that describes a representation of floating-point numbers and allows other values. The characteristics provide information about an implementation's floating-point arithmetic.[16] An implementation that defines **__STDC_IEC_60559_BFP__** or **__STDC_IEC_559__** shall implement floating types and arithmetic conforming to ISO/IEC 60559 as specified in Annex F of this document. An implementation that defines **__STDC_IEC_60559_COMPLEX__** or **__STDC_IEC_559_COMPLEX__** shall implement complex types and arithmetic conforming to ISO/IEC 60559 as specified in Annex G of this document.

2 The following parameters are used to define the model for each floating type:

s	sign (± 1)
b	base or radix of exponent representation (an integer > 1)
e	exponent (an integer between a minimum $e_{\min}$ and a maximum $e_{\max}$)
p	precision (the number of base-b digits in the significand)
f_k	nonnegative integers less than b (the significand digits)

For each floating type, the parameters b, p, $e_{\min}$, and $e_{\max}$ are fixed constants.

3 For each floating type, a *floating-point number* (x) is defined by the following model:

$$x = sb^e \sum_{k=1}^{p} f_k b^{-k}, \quad e_{\min} \le e \le e_{\max}$$

4 Model floating-point numbers x with $f_1 > 0$ are called *normalized floating-point numbers*.

5 Model floating-point numbers $x \neq 0$ with $f_1 = 0$ and $e = e_{\min}$ are called *subnormal floating-point numbers*.

6 Model floating-point numbers $x \neq 0$ with $f_1 = 0$ and $e > e_{\min}$ are called *unnormalized floating-point numbers*.

7 Model floating-point numbers x with all $f_k = 0$ are zeros.

8 Floating types shall be able to represent signed zeros or an unsigned zero and all normalized floating-point numbers. In addition, floating types may be able to contain other kinds of floating-point numbers, such as subnormal floating-point numbers and unnormalized floating-point numbers, and values that are not floating-point numbers, such as NaNs and (signed and unsigned) infinities.

9 NOTE 1 Some implementations have types that include finite numbers with range and/or precision that are not covered by the model.

10 A *NaN* is a value signifying Not-a-Number. A *quiet NaN* propagates through almost every arithmetic operation without raising a floating-point exception; a *signaling NaN* generally raises a floating-point exception when occurring as an arithmetic operand.

11 NOTE 2 ISO/IEC 60559 specifies quiet and signaling NaNs. For implementations that do not support ISO/IEC 60559, the terms quiet NaN and signaling NaN are intended to apply to values with similar behavior.

12 Wherever values are unsigned, any requirement in this document to get the sign shall produce an unspecified sign, and any requirement to set the sign shall be ignored, unless otherwise specified.

13 NOTE 3 Bit representations of floating-point values can include a sign bit, even if the values can be regarded as unsigned; ISO/IEC 60559 NaNs are such values.

14 Whether and in what cases subnormal numbers are treated as zeros is implementation-defined. Subnormal numbers that in some cases are treated by arithmetic operations as zeros are properly

[16] The floating-point model is intended to clarify the description of each floating-point characteristic and does not require the floating-point arithmetic of the implementation to be identical.

classified as subnormal. However, object representations that could represent subnormal numbers but that are always treated by arithmetic operations as zeros are non-canonical zeros, and the values are properly classified as zero, not subnormal. ISO/IEC 60559 arithmetic (with default exception handling) always treats subnormal numbers as nonzero.

15 A value is negative if and only if it compares less than 0. Thus, negative zeros and NaNs are not negative values.

16 An implementation may prefer particular representations of values that have multiple representations in a floating type, 6.2.6.1 not withstanding. The preferred representations of a floating type, including unique representations of values in the type, are called *canonical*. A floating type may also contain *non-canonical* representations, for example, redundant representations of some or all its values, or representations that are extraneous to the floating-point model. Typically, floating-point operations deliver results with canonical representations. ISO/IEC 60559 operations deliver results with canonical representations, unless specified otherwise.

17 NOTE 4 The library operations **`iscanonical`** and **`canonicalize`** distinguish canonical (preferred) representations, but this distinction alone does not imply that canonical and non-canonical representations are of different values.

18 NOTE 5 Some of the values in the ISO/IEC 60559 decimal formats have non-canonical representations (as well as a canonical representation).

19 The minimum range of representable values for a floating type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of that type is extended to all negative real numbers; likewise, if positive infinity is representable in a type, the range of that type is extended to all positive real numbers.

20 The accuracy of the floating-point operations (`+`, `-`, `*`, `/`) and of most of the library functions in `<math.h>` and `<complex.h>` that return floating-point results is implementation-defined, as is the accuracy of the conversion between floating-point internal representations and string representations performed by the library functions in `<stdio.h>`, `<stdlib.h>`, and `<wchar.h>`. The implementation may state that the accuracy is unknown. Decimal floating-point operations have stricter requirements.

21 All integer values in the `<float.h>` header, except **`FLT_ROUNDS`**, shall be constant expressions suitable for use in conditional expression inclusion preprocessing directives; all floating values shall be arithmetic constant expressions. All except **`CR_DECIMAL_DIG`** (F.5), **`DECIMAL_DIG`**, **`DEC_EVAL_METHOD`**, **`FLT_EVAL_METHOD`**, **`FLT_RADIX`**, and **`FLT_ROUNDS`** have separate names for all floating types. The floating-point model representation is provided for all values except **`DEC_EVAL_METHOD`**, **`FLT_EVAL_METHOD`** and **`FLT_ROUNDS`**.

22 The remainder of this subclause specifies characteristics of standard floating types.

23 The rounding mode for floating-point addition for standard floating types is characterized by the implementation-defined value of **`FLT_ROUNDS`**. Evaluation of **`FLT_ROUNDS`** correctly reflects any execution-time change of rounding mode through the function **`fesetround`** in `<fenv.h>`.

−1	indeterminable
0	toward zero
1	to nearest, ties to even
2	toward positive infinity
3	toward negative infinity
4	to nearest, ties away from zero

All other values for **`FLT_ROUNDS`** characterize implementation-defined rounding behavior.

24 Whether a type has the same base (b), precision (p), and exponent range ($e_{\min} \le e \le e_{\max}$) as an ISO/IEC 60559 format is characterized by the implementation-defined values of **FLT_IS_IEC_60559**, **DBL_IS_IEC_60559** and **LDBL_IS_IEC_60559** (this does not imply conformance to Annex F):

0 type does not have the precision and exponent range of an ISO/IEC 60559 format

1 type has the precision and exponent range of an ISO/IEC 60559 format

25 NOTE 6 Outside of the normalized floating-point numbers, the representability of values (e.g. negative zero) of the ISO/IEC 60559 format is not implied.

26 The values of floating type yielded by operators subject to the usual arithmetic conversions, including the values yielded by the implicit conversion of operands, and the values of floating constants are evaluated to a format whose range and precision may be greater than required by the type. Such a format is called an *evaluation format*. In all cases, assignment and cast operators yield values in the format of the type. The extent to which evaluation formats are used is characterized by the value of **FLT_EVAL_METHOD**:[17)]

−1 indeterminable;

0 evaluate all operations and constants just to the range and precision of the type;

1 evaluate operations and constants of type **float** and **double** to the range and precision of the **double** type, evaluate **long double** operations and constants to the range and precision of the **long double** type;

2 evaluate all operations and constants to the range and precision of the **long double** type.

All other negative values for **FLT_EVAL_METHOD** characterize implementation-defined behavior. The value of **FLT_EVAL_METHOD** does not characterize values returned by function calls (see 6.8.7.5, F.6).

27 The presence or absence of subnormal numbers is characterized by the implementation-defined values of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM**:

−1 indeterminable

0 absent (type does not support subnormal numbers)

1 present (type does support subnormal numbers)

The use of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM** macros is an obsolescent feature.

28 Each of the signaling NaN macros

```
FLT_SNAN
DBL_SNAN
LDBL_SNAN
```

is defined if and only if the respective type contains signaling NaNs. They expand to a constant expression of the respective type representing a signaling NaN. If an optional unary + or - operator followed by a signaling NaN macro is used as an initializer that is evaluated at translation time, the object is initialized with a signaling NaN value.

29 The macro

```
INFINITY
```

[17)] The evaluation method determines evaluation formats of expressions involving all floating types, not just real types. For example, if **FLT_EVAL_METHOD** is 1, then the product of two **float _Complex** operands is represented in the **double _Complex** format, and its parts are evaluated to **double**.

is defined if and only if the implementation supports an infinity for the type **`float`**. It expands to a constant expression of type **`float`** representing positive or unsigned infinity.

30 The macro

```
NAN
```

is defined if and only if the implementation supports quiet NaNs for the **`float`** type. It expands to a constant expression of type **`float`** representing a quiet NaN.

31 The values given in the following list shall be replaced by constant expressions with implementation-defined values that are greater or equal in magnitude (absolute value) to those shown, with the same sign:

— radix of exponent representation, b

```
FLT_RADIX                       2
```

— number of base-**`FLT_RADIX`** digits in the floating-point significand, p

```
FLT_MANT_DIG
DBL_MANT_DIG
LDBL_MANT_DIG
```

— number of decimal digits, n, such that any floating-point number with p radix b digits can be rounded to a floating-point number with n decimal digits and back again without change to the value,

$$\begin{cases} p \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lceil 1 + p \log_{10} b \rceil & \text{otherwise} \end{cases}$$

```
FLT_DECIMAL_DIG                 6
DBL_DECIMAL_DIG                10
LDBL_DECIMAL_DIG               10
```

— number of decimal digits, n, such that any floating-point number in the widest of the supported floating types and the supported ISO/IEC 60559 encodings with $p_{\max}$ radix b digits can be rounded to a floating-point number with n decimal digits and back again without change to the value,

$$\begin{cases} p_{\max} \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lceil 1 + p_{\max} \log_{10} b \rceil & \text{otherwise} \end{cases}$$

```
DECIMAL_DIG                    10
```

This is an obsolescent feature, see 7.33.8.

— number of decimal digits, q, such that any floating-point number with q decimal digits can be rounded into a floating-point number with p radix b digits and back again without change to the q decimal digits,

$$\begin{cases} p \log_{10} b & \text{if } b \text{ is a power of } 10 \\ \lfloor (p-1) \log_{10} b \rfloor & \text{otherwise} \end{cases}$$

```
FLT_DIG                         6
DBL_DIG                        10
LDBL_DIG                       10
```

— minimum negative integer such that **FLT_RADIX** raised to one less than that power is a normalized floating-point number, $e_{\min}$

```
FLT_MIN_EXP
DBL_MIN_EXP
LDBL_MIN_EXP
```

— minimum negative integer such that 10 raised to that power is in the range of normalized floating-point numbers, $\lceil \log_{10} b^{e_{\min}-1} \rceil$

```
FLT_MIN_10_EXP          -37
DBL_MIN_10_EXP          -37
LDBL_MIN_10_EXP         -37
```

— maximum integer such that **FLT_RADIX** raised to one less than that power is a representable finite floating-point number; if that representable finite floating-point number is normalized, the value of the macro is $e_{\max}$

```
FLT_MAX_EXP
DBL_MAX_EXP
LDBL_MAX_EXP
```

— maximum integer such that 10 raised to that power is in the range of representable finite floating-point numbers, $\lfloor \log_{10}((1 - b^{-p})b^{e_{\max}}) \rfloor$

```
FLT_MAX_10_EXP          +37
DBL_MAX_10_EXP          +37
LDBL_MAX_10_EXP         +37
```

32 The values given in the following list shall be replaced by constant expressions with implementation-defined values that are greater than or equal to those shown:

— maximum representable finite floating-point number; if that number is normalized, its value is $(1 - b^{-p})b^{e_{\max}}$

```
FLT_MAX                 1E+37
DBL_MAX                 1E+37
LDBL_MAX                1E+37
```

— maximum normalized floating-point number, $(1 - b^{-p})b^{e_{\max}}$

```
FLT_NORM_MAX            1E+37
DBL_NORM_MAX            1E+37
LDBL_NORM_MAX           1E+37
```

33 The values given in the following list shall be replaced by constant expressions with implementation-defined (positive) values that are less than or equal to those shown:

— the difference between 1 and the least normalized value greater than 1 that is representable in the given floating type, b^{1-p}

```
FLT_EPSILON             1E-5
DBL_EPSILON             1E-9
LDBL_EPSILON            1E-9
```

— minimum normalized positive floating-point number, $b^{e_{\min}-1}$

```
FLT_MIN                 1E-37
DBL_MIN                 1E-37
LDBL_MIN                1E-37
```

— minimum positive floating-point number

```
FLT_TRUE_MIN            1E-37
DBL_TRUE_MIN            1E-37
LDBL_TRUE_MIN           1E-37
```

Recommended practice

34 Conversion between real floating type and decimal character sequence with at most *T*`_DECIMAL_DIG` digits should be correctly rounded, where *T* is the macro prefix for the type. This assures conversion from real floating type to decimal character sequence with *T*`_DECIMAL_DIG` digits and back, using to-nearest rounding, is the identity function.

35 EXAMPLE 1 The following describes an artificial floating-point representation that meets the minimum requirements of this document, and the appropriate values in a <float.h> header for type **float**:

$$x = s16^e \sum_{k=1}^{6} f_k 16^{-k}, \quad -31 \le e \le +32$$

```
FLT_RADIX                  16
FLT_MANT_DIG                6
FLT_EPSILON          9.53674316E-07F
FLT_DECIMAL_DIG             9
FLT_DIG                     6
FLT_MIN_EXP               -31
FLT_MIN              2.93873588E-39F
FLT_MIN_10_EXP            -38
FLT_MAX_EXP               +32
FLT_MAX              3.40282347E+38F
FLT_MAX_10_EXP            +38
```

36 EXAMPLE 2

The following describes floating-point representations that also meet the requirements for binary32 and binary64 numbers in ISO/IEC 60559,[18] and the appropriate values in a <float.h> header for types **float** and **double**. Note that the decimal floating constants may not give correct values (and hence are not appropriate values in a <float.h> header) if `FLT_EVAL_METHOD` is not 0 or if a translation-time rounding mode other than the ISO/IEC 60559 default is supported (either as the default or as a constant rounding mode set by an `FENV_ROUND` pragma). The hexadecimal floating constants are correct in all such cases because their values are exactly representable in the type.

$$x_f = s2^e \sum_{k=1}^{24} f_k 2^{-k}, \quad -125 \le e \le +128$$

$$x_d = s2^e \sum_{k=1}^{53} f_k 2^{-k}, \quad -1021 \le e \le +1024$$

```
FLT_IS_IEC_60559            1
FLT_RADIX                   2
FLT_MANT_DIG               24
FLT_EPSILON          1.19209290E-07F // decimal constant
FLT_EPSILON                0X1P-23F // hex constant
```

[18] The floating-point model in ISO/IEC 60559 sums powers of *b* from zero, so the values of the exponent limits are one less than shown here.

```
FLT_DECIMAL_DIG                    9
FLT_DIG                            6
FLT_MIN_EXP                     -125
FLT_MIN              1.17549435E-38F // decimal constant
FLT_MIN                    0X1P-126F // hex constant
FLT_TRUE_MIN         1.40129846E-45F // decimal constant
FLT_TRUE_MIN               0X1P-149F // hex constant
FLT_HAS_SUBNORM                    1
FLT_MIN_10_EXP                   -37
FLT_MAX_EXP                     +128
FLT_MAX              3.40282347E+38F // decimal constant
FLT_MAX              0X1.fffffeP127F // hex constant
FLT_MAX_10_EXP                   +38
DBL_MANT_DIG                      53
DBL_IS_IEC_60559                   1
DBL_EPSILON   2.2204460492503131E-16 // decimal constant
DBL_EPSILON                 0X1P-52 // hex constant
DBL_DECIMAL_DIG                   17
DBL_DIG                           15
DBL_MIN_EXP                   -1021
DBL_MIN      2.2250738585072014E-308 // decimal constant
DBL_MIN                    0X1P-1022 // hex constant
DBL_TRUE_MIN 4.9406564584124654E-324 // decimal constant
DBL_TRUE_MIN               0X1P-1074 // hex constant
DBL_HAS_SUBNORM                    1
DBL_MIN_10_EXP                  -307
DBL_MAX_EXP                    +1024
DBL_MAX      1.7976931348623157E+308 // decimal constant
DBL_MAX       0X1.fffffffffffffP1023 // hex constant
DBL_MAX_10_EXP                  +308
```

Forward references: conditional inclusion (6.10.2), predefined macro names (6.10.10), complex arithmetic <complex.h> (7.3), extended multibyte and wide character utilities <wchar.h> (7.31), floating-point environment <fenv.h> (7.6), general utilities <stdlib.h> (7.24), input/output <stdio.h> (7.23), mathematics <math.h> (7.12), ISO/IEC 60559 floating-point arithmetic (Annex F), ISO/IEC 60559-compatible complex arithmetic (Annex G).

5.2.5.3.4 Characteristics of decimal floating types in <float.h>

1 This subclause specifies macros in <float.h> that provide characteristics of decimal floating types (an optional feature) in terms of the model presented in 5.2.5.3.3. An implementation shall provide these macros if and only if it defines **__STDC_IEC_60559_DFP__**. The prefixes **DEC32_**, **DEC64_**, and **DEC128_** denote the types **_Decimal32**, **_Decimal64**, and **_Decimal128** respectively.

2 **DEC_EVAL_METHOD** is the decimal floating-point analog of **FLT_EVAL_METHOD** (5.2.5.3.3). Its implementation-defined value characterizes the use of evaluation formats for decimal floating types:

−1 indeterminable;

0 evaluate all operations and constants just to the range and precision of the type;

1 evaluate operations and constants of type **_Decimal32** and **_Decimal64** to the range and precision of the **_Decimal64** type, evaluate **_Decimal128** operations and constants to the range and precision of the **_Decimal128** type;

2 evaluate all operations and constants to the range and precision of the **_Decimal128** type.

3 Each of the decimal signaling NaN macros

```
DEC32_SNAN
DEC64_SNAN
```

```
DEC128_SNAN
```

expands to a constant expression of the respective decimal floating type representing a signaling NaN. If an optional unary + or - operator followed by a signaling NaN macro is used for initializing an object of the same type that has static or thread storage duration, the object is initialized with a signaling NaN value.

4 The macro

```
DEC_INFINITY
```

expands to a constant expression of type **_Decimal32** representing positive infinity.

5 The macro

```
DEC_NAN
```

expands to a constant expression of type **_Decimal32** representing a quiet NaN.

6 The integer values given in the following lists shall be replaced by constant expressions suitable for use in conditional expression inclusion preprocessing directives:

— radix of exponent representation, b(=10)

For the standard floating types, this value is implementation-defined and is specified by the macro **FLT_RADIX**. For the decimal floating types there is no corresponding macro, since the value 10 is an inherent property of the types. Wherever **FLT_RADIX** appears in a description of a function that has versions that operate on decimal floating types, it is noted that for the decimal floating-point versions the value used is implicitly 10, rather than **FLT_RADIX**.

— number of digits in the coefficient

```
DEC32_MANT_DIG     7
DEC64_MANT_DIG     16
DEC128_MANT_DIG    34
```

— minimum exponent

```
DEC32_MIN_EXP      -94
DEC64_MIN_EXP      -382
DEC128_MIN_EXP     -6142
```

— maximum exponent

```
DEC32_MAX_EXP      97
DEC64_MAX_EXP      385
DEC128_MAX_EXP     6145
```

— maximum representable finite decimal floating-point number (there are 6, 15 and 33 9's after the decimal points respectively)

```
DEC32_MAX          9.999999E96DF
DEC64_MAX          9.999999999999999E384DD
DEC128_MAX         9.999999999999999999999999999999999E6144DL
```

— the difference between 1 and the least value greater than 1 that is representable in the given floating type


```
DEC32_EPSILON      1E-6DF
DEC64_EPSILON      1E-15DD
DEC128_EPSILON     1E-33DL
```

— minimum normalized positive decimal floating-point number

```
DEC32_MIN          1E-95DF
DEC64_MIN          1E-383DD
DEC128_MIN         1E-6143DL
```

— minimum positive subnormal decimal floating-point number

```
DEC32_TRUE_MIN     0.000001E-95DF
DEC64_TRUE_MIN     0.000000000000001E-383DD
DEC128_TRUE_MIN    0.00000000000000000000000000000001E-6143DL
```

7 For decimal floating-point arithmetic, it is often convenient to consider an alternate equivalent model where the significand is represented with integer rather than fraction digits. With s, b, e, p, and f_k as defined in 5.2.5.3.3, a floating-point number x is defined by the model:

$$x = s \cdot b^{(e-p)} \sum_{k=1}^{p} f_k \cdot b^{(p-k)}$$

8 With b fixed to 10, a decimal floating-point number x is thus:

$$x = s \cdot 10^{(e-p)} \sum_{k=1}^{p} f_k \cdot 10^{(p-k)}$$

The *quantum exponent* is $q = e - p$ and the *coefficient* is $c = f_1 f_2 \cdots f_p$, which is an integer between 0 and $10^p - 1$, inclusive. Thus, $x = s \cdot c \cdot 10^q$ is represented by the triple of integers (s, c, q). The *quantum* of x is 10^q, which is the value of a unit in the last place of the coefficient.

Table 5.1: Quantum exponent ranges

Type	**_Decimal32**	**_Decimal64**	**_Decimal128**
Maximum Quantum Exponent ($q_{\max}$)	90	369	6111
Minimum Quantum Exponent ($q_{\min}$)	-101	-398	-6176

9 For binary floating-point arithmetic following ISO/IEC 60559, representations in the model described in 5.2.5.3.3 that have the same numerical value are indistinguishable in the arithmetic. However, for decimal floating-point arithmetic, representations that have the same numerical value but different quantum exponents, e.g. $(+1, 10, -1)$ representing 1.0 and $(+1, 100, -2)$ representing 1.00, are distinguishable. To facilitate exact fixed-point calculation, operation results that are of decimal floating type have a *preferred quantum exponent*, as specified in ISO/IEC 60559, which is determined by the quantum exponents of the operands if they have decimal floating types (or by specific rules for conversions from other types). Table 5.2 gives rules for determining preferred quantum exponents for results of ISO/IEC 60559 operations, and for other operations specified in this document. When exact, these operations produce a result with their preferred quantum exponent, or as close to it as possible within the limitations of the type. When inexact, these operations produce a result with the least possible quantum exponent. For example, the preferred quantum exponent for addition is the minimum of the quantum exponents of the operands. Hence $(+1, 123, -2) + (+1, 4000, -3) = (+1, 5230, -3)$ or $1.23 + 4.000 = 5.230$.

10 Table 5.2 shows, for each operation delivering a result in decimal floating-point format, how the preferred quantum exponents of the operands, $Q(\mathtt{x})$, $Q(\mathtt{y})$, etc., determine the preferred quantum

exponent of the operation result, provided the table formula is defined for the arguments. For the cases where the formula is undefined and the function result is $\pm\infty$, the preferred quantum exponent is immaterial because the quantum exponent of $\pm\infty$ is defined to be infinity. For the other cases where the formula is undefined and the function result is finite, the preferred quantum exponent is unspecified.

11 NOTE Although unspecified in ISO/IEC 60559, a preferred quantum exponent of 0 for these cases would be a reasonable implementation choice.

Table 5.2: Preferred quantum exponents

Operation	Preferred quantum exponent of result
roundeven, **round**, **trunc**, **ceil**, **floor**, **rint**, **nearbyint**	$\max(Q(\texttt{x}), 0)$
nextup, **nextdown**, **nextafter**, **nexttoward**	least possible
remainder	$\min(Q(\texttt{x}), Q(\texttt{y}))$
fmin, **fmax**, **fminimum**, **fmaximum**, **fminimum_mag**, **fmaximum_mag**, **fminimum_num**, **fmaximum_num**, **fminimum_mag_num**, **fmaximum_mag_num**	$Q(\texttt{x})$ if x gives the result, $Q(\texttt{y})$ if y gives the result
scalbn, **scalbln**	$Q(\texttt{x}) + \texttt{n}$
ldexp	$Q(\texttt{x}) + \texttt{p}$
logb	0
postfix ++ operator, postfix -- operator, prefix ++ operator, prefix -- operator	$\min(Q(\texttt{x}), 0)$
+, **d32add**, **d64add**	$\min(Q(\texttt{x}), Q(\texttt{y}))$
-, **d32sub**, **d64sub**	$\min(Q(\texttt{x}), Q(\texttt{y}))$
*, **d32mul**, **d64mul**	$Q(\texttt{x}) + Q(\texttt{y})$
/, **d32div**, **d64div**	$Q(\texttt{x}) - Q(\texttt{y})$
sqrt, **d32sqrt**, **d64sqrt**	$\lfloor Q(\texttt{x})/2 \rfloor$
fma, **d32fma**, **d64fma**	$\min(Q(\texttt{x}) + Q(\texttt{y}), Q(\texttt{z}))$
conversion from integer type	0
exact conversion from non-decimal floating type	0
inexact conversion from non-decimal floating type	least possible
conversion between decimal floating types	$Q(\texttt{x})$
*cx returned by **canonicalize**	$Q(*\texttt{x})$
strto, **wcsto**, **scanf**, floating constants of decimal floating type	see 7.24.1.6
-(x), +(x)	$Q(\texttt{x})$
fabs	$Q(\texttt{x})$
copysign	$Q(\texttt{x})$
quantize	$Q(\texttt{y})$
quantum	$Q(\texttt{x})$
*encptr returned by **encodedec**, **encodebin**	$Q(*\texttt{xptr})$
*xptr returned by **decodedec**, **decodebin**	$Q(*\texttt{encptr})$
fmod	$\min(Q(\texttt{x}), Q(\texttt{y}))$
fdim	$\min((Q(\texttt{x}), Q(\texttt{y}))$ if $\texttt{x} > \texttt{y}$, 0 if $\texttt{x} \leq \texttt{y}$
cbrt	$\lfloor Q(\texttt{x})/3 \rfloor$
hypot	$\min(Q(\texttt{x}), Q(\texttt{y}))$
pow	$\lfloor \texttt{y} \times Q(\texttt{x}) \rfloor$
modf	$Q(\texttt{value})$
*iptr returned by **modf**	$\max(Q(\texttt{value}), 0)$

frexp	Q(value) if value $= 0$, –(length of coefficient of value) otherwise
$*$res returned by **setpayload**, **setpayloadsig**	0 if pl does not represent a valid payload, not applicable otherwise (NaN returned)
getpayload	0 if $*$x is a NaN, unspecified otherwise
compoundn	$\lfloor$n $\times \min(0, Q($x$))\rfloor$
pown	$\lfloor$n $\times Q($x$)\rfloor$
powr	$\lfloor$y $\times Q($x$)\rfloor$
rootn	$\lfloor Q($x$)/$n$\rfloor$
rsqrt	$-\lfloor Q($x$)/2\rfloor$
transcendental functions	0

A function family listed in Table 5.2 indicates the functions for all decimal floating types, where the function family is represented by the name of the functions without a suffix. For example, **ceil** indicates the functions **ceild32**, **ceild64**, and **ceild128**.

Forward references: extended multibyte and wide character utilities <wchar.h> (7.31), floating-point environment <fenv.h> (7.6), general utilities <stdlib.h> (7.24), input/output <stdio.h> (7.23), mathematics <math.h> (7.12), type-generic mathematics <tgmath.h> (7.27), ISO/IEC 60559 floating-point arithmetic (Annex F).

6. Language

6.1 Notation

1 In the syntax notation used in this clause, syntactic categories (nonterminals) are indicated by *italic type*, and literal words and character set members (terminals) by **bold type**. A colon (*:*) following a nonterminal introduces its definition. Alternative definitions are listed on separate lines, except when prefaced by the words "one of". An optional symbol is indicated by the subscript "opt", so that

{ *expression*$_{\text{opt}}$ **}**

indicates an optional expression enclosed in braces.

2 When syntactic categories are referred to in the main text, they are not italicized and words are separated by spaces instead of hyphens.

3 A summary of the language syntax is given in Annex A.

6.2 Concepts

6.2.1 Scopes of identifiers, type names, and compound literals

1 An identifier can denote:

— a standard attribute, an attribute prefix, or an attribute name;

— an object;

— a function;

— a tag or a member of a structure, union, or enumeration;

— a typedef name;

— a label name;

— a macro name;

— or, a macro parameter.

The same identifier can denote different entities at different points in the program. A member of an enumeration is called an *enumeration constant*. Macro names and macro parameters are not considered further here, because prior to the semantic phase of program translation any occurrences of macro names in the source file are replaced by the preprocessing token sequences that constitute their macro definitions.

2 For each different entity that an identifier designates, the identifier is *visible* (i.e. can be used) only within a region of program text called its *scope*. Different entities designated by the same identifier either have different scopes or are in different name spaces. There are four kinds of scopes: function, file, block, and function prototype. (A *function prototype* is a declaration of a function.)

3 A label name is the only kind of identifier that has *function scope*. It can be used (in a **`goto`** statement) anywhere in the function in which it appears, and is declared implicitly by its syntactic appearance (followed by a `:` and a statement).

4 Every other identifier has scope determined by the placement of its declaration (in a declarator or type specifier). If the declarator or type specifier that declares the identifier appears outside of any block or list of parameters, the identifier has *file scope*, which terminates at the end of the translation unit. If the declarator or type specifier that declares the identifier appears inside a block or within the list of parameter declarations in a function definition, the identifier has *block scope*, which terminates at the end of the associated block. If the declarator or type specifier that declares

the identifier appears within the list of parameter declarations in a function prototype (not part of a function definition), the identifier has *function prototype scope*, which terminates at the end of the function declarator. If an identifier designates two different entities in the same name space, the scopes can overlap. If so, the scope of one entity (the *inner scope*) will end strictly before the scope of the other entity (the *outer scope*). Within the inner scope, the identifier designates the entity declared in the inner scope; the entity declared in the outer scope is *hidden* (and not visible) within the inner scope.

5 Unless explicitly stated otherwise, where this document uses the term "identifier" to refer to an entity (as opposed to the syntactic construct), it refers to the entity in the relevant name space whose declaration is visible at the point the identifier occurs.

6 Two identifiers have the *same scope* if and only if their scopes terminate at the same point.

7 Structure, union, and enumeration tags have scope that begins just after the appearance of the tag in a type specifier that declares the tag. Each enumeration constant has scope that begins just after the appearance of its defining enumerator in an enumerator list. An ordinary identifier that has an underspecified definition has scope that starts when the definition is completed; if the same ordinary identifier declares another entity with a scope that encloses the current block, that declaration is hidden as soon as the inner declarator is completed.[19] Any other identifier has scope that begins just after the completion of its declarator.

8 As a special case, a type name (which is not a declaration of an identifier) is considered to have a scope that begins just after the place within the type name where the omitted identifier would appear were it not omitted. A compound literal (which is an expression that provides access to an anonymous object) is associated with the scope of the type name used in its definition; that scope is either file scope, function prototype scope, or block scope.

Forward references: declarations (6.7), function calls (6.5.3.3), function calls (6.5.3.6), function definitions (6.9.2), identifiers (6.4.2), macro replacement (6.10.5), name spaces of identifiers (6.2.3), source file inclusion (6.10.3), statements and blocks (6.8).

6.2.2 Linkages of identifiers

1 An identifier declared in different scopes or in the same scope more than once can be made to refer to the same object or function by a process called *linkage*.[20] There are three kinds of linkage: external, internal, and none.

2 In the set of translation units and libraries that constitutes an entire program, each declaration of a particular identifier with *external linkage* denotes the same object or function. Within one translation unit, each declaration of an identifier with *internal linkage* denotes the same object or function. Each declaration of an identifier with *no linkage* denotes a unique entity.

3 If the declaration of a file scope identifier for:

- an object contains any of the storage-class specifiers **`static`** or **`constexpr`**;
- or, a function contains the storage-class specifier **`static`**,

then the identifier has internal linkage.[21]

4 For an identifier declared with the storage-class specifier **`extern`** in a scope in which a prior declaration of that identifier is visible,[22] if the prior declaration specifies internal or external linkage, the linkage of the identifier at the later declaration is the same as the linkage specified at the prior declaration. If no prior declaration is visible, or if the prior declaration specifies no linkage, then the identifier has external linkage.

5 If the declaration of an identifier for a function has no storage-class specifier, its linkage is determined exactly as if it were declared with the storage-class specifier **`extern`**. If the declaration of an identifier

[19] That means, that the outer declaration is not visible for the initializer.
[20] There is no linkage between different identifiers.
[21] A function declaration can contain the storage-class specifier **`static`** only if it is at file scope; see 6.7.2.
[22] As specified in 6.2.1, the later declaration can hide the prior declaration.

for an object has file scope and does not contain the storage-class specifier **`static`** or **`constexpr`**, its linkage is external.

6 The following identifiers have no linkage: an identifier declared to be anything other than an object or a function; an identifier declared to be a function parameter; a block scope identifier for an object declared without the storage-class specifier **`extern`**.

7 If, within a translation unit, the same identifier appears with both internal and external linkage, the behavior is undefined.

Forward references: declarations (6.7), expressions (6.5.1), external definitions (6.9), statements (6.8).

6.2.3 Name spaces of identifiers

1 If more than one declaration of a particular identifier is visible at any point in a translation unit, the syntactic context disambiguates uses that refer to different entities. Thus, there are separate *name spaces* for various categories of identifiers, as follows:

— *label names* (disambiguated by the syntax of the label declaration and use);

— the *tags* of structures, unions, and enumerations (disambiguated by following any[23] of the keywords **`struct`**, **`union`**, or **`enum`**);

— the *members* of structures or unions; each structure or union has a separate name space for its members (disambiguated by the type of the expression used to access the member via the `.` or `->` operator);

— standard attributes and attribute prefixes (disambiguated by the syntax of the attribute specifier and name of the attribute token) (6.7.13);

— the trailing identifier in an attribute prefixed token; each attribute prefix has a separate name space for the implementation-defined attributes that it introduces (disambiguated by the attribute prefix and the trailing identifier token);

— all other identifiers, called *ordinary identifiers* (declared in ordinary declarators or as enumeration constants).

Forward references: enumeration specifiers (6.7.3.3), labeled statements (6.8.2), structure and union specifiers (6.7.3.2), structure and union members (6.5.3.4), tags (6.7.3.4), the **`goto`** statement (6.8.7.2).

6.2.4 Storage durations of objects

1 An object has a *storage duration* that determines its lifetime. There are four storage durations: static, thread, automatic, and allocated. Allocated storage is described in 7.24.3.

2 The *lifetime* of an object is the portion of program execution during which storage is guaranteed to be reserved for it. An object exists, has a constant address,[24] and retains its last-stored value throughout its lifetime.[25] If an object is referred to outside of its lifetime, the behavior is undefined. If a pointer value is used in an evaluation after the object the pointer points to (or just past) reaches the end of its lifetime, the behavior is undefined. The representation of a pointer object becomes indeterminate when the object the pointer points to (or just past) reaches the end of its lifetime.

3 An object whose identifier is declared without the storage-class specifier **`thread_local`**, and either with external or internal linkage or with the storage-class specifier **`static`**, has *static storage duration*. Its lifetime is the entire execution of the program and its stored value is initialized only once, prior to program startup.

4 An object whose identifier is declared with the storage-class specifier **`thread_local`** has *thread storage duration*. Its explicit or implicit initializer is evaluated prior to program execution, its lifetime

[23] There is only one name space for tags even though three are possible.

[24] The term "constant address" means that two pointers to the object constructed at possibly different times will compare equal. The address can be different during two different executions of the same program.

[25] In the case of a volatile object, the last store is not required to be explicit in the program.

is the entire execution of the thread for which it is created, and its stored value is initialized with the previously determined value when the thread is started. There is a distinct object per thread, and use of the declared name in an expression refers to the object associated with the thread evaluating the expression. The result of attempting to indirectly access an object with thread storage duration from a thread other than the one with which the object is associated is implementation-defined.

5 An object whose identifier is declared with no linkage and without the storage-class specifier **`static`** has *automatic storage duration*, as do some compound literals. The result of attempting to indirectly access an object with automatic storage duration from a thread other than the one with which the object is associated is implementation-defined.

6 For such an object that does not have a variable length array type, its lifetime extends from entry into the block with which it is associated until execution of that block ends in any way. (Entering an enclosed block or calling a function suspends, but does not end, execution of the current block.) If the block is entered recursively, a new instance of the object is created each time. The initial representation of the object is indeterminate. If an initialization is specified for the object and it is not specified with **`constexpr`**, it is performed each time the declaration or compound literal is reached in the execution of the block; if it is specified with **`constexpr`** the initializer is evaluated once at translation time and the new instance of the object is initialized to that fixed value each time the specification is reached; otherwise, the representation of the object becomes indeterminate each time the declaration is reached.

7 For such an object that does have a variable length array type, its lifetime extends from the declaration of the object until execution of the program leaves the scope of the declaration.[26)] If the scope is entered recursively, a new instance of the object is created each time. The initial representation of the object is indeterminate.

8 A non-lvalue expression with structure or union type, where the structure or union contains a member with array type (including, recursively, members of all contained structures and unions) refers to an object with automatic storage duration and *temporary lifetime*.[27)] Its lifetime begins when the expression is evaluated and its initial value is the value of the expression. Its lifetime ends when the evaluation of the containing full expression ends. Any attempt to modify an object with temporary lifetime results in undefined behavior. An object with temporary lifetime behaves as if it were declared with the type of its value for the purposes of effective type. Such an object may not have a unique address.

Forward references: array declarators (6.7.7.3), compound literals (6.5.3.6), declarators (6.7.7), function calls (6.5.3.3), initialization (6.7.11), statements (6.8), effective type (6.5.1).

6.2.5 Types

1 The meaning of a value stored in an object or returned by a function is determined by the *type* of the expression used to access it. (An identifier declared to be an object is the simplest such expression; the type is specified in the declaration of the identifier.) Types are partitioned into *object types* (types that describe objects) and *function types* (types that describe functions). At various points within a translation unit an object type may be *incomplete*[28)] (lacking sufficient information to determine the size of objects of that type) or *complete* (having sufficient information).[29)]

2 An object declared as type **`bool`** is large enough to store the values **`false`** and **`true`**.

3 An object declared as type **`char`** is large enough to store any member of the basic execution character set. If a member of the basic execution character set is stored in a **`char`** object, its value is guaranteed to be nonnegative. If any other character is stored in a **`char`** object, the resulting value is implementation-defined but shall be within the range of values that can be represented in that type.

[26)] Leaving the innermost block containing the declaration, or jumping to a point in that block or an embedded block prior to the declaration, leaves the scope of the declaration.

[27)] The address of such an object is taken implicitly when an array member is accessed.

[28)] An incomplete type can only be used when the size of an object of that type is not needed. It is not needed, for example, when a typedef name is declared to be a specifier for a structure or union, or when a pointer to or a function returning a structure or union is being declared. The specification has to be complete before such a function is called or defined.

[29)] A type can be incomplete or complete throughout an entire translation unit, or it can change states at different points within a translation unit.

4 There are five *standard signed integer types*, designated as **`signed char`**, **`short int`**, **`int`**, **`long int`**, and **`long long int`**. (These and other types may be designated in several additional ways, as described in 6.7.3.)

5 A *bit-precise signed integer type* is designated as **`_BitInt`**`(`N`)` where N is an integer constant expression that specifies the number of bits that are used to represent the type, including the sign bit. Each value of N designates a distinct type.[30)]

6 There may also be implementation-defined *extended signed integer types*.[31)] The standard signed integer types, bit-precise signed integer types, and extended signed integer types are collectively called *signed integer types*.[32)]

7 An object declared as type **`signed char`** occupies the same amount of storage as a "plain" **`char`** object. A "plain" **`int`** object has the natural size suggested by the architecture of the execution environment (large enough to contain any value in the range **`INT_MIN`** to **`INT_MAX`** as defined in the header `<limits.h>`).

8 For each of the signed integer types, there is a *corresponding* (but different) *unsigned integer type* (designated with the keyword **`unsigned`**) that uses the same amount of storage (including sign information) and has the same alignment requirements. The type **`bool`** and the unsigned integer types that correspond to the standard signed integer types are the *standard unsigned integer types*. The unsigned integer types that correspond to the extended signed integer types are the *extended unsigned integer types*. In addition to the unsigned integer types that correspond to the bit-precise signed integer types there is the type **`unsigned _BitInt`**`(1)`, which uses one bit to represent the type. Collectively, **`unsigned _BitInt`**`(1)` and the unsigned integer types that correspond to the bit-precise signed integer types are the *bit-precise unsigned integer types*. The standard unsigned integer types, bit-precise unsigned integer types, and extended unsigned integer types are collectively called *unsigned integer types*.[33)]

9 The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*; the bit-precise signed integer types and bit-precise unsigned integer types are collectively called the *bit-precise integer types*; the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*.

10 For any two integer types with the same signedness and different integer conversion rank (see 6.3.1.1), the range of values of the type with smaller integer conversion rank is a subrange of the values of the other type.

11 The range of nonnegative values of a signed integer type is a subrange of the corresponding unsigned integer type, and the representation of the same value in each type is the same.[34)] The range of representable values for the unsigned type is 0 to $2^N - 1$ (inclusive). A computation involving unsigned operands can never produce an overflow, because arithmetic for the unsigned type is performed modulo 2^N.

12 There are three *standard floating types*, designated as **`float`**, **`double`**, and **`long double`**.[35)] The set of values of the type **`float`** is a subset of the set of values of the type **`double`**; the set of values of the type **`double`** is a subset of the set of values of the type **`long double`**.

13 There are three *decimal floating types*, designated as **`_Decimal32`**, **`_Decimal64`**, and **`_Decimal128`**. Respectively, they have the ISO/IEC 60559 formats: decimal32,[36)] decimal64, and decimal128. (Decimal floating types are a conditional feature that implementations may not support; see 6.10.10.4.)

[30)] Thus, **`_BitInt`**`(3)` is not the same type as **`_BitInt`**`(4)`.

[31)] Implementation-defined keywords have the form of an identifier reserved for any use as described in 7.1.3.

[32)] Any statement in this document about signed integer types also applies to the bit-precise signed integer types and the extended signed integer types, unless otherwise noted.

[33)] Any statement in this document about unsigned integer types also applies to the bit-precise unsigned integer types and the extended unsigned integer types, unless otherwise specified.

[34)] The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

[35)] See "future language directions" (6.11.1).

[36)] ISO/IEC 60559 specifies decimal32 as a data-interchange format that does not require arithmetic support; however, **`_Decimal32`** is a fully supported arithmetic type.

14 The standard floating types and the decimal floating types are collectively called the *real floating types*..

15 There are three *complex types*, designated as **float _Complex**, **double _Complex**, and **long double _Complex**.[37] (Complex types are a conditional feature that implementations may not support; see 6.10.10.4.) The real floating and complex types are collectively called the *floating types*.

16 For each floating type there is a *corresponding real type*, which is always a real floating type. For real floating types, it is the same type. For complex types, it is the type given by deleting the keyword **_Complex** from the type name.

17 Each complex type has the same representation and alignment requirements as an array type containing exactly two elements of the corresponding real type; the first element is equal to the real part, and the second element to the imaginary part, of the complex number.

18 The type **char**, the signed and unsigned integer types, and the floating types are collectively called the *basic types*. The basic types are complete object types. Even if the implementation defines two or more basic types to have the same representation, they are nevertheless distinct types.

19 NOTE An implementation can define new keywords that provide alternative ways to designate a basic (or any other) type; this does not violate the requirement that all basic types be different. Implementation-defined keywords have the form of an identifier reserved for any use as described in 7.1.3.

20 The three types **char**, **signed char**, and **unsigned char** are collectively called the *character types*. The implementation shall define **char** to have the same range, representation, and behavior as either **signed char** or **unsigned char**.[38]

21 An *enumeration* comprises a set of named integer constant values. Each distinct enumeration constitutes a different *enumerated type*.

22 The type **char**, the signed and unsigned integer types, and the enumerated types are collectively called *integer types*. The integer and real floating types are collectively called *real types*.

23 Integer and floating types are collectively called *arithmetic types*. Each arithmetic type belongs to one *type domain*: the *real type domain* comprises the real types, the *complex type domain* comprises the complex types.

24 The **void** type comprises an empty set of values; it is an incomplete object type that cannot be completed.

25 Any number of *derived types* can be constructed from the object and function types, as follows:

— An *array type* describes a contiguously allocated nonempty set of objects with a particular member object type, called the *element type*. The element type shall be complete whenever the array type is specified. Array types are characterized by their element type and by the number of elements in the array. An array type is said to be derived from its element type, and if its element type is *T*, the array type is sometimes called "array of *T*". The construction of an array type from an element type is called "array type derivation".

— A *structure type* describes a sequentially allocated nonempty set of member objects (and, in certain circumstances, an incomplete array), each of which has an optionally specified name and possibly distinct type.

— A *union type* describes an overlapping nonempty set of member objects, each of which has an optionally specified name and possibly distinct type.

— A *function type* describes a function with specified return type. A function type is characterized by its return type and the number and types of its parameters. A function type is said to be derived from its return type, and if its return type is *T*, the function type is sometimes called "function returning *T*". The construction of a function type from a return type is called "function type derivation".

[37] A specification for imaginary types is in Annex G.

[38] **CHAR_MIN**, defined in <limits.h>, will have one of the values 0 or **SCHAR_MIN**, and this can be used to distinguish the two options. Irrespective of the choice made, **char** is a separate type from the other two and is not compatible with either.

— A *pointer type* may be derived from a function type or an object type, called the *referenced type*. A pointer type describes an object whose value provides a reference to an entity of the referenced type. A pointer type derived from the referenced type T is sometimes called "pointer to T". The construction of a pointer type from a referenced type is called "pointer type derivation". A pointer type is a complete object type.

— An *atomic type* describes the type designated by the construct **_Atomic**(*type-name*). (Atomic types are a conditional feature that implementations may not support; see 6.10.10.4.)

These methods of constructing derived types can be applied recursively.

26 Arithmetic types, pointer types, and the **nullptr_t** type are collectively called *scalar types*. Array and structure types are collectively called *aggregate types*.[39)]

27 An array type of unknown size is an incomplete type. It is completed, for an identifier of that type, by specifying the size in a later declaration (with internal or external linkage). A structure or union type of unknown content (as described in 6.7.3.4) is an incomplete type. It is completed, for all declarations of that type, by declaring the same structure or union tag with its defining content later in the same scope.

28 A complete type shall have a size that is less than or equal to **SIZE_MAX**. A type has *known constant size* if it is complete and is not a variable length array type.

29 Array, function, and pointer types are collectively called *derived declarator types*. A *declarator type derivation* from a type T is the construction of a derived declarator type from T by the application of an array-type, a function-type, or a pointer-type derivation to T.

30 A type is characterized by its *type category*, which is either the outermost derivation of a derived type (as noted previously in this subclause in the construction of derived types), or the type itself if the type consists of no derived types.

31 Any type so far mentioned is an *unqualified type*. Each unqualified type has several *qualified versions* of its type,[40)] corresponding to the combinations of one, two, or all three of the **const**, **volatile**, and **restrict** qualifiers. The qualified or unqualified versions of a type are distinct types that belong to the same type category and have the same representation and alignment requirements.[41)] An array and its element type are always considered to be identically qualified.[42)] Any other derived type is not qualified by the qualifiers (if any) of the type from which it is derived.

32 Further, there is the **_Atomic** qualifier. The presence of the **_Atomic** qualifier designates an atomic type. The size, representation, and alignment of an atomic type may not be the same as those of the corresponding unqualified type. Therefore, this document explicitly uses the phrase "atomic, qualified, or unqualified type" whenever the atomic version of a type is permitted along with the other qualified versions of a type. The phrase "qualified or unqualified type", without specific mention of atomic, does not include the atomic types.

33 A pointer to **void** shall have the same representation and alignment requirements as a pointer to a character type.[41)] Similarly, pointers to qualified or unqualified versions of compatible types shall have the same representation and alignment requirements. All pointers to structure types shall have the same representation and alignment requirements as each other. All pointers to union types shall have the same representation and alignment requirements as each other. Pointers to other types may not have the same representation or alignment requirements.

34 EXAMPLE 1 The type designated as "**float** *" has type "pointer to **float**". Its type category is pointer, not a floating type. The const-qualified version of this type is designated as "**float * const**" whereas the type designated as "**const float** *" is not a qualified type — its type is "pointer to const-qualified **float**" and is a pointer to a qualified type.

[39)] Note that aggregate type does not include union type because an object with union type can only contain one member at a time.

[40)] See 6.7.4 regarding qualified array and function types.

[41)] The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

[42)] This does not apply to the **_Atomic** qualifier. Note that qualifiers do not have any direct effect on the array type itself, but affect conversion rules for pointer types that reference an array type.

35 EXAMPLE 2 The type designated as "`struct tag (*[5])(float)`" has type "array of pointer to function returning `struct tag`". The array has length five and the function has a single parameter of type `float`. Its type category is array.

Forward references: compatible type and composite type (6.2.7), declarations (6.7).

6.2.6 Representations of types

6.2.6.1 General

1 The representations of all types are unspecified except as stated in this subclause.

2 Except for bit-fields, objects are composed of contiguous sequences of one or more bytes, the number, order, and encoding of which are either explicitly specified or implementation-defined.

3 Values stored in unsigned bit-fields and objects of type `unsigned char` shall be represented using a pure binary notation.

4 Values stored in non-bit-field objects of any other object type are represented using $n \times$ `CHAR_BIT` bits, where n is the size of an object of that type, in bytes. An object that has the value may be copied into an object of type `unsigned char [`n`]` (e.g. by `memcpy`); the resulting set of bytes is called the *object representation* of the value. Values stored in bit-fields consist of m bits, where m is the size specified for the bit-field. The object representation is the set of m bits the bit-field comprises in the addressable storage unit holding it. Two values (other than NaNs) with the same object representation compare equal, but values that compare equal may have different object representations.

5 Certain object representations do not represent a value of the object type. If such a representation is read by an lvalue expression that does not have character type, the behavior is undefined. If such a representation is produced by a side effect that modifies all or any part of the object by an lvalue expression that does not have character type, the behavior is undefined.[43] Such a representation is called a non-value representation.

6 When a value is stored in an object of structure or union type, including in a member object, the bytes of the object representation that correspond to any padding bytes take unspecified values (e.g. structure and union assignment may or may not copy any padding bits). The object representation of a structure or union object is never a non-value representation, even though the byte range corresponding to a member of the structure or union object may be a non-value representation for that member.

7 When a value is stored in a member of an object of union type, the bytes of the object representation that do not correspond to that member but do correspond to other members take unspecified values.

8 Where an operator is applied to a value that has more than one object representation, which object representation is used shall not affect the value of the result.[44] Where a value is stored in an object using a type that has more than one object representation for that value, it is unspecified which representation is used, but a non-value representation shall not be generated.

9 Loads and stores of objects with atomic types are done with `memory_order_seq_cst` semantics.

Forward references: declarations (6.7), expressions (6.5.1), lvalues, arrays, and function designators (6.3.2.1), order and consistency (7.17.3).

6.2.6.2 Integer types

1 For unsigned integer types the bits of the object representation shall be divided into two groups: value bits and padding bits. If there are N value bits, each bit shall represent a different power of 2 between 1 and 2^{N-1}, so that objects of that type shall be capable of representing values from 0 to $2^N - 1$ using a pure binary representation; this shall be known as the value representation. The values of any padding bits are unspecified. The number of value bits N is called the *width* of the unsigned integer type. The type `bool` shall have one value bit and `(sizeof(bool)*CHAR_BIT)- 1`

[43] Thus, an automatic variable can be initialized to a non-value representation without causing undefined behavior, but the value of the variable cannot be used until a proper value is stored in it.

[44] It is possible for objects `x` and `y` with the same effective type `T` to have the same value when they are accessed as objects of type `T`, but to have different values in other contexts. In particular, if == is defined for type `T`, then `x == y` does not imply that `memcmp(&x, &y, sizeof(T))== 0`. Furthermore, `x == y` does not necessarily imply that `x` and `y` have the same value; other operations on values of type `T` can distinguish between them.

padding bits. Otherwise, there is no requirement to have any padding bits; **`unsigned char`** shall not have any padding bits.

2 For signed integer types, the bits of the object representation shall be divided into three groups: value bits, padding bits, and the sign bit. If the corresponding unsigned type has width N, the signed type uses the same number of N bits, its *width*, as value bits and sign bit. $N - 1$ are value bits and the remaining bit is the sign bit. Each bit that is a value bit shall have the same value as the same bit in the object representation of the corresponding unsigned type. If the sign bit is zero, it shall not affect the resulting value. If the sign bit is one, it has value $-(2^{N-1})$. There may or may not be any padding bits; **`signed char`** shall not have any padding bits.

3 The values of any padding bits are unspecified. A valid object representation of a signed integer type where the sign bit is zero is a valid object representation of the corresponding unsigned type, and shall represent the same value. For any integer type, the object representation where all the bits are zero shall be a representation of the value zero in that type.

4 The *precision* of an integer type is the number of value bits.

5 NOTE 1 Some combinations of padding bits may generate non-value representations, for example, if one padding bit is a parity bit. Regardless, no arithmetic operation on valid values can generate a non-value representation other than as part of an exceptional condition such as an integer overflow. All other combinations of padding bits are alternative object representations of the value specified by the value bits.

6 NOTE 2 The sign representation defined in this document is called *two's complement*. Previous editions of this document (specifically ISO/IEC 9899:2018 and prior editions) additionally allowed other sign representations.

7 NOTE 3 For unsigned integer types the width and precision are the same, while for signed integer types the width is one greater than the precision.

6.2.7 Compatible type and composite type

1 Two types are *compatible types* if they are the same. Additional rules for determining whether two types are compatible are described in 6.7.3 for type specifiers, in 6.7.4 for type qualifiers, and in 6.7.7 for declarators.[45)] Moreover, two complete structure, union, or enumerated types declared with the same tag are compatible if members satisfy the following requirements:

— there shall be a one-to-one correspondence between their members such that each pair of corresponding members are declared with compatible types;

— if one member of the pair is declared with an alignment specifier, the other is declared with an equivalent alignment specifier;

— and, if one member of the pair is declared with a name, the other is declared with the same name.

For two structures, corresponding members shall be declared in the same order. For two unions declared in the same translation unit, corresponding members shall be declared in the same order. For two structures or unions, corresponding bit-fields shall have the same widths. For two enumerations, corresponding members shall have the same values; if one has a fixed underlying type, then the other shall have a compatible fixed underlying type. For determining type compatibility, anonymous structures and unions are considered a regular member of the containing structure or union type, and the type of an anonymous structure or union is considered compatible to the type of another anonymous structure or union, respectively, if their members fulfill the preceding requirements.

Furthermore, two structure, union, or enumerated types declared in separate translation units are compatible in the following cases:

— both are declared without tags and they fulfill the preceding requirements;

— both have the same tag and are completed somewhere in their respective translation units and they fulfill the preceding requirements;

[45)] Two types need not be identical to be compatible.

— both have the same tag and at least one of the two types is not completed in its translation unit.

Otherwise, the structure, union, or enumerated types are incompatible.[46)]

2 All declarations that refer to the same object or function shall have compatible type; otherwise, the behavior is undefined.

3 A *composite type* can be constructed from two types that are compatible. If both types are the same type, the composite type is this type. Otherwise, it is a type that is compatible with both and satisfies the following conditions:

— If both types are structure types or both types are union types, the composite type is determined recursively by forming the composite types of their members.

— If both types are array types, the following rules are applied:

- If one type is an array of known constant size, the composite type is an array of that size.
- Otherwise, if one type is a variable length array whose size is specified by an expression that is not evaluated, the behavior is undefined.
- Otherwise, if one type is a variable length array whose size is specified, the composite type is a variable length array of that size.
- Otherwise, if one type is a variable length array of unspecified size, the composite type is a variable length array of unspecified size.
- Otherwise, both types are arrays of unknown size and the composite type is an array of unknown size.

The element type of the composite type is the composite type of the two element types.

— If both types are function types, the type of each parameter in the composite parameter type list is the composite type of the corresponding parameters.

— If one of the types has a standard attribute, the composite type also has that attribute.

— If both types are enumerated types, the composite type is an enumerated type.

— If one type is an enumerated type and the other is an integer type other than an enumerated type, it is implementation-defined whether or not the composite type is an enumerated type.

These rules apply recursively to the types from which the two types are derived.

4 If any of the original types satisfies all requirements of the composite type, it is unspecified whether the composite type is one of these types or a different type that satisfies the requirements.[47)]

5 For an identifier with internal or external linkage declared in a scope in which a prior declaration of that identifier is visible,[48)] if the prior declaration specifies internal or external linkage, the type of the identifier at the later declaration becomes the composite type.

6 EXAMPLE Given the following two file scope declarations:

```
int f(int (*)(char *), double (*)[3]);
int f(int (*)(char *), double (*)[]);
```

The resulting composite type for the function is:

```
int f(int (*)(char *), double (*)[3]);
```

Forward references: array declarators (6.7.7.3).

[46)] A structure, union, or enumerated type without a tag or an incomplete structure, union or enumerated type is not compatible with any other structure, union or enum type declared in the same translation unit.

[47)] The notion of "same type" affects redeclarations of typedef names and tags in the same scope.

[48)] As specified in 6.2.1, the later declaration can hide the prior declaration.

6.2.8 Alignment of objects

1 Complete object types have alignment requirements which place restrictions on the addresses at which objects of that type may be allocated. An alignment is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated. An object type imposes an alignment requirement on every object of that type: stricter alignment can be requested using the **alignas** keyword.

2 A *fundamental alignment* is a valid alignment less than or equal to **alignof**(**max_align_t**). Fundamental alignments shall be supported by the implementation for objects of all storage durations. The alignment requirements of the following types shall be fundamental alignments:

— all atomic, qualified, or unqualified basic types;

— all atomic, qualified, or unqualified enumerated types;

— all atomic, qualified, or unqualified pointer types;

— all array types whose element type has a fundamental alignment requirement;

— all types specified in Clause 7 as complete object types;

— all structure or union types whose elements have types with fundamental alignment requirements and none of whose elements have an alignment specifier specifying an alignment that is not a fundamental alignment.

3 An *extended alignment* is represented by an alignment greater than **alignof**(**max_align_t**). It is implementation-defined whether any extended alignments are supported and the storage durations for which they are supported. A type having an extended alignment requirement is an *over-aligned* type.[49)]

4 Alignments are represented as values of the type **size_t**. Valid alignments include only fundamental alignments, plus an additional implementation-defined set of values, which may be empty. Every valid alignment value shall be a nonnegative integral power of two.

5 Alignments have an order from *weaker* to *stronger* or *stricter* alignments. Stricter alignments have larger alignment values. An address that satisfies an alignment requirement also satisfies any weaker valid alignment requirement.

6 The alignment requirement of a complete type can be queried using an **alignof** expression. The types **char**, **signed char**, and **unsigned char** shall have the weakest alignment requirement.

7 Comparing alignments is meaningful and provides the obvious results:

— Two alignments are equal when their numeric values are equal.

— Two alignments are different when their numeric values are not equal.

— When an alignment is larger than another it represents a stricter alignment.

6.2.9 Encodings

1 The *literal encoding* is an implementation-defined mapping of the characters of the execution character set to the values in a character constant (6.4.4.5) or string literal (6.4.5). It shall support a mapping from all the basic execution character set values into the implementation-defined encoding. It may contain multibyte character sequences (5.2.2).

2 The *wide literal encoding* is an implementation-defined mapping of the characters of the execution character set to the values in a **wchar_t** character constant (6.4.4.5) or a **wchar_t** string literal (6.4.5). It shall support a mapping from all the basic execution character set values into the implementation-defined encoding. The mapping shall produce values identical to the literal encoding for all the basic execution character set values if an implementation does not define **__STDC_MB_MIGHT_NEQ_WC__**. One or more values may map to one or more values of the extended execution character set.

[49)] Every over-aligned type is, or contains, a structure or union type with a member to which an extended alignment has been applied.

6.3 Conversions

1 Several operators convert operand values from one type to another automatically. This subclause specifies the result required from such an *implicit conversion*, as well as those that result from a cast operation (an *explicit conversion*). The list in 6.3.1.8 summarizes the conversions performed by most ordinary operators; it is supplemented as required by the discussion of each operator in 6.5.1.

2 Unless explicitly stated otherwise, conversion of an operand value to a compatible type causes no change to the value or the representation.

Forward references: cast operators (6.5.5).

6.3.1 Arithmetic operands

6.3.1.1 Boolean, characters, and integers

1 Every integer type has an *integer conversion rank* defined as follows:

- No two signed integer types shall have the same rank, even if they have the same representation.
- The rank of a signed integer type shall be greater than the rank of any signed integer type with less precision.
- The rank of **`long long int`** shall be greater than the rank of **`long int`**, which shall be greater than the rank of **`int`**, which shall be greater than the rank of **`short int`**, which shall be greater than the rank of **`signed char`**.
- The rank of a bit-precise signed integer type shall be greater than the rank of any standard integer type with less width or any bit-precise integer type with less width.
- The rank of any unsigned integer type shall equal the rank of the corresponding signed integer type, if any.
- The rank of any standard integer type shall be greater than the rank of any extended integer type with the same width or bit-precise integer type with the same width.
- The rank of any bit-precise integer type relative to an extended integer type of the same width is implementation-defined.
- The rank of **`char`** shall equal the rank of **`signed char`** and **`unsigned char`**.
- The rank of **`bool`** shall be less than the rank of all other standard integer types.
- The rank of any enumerated type shall equal the rank of the compatible integer type (see 6.7.3.3).
- The rank of any extended signed integer type relative to another extended signed integer type with the same precision is implementation-defined, but still subject to the other rules for determining the integer conversion rank.
- For all integer types `T1`, `T2`, and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` has greater rank than `T3`.

2 The following may be used in an expression wherever an **`int`** or **`unsigned int`** may be used:

- An object or expression with an integer type (other than **`int`** or **`unsigned int`**) whose integer conversion rank is less than or equal to the rank of **`int`** and **`unsigned int`**.
- A bit-field of type **`bool`**, **`int`**, **`signed int`**, or **`unsigned int`**.

The value from a bit-field of a bit-precise integer type is converted to the corresponding bit-precise integer type. If the original type is not a bit-precise integer type (6.2.5): if an **int** can represent all values of the original type (as restricted by the width, for a bit-field), the value is converted to an **int**;[50] otherwise, it is converted to an **unsigned int**. These are called the *integer promotions*. All other types are unchanged by the integer promotions.

3 NOTE The integer promotions are applied only:

1. as part of the usual arithmetic conversions,
2. to certain argument expressions,
3. to the operands of the unary +, -, and ~ operators,
4. and to both operands of the shift operators,

as specified by their respective subclauses.

4 The integer promotions preserve value including sign. As discussed earlier, whether a "plain" **char** can hold negative values is implementation-defined.

Forward references: enumeration specifiers (6.7.3.3), structure and union specifiers (6.7.3.2).

6.3.1.2 Boolean type

1 When any scalar value is converted to **bool**, the result is **false** if the value is a zero (for arithmetic types), null (for pointer types), or the scalar has type **nullptr_t**; otherwise, the result is **true**.

6.3.1.3 Signed and unsigned integers

1 When a value with integer type is converted to another integer type other than **bool**, if the value can be represented by the new type, it is unchanged.

2 Otherwise, if the new type is unsigned, the value is converted by repeatedly adding or subtracting one more than the maximum value that can be represented in the new type until the value is in the range of the new type.[51]

3 Otherwise, the new type is signed and the value cannot be represented in it; either the result is implementation-defined or an implementation-defined signal is raised.

6.3.1.4 Real floating and integer

1 When a finite value of standard floating type is converted to an integer type other than **bool**, the fractional part is discarded (i.e. the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the behavior is undefined.[52]

2 When a finite value of decimal floating type is converted to an integer type other than **bool**, the fractional part is discarded (i.e. the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the "invalid" floating-point exception shall be raised and the result of the conversion is unspecified.

3 When a value of integer type is converted to a standard floating type, if the value being converted can be represented exactly in the new type, it is unchanged. If the value being converted is in the range of values that can be represented but cannot be represented exactly, the result is either the nearest higher or nearest lower representable value, chosen in an implementation-defined manner. If the value being converted is outside the range of values that can be represented, the behavior is undefined. Results of some implicit conversions may be represented in greater range and precision than that required by the new type (see 6.3.1.8 and 6.8.7.5).

4 When a value of integer type is converted to a decimal floating type, if the value being converted can be represented exactly in the new type, it is unchanged. If the value being converted cannot be represented exactly, the result shall be correctly rounded with exceptions raised as specified in ISO/IEC 60559.

[50] E.g. **unsigned _BitInt(7): 2** is a bit-field that can hold the values 0, 1, 2, 3, and converts to **unsigned _BitInt(7)**.

[51] The rules describe arithmetic on the mathematical value, not the value of a given type of expression.

[52] The remaindering operation performed when a value of integer type is converted to unsigned type need not be performed when a value of real floating type is converted to unsigned type. Thus, the range of portable real floating values is $(-1, \mathbf{U}type\_\mathbf{MAX} + 1)$.

6.3.1.5 Real floating types

1 When a value of real floating type is converted to a real floating type, if the value being converted can be represented exactly in the new type, it is unchanged.

2 When a value of real floating type is converted to a standard floating type, if the value being converted is in the range of values that can be represented but cannot be represented exactly, the result is either the nearest higher or nearest lower representable value, chosen in an implementation-defined manner. If the value being converted is outside the range of values that can be represented, the behavior is undefined.

3 When a value of real floating type is converted to a decimal floating type, if the value being converted cannot be represented exactly, the result is correctly rounded with exceptions raised as specified in ISO/IEC 60559.

4 Results of some implicit conversions may be represented in greater range and precision than that required by the new type (see 6.3.1.8 and 6.8.7.5).

6.3.1.6 Complex types

1 When a value of complex type is converted to another complex type, both the real and imaginary parts follow the conversion rules for the corresponding real types.

6.3.1.7 Real and complex

1 When a value of real type is converted to a complex type, the real part of the complex result value is determined by the rules of conversion to the corresponding real type and the imaginary part of the complex result value is a positive zero or an unsigned zero.

2 When a value of complex type is converted to a real type other than **`bool`**,[53] the imaginary part of the complex value is discarded and the value of the real part is converted according to the conversion rules for the corresponding real type.

6.3.1.8 Usual arithmetic conversions

1 Many operators that expect operands of arithmetic type cause conversions and yield result types in a similar way. The purpose is to determine a *common real type* for the operands and result. For the specified operands, each operand is converted, without change of type domain, to a type whose corresponding real type is the common real type. Unless explicitly stated otherwise, the common real type is also the corresponding real type of the result, whose type domain is the type domain of the operands if they are the same, and complex otherwise. This pattern is called the *usual arithmetic conversions*:

> If one operand has decimal floating type, the other operand shall not have standard floating, complex, or imaginary type.
>
> First, if the type of either operand is **`_Decimal128`**, the other operand is converted to **`_Decimal128`**.
>
> Otherwise, if the type of either operand is **`_Decimal64`**, the other operand is converted to **`_Decimal64`**.
>
> Otherwise, if the type of either operand is **`_Decimal32`**, the other operand is converted to **`_Decimal32`**.
>
> Otherwise, if the corresponding real type of either operand is **`long double`**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **`long double`**.
>
> Otherwise, if the corresponding real type of either operand is **`double`**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **`double`**.

[53] See 6.3.1.2.

Otherwise, if the corresponding real type of either operand is **float**, the other operand is converted, without change of type domain, to a type whose corresponding real type is **float**.[54)]

Otherwise, if any of the two types is an enumeration, it is converted to its underlying type. Then, the integer promotions are performed on both operands. Next, the following rules are applied to the promoted operands:

If both operands have the same type, then no further conversion is needed.

Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank is converted to the type of the operand with greater rank.

Otherwise, if the operand that has unsigned integer type has rank greater or equal to the rank of the type of the other operand, then the operand with signed integer type is converted to the type of the operand with unsigned integer type.

Otherwise, if the type of the operand with signed integer type can represent all the values of the type of the operand with unsigned integer type, then the operand with unsigned integer type is converted to the type of the operand with signed integer type.

Otherwise, both operands are converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

2 The values of floating operands and of the results of floating expressions may be represented in greater range and precision than that required by the type; the types are not changed thereby. See 5.2.5.3.3 regarding evaluation formats.

3 EXAMPLE One consequence of **_BitInt** being exempt from the integer promotion rules (6.3.1) is that a **_BitInt** operand of a binary operator is not always promoted to an **int** or **unsigned int** as part of the usual arithmetic conversions. Instead, a lower-ranked operand is converted to the higher-rank operand type and the result of the operation is the higher-ranked type.

```
_BitInt(2) a2 = 1;
_BitInt(3) a3 = 2;
_BitInt(33) a33 = 1;
signed char c = 3;

a2 * a3; /* As part of the multiplication, a2 is converted to
            _BitInt(3) and the result type is _BitInt(3). */
a2 * c;  /* As part of the multiplication, c is promoted to int,
            a2 is converted to int and the result type is int. */
a33 * c; /* As part of the multiplication, c is promoted to int.
            Then, provided int has a width of at most 32,
            it is converted to _BitInt(33) and the result type
            is _BitInt(33). */

void func(_BitInt(8) a8, _BitInt(24) a24) {
     /* Cast one of the operands to 32-bits to guarantee the
        result of the multiplication can contain all possible values. */
     _BitInt(32) a32 = a8 * (_BitInt(32))a24;
}
```

6.3.2 Other operands

6.3.2.1 Lvalues, arrays, and function designators

1 An *lvalue* is an expression (with an object type other than **void**) that potentially designates an object;[55)] if an lvalue does not designate an object when it is evaluated, the behavior is undefined.

[54)] For example, addition of a **double _Complex** and a **float** entails just the conversion of the **float** operand to **double** (and yields a **double _Complex** result).

[55)] The name "lvalue" comes originally from the assignment expression `E1 = E2`, in which the left operand `E1` is required to be a (modifiable) lvalue. It is perhaps better considered as representing an object "locator value". What is sometimes called

When an object is said to have a particular type, the type is specified by the lvalue used to designate the object. A *modifiable lvalue* is an lvalue that does not have array type, does not have an incomplete type, does not have a const-qualified type, and if it is a structure or union, does not have any member (including, recursively, any member or element of all contained aggregates or unions) with a const-qualified type.

2 Except when it is the operand of the **sizeof** operator, or the typeof operators, the unary & operator, the ++ operator, the -- operator, or the left operand of the . operator or an assignment operator, an lvalue that does not have array type is converted to the value stored in the designated object (and is no longer an lvalue); this is called *lvalue conversion*. If the lvalue has qualified type, the value has the unqualified version of the type of the lvalue; additionally, if the lvalue has atomic type, the value has the non-atomic version of the type of the lvalue; otherwise, the value has the type of the lvalue. If the lvalue has an incomplete type and does not have array type, the behavior is undefined. If the lvalue designates an object of automatic storage duration that could have been declared with the **register** storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

3 Except when it is the operand of the **sizeof** operator, or typeof operators, or the unary & operator, or is a string literal used to initialize an array, an expression that has type "array of *type*" is converted to an expression with type "pointer to *type*" that points to the initial element of the array object and is not an lvalue. If the array object has register storage class, the behavior is undefined.

4 A *function designator* is an expression that has function type. Except when it is the operand of the **sizeof** operator,[56] a typeof operator, or the unary & operator, a function designator with type "function returning *type*" is converted to an expression that has type "pointer to function returning *type*".

Forward references: address and indirection operators (6.5.4.2), assignment operators (6.5.17), common definitions <stddef.h> (7.21), initialization (6.7.11), postfix increment and decrement operators (6.5.3.5), prefix increment and decrement operators (6.5.4.1), the **sizeof** and **alignof** operators (6.5.4.4), structure and union members (6.5.3.4).

6.3.2.2 void

1 The (nonexistent) value of a *void expression* (an expression that has type **void**) shall not be used in any way, and implicit or explicit conversions (except to **void**) shall not be applied to such an expression. If an expression of any other type is evaluated as a void expression, its value or designator is discarded. (A void expression is evaluated for its side effects.)

6.3.2.3 Pointers

1 A pointer to **void** may be converted to or from a pointer to any object type. A pointer to any object type may be converted to a pointer to **void** and back again; the result shall compare equal to the original pointer.

2 For any qualifier *q*, a pointer to a non-*q*-qualified type may be converted to a pointer to the *q*-qualified version of the type; the values stored in the original and converted pointers shall compare equal.

3 An integer constant expression with the value 0, such an expression cast to type **void** *, or the predefined constant **nullptr** is called a *null pointer constant*.[57] If a null pointer constant or a value of the type **nullptr_t** (which is necessarily the value **nullptr**) is converted to a pointer type, the resulting pointer, called a *null pointer*, is guaranteed to compare unequal to a pointer to any object or function.

4 Conversion of a null pointer to another pointer type yields a null pointer of that type. Any two null pointers shall compare equal.

5 An integer may be converted to any pointer type. Except as previously specified, the result is

"rvalue" is in this document described as the "value of an expression".

An obvious example of an lvalue is an identifier of an object. As a further example, if E is a unary expression that is a pointer to an object, *E is an lvalue that designates the object to which E points.

[56] Because this conversion does not occur, the operand of the **sizeof** operator remains a function designator and violates the constraints in 6.5.4.4

[57] The macro **NULL** is defined in <stddef.h> (and other headers) as a null pointer constant; see 7.21.

implementation-defined, may not be correctly aligned, may not point to an entity of the referenced type, and can produce an indeterminate representation when stored into an object.[58)]

6 Any pointer type may be converted to an integer type. Except as previously specified, the result is implementation-defined. If the result cannot be represented in the integer type, the behavior is undefined. The result is not required to be in the range of values of any integer type.

7 A pointer to an object type may be converted to a pointer to a different object type. If the resulting pointer is not correctly aligned[59)] for the referenced type, the behavior is undefined. Otherwise, when converted back again, the result shall compare equal to the original pointer. When a pointer to an object is converted to a pointer to a character type, the result points to the lowest addressed byte of the object. Successive increments of the result, up to the size of the object, yield pointers to the remaining bytes of the object.

8 A pointer to a function of one type may be converted to a pointer to a function of another type and back again; the result shall compare equal to the original pointer. If a converted pointer is used to call a function whose type is not compatible with the referenced type, the behavior is undefined.

6.3.2.4 `nullptr_t`

1 The type **`nullptr_t`** may be converted to **`void`**, **`bool`** or to a pointer type; the result is a **`void`** expression, **`false`**, or a null pointer value, respectively.

2 A null pointer constant or value of type **`nullptr_t`** may be converted to **`nullptr_t`**.

Forward references: cast operators (6.5.5), equality operators (6.5.10), integer types capable of holding object pointers (7.22.1.4), simple assignment (6.5.17.2), the **`nullptr_t`** type (7.21.2).

[58)] The mapping functions for converting a pointer to an integer or an integer to a pointer are intended to be consistent with the addressing structure of the execution environment.

[59)] In general, the concept "correctly aligned" is transitive: if a pointer to type A is correctly aligned for a pointer to type B, which in turn is correctly aligned for a pointer to type C, then a pointer to type A is correctly aligned for a pointer to type C.

6.4 Lexical elements

Syntax

1 *token:*
keyword
identifier
constant
string-literal
punctuator

preprocessing-token:
header-name
identifier
pp-number
character-constant
string-literal
punctuator
each universal character name that cannot be one of the above
each non-white-space character that cannot be one of the above

Constraints

2 Each preprocessing token that is converted to a token shall have the lexical form of a keyword, an identifier, a constant, a string literal, or a punctuator. A single universal character name shall match one of the other preprocessing token categories.

Semantics

3 A *token* is the minimal lexical element of the language in translation phases 7 and 8 (5.1.1.2). The categories of tokens are: keywords, identifiers, constants, string literals, and punctuators. A preprocessing token is the minimal lexical element of the language in translation phases 3 through 6. The categories of preprocessing tokens are: header names, identifiers, preprocessing numbers, character constants, string literals, punctuators, and both single universal character names as well as single non-white-space characters that do not lexically match the other preprocessing token categories.[60)] If a `'` or a `"` character matches the last category, the behavior is undefined. Preprocessing tokens can be separated by *white space*; this consists of comments (described later), or *white-space characters* (space, horizontal tab, new-line, vertical tab, and form-feed), or both. As described in 6.10, in certain circumstances during translation phase 4, white space (or the absence thereof) serves as more than preprocessing token separation. White space may appear within a preprocessing token only as part of a header name or between the quotation characters in a character constant or string literal.

4 If the input stream has been parsed into preprocessing tokens up to a given character, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token. There is one exception to this rule: header name preprocessing tokens are recognized only within **`#include`** and **`#embed`** preprocessing directives, in **`__has_include`** and **`__has_embed`** expressions, as well as in implementation-defined locations within **`#pragma`** directives. In such contexts, a sequence of characters that could be either a header name or a string literal is recognized as the former.

5 EXAMPLE 1 The program fragment `1Ex` is parsed as a preprocessing number token (one that is not a valid floating or integer constant token), even though a parse as the pair of preprocessing tokens `1` and `Ex` can produce a valid expression (for example, if `Ex` were a macro defined as `+1`). Similarly, the program fragment `1E1` is parsed as a preprocessing number (one that is a valid floating constant token), whether or not `E` is a macro name.

6 EXAMPLE 2 The program fragment `x+++++y` is parsed as `x ++ ++ + y`, which violates a constraint on increment operators, even though the parse `x ++ + ++ y` can yield a correct expression.

[60)] An additional category, placemarkers, is used internally in translation phase 4 (see 6.10.5.3); it cannot occur in source files.

Forward references: character constants (6.4.4.5), comments (6.4.9), expressions (6.5.1), floating constants (6.4.4.3), header names (6.4.7), macro replacement (6.10.5), postfix increment and decrement operators (6.5.3.5), prefix increment and decrement operators (6.5.4.1), preprocessing directives (6.10), preprocessing numbers (6.4.8), string literals (6.4.5).

6.4.1 Keywords

Syntax

1 *keyword:* one of

alignas	**do**	**int**	**struct**	**while**
alignof	**double**	**long**	**switch**	**_Atomic**
auto	**else**	**nullptr**	**thread_local**	**_BitInt**
bool	**enum**	**register**	**true**	**_Complex**
break	**extern**	**restrict**	**typedef**	**_Decimal128**
case	**false**	**return**	**typeof**	**_Decimal32**
char	**float**	**short**	**typeof_unqual**	**_Decimal64**
const	**for**	**signed**	**union**	**_Generic**
constexpr	**goto**	**sizeof**	**unsigned**	**_Imaginary**
continue	**if**	**static**	**void**	**_Noreturn**
default	**inline**	**static_assert**	**volatile**	

Semantics

2 The previously listed tokens (case sensitive) are reserved (in translation phases 7 and 8) for use as keywords except in an attribute token, and shall not be used otherwise. The keyword **_Imaginary** is reserved for specifying imaginary types.[61)]

3 The following table provides alternate spellings for certain keywords. These can be used wherever the keyword can.[62)]

Keyword	Alternative Spelling
alignas	**_Alignas**
alignof	**_Alignof**
bool	**_Bool**
static_assert	**_Static_assert**
thread_local	**_Thread_local**

The spelling of these keywords, their alternate forms, and of **false** and **true** inside expressions that are subject to the **#** and **##** preprocessing operators is unspecified.[63)]

[61)] One possible specification for imaginary types appears in Annex G.

[62)] These alternative keywords are obsolescent features and should not be used for new code and development.

[63)] The intent of this specification is to allow but not force the implementation of the corresponding feature by means of a predefined macro.

6.4.2 Identifiers

6.4.2.1 General

Syntax

1 *identifier:*
identifier-start
identifier identifier-continue

identifier-start:
nondigit
XID_Start character
universal character name of class XID_Start

identifier-continue:
digit
nondigit
XID_Continue character
universal character name of class XID_Continue

nondigit: one of

```
_ a b c d e f g h i j k l m
  n o p q r s t u v w x y z
  A B C D E F G H I J K L M
  N O P Q R S T U V W X Y Z
```

digit: one of

```
0 1 2 3 4 5 6 7 8 9
```

Semantics

2 An XID_Start character is an implementation-defined character whose corresponding code point in ISO/IEC 10646 has the XID_Start property. An XID_Continue character is an implementation-defined character whose corresponding code point in ISO/IEC 10646 has the XID_Continue property. An identifier is a sequence of one identifier start character followed by 0 or more identifier continue characters, which designates one or more entities as described in 6.2.1. It is implementation-defined if a $ (U+0024, DOLLAR SIGN) may be used as a nondigit character. Lowercase and uppercase letters are distinct. There is no specific limit on the maximum length of an identifier.

3 The character classes XID_Start and XID_Continue are Derived Core Properties as described by UAX #44.[64)] Each character and universal character name in an identifier shall designate a character whose encoding in ISO/IEC 10646 has the XID_Continue property. The initial character (which may be a universal character name) shall designate a character whose encoding in ISO/IEC 10646 has the XID_Start property. An identifier shall conform to Normalization Form C as specified in ISO/IEC 10646. Annex D provides an overview of the conforming identifiers.

4 NOTE 1 Uppercase and lowercase letters are considered different for all identifiers.

5 NOTE 2 In translation phase 4, the term identifier also includes those preprocessing tokens (6.4.8) differentiated as keywords (6.4.1) in the later translation phase 7 (5.1.1.2).

[64)] On systems that cannot accept extended characters in external identifiers, an encoding of the universal character name may be used in forming such identifiers. For example, some otherwise unused character or sequence of characters may be used to encode the u in a universal character name.

6 When preprocessing tokens are converted to tokens during translation phase 7, if a preprocessing token could be converted to either a keyword or an identifier, it is converted to a keyword except in an attribute token.

7 Some identifiers are reserved.

— All identifiers that begin with a double underscore (__) or begin with an underscore (_) followed by an uppercase letter are reserved for any use, except those identifiers which are lexically identical to keywords.[65)]

— All identifiers that begin with an underscore are reserved for use as identifiers with file scope in both the ordinary and tag name spaces.

Other identifiers may be reserved, see 7.1.3.

8 If the program declares or defines an identifier in a context in which it is reserved (other than as allowed by 7.1.4), the behavior is undefined.

9 If the program defines a reserved identifier or standard attribute token described in 6.7.13.2 as a macro name, or removes (with **`#undef`**) any macro definition of an identifier in the first group listed previously or standard attribute token described in 6.7.13.2, the behavior is undefined.

10 Some identifiers may be potentially reserved. A *potentially reserved identifier* is an identifier which is not reserved unless made so by an implementation providing the identifier (7.1.3) but is anticipated to become reserved by an implementation or a future version of this document. An identifier that this document describes as optional:

— If it is defined as a macro it is reserved.

— Otherwise, if the definition is given in clauses 1 to 6 it is reserved.

— Otherwise, it is potentially reserved.

Recommended practice

11 Implementations are encouraged to issue a diagnostic message when a potentially reserved identifier is declared or defined for any use that is not implementation-compatible (see subsequent description in this subclause) in a context where the potentially reserved identifier may be reserved under a conforming implementation. This brings attention to a potential conflict when porting a program to a future revision of this document.

12 An implementation-compatible use of a potentially reserved identifier is a declaration of an external name where the name is provided by the implementation as an external name and where the declaration declares an object or function with a type that is compatible with the type of the object or function provided by the implementation under that name.

Implementation limits

13 As discussed in 5.2.5.2, an implementation may limit the number of significant initial characters in an identifier; the limit for an *external name* (an identifier that has external linkage) may be more restrictive than that for an *internal name* (a macro name or an identifier that does not have external linkage). The number of significant characters in an identifier is implementation-defined.

14 Any identifiers that differ in a significant character are different identifiers. If two identifiers differ only in nonsignificant characters, the behavior is undefined.

Forward references: universal character names (6.4.3), macro replacement (6.10.5), reserved library identifiers (7.1.3), use of library functions (7.1.4), attributes (6.7.13.2).

6.4.2.2 Predefined identifiers

Semantics

1 The identifier **`__func__`** shall be implicitly declared by the translator as if, immediately following the opening brace of each function definition, the declaration

[65)] This allows a reserved identifier that matches the spelling of a keyword to be used as a macro name by the program.


```
static const char __func__[] = "function-name";
```

appeared, where *function-name* is the name of the lexically-enclosing function.[66)]

2 This name is encoded as if the implicit declaration had been written in the source character set and then translated into the execution character set as indicated in translation phase 5.

3 EXAMPLE The following code fragment can be used as an example:

```
#include <stdio.h>
void myfunc(void)
{
      printf("%s\n", __func__);
      /* ... */
}
```

Each time the function is called, it will print to the standard output stream:

```
myfunc
```

Forward references: function definitions (6.9.2).

6.4.3 Universal character names

Syntax

1 *universal-character-name:*
\u *hex-quad*
\U *hex-quad hex-quad*

hex-quad:
hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

Constraints

2 A universal character name shall not designate a code point where the hexadecimal value is:

— in the range D800 through DFFF inclusive; or

— greater than 10FFFF.[67)]

A universal character name outside the c-char sequence of a character constant, or the s-char sequence of a string literal shall not designate a control character or a character in the basic character set.

Description

3 Universal character names may be used in identifiers, character constants, and string literals to designate characters that are not in the basic character set.

Semantics

4 A universal character name designates the character in ISO/IEC 10646 whose code point is the hexadecimal value represented by the sequence of hexadecimal digits in the universal character name.

[66)] Since the name `__func__` is reserved for any use by the implementation (7.1.3), if any other identifier is explicitly declared using the name `__func__`, the behavior is undefined.

[67)] The disallowed characters are the characters in the basic character set and the code positions reserved by ISO/IEC 10646 for control characters, the character DELETE, the S-zone (reserved for use by UTF-16), and characters too large to be encoded by ISO/IEC 10646. Disallowed universal character escape sequences can still be specified with hexadecimal and octal escape sequences (6.4.4.5).

6.4.4 Constants

6.4.4.1 General

Syntax

1 *constant:*
integer-constant
floating-constant
enumeration-constant
character-constant
predefined-constant

Constraints

2 Each constant shall have a type and the value of a constant shall be in the range of representable values for its type.

Semantics

3 Each constant has a type, determined by its form and value, as detailed later.

6.4.4.2 Integer constants

Syntax

1 *integer-constant:*
decimal-constant *integer-suffix*$_{\text{opt}}$
octal-constant *integer-suffix*$_{\text{opt}}$
hexadecimal-constant *integer-suffix*$_{\text{opt}}$
binary-constant *integer-suffix*$_{\text{opt}}$

decimal-constant:
nonzero-digit
decimal-constant **'**$_{\text{opt}}$ *digit*

octal-constant:
0
octal-constant **'**$_{\text{opt}}$ *octal-digit*

hexadecimal-constant:
hexadecimal-prefix *hexadecimal-digit-sequence*

binary-constant:
binary-prefix *binary-digit*
binary-constant **'**$_{\text{opt}}$ *binary-digit*

hexadecimal-prefix: one of
0x 0X

binary-prefix: one of
0b 0B

nonzero-digit: one of
1 2 3 4 5 6 7 8 9

octal-digit: one of
0 1 2 3 4 5 6 7

hexadecimal-digit-sequence:
hexadecimal-digit
hexadecimal-digit-sequence **'**$_{\text{opt}}$ *hexadecimal-digit*

hexadecimal-digit: one of
0 1 2 3 4 5 6 7 8 9
a b c d e f
A B C D E F

binary-digit: one of
0 1

integer-suffix:
unsigned-suffix long-suffix$_{\text{opt}}$
unsigned-suffix long-long-suffix
unsigned-suffix bit-precise-int-suffix
long-suffix unsigned-suffix$_{\text{opt}}$
long-long-suffix unsigned-suffix$_{\text{opt}}$
bit-precise-int-suffix unsigned-suffix$_{\text{opt}}$

bit-precise-int-suffix: one of
wb WB

unsigned-suffix: one of
u U

long-suffix: one of
l L

long-long-suffix: one of
ll LL

Description

2 An integer constant begins with a digit, but has no period or exponent part. It may have a prefix that specifies its base and a suffix that specifies its type. An optional separating single quote character

(') in an integer or floating constant is called a digit separator. Digit separators are ignored when determining the value of the constant.

3 EXAMPLE 1 The following integer constants use digit separators; the comment associated with each constant shows the equivalent constant without digit separators.

```
0b11'10'11'01 /* 0b11101101 */
'1'2 /* character constant '1' followed by integer constant 2,
      not the integer constant 12 */
11'22 /* 1122 */
0x'FFFF'FFFF /* invalid hexadecimal constant (' cannot appear after 0x) */
0x1'2'3'4AB'C'D /* 0x1234ABCD */
```

4 A decimal constant begins with a nonzero digit and consists of a sequence of decimal digits. An octal constant consists of the prefix **`0`** optionally followed by a sequence of the digits **`0`** through **`7`** only. A hexadecimal constant consists of the prefix **`0x`** or **`0X`** followed by a sequence of the decimal digits and the letters **`a`** (or **`A`**) through **`f`** (or **`F`**) with values 10 through 15 respectively. A binary constant consists of the prefix **`0b`** or **`0B`** followed by a sequence of the digits **`0`** or **`1`**.

Semantics

5 The value of a decimal constant is computed base 10; that of an octal constant, base 8; that of a hexadecimal constant, base 16; that of a binary constant, base 2. The lexically first digit is the most significant.

6 The type of an integer constant is the first of the corresponding list in which its value can be represented.

Suffix	Decimal Constant	Octal, Hexadecimal or Binary Constant
none	**`int`** **`long int`** **`long long int`**	**`int`** **`unsigned int`** **`long int`** **`unsigned long int`** **`long long int`** **`unsigned long long int`**
`u` or **`U`**	**`unsigned int`** **`unsigned long int`** **`unsigned long long int`**	**`unsigned int`** **`unsigned long int`** **`unsigned long long int`**
`l` or **`L`**	**`long int`** **`long long int`**	**`long int`** **`unsigned long int`** **`long long int`** **`unsigned long long int`**
Both **`u`** or **`U`** and **`l`** or **`L`**	**`unsigned long int`** **`unsigned long long int`**	**`unsigned long int`** **`unsigned long long int`**
`ll` or **`LL`**	**`long long int`**	**`long long int`** **`unsigned long long int`**
Both **`u`** or **`U`** and **`ll`** or **`LL`**	**`unsigned long long int`**	**`unsigned long long int`**
`wb` or **`WB`**	**`_BitInt`**`(N)` where the width `N` is the smallest `N` greater than 1 which can accommodate the value and the sign bit.	**`_BitInt`**`(N)` where the width `N` is the smallest `N` greater than 1 which can accommodate the value and the sign bit.
Both **`u`** or **`U`** and **`wb`** or **`WB`**	**`unsigned _BitInt`**`(N)` where the width `N` is the smallest `N` greater than 0 which can accommodate the value.	**`unsigned _BitInt`**`(N)` where the width `N` is the smallest `N` greater than 0 which can accommodate the value.

7 If an integer constant that does not have suffixes **`wb`**, **`WB`**, **`uwb`**, or **`UWB`** cannot be represented by any type in its list, it may have an extended integer type, if the extended integer type can represent

its value. If all the types in the list for the constant are signed, the extended integer type shall be signed. If all the types in the list for the constant are unsigned, the extended integer type shall be unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. If an integer constant cannot be represented by any type in its list and has no extended integer type, then the integer constant has no type.

8 EXAMPLE 2 The **wb** suffix results in an **_BitInt** that includes space for the sign bit even if the value of the constant is positive or was specified in binary, octal, or hexadecimal notation.

```
-3wb   /* Yields a _BitInt(3) that is then arithmetically negated;
          two value bits, one sign bit */
-0x3wb /* Yields a _BitInt(3) that is then arithmetically negated;
          two value bits, one sign bit */
3wb    /* Yields a _BitInt(3); two value bits, one sign bit */
3uwb   /* Yields an unsigned _BitInt(2) */
-3uwb  /* Yields an unsigned _BitInt(2) that is then arithmetically
           negated, resulting in wraparound */
```

Forward references: preprocessing numbers (6.4.8), numeric conversion functions (7.24.1).

6.4.4.3 Floating constants

Syntax

1 *floating-constant:*
decimal-floating-constant
hexadecimal-floating-constant

decimal-floating-constant:
fractional-constant *exponent-part*$_{\text{opt}}$ *floating-suffix*$_{\text{opt}}$
digit-sequence *exponent-part* *floating-suffix*$_{\text{opt}}$

hexadecimal-floating-constant:
hexadecimal-prefix *hexadecimal-fractional-constant*
binary-exponent-part *floating-suffix*$_{\text{opt}}$
hexadecimal-prefix *hexadecimal-digit-sequence*
binary-exponent-part *floating-suffix*$_{\text{opt}}$

fractional-constant:
digit-sequence$_{\text{opt}}$ **.** *digit-sequence*
digit-sequence **.**

exponent-part:
e *sign*$_{\text{opt}}$ *digit-sequence*
E *sign*$_{\text{opt}}$ *digit-sequence*

sign: one of
+ -

digit-sequence:
digit

digit-sequence **'**$_{\text{opt}}$ *digit*

hexadecimal-fractional-constant:
hexadecimal-digit-sequence$_{\text{opt}}$ **.** *hexadecimal-digit-sequence*
hexadecimal-digit-sequence **.**

binary-exponent-part:
p *sign*$_{\text{opt}}$ *digit-sequence*
P *sign*$_{\text{opt}}$ *digit-sequence*

floating-suffix: one of
f l F L df dd dl DF DD DL

Constraints

2 A floating suffix **df**, **dd**, **dl**, **DF**, **DD**, or **DL** shall not be used in a hexadecimal floating constant.

Description

3 A floating constant has a *significand part* that may be followed by an *exponent part* and a suffix that specifies its type. The components of the significand part may include a digit sequence representing the whole-number part, followed by a period (**.**), followed by a digit sequence representing the fraction part. Digit separators (6.4.4.2) are ignored when determining the value of the constant. The components of the exponent part are an **e**, **E**, **p**, or **P** followed by an exponent consisting of an optionally signed digit sequence. Either the whole-number part or the fraction part has to be present; for decimal floating constants, either the period or the exponent part has to be present.

Semantics

4 The significand part is interpreted as a (decimal or hexadecimal) rational number; the digit sequence in the exponent part is interpreted as a decimal integer. For decimal floating constants, the exponent indicates the power of 10 by which the significand part is to be scaled. For hexadecimal floating constants, the exponent indicates the power of 2 by which the significand part is to be scaled. For decimal floating constants, and also for hexadecimal floating constants when **FLT_RADIX** is not a power of 2, the result is either the nearest representable value, or the larger or smaller representable value immediately adjacent to the nearest representable value, chosen in an implementation-defined manner. For hexadecimal floating constants when **FLT_RADIX** is a power of 2, the result is correctly rounded.

5 An unsuffixed floating constant has type **double**. If suffixed by a floating suffix it has a type according to Table 6.1.

Table 6.1: Suffixes for floating constants

Suffix	Type
f, **F**	**float**
l, **L**	**long double**
df, **DF**	**_Decimal32**
dd, **DD**	**_Decimal64**
dl, **DL**	**_Decimal128**

6 The values of floating constants may be represented in greater range and precision than that required by the type (determined by the suffix); the types are not changed thereby. See 5.2.5.3.3 regarding

evaluation formats.[68)]

7 Floating constants of decimal floating type that have the same numerical value but different quantum exponents have distinguishable internal representations. The value shall be correctly rounded as specified in ISO/IEC 60559. The coefficient c and the quantum exponent q of a finite converted decimal floating-point number (see 5.2.5.3.4) are determined as follows:

— q is set to the value of $sign_{opt}$ *digit-sequence* in the exponent part, if any, or to 0, otherwise.

— If there is a fractional constant, q is decreased by the number of digits to the right of the period and the period is removed to form a digit sequence.

— c is set to the value of the digit sequence (after any period has been removed).

— Rounding required because of insufficient precision or range in the type of the result will round c to the full precision available in the type, and will adjust q accordingly within the limits of the type, provided the rounding does not yield an infinity (in which case the result is an appropriately signed internal representation of infinity). If the full precision of the type would require q to be smaller than the minimum for the type, then q is pinned at the minimum and c is adjusted through the subnormal range accordingly, perhaps to zero.

8 Floating constants are converted to internal format as if at translation-time. The conversion of a floating constant shall not raise an exceptional condition or a floating-point exception at execution time. All floating constants of the same source form[69)] shall convert to the same internal format and, provided they are subject to the same translation-time rounding direction (either the default or a constant rounding mode set by an **FENV_ROUND** or **FENV_DEC_ROUND** pragma), to the same value.

9 EXAMPLE Following are floating constants of type **_Decimal64** and their values as triples (s, c, q). Note that for **_Decimal64**, the precision (maximum coefficient length) is 16 and the quantum exponent range is $-398 \leq q \leq 369$.

`0.dd`	$(+1, 0, 0)$
`0.00dd`	$(+1, 0, -2)$
`123.dd`	$(+1, 123, 0)$
`1.23E3dd`	$(+1, 123, 1)$
`1.23E+3dd`	$(+1, 123, 1)$
`12.3E+7dd`	$(+1, 123, 6)$
`12.0dd`	$(+1, 120, -1)$
`12.3dd`	$(+1, 123, -1)$
`0.00123dd`	$(+1, 123, -5)$
`1.23E-12dd`	$(+1, 123, -14)$
`1234.5E-4dd`	$(+1, 12345, -5)$
`0E+7dd`	$(+1, 0, 7)$
`12345678901234567890.dd`	$(+1, 1234567890123457, 4)$ assuming default rounding and **DEC_EVAL_METHOD** is 0 or 1[70)]
`1234E-400dd`	$(+1, 12, -398)$ assuming default rounding and **DEC_EVAL_METHOD** is 0 or 1
`1234E-402dd`	$(+1, 0, -398)$ assuming default rounding and **DEC_EVAL_METHOD** is 0 or 1
`1000.dd`	$(+1, 1000, 0)$
`.0001dd`	$(+1, 1, -4)$
`1000.e0dd`	$(+1, 1000, 0)$
`.0001e0dd`	$(+1, 1, -4)$
`1000.0dd`	$(+1, 10000, -1)$
`0.0001dd`	$(+1, 1, -4)$
`1000.00dd`	$(+1, 100\,000, -2)$
`00.0001dd`	$(+1, 1, -4)$
`001000.dd`	$(+1, 1000, 0)$

[68)] Hexadecimal floating constants can be used to obtain exact values in the semantic type that are independent of the evaluation format. Casts produce values in the semantic type, though depend on the rounding mode and may raise the inexact floating-point exception.

[69)] `1.23`, `1.230`, `123e-2`, `123e-02`, and `1.23L` are all different source forms and thus need not convert to the same internal format and value.

`001000.0dd`	$(+1, 10000, -1)$
`001000.00dd`	$(+1, 100\,000, -2)$
`00.00dd`	$(+1, 0, -2)$
`00.dd`	$(+1, 0, 0)$
`.00dd`	$(+1, 0, -2)$
`00.00e-5dd`	$(+1, 0, -7)$
`00.e-5dd`	$(+1, 0, -5)$
`.00e-5dd`	$(+1, 0, -7)$

Recommended practice

10 The implementation should produce a diagnostic message if a hexadecimal constant cannot be represented exactly in its evaluation format; the implementation should then proceed with the translation of the program.

11 The translation-time conversion of floating constants should match the execution-time conversion of character strings by library functions, such as **`strtod`**, given matching inputs suitable for both conversions, the same result format, and default execution-time rounding.[71)]

12 NOTE Floating constants do not include a sign and are arithmetically negated by the unary - operator (6.5.4.3) which arithmetically negates the rounded value of the constant. In contrast, the numeric conversion functions in the `strto` family (7.24.1.5, 7.24.1.6) can include the sign as part of the input value and convert and round the arithmetically negated input. Implementations conforming to Annex F have this behavior. Negating before rounding and negating after rounding may yield different results, depending on the rounding direction and whether the results are correctly rounded. For example, the results are the same when both are correctly rounded using rounding to nearest or rounding toward zero, but the results are different when they are inexact and correctly rounded using rounding toward positive infinity or rounding toward negative infinity.

Conversions yielding exact results are not affected by the order of negating and rounding. For types with radix 10, decimal floating constants expressed within the precision and range of the evaluation format convert exactly. For types whose radix is a power of 2, hexadecimal floating constants expressed within the precision and range of the evaluation format convert exactly.

Forward references: preprocessing numbers (6.4.8), numeric conversion functions (7.24.1), the `strto` function family (7.24.1.5, 7.24.1.6).

6.4.4.4 Enumeration constants

Syntax

1 *enumeration-constant:*
identifier

Semantics

2 An identifier declared as an enumeration constant for an enumeration without a fixed underlying type has either type **`int`** or the enumerated type, as defined in 6.7.3.3. An identifier declared as an enumeration constant for an enumeration with a fixed underlying type has the associated enumerated type.

3 An enumeration constant may be used in an expression (or constant expression) wherever a value of an integer type may be used.

Forward references: enumeration specifiers (6.7.3.3).

6.4.4.5 Character constants

Syntax

1 *character-constant:*
encoding-prefix$_{\text{opt}}$ **'** *c-char-sequence* **'**

[70)] That is, assuming the default translation rounding-direction mode is not changed by an **`FENV_DEC_ROUND`** pragma (7.6.3).
[71)] The specification for the library functions recommends more accurate conversion than required for floating constants (see 7.24.1.5).

encoding-prefix: one of
u8 **u** **U** **L**

c-char-sequence:
c-char
c-char-sequence c-char

c-char:
any member of the source character set except
the single-quote ', backslash \, or new-line character
escape-sequence

escape-sequence:
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence
universal-character-name

simple-escape-sequence: one of
\' \" \? \\
\a \b \f \n \r \t \v

octal-escape-sequence:
\ *octal-digit*
\ *octal-digit octal-digit*
\ *octal-digit octal-digit octal-digit*

hexadecimal-escape-sequence:
\x *hexadecimal-digit*
hexadecimal-escape-sequence hexadecimal-digit

Description

2 An *integer character constant* is a sequence of one or more multibyte characters enclosed in single-quotes, as in 'x'. A *UTF-8 character constant* is the same, except prefixed by **u8**. A ***wchar_t*** *character constant* is prefixed by the letter **L**. A *UTF-16 character constant* is prefixed by the letter **u**. A *UTF-32 character constant* is prefixed by the letter **U**. Collectively, **wchar_t**, UTF-16, and UTF-32 character constants are called *wide character constants*. With a few exceptions detailed later, the elements of the sequence are any members of the source character set; they are mapped in an implementation-defined manner to members of the execution character set.

3 The single-quote ', the double-quote ", the question-mark ?, the backslash \, and arbitrary integer values are representable according to the following table of escape sequences:

single quote '	\'
double quote "	\"
question mark ?	\?
backslash \	\\
octal character	*octal digits*
hexadecimal character	\x *hexadecimal digits*

4 The double-quote " and question-mark ? are representable either by themselves or by the escape sequences \" and \?, respectively, but the single-quote ' and the backslash \ shall be represented, respectively, by the escape sequences \' and \\.

5 The octal digits that follow the backslash in an octal escape sequence are taken to be part of the construction of a single character for an integer character constant or of a single wide character for a wide character constant. The numerical value of the octal integer so formed specifies the value of the desired character or wide character.

6 The hexadecimal digits that follow the backslash and the letter x in a hexadecimal escape sequence are taken to be part of the construction of a single character for an integer character constant or of a single wide character for a wide character constant. The numerical value of the hexadecimal integer so formed specifies the value of the desired character or wide character.

7 Each octal or hexadecimal escape sequence is the longest sequence of characters that can constitute the escape sequence.

8 In addition, characters not in the basic character set are representable by universal character names and certain non-graphic characters are representable by escape sequences consisting of the backslash \ followed by a lowercase letter: \a, \b, \f, \n, \r, \t, and \v.[72)]

Constraints

9 The value of an octal or hexadecimal escape sequence shall be in the range of representable values for the corresponding type:

Prefix	Corresponding Type
none	**unsigned char**
u8	**char8_t**
L	the unsigned type corresponding to **wchar_t**
u	**char16_t**
U	**char32_t**

10 A UTF-8, UTF-16, or UTF-32 character constant shall not contain more than one character.[73)] The value shall be representable with a single UTF-8, UTF-16, or UTF-32 code unit, respectively.

Semantics

11 An integer character constant has type **int**. The value of an integer character constant containing a single character that maps to a single value in the literal encoding (6.2.9) is the numerical value of the representation of the mapped character in the literal encoding interpreted as an integer. The value of an integer character constant containing more than one character (e.g. 'ab'), or containing a character or escape sequence that does not map to a single value in the literal encoding, is implementation-defined. If an integer character constant contains a single character or escape sequence, its value is the one that results when an object with type **char** whose value is that of the single character or escape sequence is converted to type **int**.

12 A UTF-8 character constant has type **char8_t**. If the UTF-8 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-8 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-8 code unit. Otherwise, the value of the UTF-8 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

13 A UTF-16 character constant has type **char16_t** which is an unsigned integer type defined in the

[72)] The semantics of these characters were discussed in 5.2.3. If any other character follows a backslash, the result is not a token and a diagnostic is required. See "future language directions" (6.11.4).

[73)] For example u8'ab' violates this constraint.

<uchar.h> header. If the UTF-16 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-16 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-16 code unit. Otherwise, the value of the UTF-16 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

14 A UTF-32 character constant has type **char32_t** which is an unsigned integer type defined in the <uchar.h> header. If the UTF-32 character constant is not produced through a hexadecimal or octal escape sequence, the value of a UTF-32 character constant is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-32 code unit. Otherwise, the value of the UTF-32 character constant is the numeric value specified in the hexadecimal or octal escape sequence.

15 A **wchar_t** character constant has type **wchar_t**, an integer type defined in the <stddef.h> header. The value of a **wchar_t** character constant containing a single multibyte character that maps to a single member of the extended execution character set is the wide character corresponding to that multibyte character in the implementation-defined wide literal encoding (6.2.9). The value of a **wchar_t** character constant containing more than one multibyte character or a single multibyte character that maps to multiple members of the extended execution character set, or containing a multibyte character or escape sequence not represented in the extended execution character set, is implementation-defined.

16 EXAMPLE 1 The construction `'\0'` is commonly used to represent the null character.

17 EXAMPLE 2 Implementations that use eight bits for objects that have type **char** can furnish certain values in an variety of ways. In an implementation in which type **char** has the same range of values as **signed char**, the integer character constant `'\xFF'` has the value -1; if type **char** has the same range of values as **unsigned char**, the character constant `'\xFF'` has the value $+255$.

18 EXAMPLE 3 Even if eight bits are used for objects that have type **char**, the construction `'\x123'` specifies an integer character constant containing only one character, since a hexadecimal escape sequence is terminated only by a non-hexadecimal character. To specify an integer character constant containing the two characters whose values are `'\x12'` and `'3'`, the construction `'\0223'` can be used, since an octal escape sequence is terminated after three octal digits. (The value of this two-character integer character constant is implementation-defined.)

19 EXAMPLE 4 Even if 12 or more bits are used for objects that have type **wchar_t**, the construction `L'\1234'` specifies the implementation-defined value that results from the combination of the values `0123` and `'4'`.

Forward references: common definitions <stddef.h> (7.21), the **mbtowc** function (7.24.7.2), Unicode utilities <uchar.h> (7.30).

6.4.4.6 Predefined constants

Syntax

1 *predefined-constant:*

false
true
nullptr

Description

2 Some keywords represent constants of a specific value and type.

3 The keywords **false** and **true** are constants of type **bool** with a value of `0` for **false** and `1` for **true**.[74)]

4 The keyword **nullptr** represents a null pointer constant. Details of its type are described in 7.21.2.

[74)] The constants **false** and **true** promote to type **int**, see 6.3.1.1. When used for arithmetic, in translation phase 4 (5.1.1.2), they are signed values and the result of such arithmetic is consistent with the results of later translation phases.

6.4.5 String literals

Syntax

1 *string-literal:*

encoding-prefix$_{\text{opt}}$ **"** *s-char-sequence*$_{\text{opt}}$ **"**

s-char-sequence:

s-char
s-char-sequence s-char

s-char:

any member of the source character set except
the double-quote `"`, backslash `\`, or new-line character
escape-sequence

Constraints

2 If a sequence of adjacent string literal tokens includes prefixed string literal tokens, the prefixed tokens shall all have the same prefix.

Description

3 A *character string literal* is a sequence of zero or more multibyte characters enclosed in double-quotes, as in `"xyz"`. A *UTF-8 string literal* is the same, except prefixed by **`u8`**. A ***`wchar_t`*** *string literal* is the same, except prefixed by **`L`**. A *UTF-16 string literal* is the same, except prefixed by **`u`**. A *UTF-32 string literal* is the same, except prefixed by **`U`**. Collectively, **`wchar_t`**, UTF-16, and UTF-32 string literals are called *wide string literals*.

4 The same considerations apply to each element of the sequence in a string literal as if it were in an integer character constant (for a character or UTF-8 string literal) or a wide character constant (for a wide string literal), except that the single-quote `'` is representable either by itself or by the escape sequence `\'`, but the double-quote `"` shall be represented by the escape sequence `\"`.

Semantics

5 In translation phase 6 (5.1.1.2), the multibyte character sequences specified by any sequence of adjacent character and identically-prefixed string literal tokens are concatenated into a single multibyte character sequence. If any of the tokens has an encoding prefix, the resulting multibyte character sequence is treated as having the same prefix; otherwise, it is treated as a character string literal.

6 In translation phase 7 (5.1.1.2), a byte or code of value zero is appended to each multibyte character sequence that results from a string literal or literals.[75)] The multibyte character sequence is then used to initialize an array of static storage duration and length just sufficient to contain the sequence. For character string literals, the array elements have type **`char`**, and are initialized with the individual bytes of the multibyte character sequence corresponding to the literal encoding (6.2.9). For UTF-8 string literals, the array elements have type **`char8_t`**, and are initialized with the characters of the multibyte character sequence, as encoded in UTF-8. For wide string literals prefixed by the letter **`L`**, the array elements have type **`wchar_t`** and are initialized with the sequence of wide characters corresponding to the wide literal encoding. For wide string literals prefixed by the letter **`u`** or **`U`**, the array elements have type **`char16_t`** or **`char32_t`**, respectively, and are initialized sequence of wide characters corresponding to UTF-16 and UTF-32 encoded text, respectively. The value of a string literal containing a multibyte character or escape sequence not represented in the execution character set is implementation-defined. Any hexadecimal escape sequence or octal escape sequence specified in a **`u8`**, **`u`**, or **`U`** string specifies a single **`char8_t`**, **`char16_t`**, or **`char32_t`** value and may result in the full character sequence not being valid UTF-8, UTF-16, or UTF-32.

[75)] A string literal may not be a string (see 7.1.1), because a null character can be embedded in it by a `\0` escape sequence.

7 It is unspecified whether these arrays are distinct provided their elements have the appropriate values. If the program attempts to modify such an array, the behavior is undefined.

8 EXAMPLE 1 This pair of adjacent character string literals

```
"\x12" "3"
```

produces a single character string literal containing the two characters whose values are `'\x12'` and `'3'`, because escape sequences are converted into single members of the execution character set just prior to adjacent string literal concatenation.

9 EXAMPLE 2 Each of the sequences of adjacent string literal tokens

```
"a" "b" L"c"
"a" L"b" "c"
L"a" "b" L"c"
L"a" L"b" L"c"
```

is equivalent to the string literal

```
L"abc"
```

Likewise, each of the sequences

```
"a" "b" u"c"
"a" u"b" "c"
u"a" "b" u"c"
u"a" u"b" u"c"
```

is equivalent to

```
u"abc"
```

Forward references: common definitions `<stddef.h>` (7.21), the **`mbstowcs`** function (7.24.8.1), Unicode utilities `<uchar.h>` (7.30).

6.4.6 Punctuators

Syntax

1 *punctuator:* one of

```
[  ]  (  )  {  }  .   ->
++  --  &  *  +  -  ~  !
/  %  <<  >>  <  >  <=  >=  ==  !=  ^  |  &&  ||
?  :  ::  ;  ...
=  *=  /=  %=  +=  -=  <<=  >>=  &=  ^=  |=
,  #  ##
<:  :>  <%  %>  %:  %:%:
```

Semantics

2 A punctuator is a symbol that has independent syntactic and semantic significance. Depending on context, it may specify an operation to be performed (which in turn may yield a value or a function designator, produce a side effect, or some combination thereof) in which case it is known as an *operator* (other forms of operator also exist in some contexts). An *operand* is an entity on which an operator acts.

3 In all aspects of the language, the six tokens[76)]

```
<:  :>  <%  %>  %:  %:%:
```

[76)] These tokens are sometimes called "digraphs".

behave, respectively, the same as the six tokens

[ ] { } # ##

except for their spelling.[77)]

Forward references: expressions (6.5.1), declarations (6.7), preprocessing directives (6.10), statements (6.8).

6.4.7 Header names

Syntax

1 *header-name:*
< *h-char-sequence* **>**
" *q-char-sequence* **"**

h-char-sequence:
h-char
h-char-sequence h-char

h-char:
any member of the source character set except
the new-line character and >

q-char-sequence:
q-char
q-char-sequence q-char

q-char:
any member of the source character set except
the new-line character and **"**

Semantics

2 The sequences in both forms of header names are mapped in an implementation-defined manner to headers or external source file names as specified in 6.10.3.

3 If the characters `'`, \, `"`, `//`, or `/*` occur in the sequence between the < and > delimiters, the behavior is undefined. Similarly, if the characters `'`, \, `//`, or `/*` occur in the sequence between the `"` delimiters, the behavior is undefined.[78)]

Header name preprocessing tokens are recognized only within **`#include`** and **`#embed`** preprocessing directives, in **`__has_include`** and **`__has_embed`** expressions, as well as in implementation-defined locations within **`#pragma`** directives.[79)]

4 EXAMPLE The following sequence of characters:

```
0x3<1/a.h>1e2
#include <1/a.h>
#define const.member@$
```

[77)] Thus [and <: behave differently when "stringized" (see 6.10.5.2), but can otherwise be freely interchanged.
[78)] Thus, sequences of characters that resemble escape sequences cause undefined behavior.
[79)] For an example of a header name preprocessing token used in a **`#pragma`** directive, see 6.10.11.

forms the following sequence of preprocessing tokens (with each individual preprocessing token delimited by a { on the left and a } on the right).

```
{0x3}{<}{1}{/}{a}{.}{h}{>}{1e2}
{#}{include} {<1/a.h>}
{#}{define} {const}{.}{member}{@}{$}
```

Forward references: source file inclusion (6.10.3).

6.4.8 Preprocessing numbers

Syntax

1 *pp-number:*
digit
. *digit*
pp-number identifier-continue
pp-number **'** *digit*
pp-number **'** *nondigit*
pp-number **e** *sign*
pp-number **E** *sign*
pp-number **p** *sign*
pp-number **P** *sign*
pp-number **.**

Description

2 A preprocessing number begins with a digit optionally preceded by a period (**.**) and may be followed by valid identifier characters and the character sequences **e+**, **e-**, **E+**, **E-**, **p+**, **p-**, **P+**, or **P-**.

3 Preprocessing number tokens lexically include all floating and integer constant tokens.

Semantics

4 A preprocessing number does not have type or a value; it acquires both after a successful conversion (as part of translation phase 7 (5.1.1.2)) to a floating constant token or an integer constant token.

6.4.9 Comments

1 Except within a character constant, a string literal, or a comment, the characters `/*` introduce a comment. The contents of such a comment are examined only to identify multibyte characters and to find the characters `*/` that terminate it.[80)]

2 Except within a character constant, a string literal, or a comment, the characters `//` introduce a comment that includes all multibyte characters up to, but not including, the next new-line character. The contents of such a comment are examined only to identify multibyte characters and to find the terminating new-line character.

3 EXAMPLE

```
"a//b"                  // four-character string literal
#include "//e"          // undefined behavior
// */                   // comment, not syntax error
f = g/**//h;            // equivalent to f = g / h;
//\
i();                    // part of a two-line comment
/\
/ j();                  // part of a two-line comment
#define glue(x,y) x##y
glue(/,/) k();          // syntax error, not comment
/*//*/ l();             // equivalent to l();
```

[80)] Thus, `/* ... */` comments do not nest.

```
    m = n//**/o
      + p;                    // equivalent to m = n + p;
```

6.5 Expressions

6.5.1 General

1 An *expression* is a sequence of operators and operands that specifies computation of a value, or that designates an object or a function, or that generates side effects, or that performs a combination thereof. The value computations of the operands of an operator are sequenced before the value computation of the result of the operator.

2 If a side effect on a scalar object is unsequenced relative to either a different side effect on the same scalar object or a value computation using the value of the same scalar object, the behavior is undefined. If there are multiple allowable orderings of the subexpressions of an expression, the behavior is undefined if such an unsequenced side effect occurs in any of the orderings.[81)]

3 The grouping of operators and operands is indicated by the syntax.[82)] Except as specified later, side effects and value computations of subexpressions are unsequenced.[83)]

4 Some operators (the unary operator ~, and the binary operators <<, >>, &, ^, and |, collectively described as *bitwise operators*) are required to have operands that have integer type. These operators yield values that depend on the internal representations of integers, and have implementation-defined and undefined aspects for signed types.

5 If an *exceptional condition* occurs during the evaluation of an expression (that is, if the result is not mathematically defined or not in the range of representable values for its type), the behavior is undefined.

6 The *effective type* of an object for an access to its stored value is the declared type of the object, if any.[84)] If a value is stored into an object having no declared type through an lvalue having a type that is not a non-atomic character type, then the type of the lvalue becomes the effective type of the object for that access and for subsequent accesses that do not modify the stored value. If a value is copied into an object having no declared type using **`memcpy`** or **`memmove`**, or is copied as an array of character type, then the effective type of the modified object for that access and for subsequent accesses that do not modify the value is the effective type of the object from which the value is copied, if it has one. For all other accesses to an object having no declared type, the effective type of the object is simply the type of the lvalue used for the access.

7 An object shall have its stored value accessed only by an lvalue expression that has one of the following types:[85)]

— a type compatible with the effective type of the object,

— a qualified version of a type compatible with the effective type of the object,

[81)] This paragraph renders undefined statement expressions such as

```
i = ++i + 1;
a[i++] = i;
```

while allowing

```
i = i + 1;
a[i] = i;
```

[82)] The syntax specifies the precedence of operators in the evaluation of an expression, which is the same as the order of the major subclauses of this subclause, highest precedence first. Thus, for example, the expressions allowed as the operands of the binary + operator (6.5.7) are those expressions defined in 6.5.2 through 6.5.7. The exceptions are cast expressions (6.5.5) as operands of unary operators (6.5.4), and an operand contained between any of the following pairs of operators: grouping parentheses `()` (6.5.2), generic selection parentheses `()` (6.5.2.1), subscripting brackets code[] (6.5.3.2), function-call parentheses `()` (6.5.3.3), and the conditional operator `?:` (6.5.16).

Within each major subclause, the operators have the same precedence. Left- or right-associativity is indicated in each subclause by the syntax for the expressions discussed therein.

[83)] In an expression that is evaluated more than once during the execution of a program, unsequenced and indeterminately sequenced evaluations of its subexpressions can be performed inconsistently in different evaluations.

[84)] Allocated objects have no declared type.

[85)] The intent of this list is to specify those circumstances in which an object can or cannot be aliased.

— the signed or unsigned type compatible with the underlying type of the effective type of the object,

— the signed or unsigned type compatible with a qualified version of the underlying type of the effective type of the object,

— an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or

— a character type.

8 A floating expression may be *contracted*, that is, evaluated as though it were a single operation, thereby omitting rounding errors implied by the source code and the expression evaluation method.[86] The `FP_CONTRACT` pragma in `<math.h>` provides a way to disallow contracted expressions. Otherwise, whether and how expressions are contracted is implementation-defined.[87]

9 Operators involving decimal floating types are evaluated according to the semantics of ISO/IEC 60559, including production of results with the preferred quantum exponent as specified in ISO/IEC 60559.

Forward references: the `FP_CONTRACT` pragma (7.12.2), copying functions (7.26.2).

6.5.2 Primary expressions

Syntax

1 *primary-expression:*
identifier
constant
string-literal
(*expression* **)**
generic-selection

Constraints

2 The identifier in an identifier primary expression shall have a visible declaration as an ordinary identifier that declares an object or a function.[88]

Semantics

3 An identifier primary expression designating an object is an lvalue. An identifier primary expression designating a function is a function designator.

4 A constant is a primary expression. Its type depends on its form and value, as detailed in 6.4.4.

5 A string literal is a primary expression. It is an lvalue with type as detailed in 6.4.5.

6 A *parenthesized expression* is a primary expression. Its type, value, and semantics are identical to those of the unparenthesized expression.

7 A generic selection is a primary expression. Its type, value, and semantics depend on the selected generic association, as detailed in the following subclause.

Forward references: declarations (6.7).

[86] The intermediate operations in the contracted expression are evaluated as if to infinite range and precision, while the final operation is rounded to the format determined by the expression evaluation method. A contracted expression may also omit the raising of floating-point exceptions.

[87] This license is specifically intended to allow implementations to exploit fast machine instructions that combine multiple C operators. As contractions potentially undermine predictability, and can even decrease accuracy for containing expressions, their use needs to be well-defined and clearly documented.

[88] An identifier designating an enumeration constant is a primary expression through the constant production, not the identifier production.

6.5.2.1 Generic selection

Syntax

1 *generic-selection:*
_Generic (*assignment-expression* **,** *generic-assoc-list* **)**
generic-assoc-list:
generic-association
generic-assoc-list **,** *generic-association*
generic-association:
type-name **:** *assignment-expression*
default : *assignment-expression*

Constraints

2 A generic selection shall have no more than one **default** generic association. The type name in a generic association shall specify a complete object type other than a variably modified type. No two generic associations in the same generic selection shall specify compatible types. The type of the controlling expression is the type of the expression as if it had undergone an lvalue conversion,[89] array to pointer conversion, or function to pointer conversion. That type shall be compatible with at most one of the types named in the generic association list. If a generic selection has no **default** generic association, its controlling expression shall have type compatible with exactly one of the types named in its generic association list.

Semantics

3 The controlling expression of a generic selection is not evaluated. If a generic selection has a generic association with a type name that is compatible with the type of the controlling expression, then the result expression of the generic selection is the expression in that generic association. Otherwise, the result expression of the generic selection is the expression in the **default** generic association. None of the expressions from any other generic association of the generic selection is evaluated.

4 The type and value of a generic selection are identical to those of its result expression. It is an lvalue, a function designator, or a void expression if its result expression is, respectively, an lvalue, a function designator, or a void expression.

5 EXAMPLE A **cbrt** type-generic macro could be implemented as follows:

```
#define cbrt(X) _Generic((X),                   \
                         long double: cbrtl,    \
                         default: cbrt,         \
                         float: cbrtf           \
                         )(X)
```

7.27 shows how such a macro could be implemented with the required rounding properties.

6.5.3 Postfix operators

6.5.3.1 General

Syntax

1 *postfix-expression:*
primary-expression
postfix-expression **[** *expression* **]**
postfix-expression **(** *argument-expression-list*$_{\text{opt}}$ **)**
postfix-expression **.** *identifier*
postfix-expression **->** *identifier*
postfix-expression **++**
postfix-expression **--**
compound-literal

[89] An lvalue conversion drops type qualifiers.

argument-expression-list:
assignment-expression
argument-expression-list **,** *assignment-expression*

6.5.3.2 Array subscripting

Constraints

1 One of the expressions shall have type "pointer to complete object *type*", the other expression shall have integer type, and the result has type "*type*".

Semantics

2 A postfix expression followed by an expression in square brackets `[]` is a subscripted designation of an element of an array object. The definition of the subscript operator `[]` is that `E1[E2]` is identical to `(*((E1)+(E2)))`. Because of the conversion rules that apply to the binary + operator, if `E1` is an array object (equivalently, a pointer to the initial element of an array object) and `E2` is an integer, `E1[E2]` designates the `E2`-th element of `E1` (counting from zero).

3 Successive subscript operators designate an element of a multidimensional array object. If `E` is an n-dimensional array ($n \geq 2$) with dimensions $i \times j \times \cdots \times k$, then `E` (used as other than an lvalue) is converted to a pointer to an $(n-1)$-dimensional array with dimensions $j \times \cdots \times k$. If the unary $*$ operator is applied to this pointer explicitly, or implicitly as a result of subscripting, the result is the referenced $(n-1)$-dimensional array, which itself is converted into a pointer if used as other than an lvalue. It follows from this that arrays are stored in row-major order (last subscript varies fastest).

4 EXAMPLE The following snippet has an array object defined by the declaration:

```
int x[3][5];
```

Here `x` is a 3×5 array of objects of type **`int`**; more precisely, `x` is an array of three element objects, each of which is an array of five objects of type **`int`**. In the expression `x[i]`, which is equivalent to `(*((x)+(i)))`, `x` is first converted to a pointer to the initial array of five objects of type **`int`**. Then `i` is adjusted according to the type of `x`, which conceptually entails multiplying `i` by the size of the object to which the pointer points, namely an array of five **`int`** objects. The results are added and indirection is applied to yield an array of five objects of type **`int`**. When used in the expression `x[i][j]`, that array is in turn converted to a pointer to the first of the objects of type **`int`**, so `x[i][j]` yields an **`int`**.

Forward references: additive operators (6.5.7), address and indirection operators (6.5.4.2), array declarators (6.7.7.3).

6.5.3.3 Function calls

Constraints

1 The expression that denotes the called function[90] shall have type pointer to function returning **`void`** or returning a complete object type other than an array type.

2 The number of arguments shall agree with the number of parameters. Each argument shall have a type such that its value may be assigned to an object with the unqualified version of the type of its corresponding parameter

Semantics

3 A postfix expression followed by parentheses `()` containing a possibly empty, comma-separated list of expressions is a function call. The postfix expression denotes the called function. The list of expressions specifies the arguments to the function.

4 An argument may be an expression of any complete object type. In preparing for the call to a function, the arguments are evaluated, and each parameter is assigned the value of the corresponding argument.[91]

[90] Most often, this is the result of converting an identifier that is a function designator.

[91] A function can change the values of its parameters, but these changes cannot affect the values of the arguments. On the other hand, it is possible to pass a pointer to an object, and the function can then change the value of the object pointed to. A

5 If the expression that denotes the called function has type pointer to function returning an object type, the function call expression has the same type as that object type, and has the value determined as specified in 6.8.7.5. Otherwise, the function call has type **void**.

6 The arguments are implicitly converted, as if by assignment, to the types of the corresponding parameters, taking the type of each parameter to be the unqualified version of its declared type. The ellipsis notation in a function prototype declarator causes argument type conversion to stop after the last declared parameter, if present. The integer promotions are performed on each trailing argument, and trailing arguments that have type **float** are promoted to **double**. These are called the *default argument promotions*. No other conversions are performed implicitly.

7 If the function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function, the behavior is undefined.

8 There is a sequence point after the evaluations of the function designator and the actual arguments but before the actual call. Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or after the execution of the body of the called function is indeterminately sequenced with respect to the execution of the called function.[92)]

9 Recursive function calls shall be permitted, both directly and indirectly through any chain of other functions.

10 EXAMPLE In the function call

```
(*pf[f1()]) (f2(), f3() + f4())
```

the functions `f1`, `f2`, `f3`, and `f4` can be called in any order. All side effects are completed before the function pointed to by `pf[f1()]` is called.

Forward references: function declarators (6.7.7.4), function definitions (6.9.2), the **return** statement (6.8.7.5), simple assignment (6.5.17.2).

6.5.3.4 Structure and union members

Constraints

1 The first operand of the `.` operator shall have an atomic, qualified, or unqualified structure or union type, and the second operand shall name a member of that type.

2 The first operand of the `->` operator shall have type "pointer to atomic, qualified, or unqualified structure" or "pointer to atomic, qualified, or unqualified union", and the second operand shall name a member of the type pointed to.

Semantics

3 A postfix expression followed by the `.` operator and an identifier designates a member of a structure or union object. The value is that of the named member,[93)] and is an lvalue if the first expression is an lvalue. If the first expression has qualified type, the result has the so-qualified version of the type of the designated member.

4 A postfix expression followed by the `->` operator and an identifier designates a member of a structure or union object. The value is that of the named member of the object to which the first expression points, and is an lvalue.[94)] If the first expression is a pointer to a qualified type, the result has the so-qualified version of the type of the designated member.

5 Accessing a member of an atomic structure or union object results in undefined behavior.[95)]

parameter declared to have array or function type is adjusted to have a pointer type as described in 6.7.7.4.

[92)] In other words, function executions do not interleave with each other.

[93)] If the member used to read the contents of a union object is not the same as the member last used to store a value in the object the appropriate part of the object representation of the value is reinterpreted as an object representation in the new type as described in 6.2.6 (a process sometimes called type punning). This may be a non-value representation.

[94)] If `&E` is a valid pointer expression (where & is the address of operator, which generates a pointer to its operand), the expression `(&E)->MOS` is the same as `E.MOS`.

[95)] For example, a data race would occur if access to the entire structure or union in one thread conflicts with access to a member from another thread, where at least one access is a modification. Members can be safely accessed using a non-atomic object which is assigned to or from the atomic object.

6 One special guarantee is made to simplify the use of unions: if a union contains several structures that share a common initial sequence (see following sentence), and if the union object currently contains one of these structures, it is permitted to inspect the common initial part of any of them anywhere that a declaration of the completed type of the union is visible. Two structures share a *common initial sequence* if corresponding members have compatible types (and, for bit-fields, the same widths) for a sequence of one or more initial members.

7 EXAMPLE 1 If `f` is a function returning a structure or union, and `x` is a member of that structure or union, `f().x` is a valid postfix expression but is not an lvalue.

8 EXAMPLE 2 In:

```
struct s { int i; const int ci; };
struct s s;
const struct s cs;
volatile struct s vs;
```

the various members have the types:

```
s.i    int
s.ci   const int
cs.i   const int
cs.ci  const int
vs.i   volatile int
vs.ci  volatile const int
```

9 EXAMPLE 3 The following is a valid fragment:

```
union {
      struct {
            int    alltypes;
      } n;
      struct {
            int    type;
            int    intnode;
      } ni;
      struct {
            int    type;
            double doublenode;
      } nf;
} u;
u.nf.type = 1;
u.nf.doublenode = 3.14;
/* ... */
if (u.n.alltypes == 1)
      if (sin(u.nf.doublenode) == 0.0)
            /* ... */
```

The following is not a valid fragment (because the union type is not visible within function `f`):

```
struct t1 { int m; };
struct t2 { int m; };
int f(struct t1 *p1, struct t2 *p2)
{
      if (p1->m  <  0)
            p2->m = -p2->m;
      return p1->m;
}
int g()
{
      union {
            struct t1 s1;
            struct t2 s2;
```

```
        } u;
        /* ... */
        return f(&u.s1, &u.s2);
}
```

Forward references: address and indirection operators (6.5.4.2), structure and union specifiers (6.7.3.2).

6.5.3.5 Postfix increment and decrement operators

Constraints

1 The operand of the postfix increment or decrement operator shall have atomic, qualified, or unqualified real or pointer type, and shall be a modifiable lvalue.

Semantics

2 The result of the postfix ++ operator is the value of the operand. As a side effect, the value of the operand object is incremented (that is, the value 1 of the appropriate type is added to it). See the discussions of additive operators and compound assignment for information on constraints, types, and conversions and the effects of operations on pointers. The value computation of the result is sequenced before the side effect of updating the stored value of the operand. With respect to an indeterminately sequenced function call, the operation of postfix ++ is a single evaluation. Postfix ++ on an object with atomic type is a read-modify-write operation with `memory_order_seq_cst` memory order semantics.[96)]

3 The postfix -- operator is analogous to the postfix ++ operator, except that the value of the operand is decremented (that is, the value 1 of the appropriate type is subtracted from it).

Forward references: additive operators (6.5.7), compound assignment (6.5.17.3).

6.5.3.6 Compound literals

Syntax

1 *compound-literal:*
(*storage-class-specifiers*$_{\text{opt}}$ *type-name* **)** *braced-initializer*

storage-class-specifiers:
storage-class-specifier
storage-class-specifiers storage-class-specifier

Constraints

2 The type name shall specify a complete object type or an array of unknown size, but not a variable length array type.

3 All the constraints for initializer lists in 6.7.11 also apply to compound literals.

4 If the compound literal is associated with file scope or block scope (see 6.2.1) the storage-class specifiers `SC` (possibly empty),[97)] type name `T`, and initializer list, if any, shall be such that they are

[96)] Where a pointer to an atomic object can be formed and `E` has integer type, `E++` is equivalent to the following code sequence where *T* is the type of `E`:

```
T *addr = &E;
T old = *addr;
T new;
do {
      new = old + 1;
} while (!atomic_compare_exchange_strong(addr, &old, new));
```

with `old` being the result of the operation.

Special care is necessary if `E` has floating type; see 6.5.17.3.

[97)] If the storage-class specifiers contain the same storage-class specifier more than once, the following constraint is violated.

valid specifiers for an object definition in file scope or block scope, respectively, of the following form,

```
SC typeof(T) ID = { IL };
```

where `ID` is an identifier that is unique for the whole program and where `IL` is a (possibly empty) initializer list with nested structure, designators, values and types as the initializer list of the compound literal. All the constraints for storage-class specifiers in 6.7.2 also apply correspondingly to compound literals. If the compound literal is associated with function prototype scope, constraints as if in block scope apply.

Semantics

5 For a *compound literal* associated with function prototype scope:

— the type is determined as if in block scope and no object is created;

— if it is a compound literal constant it is evaluated at translation time;

— if it is not a compound literal constant, neither the compound literal as a whole nor any of the initializers are evaluated.

Otherwise, a compound literal provides access to an unnamed object whose value, type, storage duration, initializer, and other properties are as if given by the definition syntax in the constraints.

6 If the storage duration is automatic, the lifetime of the instance of the unnamed object is the current execution of the enclosing block.[98)]

7 If the storage-class specifiers are absent or contain **`constexpr`**, **`static`**, **`register`**, or **`thread_local`** the behavior is as if the object were declared and initialized in the corresponding scope with these storage-class specifiers; if another storage-class specifier is present, the behavior is undefined. If the storage-class specifier **`constexpr`** is present, the initializer is evaluated at translation time. Otherwise, if the storage duration is automatic, the initializer is evaluated at each evaluation of the compound literal; if the storage duration is static or thread the initializer is (as if) evaluated once prior to program startup.

8 The value of the compound literal is that of an lvalue corresponding to the unnamed object.

9 All the semantic rules for initializer lists in 6.7.11 also apply to compound literals.[99)]

10 String literals, and compound literals with **`const`**-qualified types, including those specified with **`constexpr`**, are not required to designate distinct objects.[100)]

11 EXAMPLE 1 The following 2 functions can be used as an example:

```
int f(int*);
int g(char * para[f((int[27]){ 0, })]) {
      /* ... */
      return 0;
}
```

Here, each call to `g` creates an unnamed object of type **`int`**`[27]` to determine the variably-modified type of `para` for the duration of the call. During that determination, a pointer to the object is passed into a call to the function `f`. If a pointer to the object is kept by `f`, access to that object is possible during the whole execution of the call to `g`. The lifetime of the object ends with the end of the call to `g`; for any access after that, the behavior is undefined.

12 EXAMPLE 2 The file scope definition

[98)] Note that this differs from a cast expression. For example, a cast specifies a conversion to scalar types or **`void`** only, and the result of a cast expression is not an lvalue.

[99)] For example, subobjects without explicit initializers are initialized to zero.

[100)] This allows implementations to share storage for string literals and constant compound literals with the same or overlapping representations.


```
int *p = (int []){2, 4};
```

initializes `p` to point to the first element of an array of two **`int`** s, the first having the value two and the second, four. The expressions in this compound literal are required to be constant. The unnamed object has static storage duration.

13 EXAMPLE 3 In contrast, in

```
void f(void)
{
      int *p;
      /*...*/
      p = (int [2]){*p};
      /*...*/
}
```

`p` is assigned the address of the first element of an array of two **`int`** s, the first having the value previously pointed to by `p` and the second, zero. The initializer expression in this compound literal is not required to be constant. The unnamed object has automatic storage duration.

14 EXAMPLE 4 Initializers with designations can be combined with compound literals. Structure objects created using compound literals can be passed to functions without depending on member order:

```
drawline((struct point){.x=1, .y=1},
      (struct point){.x=3, .y=4});
```

Or, if `drawline` instead expected pointers to **`struct`** `point`:

```
drawline(&(struct point){.x=1, .y=1},
      &(struct point){.x=3, .y=4});
```

15 EXAMPLE 5 A read-only compound literal can be specified through constructions like:

```
(const float []){1e0, 1e1, 1e2, 1e3, 1e4, 1e5, 1e6}
```

16 EXAMPLE 6 The following three expressions have different meanings:

```
"/tmp/fileXXXXXX"
(char []){"/tmp/fileXXXXXX"}
(const char []){"/tmp/fileXXXXXX"}
```

The first always has static storage duration and has type array of **`char`**, but could or could not be modifiable; the last two have automatic storage duration when they occur within the body of a function, and the first of these two is modifiable.

17 EXAMPLE 7 Like string literals, const-qualified compound literals can be placed into read-only memory and can even be shared. For example,

```
(const char []){"abc"} == "abc"
```

may yield 1 if the literals' storage is shared.

18 EXAMPLE 8 Since compound literals are unnamed, a single compound literal cannot specify a circularly linked object. For example, there is no way to write a self-referential compound literal that could be used as the function argument in place of the named object `endless_zeros` in the following snippet:

```
      struct int_list { int car; struct int_list *cdr; };
      struct int_list endless_zeros = {0, &endless_zeros};
      eval(endless_zeros);
```

19 EXAMPLE 9 Each compound literal creates only a single object in a given scope:

```
struct s { int i; };

int f (void)
{
      struct s *p = 0, *q;
      int j = 0;

again:
      q = p, p = &((struct s){ j++ });
      if (j  <  2) goto again;

      return p == q && q->i == 1;
}
```

The function `f()` always returns the value 1.

20 Note that if an iteration statement were used instead of an explicit **goto** and a label, the lifetime of the unnamed object would be the body of the loop only, and on entry next time around `p` would have indeterminate representation, which would result in undefined behavior.

Forward references: type names (6.7.8), initialization (6.7.11).

6.5.4 Unary operators

Syntax

1 *unary-expression:*
postfix-expression
++ *unary-expression*
-- *unary-expression*
unary-operator cast-expression
sizeof *unary-expression*
sizeof (*type-name* **)**
alignof (*type-name* **)**

unary-operator: one of
& * + - ~ !

6.5.4.1 Prefix increment and decrement operators

Constraints

1 The operand of the prefix increment or decrement operator shall have atomic, qualified, or unqualified real or pointer type, and shall be a modifiable lvalue.

Semantics

2 The value of the operand of the prefix ++ operator is incremented. The result is the new value of the operand after incrementation. The expression ++`E` is equivalent to `(E+=1)`, where the value `1` is of the appropriate type. See the discussions of additive operators and compound assignment for information on constraints, types, side effects, and conversions and the effects of operations on pointers.

3 The prefix `--` operator is analogous to the prefix ++ operator, except that the value of the operand is decremented.

Forward references: additive operators (6.5.7), compound assignment (6.5.17.3).

6.5.4.2 Address and indirection operators

Constraints

1 The operand of the unary & operator shall be either a function designator, the result of a `[]` or unary * operator, or an lvalue that designates an object that is not a bit-field and is not declared with the

register storage-class specifier.

2 The operand of the unary * operator shall have pointer type.

Semantics

3 The unary & operator yields the address of its operand. If the operand has type "*type*", the result has type "pointer to *type*". If the operand is the result of a unary * operator, neither that operator nor the & operator is evaluated and the result is as if both were omitted, except that the constraints on the operators still apply and the result is not an lvalue. Similarly, if the operand is the result of a `[]` operator, neither the & operator nor the unary * that is implied by the `[]` is evaluated and the result is as if the & operator were removed and the `[]` operator were changed to a + operator. Otherwise, the result is a pointer to the object or function designated by its operand.

4 The unary * operator denotes indirection. If the operand points to a function, the result is a function designator; if it points to an object, the result is an lvalue designating the object. If the operand has type "pointer to *type*", the result has type "*type*". If an invalid value has been assigned to the pointer, the behavior of the unary * operator is undefined.[101)]

Forward references: storage-class specifiers (6.7.2), structure and union specifiers (6.7.3.2).

6.5.4.3 Unary arithmetic operators

Constraints

1 The operand of the unary + or - operator shall have arithmetic type; of the ~ operator, integer type; of the ! operator, scalar type.

Semantics

2 The result of the unary + operator is the value of its (promoted) operand. The integer promotions are performed on the operand, and the result has the promoted type.

3 The result of the unary - operator is the negative of its (promoted) operand. The integer promotions are performed on the operand, and the result has the promoted type.

4 The result of the ~ operator is the bitwise complement of its (promoted) operand (that is, each bit in the result is set if and only if the corresponding bit in the converted operand is not set). The integer promotions are performed on the operand, and the result has the promoted type. If the promoted type is an unsigned type, the expression `~E` is equivalent to the maximum value representable in that type minus `E`.

5 The result of the logical negation operator ! is 0 if the value of its operand compares unequal to 0, 1 if the value of its operand compares equal to 0. The result has type **int**. The expression `!E` is equivalent to `(0==E)`.

6.5.4.4 The sizeof and alignof operators

Constraints

1 The **sizeof** operator shall not be applied to an expression that has function type or an incomplete type, to the parenthesized name of such a type, or to an expression that designates a bit-field member. The **alignof** operator shall not be applied to a function type or an incomplete type.

Semantics

2 The **sizeof** operator yields the size (in bytes) of its operand, which may be an expression or the parenthesized name of a type. The size is determined from the type of the operand. The result is an integer. If the type of the operand is a variable length array type, the operand is evaluated; otherwise, the operand is not evaluated and the result is an integer constant.

3 The **alignof** operator yields the alignment requirement of its operand type. The operand is not

[101)] Thus, `&*E` is equivalent to `E` (even if `E` is a null pointer), and `&(E1[E2])` to `((E1)+(E2))`. It is always true that if `E` is a function designator or an lvalue that is a valid operand of the unary & operator, `*&E` is a function designator or an lvalue equal to `E`. If `*P` is an lvalue and `T` is the name of an object pointer type, `*(T)P` is an lvalue that has a type compatible with that to which `T` points.

Among the invalid values for dereferencing a pointer by the unary * operator are a null pointer, an address inappropriately aligned for the type of object pointed to, and the address of an object after the end of its lifetime.

evaluated and the result is an integer constant expression. When applied to an array type, the result is the alignment requirement of the element type.

4 When **sizeof** is applied to an operand that has type **char**, **unsigned char**, or **signed char**, (or a qualified version thereof) the result is 1. When applied to an operand that has array type, the result is the total number of bytes in the array.[102] When applied to an operand that has structure or union type, the result is the total number of bytes in such an object, including internal and trailing padding.

5 The value of the result of both operators is implementation-defined, and its type (an unsigned integer type) is **size_t**, defined in <stddef.h> (and other headers).

6 EXAMPLE 1 A principal use of the **sizeof** operator is in communication with routines such as storage allocators and I/O systems. A storage-allocation function can accept a size (in bytes) of an object to allocate and return a pointer to **void**. For example:

```
extern void *alloc(size_t);
double *dp = alloc(sizeof *dp);
```

The implementation of the alloc function presumably ensures that its return value is aligned suitably for conversion to a pointer to **double**.

7 EXAMPLE 2 Another use of the **sizeof** operator is to compute the number of elements in an array:

```
sizeof array / sizeof array[0]
```

8 EXAMPLE 3 In this example, the size of a variable length array is computed and returned from a function:

```
#include <stddef.h>

size_t fsize3(int n)
{
      char b[n+3];       // variable length array
      return sizeof b;   // execution time sizeof
}

int main(void)
{
      size_t size;
      size = fsize3(10); // fsize3 returns 13
      return 0;
}
```

Forward references: common definitions <stddef.h> (7.21), declarations (6.7), structure and union specifiers (6.7.3.2), type names (6.7.8), array declarators (6.7.7.3).

6.5.5 Cast operators

Syntax

1 *cast-expression:*

unary-expression
(*type-name* **)** *cast-expression*

Constraints

2 Unless the type name specifies a void type, the type name shall specify atomic, qualified, or unqualified scalar type, and the operand shall have scalar type.

3 Conversions that involve pointers, other than where permitted by the constraints of 6.5.17.2, shall be

[102] When applied to a parameter declared to have array or function type, the **sizeof** operator yields the size of the adjusted (pointer) type (see 6.9.2).

specified by means of an explicit cast.

4 A pointer type shall not be converted to any floating type. A floating type shall not be converted to any pointer type. The type **nullptr_t** shall not be converted to any type other than **void**, **bool** or a pointer type. If the target type is **nullptr_t**, the cast expression shall be a null pointer constant or have type **nullptr_t**.

Semantics

5 Size expressions and typeof operators contained in a type name used with a cast operator are evaluated whenever the cast expression is evaluated.

6 Preceding an expression by a parenthesized type name converts the value of the expression to the unqualified, non-atomic version of the named type. This construction is called a *cast*.[103] A cast that specifies no conversion has no effect on the type or value of an expression.

7 If the value of the expression is represented with greater range or precision than required by the type named by the cast (6.3.1.8), then the cast specifies a conversion even if the type of the expression is the same as the named type and removes any extra range and precision.

Forward references: equality operators (6.5.10), function declarators (6.7.7.4), simple assignment (6.5.17.2), type names (6.7.8).

6.5.6 Multiplicative operators

Syntax

1 *multiplicative-expression:*

cast-expression
multiplicative-expression `*` *cast-expression*
multiplicative-expression `/` *cast-expression*
multiplicative-expression `%` *cast-expression*

Constraints

2 Each of the operands shall have arithmetic type. The operands of the `%` operator shall have integer type.

3 If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

4 The usual arithmetic conversions are performed on the operands.

5 The result of the binary `*` operator is the product of the operands.

6 The result of the `/` operator is the quotient from the division of the first operand by the second; the result of the `%` operator is the remainder. In both operations, if the value of the second operand is zero, the behavior is undefined.

7 When integers are divided, the result of the `/` operator is the algebraic quotient with any fractional part discarded.[104] If the quotient `a/b` is representable, the expression `(a/b)*b + a%b` shall equal `a`; otherwise, the behavior of both `a/b` and `a%b` is undefined.

6.5.7 Additive operators

Syntax

1 *additive-expression:*

multiplicative-expression
additive-expression `+` *multiplicative-expression*
additive-expression `-` *multiplicative-expression*

[103] A cast does not yield an lvalue.
[104] This is often called "truncation toward zero".

Constraints

2 For addition, either both operands shall have arithmetic type, or one operand shall be a pointer to a complete object type and the other shall have integer type. (Incrementing is equivalent to adding 1.)

3 For subtraction, one of the following shall hold:

— both operands have arithmetic type;

— both operands are pointers to qualified or unqualified versions of compatible complete object types; or

— the left operand is a pointer to a complete object type and the right operand has integer type.

(Decrementing is equivalent to subtracting 1.)

4 If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

5 If both operands have arithmetic type, the usual arithmetic conversions are performed on them.

6 The result of the binary + operator is the sum of the operands.

7 The result of the binary - operator is the difference resulting from the subtraction of the second operand from the first.

8 For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

9 When an expression that has integer type is added to or subtracted from a pointer, the result has the type of the pointer operand. If the pointer operand points to an element of an array object, and the array is large enough, the result points to an element offset from the original element such that the difference of the subscripts of the resulting and original array elements equals the integer expression. In other words, if the expression `P` points to the i-th element of an array object, the expressions `(P)+N` (equivalently, `N+(P)`) and `(P)-N` (where `N` has the value n) point to, respectively, the $i+n$-th and $i-n$-th elements of the array object, provided they exist. Moreover, if the expression `P` points to the last element of an array object, the expression `(P)+1` points one past the last element of the array object, and if the expression `Q` points one past the last element of an array object, the expression `(Q)-1` points to the last element of the array object. If the pointer operand and the result do not point to elements of the same array object or one past the last element of the array object, the behavior is undefined. If the addition or subtraction produces an overflow, the behavior is undefined. If the result points one past the last element of the array object, it shall not be used as the operand of a unary $*$ operator that is evaluated.

10 When two pointers are subtracted, both shall point to elements of the same array object, or one past the last element of the array object; the result is the difference of the subscripts of the two array elements. The size of the result is implementation-defined, and its type (a signed integer type) is **`ptrdiff_t`** defined in the `<stddef.h>` header. If the result is not representable in an object of that type, the behavior is undefined. In other words, if the expressions `P` and `Q` point to, respectively, the i-th and j-th elements of an array object, the expression `(P)-(Q)` has the value $i-j$ provided the value fits in an object of type **`ptrdiff_t`**. Moreover, if the expression `P` points either to an element of an array object or one past the last element of an array object, and the expression `Q` points to the last element of the same array object, the expression `((Q)+1)-(P)` has the same value as `((Q)-(P))+1` and as `-((P)-((Q)+1))`, and has the value zero if the expression `P` points one past the last element of the array object, even though the expression `(Q)+1` does not point to an element of the array object.[105)]

[105)] Another way to approach pointer arithmetic is first to convert the pointer(s) to character pointer(s): In this scheme the

11 EXAMPLE Pointer arithmetic is well defined with pointers to variable length array types.

```
{
      int n = 4, m = 3;
      int a[n][m];
      int (*p)[m] = a;  // p == &a[0]
      p += 1;           // p == &a[1]
      (*p)[2] = 99;     // a[1][2] == 99
      n = p - a;        // n == 1
}
```

12 If array `a` in the preceding example were declared to be an array of known constant size, and pointer `p` were declared to be a pointer to an array of the same known constant size (pointing to `a`), the results would be the same.

Forward references: array declarators (6.7.7.3), common definitions `<stddef.h>` (7.21).

6.5.8 Bitwise shift operators

Syntax

1 *shift-expression:*
additive-expression
shift-expression **<<** *additive-expression*
shift-expression **>>** *additive-expression*

Constraints

2 Each of the operands shall have integer type.

Semantics

3 The integer promotions are performed on each of the operands. The type of the result is that of the promoted left operand. If the value of the right operand is negative or is greater than or equal to the width of the promoted left operand, the behavior is undefined.

4 The result of `E1 << E2` is `E1` left-shifted `E2` bit positions; vacated bits are filled with zeros. If `E1` has an unsigned type, the value of the result is $\texttt{E1} \times 2^{\texttt{E2}}$, wrapped around. If `E1` has a signed type and nonnegative value, and $\texttt{E1} \times 2^{\texttt{E2}}$ is representable in the result type, then that is the resulting value; otherwise, the behavior is undefined.

5 The result of `E1 >> E2` is `E1` right-shifted `E2` bit positions. If `E1` has an unsigned type or if `E1` has a signed type and a nonnegative value, the value of the result is the integral part of the quotient of $\texttt{E1}/2^{\texttt{E2}}$. If `E1` has a signed type and a negative value, the resulting value is implementation-defined.

6.5.9 Relational operators

Syntax

1 *relational-expression:*
shift-expression
relational-expression **<** *shift-expression*
relational-expression **>** *shift-expression*
relational-expression **<=** *shift-expression*
relational-expression **>=** *shift-expression*

Constraints

2 One of the following shall hold:

integer expression added to or subtracted from the converted pointer is first multiplied by the size of the object originally pointed to, and the resulting pointer is converted back to the original type. For pointer subtraction, the result of the difference between the character pointers is similarly divided by the size of the object originally pointed to.

When viewed in this way, an implementation need only provide one extra byte (which can overlap another object in the program) just after the end of the object to satisfy the "one past the last element" requirements.

— both operands have real type; or

— both operands are pointers to qualified or unqualified versions of compatible object types.

3 If either operand has decimal floating type, the other operand shall not have standard floating type.

Semantics

4 If both of the operands have arithmetic type, the usual arithmetic conversions are performed. Positive zeros compare equal to negative zeros.

5 For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

6 When two pointers are compared, the result depends on the relative locations in the address space of the objects pointed to. If two pointers to object types both point to the same object, or both point one past the last element of the same array object, they compare equal. If the objects pointed to are members of the same aggregate object, pointers to structure members declared later compare greater than pointers to members declared earlier in the structure, and pointers to array elements with larger subscript values compare greater than pointers to elements of the same array with lower subscript values. All pointers to members of the same union object compare equal. If the expression `P` points to an element of an array object and the expression `Q` points to the last element of the same array object, the pointer expression `Q+1` compares greater than `P`. In all other cases, the behavior is undefined.

7 Each of the operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) shall yield 1 if the specified relation is true and 0 if it is false.[106] The result has type **`int`**.

6.5.10 Equality operators

Syntax

1 *equality-expression:*

relational-expression
equality-expression **`==`** *relational-expression*
equality-expression **`!=`** *relational-expression*

Constraints

2 One of the following shall hold:

— both operands have arithmetic type;

— both operands are pointers to qualified or unqualified versions of compatible types;

— one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **`void`**;

— both operands have type **`nullptr_t`**;

— one operand has type **`nullptr_t`** and the other is a null pointer constant; or,

— one operand is a pointer and the other is a null pointer constant or has type **`nullptr_t`**.

3 If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

[106] The expression `a<b<c` is not interpreted as in ordinary mathematics. As the syntax indicates, it means `(a<b)<c`; in other words, "if `a` is less than `b`, compare 1 to `c`; otherwise, compare 0 to `c`".

Semantics

4 The == (equal to) and != (not equal to) operators are analogous to the relational operators except for their lower precedence.[107] Each of the operators yields 1 if the specified relation is true and 0 if it is false. The result has type **int**. For any pair of operands, exactly one of the relations is true.

5 If both of the operands have arithmetic type, the usual arithmetic conversions are performed. Positive zeros compare equal to negative zeros. Values of complex types are equal if and only if both their real parts are equal and also their imaginary parts are equal. Any two values of arithmetic types from different type domains are equal if and only if the results of their conversions to the (complex) result type determined by the usual arithmetic conversions are equal. If both operands have type **nullptr_t** or one operand has type **nullptr_t** and the other is a null pointer constant, they compare equal.

6 Otherwise, at least one operand is a pointer. If one operand is a pointer and the other is a null pointer constant or has type **nullptr_t**, they compare equal if the former is a null pointer. If one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **void**, the former is converted to the type of the latter.

7 Two pointers compare equal if and only if both are null pointers, both are pointers to the same object (including a pointer to an object and a subobject at its beginning) or function, both are pointers to one past the last element of the same array object, or one is a pointer to one past the end of one array object and the other is a pointer to the start of a different array object that happens to immediately follow the first array object in the address space.[108]

8 For the purposes of these operators, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

6.5.11 Bitwise AND operator

Syntax

1 *AND-expression:*
equality-expression
AND-expression **&** *equality-expression*

Constraints

2 Each of the operands shall have integer type.

Semantics

3 The usual arithmetic conversions are performed on the operands.

4 The result of the binary & operator is the bitwise AND of the operands (that is, each bit in the result is set if and only if each of the corresponding bits in the converted operands is set).

6.5.12 Bitwise exclusive OR operator

Syntax

1 *exclusive-OR-expression:*
AND-expression
exclusive-OR-expression **^** *AND-expression*

Constraints

2 Each of the operands shall have integer type.

[107] Because of the precedences, `a<b == c<d` is 1 whenever `a<b` and `c<d` have the same truth-value.

[108] Two objects can be adjacent in memory because they are adjacent elements of a larger array or adjacent members of a structure with no padding between them, or because the implementation chose to place them so, even though they are unrelated. If prior invalid pointer operations (such as accesses outside array bounds) produced undefined behavior, subsequent comparisons also produce undefined behavior.

Semantics

3 The usual arithmetic conversions are performed on the operands.

4 The result of the ^ operator is the bitwise exclusive OR of the operands (that is, each bit in the result is set if and only if exactly one of the corresponding bits in the converted operands is set).

6.5.13 Bitwise inclusive OR operator

Syntax

1 *inclusive-OR-expression:*
exclusive-OR-expression
inclusive-OR-expression **|** *exclusive-OR-expression*

Constraints

2 Each of the operands shall have integer type.

Semantics

3 The usual arithmetic conversions are performed on the operands.

4 The result of the | operator is the bitwise inclusive OR of the operands (that is, each bit in the result is set if and only if at least one of the corresponding bits in the converted operands is set).

6.5.14 Logical AND operator

Syntax

1 *logical-AND-expression:*
inclusive-OR-expression
logical-AND-expression **&&** *inclusive-OR-expression*

Constraints

2 Each of the operands shall have scalar type.

Semantics

3 The && operator shall yield 1 if both of its operands compare unequal to 0; otherwise, it yields 0. The result has type **int**.

4 Unlike the bitwise binary & operator, the && operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares equal to 0, the second operand is not evaluated.

6.5.15 Logical OR operator

Syntax

1 *logical-OR-expression:*
logical-AND-expression
logical-OR-expression **||** *logical-AND-expression*

Constraints

2 Each of the operands shall have scalar type.

Semantics

3 The || operator shall yield 1 if either of its operands compare unequal to 0; otherwise, it yields 0. The result has type **int**.

4 Unlike the bitwise | operator, the || operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares unequal to 0, the second operand is not evaluated.

6.5.16 Conditional operator

Syntax

1 *conditional-expression:*
logical-OR-expression
logical-OR-expression **?** *expression* **:** *conditional-expression*

Constraints

2 The first operand shall have scalar type.

3 One of the following shall hold for the second and third operands:[109)]

— both operands have arithmetic type;

— both operands have compatible structure or union type;

— both operands have void type;

— both operands are pointers to qualified or unqualified versions of compatible types;

— both operands have **nullptr_t** type;

— one operand is a pointer and the other is a null pointer constant or has type **nullptr_t**; or

— one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of **void**.

4 If either of the second or third operands has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

5 The first operand is evaluated; there is a sequence point between its evaluation and the evaluation of the second or third operand (whichever is evaluated). The second operand is evaluated only if the first compares unequal to 0; the third operand is evaluated only if the first compares equal to 0; the result is the value of the second or third operand (whichever is evaluated), converted to the type described subsequently in this subclause.[110)]

6 If the second and third operands have arithmetic type, the result type is the same as if the usual arithmetic conversions were applied to both operands. If both the operands have structure or union type, the result is the composite type. If both operands have void type, the result has void type.

7 If both the second and third operands are pointers, the result type is a pointer to a type qualified with all the type qualifiers of the types referenced by both operands; if one is a null pointer constant (other than a pointer) or has type **nullptr_t** and the other is a pointer, the result type is the pointer type; if both the second and third operands have **nullptr_t** type, the result also has that type. Furthermore, if both operands are pointers to compatible types or to differently qualified versions of compatible types, the result type is a pointer to an appropriately qualified version of the composite type; if one operand is a null pointer constant, the result has the type of the other operand; otherwise, one operand is a pointer to **void** or a qualified version of **void**, in which case the result type is a pointer to an appropriately qualified version of **void**.

8 If one operand is a pointer to a variably modified type and the other operand is a null pointer constant or has type **nullptr_t**, the behavior is undefined if the type depends on an array size expression that is not evaluated.

9 EXAMPLE The common type that results when the second and third operands are pointers is determined in two independent stages. The appropriate qualifiers, for example, do not depend on whether the two pointers have compatible types.

10 Given the declarations

[109)] If a second or third operand of type **nullptr_t** is used and the other operand is not a pointer and does not have type **nullptr_t** itself, a constraint is violated even if that other operand is a null pointer constant such as `0`.
[110)] A conditional expression does not yield an lvalue.

```
const void *c_vp;
void *vp;
const int *c_ip;
volatile int *v_ip;
int *ip;
const char *c_cp;
```

the third column in the following table is the common type that is the result of a conditional expression in which the first two columns are the second and third operands (in either order):

`c_vp`	`c_ip`	`const void *`
`v_ip`	`0`	`volatile int *`
`c_ip`	`v_ip`	`const volatile int *`
`vp`	`c_cp`	`const void *`
`ip`	`c_ip`	`const int *`
`vp`	`ip`	`void *`

6.5.17 Assignment operators

6.5.17.1 General

Syntax

1 *assignment-expression:*
conditional-expression
unary-expression assignment-operator assignment-expression

assignment-operator: one of
= *= /= %= += -= <<= >>= &= ^= |=

Constraints

2 An assignment operator shall have a modifiable lvalue as its left operand.

Semantics

3 An assignment operator stores a value in the object designated by the left operand. An assignment expression has the value of the left operand after the assignment,[111] but is not an lvalue. The type of an assignment expression is the type the left operand would have after lvalue conversion. The side effect of updating the stored value of the left operand is sequenced after the value computations of the left and right operands. The evaluations of the operands are unsequenced.

6.5.17.2 Simple assignment

Constraints

1 One of the following shall hold:[112]

— the left operand has atomic, qualified, or unqualified arithmetic type, and the right operand has arithmetic type;

— the left operand has an atomic, qualified, or unqualified version of a structure or union type compatible with the type of the right operand;

— the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) both operands are pointers to qualified or unqualified versions of compatible types, and the type pointed to by the left operand has all the qualifiers of the type pointed to by the right operand;

[111] The implementation is permitted to read the object to determine the value but is not required to, even when the object has volatile-qualified type.

[112] The asymmetric appearance of these constraints with respect to type qualifiers is due to the conversion (specified in 6.3.2.1) that changes lvalues to "the value of the expression" and thus removes any type qualifiers that were applied to the type category of the expression (for example, it removes **const** but not **volatile** from the type **int volatile * const**).

— the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) one operand is a pointer to an object type, and the other is a pointer to a qualified or unqualified version of **void**, and the type pointed to by the left operand has all the qualifiers of the type pointed to by the right operand;

— the left operand has an atomic, qualified, or unqualified version of the **nullptr_t** type and the right operand is a null pointer constant or its type is **nullptr_t**;

— the left operand is an atomic, qualified, or unqualified pointer, and the right operand is a null pointer constant or its type is **nullptr_t**; or

— the left operand has type atomic, qualified, or unqualified **bool**, and the right operand is a pointer or its type is **nullptr_t**.

Semantics

2 In *simple assignment* (=), the value of the right operand is converted to the type of the assignment expression and replaces the value stored in the object designated by the left operand.[113)]

3 If the value being stored in an object is read from another object that overlaps in any way the storage of the first object, then the two objects shall occupy exactly the same storage and shall have qualified or unqualified versions of a compatible type; otherwise, the behavior is undefined.

4 EXAMPLE 1 In the program fragment

```
int f(void);
char c;
/* ... */
if ((c = f()) == -1)
      /* ... */
```

the **int** value returned by the function could be truncated when stored in the **char**, and then converted back to **int** width prior to the comparison. In an implementation in which "plain" **char** has the same range of values as **unsigned char** (and **char** is narrower than **int**), the result of the conversion cannot be negative, so the operands of the comparison can never compare equal. Therefore, for full portability, the variable c would be declared as **int**.

5 EXAMPLE 2 In the fragment:

```
char c;
int i;
long l;

l = (c = i);
```

the value of i is converted to the type of the assignment expression c = i, that is, **char** type. The value of the expression enclosed in parentheses is then converted to the type of the outer assignment expression, that is, **long int** type.

6 EXAMPLE 3 The following fragment can be used as an example:

```
const char **cpp;
char *p;
const char c = 'A';

cpp = &p;       // constraint violation
*cpp = &c;      // valid
*p = 0;         // valid
```

The first assignment is unsafe because it would allow the following valid code to attempt to change the value of the const object c.

[113)] As described in 6.2.6.1, a store to an object with atomic type is done with **memory_order_seq_cst** semantics.

6.5.17.3 Compound assignment

Constraints

1 For the operators `+=` and `-=` only, either the left operand shall be an atomic, qualified, or unqualified pointer to a complete object type, and the right shall have integer type; or the left operand shall have atomic, qualified, or unqualified arithmetic type, and the right shall have arithmetic type.

2 For the other operators, the left operand shall have atomic, qualified, or unqualified arithmetic type, and (considering the type the left operand would have after lvalue conversion) each operand shall have arithmetic type consistent with those allowed by the corresponding binary operator.

3 If either operand has decimal floating type, the other operand shall not have standard floating type, complex type, or imaginary type.

Semantics

4 A *compound assignment* of the form `E1` *op*`= E2` is equivalent to the simple assignment expression `E1 = E1` *op* `(E2)`, except that the lvalue `E1` is evaluated only once, and with respect to an indeterminately sequenced function call, the operation of a compound assignment is a single evaluation. If `E1` has an atomic type, compound assignment is a read-modify-write operation with `memory_order_seq_cst` memory order semantics.

5 NOTE Where a pointer to an atomic object can be formed and `E1` and `E2` have integer type, this is equivalent to the following code sequence where *T1* is the type of `E1` and *T2* is the type of `E2`:

```
T1 *addr = &E1;
T2 val = (E2);
T1 old = *addr;
T1 new;
do {
      new = old op val;
} while (!atomic_compare_exchange_strong(addr, &old, new));
```

with `new` being the result of the operation.

If `E1` or `E2` has floating type, then exceptional conditions or floating-point exceptions encountered during discarded evaluations of `new` would also be discarded to satisfy the equivalence of `E1` *op*`= E2` and `E1 = E1` *op* `(E2)`. For example, if Annex F is in effect, the floating types involved have ISO/IEC 60559 binary formats, and `FLT_EVAL_METHOD` is 0, the equivalent code would be:

```
#include <fenv.h>
#pragma STDC FENV_ACCESS ON
/* ... */
      fenv_t fenv;
      T1 *addr = &E1;
      T2 val = E2;
      T1 old = *addr;
      T1 new;
      feholdexcept(&fenv);
      for (;;) {
            new = old op val;
            if (atomic_compare_exchange_strong(addr, &old, new))
                  break;
            feclearexcept(FE_ALL_EXCEPT);
      }
      feupdateenv(&fenv);
```

If `FLT_EVAL_METHOD` is not 0, then *T2* is expected to be a type with the range and precision to which `E2` is evaluated to satisfy the equivalence.

6.5.18 Comma operator

Syntax

1 *expression:*

assignment-expression
expression **,** *assignment-expression*

Semantics

2 The left operand of a comma operator is evaluated as a void expression; there is a sequence point between its evaluation and that of the right operand. Then the right operand is evaluated; the result has its type and value.[114)]

3 EXAMPLE As indicated by the syntax, the comma operator (as described in this subclause) cannot appear in contexts where a comma is used to separate items in a list (such as arguments to functions or lists of initializers). On the other hand, it can be used within a parenthesized expression or within the second expression of a conditional operator in such contexts. In the function call

```
f(a, (t=3, t+2), c)
```

the function has three arguments, the second of which has the value 5.

Forward references: initialization (6.7.11).

[114)] A comma operator does not yield an lvalue.

6.6 Constant expressions

Syntax

1 *constant-expression:*
conditional-expression

Description

2 A constant expression can be evaluated during translation rather than runtime, and accordingly may be used in any place that a constant may be.

Constraints

3 Constant expressions shall not contain assignment, increment, decrement, function-call, or comma operators, except when they are contained within a subexpression that is not evaluated.[115)]

4 Each constant expression shall evaluate to a constant that is in the range of representable values for its type.

Semantics

5 An expression that evaluates to a constant is required in several contexts. If a floating expression is evaluated in the translation environment, the arithmetic range and precision shall be at least as great as if the expression were being evaluated in the execution environment.[116)]

6 A compound literal with storage-class specifier **`constexpr`** is a *compound literal constant*, as is a postfix expression that applies the `.` member access operator to a compound literal constant of structure or union type, even recursively. A compound literal constant is a constant expression with the type and value of the unnamed object.

7 An identifier that is:

— an enumeration constant,

— a predefined constant, or

— declared with storage-class specifier **`constexpr`** and has an object type,

is a *named constant*, as is a postfix expression that applies the `.` member access operator to a named constant of structure or union type, even recursively. For enumeration and predefined constants, their value and type are defined in the respective clauses; for **`constexpr`** objects, such a named constant is a constant expression with the type and value of the declared object.

8 An *integer constant expression*[117)] shall have integer type and shall only have operands that are integer constants, named and compound literal constants of integer type, character constants, **`sizeof`** expressions whose results are integer constants, **`alignof`** expressions, and floating, named, or compound literal constants of arithmetic type that are the immediate operands of casts. Cast operators in an integer constant expression shall only convert arithmetic types to integer types, except as part of an operand to the typeof operators, **`sizeof`** operator, or **`alignof`** operator.

9 More latitude is permitted for constant expressions in initializers. Such a constant expression shall be, or evaluate to, one of the following:

— a named constant,

— a compound literal constant,

[115)] The operand of a typeof (6.7.3.6), **`sizeof`**, or **`alignof`** operator is usually not evaluated (6.5.4.4).

[116)] The use of evaluation formats as characterized by **`FLT_EVAL_METHOD`** and **`DEC_EVAL_METHOD`** also applies to evaluation in the translation environment.

[117)] An integer constant expression is required in contexts such as the size of a bit-field member of a structure, the value of an enumeration constant, and the size of a non-variable length array. Further constraints that apply to the integer constant expressions used in conditional-inclusion preprocessing directives are discussed in 6.10.2.

— an arithmetic constant expression,

— a null pointer constant,

— an address constant, or

— an address constant for a complete object type plus or minus an integer constant expression.

10 An *arithmetic constant expression* shall have arithmetic type and shall only have operands that are integer constants, floating constants, named or compound literal constants of arithmetic type, character constants, **sizeof** expressions whose results are integer constants, and **alignof** expressions. Cast operators in an arithmetic constant expression shall only convert arithmetic types to arithmetic types, except as part of an operand to the typeof operators, **sizeof** operator, or **alignof** operator.

11 An *address constant* is a null pointer,[118] a pointer to an lvalue designating an object of static storage duration, or a pointer to a function designator; it shall be created explicitly using the unary & operator or an integer constant cast to pointer type, or implicitly using an expression of array or function type.

12 The array-subscript `[]` and member-access `->` operator, the address `&` and indirection `*` unary operators, and pointer casts may be used in the creation of an address constant, but the value of an object shall not be accessed by use of these operators.[119]

13 A *structure or union constant* is a named constant or compound literal constant with structure or union type, respectively.

14 An implementation may accept other forms of constant expressions; however, it is implementation-defined whether they are an integer constant expression.[120]

15 Starting from a structure or union constant, the member-access `.` operator may be used to form a named constant or compound literal constant as described previously in this subclause.

16 If the member-access operator `.` accesses a member of a union constant, the accessed member shall be the same as the member that is initialized by the union constant's initializer.

17 The semantic rules for the evaluation of a constant expression are the same as for nonconstant expressions.[121]

Forward references: array declarators (6.7.7.3), initialization (6.7.11).

[118] A named constant or compound literal constant of integer type and value zero is a null pointer constant. A named constant or compound literal constant with a pointer type and a value null is a null pointer but not a null pointer constant; it may only be used to initialize a pointer object if its type implicitly converts to the target type.

[119] Named constants or compound literal constants with arithmetic type, including names of **constexpr** objects, are valid in offset computations such as array subscripts or in pointer casts, as long as the expressions in which they occur form integer constant expressions. In contrast, names of other objects, even if **const**-qualified and with static storage duration, are not valid.

[120] For example, in the declaration `int arr_or_vla[(int)+1.0];`, while possible to be computed by some implementations as an array with a size of one, it is implementation-defined whether this results in a variable length array declaration or a declaration of an array of known constant size of automatic storage duration. The choice depends on whether `(int)+1.0` is an extended integer constant expression.

[121] Thus, in the following initialization,

```
static int i = 2 || 1 / 0;
```

the expression is a valid integer constant expression with value one.

6.7 Declarations

6.7.1 General

Syntax

1 *declaration:*
declaration-specifiers *init-declarator-list*$_{\text{opt}}$ **;**
attribute-specifier-sequence declaration-specifiers *init-declarator-list* **;**
static_assert-declaration
attribute-declaration
declaration-specifiers:
declaration-specifier *attribute-specifier-sequence*$_{\text{opt}}$
declaration-specifier *declaration-specifiers*
declaration-specifier:
storage-class-specifier
type-specifier-qualifier
function-specifier
init-declarator-list:
init-declarator
init-declarator-list **,** *init-declarator*
init-declarator:
declarator
declarator **=** *initializer*
attribute-declaration:
attribute-specifier-sequence **;**

Constraints

2 If a declaration other than a static_assert or attribute declaration does not include an init declarator list, its declaration specifiers shall include one of the following:

— a struct or union specifier or enum specifier that includes a tag, with the declaration being of a form specified in 6.7.3.4 to declare that tag;

— an enum specifier that includes an enumerator list.

3 EXAMPLE 1 The following are invalid, because the declared tag or enumeration constants are in a nested construct, rather than a declaration specifier of the declaration being of one of the given forms:

```
struct { struct s2 { int x2a; } x2b; };
typeof (struct s3 { int x3; });
alignas (struct s4 { int x4; }) int;
typeof (struct s5 *);
typeof (enum { E6 });
struct { void (*p)(struct s7 *); };
```

4 If an identifier has no linkage, there shall be no more than one declaration of the identifier (in a declarator or type specifier) with the same scope and in the same name space, except that:

— a typedef name may be redefined to denote the same type as it currently does, provided that type is not a variably modified type;

— enumeration constants and tags may be redeclared as specified in 6.7.3.3 and 6.7.3.4, respectively.

5 All declarations in the same scope that refer to the same object or function shall specify compatible types.

Semantics

6 A declaration specifies the interpretation and properties of a set of identifiers. A *definition* of an identifier is a declaration for that identifier that for:

— an object, causes storage to be reserved for that object,

— a function, includes the function body,[122)]

— an enumeration constant, is the first (or only) declaration of the identifier, or

— a typedef name, is the first (or only) declaration of the identifier.

7 The declaration specifiers consist of a sequence of specifiers, followed by an optional attribute specifier sequence. The declaration specifiers indicate the linkage, storage duration, and part of the type of the entities that the declarators denote. The init declarator list is a comma-separated sequence of declarators, each of which may have additional type information, or an initializer, or both. The declarators contain the identifiers (if any) being declared. The optional attribute specifier sequence in a declaration appertains to each of the entities declared by the declarators of the init declarator list.

8 If an identifier for an object is declared with no linkage, the type for the object shall be complete by the end of its declarator, or by the end of its init-declarator if it has an initializer. In the case of function parameters, it is the adjusted type (see 6.7.7.4) that is required to be complete.

9 The optional attribute specifier sequence terminating a sequence of declaration specifiers appertains to the type determined by the preceding sequence of declaration specifiers. The attribute specifier sequence affects the type only for the declaration it appears in, not other declarations involving the same type.

10 Except where specified otherwise, the meaning of an attribute declaration is implementation-defined.

11 EXAMPLE 2 In the declaration for an entity, attributes appertaining to that entity may appear at the start of the declaration and after the identifier for that declaration.

```
[[deprecated]] void f [[deprecated]] (void); // valid
```

12 A declaration such that the declaration specifiers contain no type specifier or that is declared with **constexpr** is said to be *underspecified*. If such a declaration is not a definition, if it declares no or more than one ordinary identifier, if the declared identifier already has a declaration in the same scope, if the declared entity is not an object, or if anywhere within the sequence of tokens making up the declaration identifiers that are not ordinary are declared, the behavior is implementation-defined.[123)]

13 EXAMPLE 3 As declarations using **constexpr** are underspecified, the following has implementation-defined behavior because tokens within the declaration declare `s` which is not an ordinary identifier:

```
constexpr typeof(struct s *) x = 0;
```

Forward references: declarators (6.7.7), enumeration specifiers (6.7.3.3), initialization (6.7.11), storage-class specifiers (6.7.2), type inference (6.7.10), type names (6.7.8), type qualifiers (6.7.4).

6.7.2 Storage-class specifiers

Syntax

1 *storage-class-specifier:*

auto
constexpr
extern
register

[122)] Function definitions have a different syntax, described in 6.9.2.

[123)] It is recommended that implementations that accept such declarations follow the semantics of the corresponding feature in ISO/IEC 14882.

```
static
thread_local
typedef
```

Constraints

2 At most, one storage-class specifier may be given in the declaration specifiers in a declaration, except that:

— **thread_local** may appear with **static** or **extern**,

— **auto** may appear with all the others except **typedef**,[124] and

— **constexpr** may appear with **auto**, **register**, or **static**.

3 In the declaration of an object with block scope, if the declaration specifiers include **thread_local**, they shall also include either **static** or **extern**. If **thread_local** appears in any declaration of an object, it shall be present in every declaration of that object.

4 **thread_local** shall not appear in the declaration specifiers of a function declaration. **auto** shall only appear in the declaration specifiers of an identifier with file scope or along with other storage-class specifiers if the type is to be inferred from an initializer.

5 An object declared with storage-class specifier **constexpr** or any of its members, even recursively, shall not have an atomic type, or a variably modified type, or a type that is **volatile** or **restrict** qualified. An initializer of floating type shall be evaluated with the translation-time floating-point environment. The declaration shall be a definition and shall have an initializer.[125] The value of any constant expressions or of any character in a string literal of the initializer shall be exactly representable in the corresponding target type; no change of value shall be applied.[126]

6 If an object or subobject declared with storage-class specifier **constexpr** has pointer, integer, or arithmetic type, any explicit initializer value for it shall be null,[127] an integer constant expression, or an arithmetic constant expression, respectively. If the object declared has real floating type, the initializer shall have integer or real floating type. If the object declared has imaginary type, the initializer shall have imaginary type. If the initializer has decimal floating type, the object declared shall have decimal floating type and the conversion shall preserve the quantum of the initializer. If the initializer has real type and a signaling NaN value, the unqualified versions of the type of the initializer and the corresponding real type of the object declared shall be compatible.

7 EXAMPLE 1 Although in the following the expression `A.p` is not a null pointer constant, only a constant expression with pointer type and a null pointer value, the member-wise initialization of `B` with `A` is valid.

```
struct s { void *p; };
constexpr struct s A = { nullptr };
constexpr struct s B = A;
```

8 EXAMPLE 2 Pointers may be initialized to eligible constant expressions, such as a null pointer constant:

```
constexpr int *p = {}; // Default initialization with a null pointer
```

9 EXAMPLE 3

[124] See “future language directions” (6.11.5).

[125] All assignment expressions of such an initializer, if any, are constant expressions or string literals, see 6.7.11.

[126] In the context of arithmetic conversions, 6.3.1 describes the details of changes of value that occur if values of arithmetic expressions are stored in the objects that for example have a different signedness, excess precision or quantum exponent. Whenever such a change of value is necessary, the constraint is violated.

[127] The named constant or compound literal constant corresponding to an object declared with storage-class specifier **constexpr** and pointer type is a constant expression with a value null, and thus a null pointer and an address constant. Thus, such a named constant is a valid initializer for other **constexpr** declarations, provided the pointer types match accordingly. However, even if it has type **void***∗* it is not a null pointer constant.


```
void f (void) {
      constexpr float f = 1.0f;
      constexpr float g = 3.0f;
      fesetround(FE_TOWARDSZERO); // does not affect
                                  // the following initialization
                                  // of "h"
      constexpr float h = f / g;
      // ...
}
```

Semantics

10 Storage-class specifiers specify various properties of identifiers and declared features:

— storage duration (**static** in block scope, **thread_local**, **auto**, **register**),

— linkage (**extern**, **static** and **constexpr** in file scope, **typedef**),

— value (**constexpr**), and

— type (**typedef**).

11 The meanings of the various linkages and storage durations were discussed in 6.2.2 and 6.2.4, **typedef** is discussed in 6.7.9, and type inference using **auto** is discussed in 6.7.10.

12 A declaration of an identifier for an object with storage-class specifier **register** suggests that access to the object be as fast as possible. The extent to which such suggestions are effective is implementation-defined.[128)]

13 The declaration of an identifier for a function that has block scope shall have no explicit storage-class specifier other than **extern**.

14 If an aggregate or union object is declared with a storage-class specifier other than **typedef**, the properties resulting from the storage-class specifier, except with respect to linkage, also apply to the members of the object, including recursively for any aggregate or union member objects.

15 If **auto** appears with another storage-class specifier, or if it appears in a declaration at file scope, it is ignored for the purposes of determining a storage duration or linkage. In this case, it indicates only that the declared type may be inferred.

16 An object declared with a storage-class specifier **constexpr** has its value permanently fixed at translation-time; if not yet present, a **const**-qualification is implicitly added to the object's type. The declared identifier is considered a constant expression of the respective kind, see 6.6.

17 NOTE 1 An object declared in block scope with a storage-class specifier **constexpr** and without **static** has automatic storage duration, the identifier has no linkage, and each instance of the object has a unique address obtainable with & (if it is not declared with the **register** specifier), if any. Such an object in file scope has static storage duration, the corresponding identifier has internal linkage, and each translation unit that sees the same textual definition implements a separate object with a distinct address.

18 NOTE 2 The constraints for **constexpr** objects are intended to enforce checks for portability at translation time.

```
constexpr unsigned int minusOne = -1;      // constraint violation
constexpr unsigned int uint_max = -1U;     // ok
constexpr double onethird       = 1.0/3.0; // possible constraint violation
constexpr double onethirdtrunc  = (double)(1.0/3.0);  // ok
constexpr _Decimal32 small      = DEC64_TRUE_MIN * 0; // constraint violation
```

[128)] The implementation can treat any **register** declaration simply as an **auto** declaration. However, whether or not addressable storage is used, the address of any part of an object declared with storage-class specifier **register** cannot be computed, either explicitly (by use of the unary & operator as discussed in 6.5.4.2) or implicitly (by converting an array name to a pointer as discussed in 6.3.2.1). Thus, the only operator that can be applied to an array declared with storage-class specifier **register** is **sizeof** and the typeof operators.

If a truncation of excess precision changes the value in the initializer of `onethird`, a constraint is violated and a diagnostic is required. In contrast to that, the explicit conversion in the initializer for `onethirdtrunc` ensures that the definition is valid. Similarly, the initializer of `small` has a quantum exponent that is larger than the largest possible quantum exponent for **`_Decimal32`**.

19 NOTE 3 Similarly, implementation-defined behavior related to the **`char`** type of the elements of the string literal `"\xFF"` may cause constraint violations at translation time:

```
constexpr char string[]            = {   "\xFF", }; // ok
constexpr char8_t u8string[]       = { u8"\xFF", }; // ok
constexpr unsigned char ucstring[] = {   "\xFF", }; // possible constraint
                                                    // violation
```

In both the string and `ucstring` initializers, the initializer is a (brace-enclosed) string literal of type **`char`**. If the type **`char`** is capable of representing negative values and its width is 8, then the preceding code is equivalent to:

```
constexpr char string[]            = {  -1, 0, }; // ok
constexpr char8_t u8string[]       = { 255, 0, }; // ok
constexpr unsigned char ucstring[] = {  -1, 0, }; // constraint violation
```

The hexadecimal escape sequence results in a value of 255. For an initializer of type **`char`**, it is converted to a signed 8-bit integer, making a value of -1. A negative value does not fit within the range of values for **`unsigned char`**, and therefore the initialization of `ucstring` is a constraint violation under the previously stated implementation conditions. In the case where **`char`** is not capable of representing negative values, the original snippet is equivalent to the following and there is no constraint violation.

```
constexpr char string[]            = { 255, 0, }; // ok
constexpr char8_t u8string[]       = { 255, 0, }; // ok
constexpr unsigned char ucstring[] = { 255, 0, }; // ok
```

20 EXAMPLE 4 An identifier declared with the **`constexpr`** specifier may have its value used in constant expressions:

```
constexpr int K = 47;
enum {
      A = K,              // valid, constant initialization
};
constexpr int L = K;     // valid, constexpr initialization
static int b    = K + 1; // valid, static initialization
int array[K];            // not a VLA
```

21 EXAMPLE 5 This example illustrates **`constexpr`** initializations involving different type domains, decimal and non-decimal floating types, NaNs and infinities, and quanta in decimal floating types.

```
#include <float.h>
#include <complex.h>

constexpr float _Complex fc1 = 1.0; // ok
constexpr float _Complex fc2 = 0.1; // constraint violation, unless double
                                    // has the same precision as float
                                    // and is evaluated with the same
                                    // precision
constexpr float _Complex fc3 = 3*I; // ok

constexpr double d1 = (double _Complex)1.0; // constraint violation
constexpr float f1 = (long double)INFINITY; // ok
constexpr float f2 = (long double)NAN;      // ok, quiet NaNs in real floating
                                            // types are considered the same
                                            // value, regardless of payloads
constexpr double d2 = DBL_SNAN; // ok
constexpr double d3 = FLT_SNAN; // constraint violation, even if float
                                // and double have the same format
```

```
constexpr double _Complex dc1 = DBL_SNAN;            // ok
constexpr double _Complex dc2 = CMPLX(DBL_SNAN, 0.); // ok
constexpr double _Complex dc3 = CMPLX(0., DBL_SNAN); // ok

constexpr _Decimal32 d321 = 1.0;        // ok
constexpr _Decimal32 d322 = 1;          // ok
constexpr _Decimal32 d323 = INFINITY;   // ok
constexpr _Decimal32 d324 = NAN;        // ok
constexpr _Decimal64 d641 = DEC64_SNAN; // ok
constexpr _Decimal64 d642 = DEC32_SNAN; // constraint violation
constexpr float f3 = 1.DF;              // constraint violation
constexpr float f4 = DEC_INFINITY;      // constraint violation
constexpr double d4 = DEC_NAN;          // constraint violation
constexpr _Decimal32 d325 = DEC64_TRUE_MIN * 0; // constraint violation,
                                                // quantum not preserved
#ifdef __STDC_IEC_60559_COMPLEX__
constexpr double d5 = (double _Imaginary)0.0; // constraint violation
constexpr double d6 = (double _Imaginary)0.0; // constraint violation
constexpr double _Imaginary di1 = 0.0*I;      // ok
constexpr double _Imaginary di2 = 0.0;        // constraint violation
#endif
```

22 EXAMPLE 6 An object declared with the **constexpr** specifier stores the exact value of its initializer, no implicit value change is applied:

```
#include <float.h>

constexpr int A          = 42LL;          // valid, 42 always fits in an int
constexpr signed short B = ULLONG_MAX;    // constraint violation, value never
                                          // fits
constexpr float C        = 47u;           // valid, exactly representable
                                          // in float

#if FLT_MANT_DIG > 24
constexpr float D = 536900000;            // constraint violation if float is
                                          // ISO/IEC 60559 binary32
#endif

#if (FLT_MANT_DIG == DBL_MANT_DIG) \
      && (0 <= FLT_EVAL_METHOD) \
      && (FLT_EVAL_METHOD <= 1)
constexpr float E = 1.0 / 3.0;            // only valid if double expressions
                                          // and float objects have the same
                                          // precision
#endif

#if FLT_EVAL_METHOD == 0
constexpr float F = 1.0f / 3.0f;          // valid, same type and precision
#else
constexpr float F = (float)(1.0f / 3.0f); // needs cast to truncate the
                                          // excess precision
#endif
```

23 EXAMPLE 7 This recursively applies to initializers for all elements of an aggregate object declared with the **constexpr** specifier:

```
constexpr static unsigned short array[] = {
      3000,    // valid, fits in unsigned short range
      300000,  // constraint violation if short is 16-bit
      -1       // constraint violation, target type is unsigned
```

```
};

struct S {
      int x, y;
};
constexpr struct S s = {
      .x = INT_MAX,         // valid
      .y = UINT_MAX,        // constraint violation
};
```

Forward references: type definitions (6.7.9), type inference (6.7.10).

6.7.3 Type specifiers

6.7.3.1 General

Syntax

1 *type-specifier:*
void
char
short
int
long
float
double
signed
unsigned
_BitInt (*constant-expression*)
bool
_Complex
_Decimal32
_Decimal64
_Decimal128
atomic-type-specifier
struct-or-union-specifier
enum-specifier
typedef-name
typeof-specifier

Constraints

2 Except where the type is inferred (6.7.10), at least one type specifier shall be given in the declaration specifiers in each declaration, and in the specifier-qualifier list in each member declaration and type name. Each list of type specifiers shall be one of the following multisets (delimited by commas, when there is more than one multiset per item); the type specifiers may occur in any order, possibly intermixed with the other declaration specifiers.

— **void**
— **char**
— **signed char**
— **unsigned char**
— **short**, **signed short**, **short int**, or **signed short int**
— **unsigned short**, or **unsigned short int**
— **int**, **signed**, or **signed int**
— **unsigned**, or **unsigned int**

— **long**, **signed long**, **long int**, or **signed long int**
— **unsigned long**, or **unsigned long int**
— **long long**, **signed long long**, **long long int**, or **signed long long int**
— **unsigned long long**, or **unsigned long long int**
— **_BitInt(** *constant-expression* **)**, or **signed _BitInt(** *constant-expression* **)**
— **unsigned _BitInt(** *constant-expression* **)**
— **float**
— **double**
— **long double**
— **_Decimal32**
— **_Decimal64**
— **_Decimal128**
— **bool**
— **float _Complex**
— **double _Complex**
— **long double _Complex**
— atomic type specifier
— struct or union specifier
— enum specifier
— typedef name
— typeof specifier

3 The type specifier **_Complex** shall not be used if the implementation does not support complex types, and the type specifiers **_Decimal32**, **_Decimal64**, and **_Decimal128** shall not be used if the implementation does not support decimal floating types (see 6.10.10.4).

4 The parenthesized constant expression that follows the **_BitInt** keyword shall be an integer constant expression *N* that specifies the width (6.2.6.2) of the type. The value of *N* for **unsigned _BitInt** shall be greater than or equal to 1. The value of *N* for **_BitInt** shall be greater than or equal to 2. The value of *N* shall be less than or equal to the value of **BITINT_MAXWIDTH** (see 5.2.5.3.2).

Semantics

5 Specifiers for structures, unions, enumerations, atomic types, and typeof specifiers are discussed in 6.7.3.2 through 6.7.3.6. Declarations of typedef names are discussed in 6.7.9. The characteristics of the other types are discussed in 6.2.5.

6 For a declaration such that the declaration specifiers contain no type specifier a mechanism to infer the type from an initializer is discussed in 6.7.10. In such a declaration, optional elements, if any, of a sequence of declaration specifiers appertain to the inferred type (for qualifiers and attribute specifiers) or to the declared objects (for alignment specifiers).

7 Each of the comma-separated multisets designates the same type, except that for bit-fields, it is implementation-defined whether the specifier **int** designates the same type as **signed int** or the same type as **unsigned int**.

Forward references: atomic type specifiers (6.7.3.5), enumeration specifiers (6.7.3.3), structure and union specifiers (6.7.3.2), tags (6.7.3.4), type definitions (6.7.9).

6.7.3.2 Structure and union specifiers

Syntax

1 *struct-or-union-specifier:*
struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ **{** *member-declaration-list* **}**

struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*

struct-or-union:
struct
union

member-declaration-list:
member-declaration
member-declaration-list *member-declaration*

member-declaration:
attribute-specifier-sequence$_{\text{opt}}$ *specifier-qualifier-list* *member-declarator-list*$_{\text{opt}}$ **;**
static_assert-declaration

specifier-qualifier-list:
type-specifier-qualifier *attribute-specifier-sequence*$_{\text{opt}}$
type-specifier-qualifier *specifier-qualifier-list*

type-specifier-qualifier:
type-specifier
type-qualifier
alignment-specifier

member-declarator-list:
member-declarator
member-declarator-list **,** *member-declarator*

member-declarator:
declarator
declarator$_{\text{opt}}$ **:** *constant-expression*

Constraints

2 A member declaration that does not declare an anonymous structure or anonymous union shall contain a member declarator list.

3 A structure or union shall not contain a member with incomplete or function type (hence, a structure shall not contain an instance of itself, but may contain a pointer to an instance of itself), except that the last member of a structure with more than one named member may have incomplete array type; such a structure (and any union containing, possibly recursively, a member that is such a structure) shall not be a member of a structure or an element of an array.

4 The expression that specifies the width of a bit-field shall be an integer constant expression with a nonnegative value that does not exceed the width of an object of the type that would be specified were the colon and expression omitted.[129)] If the value is zero, the declaration shall have no declarator.

5 A bit-field shall have a type that is a qualified or unqualified **bool**, **signed int**, **unsigned int**, a bit-precise integer type, or other implementation-defined type. It is implementation-defined whether atomic types are permitted.

6 An attribute specifier sequence shall not appear in a struct-or-union specifier without a member

[129)] While the number of bits in a **bool** object is at least **CHAR_BIT**, the width of a **bool** is just 1 bit.

declaration list, except in a declaration of the form:

struct-or-union attribute-specifier-sequence identifier **;**

The attributes in the attribute specifier sequence, if any, are thereafter considered attributes of the **struct** or **union** whenever it is named.

Semantics

7 As discussed in 6.2.5, a structure is a type consisting of a sequence of members, whose storage is allocated in an ordered sequence, and a union is a type consisting of a sequence of members whose storage overlap.

8 Structure and union specifiers have the same form. The keywords **struct** and **union** indicate that the type being specified is, respectively, a structure type or a union type.

9 The optional attribute specifier sequence in a struct-or-union specifier appertains to the structure or union type being declared. The optional attribute specifier sequence in a member declaration appertains to each of the members declared by the member declarator list; it shall not appear if the optional member declarator list is omitted. The optional attribute specifier sequence in a specifier qualifier list appertains to the type denoted by the preceding type specifier qualifiers. The attribute specifier sequence affects the type only for the member declaration or type name it appears in, not other types or declarations involving the same type.

10 The member declaration list is a sequence of declarations for the members of the structure or union. If the member declaration list does not contain any named members, either directly or via an anonymous structure or anonymous union, the behavior is undefined.[130)]

11 A member of a structure or union may have any complete object type other than a variably modified type.[131)] In addition, a member may be declared to consist of a specified number of bits (including a sign bit, if any). Such a member is called a *bit-field*;[132)] its width is preceded by a colon.

12 A bit-field is interpreted as having a signed or unsigned integer type consisting of the specified number of bits.[133)] If the value 0 or 1 is stored into a nonzero-width bit-field of type **bool**, the value of the bit-field shall compare equal to the value stored; a **bool** bit-field has the semantics of a **bool**.

13 An implementation may allocate any addressable storage unit large enough to hold a bit-field. If enough space remains, a bit-field that immediately follows another bit-field in a structure shall be packed into adjacent bits of the same unit. If insufficient space remains, whether a bit-field that does not fit is put into the next unit or overlaps adjacent units is implementation-defined. The order of allocation of bit-fields within a unit (high-order to low-order or low-order to high-order) is implementation-defined. The alignment of the addressable storage unit is unspecified.

14 A bit-field declaration with no declarator, but only a colon and a width, indicates an unnamed bit-field.[134)] As a special case, a bit-field structure member with a width of zero indicates that no further bit-field is to be packed into the unit in which the previous bit-field, if any, was placed.

15 An unnamed member whose type specifier is a structure specifier with no tag is called an *anonymous structure*; an unnamed member whose type specifier is a union specifier with no tag is called an *anonymous union*. The members of an anonymous structure or union are members of the containing structure or union, keeping their structure or union layout. This applies recursively if the containing structure or union is also anonymous.

16 Each non-bit-field member of a structure or union object is aligned in an implementation-defined manner appropriate to its type.

[130)] For further rules affecting compatibility and completeness of structure or union types, see 6.2.7 and 6.7.3.4.

[131)] A structure or union cannot contain a member with a variably modified type because member names are not ordinary identifiers as defined in 6.2.3.

[132)] The unary & (address-of) operator cannot be applied to a bit-field object; thus, there are no pointers to or arrays of bit-field objects.

[133)] As specified in 6.7.3, if the actual type specifier used is **int** or a typedef-name defined as **int**, then it is implementation-defined whether the bit-field is signed or unsigned. This includes an **int** type specifier produced using the typeof specifiers (6.7.3.6).

[134)] An unnamed bit-field structure member is useful for padding to conform to externally imposed layouts.

17 Within a structure object, the non-bit-field members and the units in which bit-fields reside have addresses that increase in the order in which they are declared. A pointer to a structure object, suitably converted, points to its initial member (or if that member is a bit-field, then to the unit in which it resides), and vice versa. There may be unnamed padding within a structure object, but not at its beginning.

18 The size of a union is sufficient to contain the largest of its members. The value of at most one of the members can be stored in a union object at any time. A pointer to a union object, suitably converted, points to each of its members (or if a member is a bit-field, then to the unit in which it resides), and vice versa. The members of a union object overlap in such a way that pointers to them when converted to pointers to character type point to the same byte. There may be unnamed padding at the end of a union object, but not at its beginning.

19 There may be unnamed padding at the end of a structure or union.

20 As a special case, the last member of a structure with more than one named member may have an incomplete array type; this is called a *flexible array member*. In most situations, the flexible array member is ignored. In particular, the size of the structure is as if the flexible array member were omitted except that it may have more trailing padding than the omission would imply. However, when a `.` (or `->`) operator has a left operand that is (a pointer to) a structure with a flexible array member and the right operand names that member, it behaves as if that member were replaced with the longest array (with the same element type) that would not make the structure larger than the object being accessed; the offset of the array shall remain that of the flexible array member, even if this would differ from that of the replacement array. If this array would have no elements, it behaves as if it had one element but the behavior is undefined if any attempt is made to access that element or to generate a pointer one past it.

21 EXAMPLE 1 The following declarations illustrate the behavior when an attribute is written on a tag declaration:

```
struct [[deprecated]] S; // valid, [[deprecated]] appertains to struct S
void f(struct S *s);     // valid, the struct S type has the [[deprecated]]
                         // attribute
struct S {               // valid, struct S inherits the [[deprecated]] attribute
      int a;              // from the previous declaration
};
void g(struct [[deprecated]] S s); // invalid
```

22 EXAMPLE 2 The following illustrates anonymous structures and unions:

```
struct v {
      union {     // anonymous union
            struct { int i, j; };   // anonymous structure
            struct { long k, l; } w;
      };
      int m;
} v1;

v1.i = 2;   // valid
v1.k = 3;   // invalid:  inner structure is not anonymous
v1.w.k = 5; // valid
```

23 EXAMPLE 3 After the declaration:

```
struct s { int n; double d[]; };
```

the structure **struct** `s` has a flexible array member `d`. A typical way to use this is:

```
int m = /* some value */;
struct s *p = malloc(sizeof(struct s) + sizeof(double [m]));
```

and assuming that the call to **malloc** succeeds, the object pointed to by `p` behaves, for most purposes, as if `p` had been declared as:


```
struct { int n; double d[m]; } *p;
```

(there are circumstances in which this equivalence is broken; in particular, the offsets of member `d` may not be the same).

24 Following the declaration of the previous example:

```
struct s t1 = { 0 };          // valid
struct s t2 = { 1, { 4.2 }};  // invalid
t1.n = 4;                     // valid
t1.d[0] = 4.2;                // may be undefined behavior
```

The initialization of `t2` is invalid (and violates a constraint) because **`struct`** `s` is treated as if it did not contain member `d`. The assignment to `t1.d[0]` is probably undefined behavior, but it is possible that

```
sizeof(struct s) >= offsetof(struct s, d) + sizeof(double)
```

in which case the assignment would be legitimate. Nevertheless, it cannot appear in strictly conforming code.

25 After the further declaration:

```
struct ss { int n; };
```

the expressions:

```
sizeof(struct s) >= sizeof(struct ss)
sizeof(struct s) >= offsetof(struct s, d)
```

are always equal to 1.

26 If **`sizeof(double)`** is 8, then after the following code is executed:

```
struct s *s1;
struct s *s2;
s1 = malloc(sizeof(struct s) + 64);
s2 = malloc(sizeof(struct s) + 46);
```

and assuming that the calls to **`malloc`** succeed, the objects pointed to by `s1` and `s2` behave, for most purposes, as if the identifiers had been declared as:

```
struct { int n; double d[8]; } *s1;
struct { int n; double d[5]; } *s2;
```

27 Following the further successful assignments:

```
s1 = malloc(sizeof(struct s) + 10);
s2 = malloc(sizeof(struct s) +  6);
```

they then behave as if the declarations were:

```
struct { int n; double d[1]; } *s1, *s2;
```

and:

```
double *dp;
dp = &(s1->d[0]); // valid
*dp = 42;         // valid
dp = &(s2->d[0]); // valid
*dp = 42;         // undefined behavior
```

28 The assignment:

```
*s1 = *s2;
```

only copies the member `n`; if any of the array elements are within the first **`sizeof`**`(`**`struct`** `s)` bytes of the structure, they are set to an indeterminate representation, that may or may not coincide with a copy of the representation of the elements of the source array.

29 EXAMPLE 4 Because members of anonymous structures and unions are considered to be members of the containing structure or union, **`struct`** `s` in the following example has more than one named member and thus the use of a flexible array member is valid:

```
struct s {
      struct { int i; };
      int a[];
};
```

Forward references: declarators (6.7.7), tags (6.7.3.4).

6.7.3.3 Enumeration specifiers

Syntax

1 *enum-specifier:*
`enum` *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ *enum-type-specifier*$_{\text{opt}}$ **`{`** *enumerator-list* **`}`**
`enum` *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ *enum-type-specifier*$_{\text{opt}}$ **`{`** *enumerator-list* **`,`** **`}`**
`enum` *identifier* *enum-type-specifier*$_{\text{opt}}$

enumerator-list:
enumerator
enumerator-list **`,`** *enumerator*

enumerator:
enumeration-constant *attribute-specifier-sequence*$_{\text{opt}}$
enumeration-constant *attribute-specifier-sequence*$_{\text{opt}}$ **`=`** *constant-expression*

enum-type-specifier:
`:` *specifier-qualifier-list*

2 All enumerations have an *underlying type*. The underlying type can be explicitly specified using an enum type specifier and is its *fixed underlying type*. If it is not explicitly specified, the underlying type is the enumeration's compatible type, which is either **`char`** or a standard or extended signed or unsigned integer type.

Constraints

3 For an enumeration with a fixed underlying type, the integer constant expression defining the value of the enumeration constant shall be representable in that fixed underlying type. If the value of an enumeration constant without a defining constant expression for an enumeration with fixed underlying type is obtained by adding 1 to the previous enumeration constant, the value of that previous enumeration constant shall not be the maximum value of the underlying type.

4 For an enumeration without a fixed underlying type, the expression that defines the value of an enumeration constant shall be an integer constant expression. For all the integer constant expressions which make up the values of the enumeration constants, there shall be a type capable of representing all the values that is a standard or extended signed or unsigned integer type, or **`char`**.

5 If an enum type specifier is present, then the longest possible sequence of tokens that can be interpreted as a specifier qualifier list is interpreted as part of the enum type specifier. It shall name an integer type that is neither an enumeration nor a bit-precise integer type. The underlying type of the enumeration is the unqualified, non-atomic version of the type specified by the type specifiers in the specifier qualifier list.[135)]

[135)] The specifier qualifier list is not a context listed in 6.7.6 as permitted for alignment specifiers, so the presence of an alignment specifier in the list violates a constraint.

6 An enum specifier of the form

enum *identifier enum-type-specifier*

may not appear except in a declaration of the form

enum *identifier enum-type-specifier* **;**

unless it is immediately followed by an opening brace, an enumerator list (with an optional ending comma), and a closing brace.

7 If two enum specifiers that include an enum type specifier declare the same type, the underlying types shall be compatible.

8 An enumeration with a fixed underlying type shall be defined with an enum type specifier. No enum specifier for an enumeration without a fixed underlying type shall include an enum type specifier.

Semantics

9 The optional attribute specifier sequence in the **enum** specifier appertains to the enumeration; the attributes in that attribute specifier sequence are thereafter considered attributes of the enumeration whenever it is named. The optional attribute specifier sequence in the enumerator appertains to that enumerator.

10 The identifiers in an enumerator list are declared as constants of the types specified later in this subclause and may appear wherever such are permitted.[136)] An enumerator with = defines its enumeration constant as the value of the constant expression. If the first enumerator has no =, the value of its enumeration constant is zero. Each subsequent enumerator with no = defines its enumeration constant as the value of the constant expression obtained by adding 1 to the value of the previous enumeration constant. (The use of enumerators with = may produce enumeration constants with values that duplicate other values in the same enumeration.) The enumerators of an enumeration are also known as its members.

11 The type for the members of an enumeration is called the *enumeration member type*.

12 During the processing of each enumeration constant in the enumerator list, the type of the enumeration constant shall be:

- the previously declared type, if it is a redeclaration of the same enumeration constant; or,
- the enumerated type, for an enumeration with fixed underlying type; or,
- **int**, if there are no previous enumeration constants in the enumerator list and no explicit = with a defining integer constant expression; or,
- **int**, if given explicitly with = and the value of the integer constant expression is representable by an **int**; or,
- the type of the integer constant expression, if given explicitly with = and if the value of the integer constant expression is not representable by **int**; or,
- the type of the value from the previous enumeration constant with one added to it. If such an integer constant expression would overflow or wraparound the value of the previous enumeration constant from the addition of one, the type takes on either:
  - a suitably sized signed integer type, excluding the bit-precise signed integer types, capable of representing the value of the previous enumeration constant plus one; or,
  - a suitably sized unsigned integer type, excluding the bit-precise unsigned integer types, capable of representing the value of the previous enumeration constant plus one.

[136)] Thus, the identifiers of enumeration constants declared in the same scope are all required to be distinct from each other and from other identifiers declared in ordinary declarators.

A signed integer type is chosen if the previous enumeration constant being added is of signed integer type. An unsigned integer type is chosen if the previous enumeration constant is of unsigned integer type. If there is no suitably sized integer type described previously which can represent the new value, then the enumeration has no type which can represent all its values.[137)]

13 For all enumerations without a fixed underlying type, each enumerated type shall be compatible with **char** or a signed or an unsigned integer type that is not **bool** or a bit-precise integer type. The choice of type is implementation-defined,[138)] but shall be capable of representing the values of all the members of the enumeration.[139)]

14 Enumeration constants can be redefined in the same scope with the same value as part of a redeclaration of the same enumerated type.

15 The enumeration member type for an enumerated type without fixed underlying type upon completion is:

— **int** if all the values of the enumeration are representable as an **int**; or,

— the enumerated type.[140)]

16 The enumeration member type for an enumerated type with fixed underlying type is the enumerated type. The enumerated type is compatible with the underlying type of the enumeration. After possible lvalue conversion a value of the enumerated type behaves the same as the value with the underlying type, in particular with all aspects of promotion, conversion, and arithmetic. Conversion to the enumerated type has the same semantics as conversion to the underlying type.[141)]

17 EXAMPLE 1 The following fragment:

```
enum hue { chartreuse, burgundy, claret=20, winedark };
enum hue col, *cp;
col = claret;
cp = &col;
if (*cp != burgundy)
      /* ... */
```

makes `hue` the tag of an enumeration, and then declares `col` as an object that has that type and `cp` as a pointer to an object that has that type. The enumerated values are in the set $\{0, 1, 20, 21\}$.

18 EXAMPLE 2 Even if the value of an enumeration constant is generated by the implicit addition of one, an enumeration with a fixed underlying type does not exhibit typical overflow behavior:

```
#include <limits.h>

enum us : unsigned short {
      us_max = USHRT_MAX,
      us_violation, /* Constraint violation:
                       USHRT_MAX + 1 would wraparound. */
      us_violation_2 = us_max + 1, /* Maybe constraint violation:
                                      USHRT_MAX + 1 may be promoted to "int", and
                                      result is too wide for the
                                      underlying type. */
      us_wraparound_to_zero = (unsigned short)(USHRT_MAX + 1) /* Okay:
```

[137)] Therefore, a constraint has been violated.

[138)] An implementation can delay the choice of which integer type until all enumeration constants have been seen.

[139)] For further rules affecting compatibility and completeness of enumerated types see 6.2.7 and 6.7.3.4.

[140)] The integer type selected during processing of the enumerator list (before completion) of the enumeration may not be the same as the compatible implementation-defined integer type selected for the completed enumeration.

[141)] This means in particular that if the compatible type is **bool**, values of the enumerated type behave in all aspects the same as **bool**, conversion to the enumerated type behaves the same as **bool** (6.3.1.2), and the members only have values **false** and **true**. If it is a signed integer type and the constant expression of an enumeration constant overflows, a constraint for constant expressions (6.6) is violated.


```
                                conversion done in constant expression
                                before conversion to underlying type:
                                unsigned semantics okay. */
};

enum ui : unsigned int {
      ui_max = UINT_MAX,
      ui_violation, /* Constraint violation:
                       UINT_MAX + 1 would wraparound. */
      ui_no_violation = ui_max + 1, /* Okay: Arithmetic performed as typical
                                       unsigned integer arithmetic: conversion
                                       from a value that is already 0 to 0. */
      ui_wraparound_to_zero = (unsigned int)(UINT_MAX + 1) /* Okay: conversion
                               done in constant expression before conversion to
                               underlying type: unsigned semantics okay. */
};

int main () {
      // Same as return 0;
      return ui_wraparound_to_zero + us_wraparound_to_zero;
}
```

19 EXAMPLE 3 The following fragment:

```
#include <limits.h>

enum E1: short;
enum E2: short;
enum E3; /* Constraint violation: E3 forward declaration. */
enum E4 : unsigned long long;

enum E1 : short { m11, m12 };
enum E1 x = m11;

enum E2 : long { m21, m22 }; /* Constraint violation: different underlying types
    */

enum E3 {
      m31,
      m32,
      m33 = sizeof(enum E3) /* Constraint violation: E3 is not complete here. */
};
enum E3 : int; /* Constraint violation: E3 previously had no underlying type */

enum E4 : unsigned long long {
      m40 = sizeof(enum E4),
      m41 = ULLONG_MAX,
      m42 /* Constraint violation: unrepresentable value (wraparound) */
};

enum E5 y; /* Constraint violation: incomplete type */
enum E6 : long int z; /* Constraint violation: enum-type-specifier
                         with identifier in declarator */
enum E7 : long int = 0; /* Syntax violation:
                           enum-type-specifier with initializer */
```

demonstrates many of the properties of multiple declarations of enumerations with underlying types. Particularly, **enum** E3 is declared and defined without an underlying type first, therefore a redeclaration with an underlying type second is a violation. Because it not complete at that time within its enumerator list, **sizeof**(**enum** E3) is a constraint violation within the **enum** E3 definition. **enum** E4 is complete as it is being defined, therefore **sizeof**(**enum** E4) is not a constraint violation.

20 EXAMPLE 4 The following fragment:

```
enum no_underlying {
      a0
};

int main (void) {
      int a = _Generic(a0,
            int: 2,
            unsigned char: 1,
            default: 0
      );
      int b = _Generic((enum no_underlying)a0,
            int: 2,
            unsigned char: 1,
            default: 0
      );
      return a + b;
}
```

demonstrates the implementation-defined nature of the underlying type of enumerations using generic selection (6.5.2.1). The value of `a` after its initialization is `2`. The value of `b` after its initialization is implementation-defined: the enumeration is compatible with a type large enough to fit the values of its enumeration constants due to the constraints. Because the only value is `0` for `a0`, `b` may hold any of `2`, `1`, or `0`.

Now, consider a similar fragment, but using a fixed underlying type:

```
enum underlying : unsigned char {
      b0
};

int main (void) {
      int a = _Generic(b0,
            int: 2,
            unsigned char: 1,
            default: 0
      );
      int b = _Generic((enum underlying)b0,
            int: 2,
            unsigned char: 1,
            default: 0
      );
      return 0;
}
```

Here, we are guaranteed that `a` and `b` are both initialized to `1`. This makes enumerations with a fixed underlying type more portable.

21 EXAMPLE 5 Enumerations with a fixed underlying type have braces and the enumerator list specified as part of their declaration if they are not a standalone declaration:

```
void f1 (enum a : long b); /* Constraint violation */
void f2 (enum c : long { x } d);
enum e : int f3(); /* Constraint violation */

typedef enum t u; /* Constraint violation: forward declaration of t. */
typedef enum v : short W; /* Constraint violation */
typedef enum q : short { s } R;

struct s1 {
      int x;
      enum e : int : 1; /* Constraint violation */
      int y;
```



```
};

enum forward; /* Constraint violation */
extern enum forward fwd_val0; /* Constraint violation: incomplete type */
extern enum forward* fwd_ptr0; /* Constraint violation: enums cannot be
                                  used like other incomplete types */
extern int* fwd_ptr0; /* Constraint violation: incompatible
                         with incomplete type. */

enum forward1 : int;
extern enum forward1 fwd_val1;
extern int fwd_val1;
extern enum forward1* fwd_ptr1;
extern int* fwd_ptr1;

int main () {
      enum e : short;
      enum e : short f = 0; /* Constraint violation */
      enum g : short { y } h = y;
      return 0;
}
```

22 EXAMPLE 6 Enumerations with a fixed underlying type are complete when the enum type specifier for that specific enumeration is complete. The enumeration `e` in this snippet:

```
enum e : typeof ((enum e : short { A })0, (short)0);
```

enum `e` is considered complete by the first opening brace within the **typeof** in this snippet.

Forward references: generic selection (6.5.2.1), tags (6.7.3.4), declarations (6.7), declarators (6.7.7), function declarators (6.7.7.4), type names (6.7.8).

6.7.3.4 Tags

Constraints

1 Where two declarations that use the same tag declare the same type, they shall both use the same choice of **struct**, **union**, or **enum**. If two declarations of the same type have a member-declaration or enumerator-list, one shall not be nested within the other and both declarations shall fulfill all requirements of compatible types (6.2.7) with the additional requirement that corresponding members of structure or union types shall have the same (and not merely compatible) types.

2 A type specifier of the form

enum *identifier*

without an enumerator list shall only appear after the type it specifies is complete.

3 A type specifier of the form

struct-or-union attribute-specifier-sequence$_{\text{opt}}$ *identifier*

shall not contain an attribute specifier sequence.[142)]

Semantics

4 All declarations of structure, union, or enumerated types that have the same scope and use the same tag declare the same type.

5 Irrespective of whether there is a tag or what other declarations of the type are in the same translation unit, the type (except enumerated types with a fixed underlying type) is incomplete until immediately after the closing brace of the list defining the content for the first time and complete thereafter.

6 Enumerated types with fixed underlying type (6.7.3.3) are complete immediately after their first

[142)] As specified in 6.7.3.2, the type specifier may be followed by a ; or a member declaration list.

associated enum type specifier ends.

7 EXAMPLE 1 The following example shows allowed redeclarations of the same structure, union, or enumerated type in the same scope:

```
struct foo { struct { int x; }; };
struct foo { struct { int x; }; };
union bar { int x; float y; };
union bar { int x; float y; };
typedef struct q { int x; } q_t;
typedef struct q { int x; } q_t;
void foo(void)
{
      struct S { int x; };
      struct T { struct S s; };
      struct S { int x; };
      struct T { struct S s; };
}
enum X { A = 1, B = 1 + 1 };
enum X { B = 2, A = 1 };

enum Q { C = 1 };
enum Q { C = C };             // ok!
```

8 EXAMPLE 2 The following example shows invalid redeclarations of the same structure, union, or enumerated type in the same scope:

```
struct foo { int (*p)[3]; };
struct foo { int (*p)[]; };    // member has different type

union bar { int x; float y; };
union bar { int z; float y; }; // member has different name

union purr { int x; float y; };
union purr { float y; int x; }; // members have different order
// "purr" only valid if each union "purr" is in
// two different translation units

typedef struct { int x; } q_t;
typedef struct { int x; } q_t; // not the same type

struct S { int x; };

void foo(void)
{
      struct T { struct S s; };
      struct S { int x; };
      struct T { struct S s; }; // struct S not the same type
}

enum X { A = 1, B = 2 };
enum X { A = 1, B = 3 };       // different enumeration constant

enum R { C = 1 };
enum Q { C = 1 };              // conflicting enumeration constant
```

9 Two declarations of structure, union, or enumerated types which are in different scopes or use different tags declare distinct types. Each declaration of a structure, union, or enumerated type which does not include a tag declares a distinct type.

10 A type specifier of the form

struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ **{** *member-declaration-list* **}**

or

enum *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ *enum-type-specifier*$_{\text{opt}}$ **{** *enumerator-list* **}**

or

enum *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*$_{\text{opt}}$ *enum-type-specifier*$_{\text{opt}}$ **{** *enumerator-list* **,** **}**

declares a structure, union, or enumerated type. The list defines the *structure content*, *union content*, or *enumeration content*. If an identifier is provided,[143)] the type specifier also declares the identifier to be the tag of that type. The optional attribute specifier sequence appertains to the structure, union, or enumerated type being declared; the attributes in that attribute specifier sequence are thereafter considered attributes of the structure, union, or enumerated type whenever it is named.

11 A declaration of the form

struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier* **;**

or

enum *identifier* *enum-type-specifier* **;**

specifies a structure, union, or enumerated type and declares the identifier as a tag of that type.[144)] The optional attribute specifier sequence appertains to the structure or union type being declared; the attributes in that attribute specifier sequence are thereafter considered attributes of the structure or union type whenever it is named.

12 If a type specifier of the form

struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*

occurs other than as part of one of the preceding forms, and no other declaration of the identifier as a tag is visible, then it declares an incomplete structure or union type, and declares the identifier as the tag of that type.[144)]

13 If a type specifier of the form

struct-or-union *attribute-specifier-sequence*$_{\text{opt}}$ *identifier*

or

enum *identifier*

occurs other than as part of one of the preceding forms, and a declaration of the identifier as a tag is visible, then it specifies the same type as that other declaration, and does not redeclare the tag.

14 EXAMPLE 3 This mechanism allows declaration of a self-referential structure.

```
struct tnode {
      int count;
      struct tnode *left, *right;
};
```

specifies a structure that contains an integer and two pointers to objects of the same type. Once this declaration has been given, the declaration

```
struct tnode s, *sp;
```

declares `s` to be an object of the given type and `sp` to be a pointer to an object of the given type. With these declarations, the expression `sp->left` refers to the left **`struct`** `tnode` pointer of the object to which `sp` points; the expression `s.right->count` designates the `count` member of the right **`struct`** `tnode` pointed to from `s`.

15 The following alternative formulation uses the **`typedef`** mechanism:

[143)] If there is no identifier, the type can, within the translation unit, only be referred to by the declaration of which it is a part. Of course, when the declaration is of a typedef name, subsequent declarations can make use of that typedef name to declare objects having the specified structure, union, or enumerated type.

[144)] A similar construction for an **`enum`** that does not contain a fixed underlying type does not exist. Enumerations with a fixed underlying type are always complete after the enum type specifier.

```
typedef struct tnode TNODE;
struct tnode {
      int count;
      TNODE *left, *right;
};
TNODE s, *sp;
```

16 EXAMPLE 4 To illustrate the use of prior declaration of a tag to specify a pair of mutually referential structures, the declarations

```
struct s1 { struct s2 *s2p; /* ... */ }; // D1
struct s2 { struct s1 *s1p; /* ... */ }; // D2
```

specify a pair of structures that contain pointers to each other. Note, however, that if `s2` were already declared as a tag in an enclosing scope, the declaration `D1` would refer to *it*, not to the tag `s2` declared in `D2`. To eliminate this context sensitivity, the declaration

```
struct s2;
```

can be inserted ahead of `D1`. This declares a new tag `s2` in the inner scope; the declaration `D2` then completes the specification of the new type.

Forward references: declarators (6.7.7), type definitions (6.7.9).

6.7.3.5 Atomic type specifiers

Syntax

1 *atomic-type-specifier:*

_Atomic (*type-name* **)**

Constraints

2 Atomic type specifiers shall not be used if the implementation does not support atomic types (see 6.10.10.4).

3 The type name in an atomic type specifier shall not refer to an array type, a function type, an atomic type, or a qualified type.

Semantics

4 The properties associated with atomic types are meaningful only for expressions that are lvalues. If the **_Atomic** keyword is immediately followed by a left parenthesis, it is interpreted as a type specifier (with a type name), not as a type qualifier.

6.7.3.6 Typeof specifiers

Syntax

1 *typeof-specifier:*

typeof (*typeof-specifier-argument* **)**
typeof_unqual (*typeof-specifier-argument* **)**

typeof-specifier-argument:

expression
type-name

2 The **typeof** and **typeof_unqual** tokens are collectively called the *typeof operators*.

Constraints

3 The typeof operators shall not be applied to an expression that designates a bit-field member.

Semantics

4 The typeof specifier applies the typeof operators to an *expression* (6.5.1) or a type name. If the typeof operators are applied to an expression, they yield the type of their operand.[145] Otherwise, they designate the same type as the type name with any nested typeof specifier evaluated.[146] If the type of the operand is a variably modified type, the operand is evaluated; otherwise, the operand is not evaluated.

5 The result of the **`typeof_unqual`** operator is the non-atomic unqualified version of the type that would result from the **`typeof`** operator.[147] The **`typeof`** operator preserves all qualifiers.

6 EXAMPLE 1 Type of an expression.

```
typeof(1+1) main () {
      return 0;
}
```

is equivalent to this program:

```
int main () {
      return 0;
}
```

7 EXAMPLE 2 The following program:

```
const _Atomic int purr = 0;
const int meow = 1;
const char* const animals[] = {
      "aardvark",
      "bluejay",
      "catte",
};

typeof_unqual(meow) main (int argc, char* argv[]) {
      typeof_unqual(purr)          plain_purr;
      typeof(_Atomic typeof(meow)) atomic_meow;
      typeof(animals)              animals_array;
      typeof_unqual(animals)       animals2_array;
      return 0;
}
```

is equivalent to this program:

```
const _Atomic int purr = 0;
const int meow = 1;
const char* const animals[] = {
      "aardvark",
      "bluejay",
      "catte",
};

int main (int argc, char* argv[]) {
      int               plain_purr;
      const _Atomic int atomic_meow;
      const char* const animals_array[3];
      const char*       animals2_array[3];
      return 0;
```

[145] When applied to a parameter declared to have array or function type, the typeof operators yield the adjusted (pointer) type (see 6.9.2).

[146] If the typeof specifier argument is itself a typeof specifier, the operand will be evaluated before evaluating the current typeof operator. This happens recursively until a typeof specifier is no longer the operand.

[147] **`_Atomic`** `(` *type-name*`)`, with parentheses, is considered an **`_Atomic`**-qualified type.

```
}
```

8 EXAMPLE 3 The equivalence between **sizeof** and **typeof**'s deduction of the type means this program has no constraint violations:

```
int main (int argc, char* argv[]) {
      static_assert(sizeof(typeof('p')) == sizeof(int));
      static_assert(sizeof(typeof('p')) == sizeof('p'));
      static_assert(sizeof(typeof((char)'p')) == sizeof(char));
      static_assert(sizeof(typeof((char)'p')) == sizeof((char)'p'));
      static_assert(sizeof(typeof("meow")) == sizeof(char[5]));
      static_assert(sizeof(typeof("meow")) == sizeof("meow"));
      static_assert(sizeof(typeof(argc)) == sizeof(int));
      static_assert(sizeof(typeof(argc)) == sizeof(argc));
      static_assert(sizeof(typeof(argv)) == sizeof(char**));
      static_assert(sizeof(typeof(argv)) == sizeof(argv));

      static_assert(sizeof(typeof_unqual('p')) == sizeof(int));
      static_assert(sizeof(typeof_unqual('p')) == sizeof('p'));
      static_assert(sizeof(typeof_unqual((char)'p')) == sizeof(char));
      static_assert(sizeof(typeof_unqual((char)'p')) == sizeof((char)'p'));
      static_assert(sizeof(typeof_unqual("meow")) == sizeof(char[5]));
      static_assert(sizeof(typeof_unqual("meow")) == sizeof("meow"));
      static_assert(sizeof(typeof_unqual(argc)) == sizeof(int));
      static_assert(sizeof(typeof_unqual(argc)) == sizeof(argc));
      static_assert(sizeof(typeof_unqual(argv)) == sizeof(char**));
      static_assert(sizeof(typeof_unqual(argv)) == sizeof(argv));
      return 0;
}
```

9 EXAMPLE 4 The following program with nested **typeof(...)**:

```
int main (int argc, char*[]) {
      float val = 6.0f;
      return (typeof(typeof_unqual(typeof(argc))))val;
}
```

is equivalent to this program:

```
int main (int argc, char*[]) {
      float val = 6.0f;
      return (int)val;
}
```

10 EXAMPLE 5 Variable length arrays with typeof operators performs the operation at execution time rather than translation time.

```
#include <stddef.h>

size_t vla_size (int n) {
      typedef char vla_type[n + 3];
      vla_type b; // variable length array
      return sizeof(
            typeof_unqual(b)
      ); // execution-time sizeof, translation-time typeof operation
}

int main () {
      return (int)vla_size(10); // vla_size returns 13
}
```

11 EXAMPLE 6 Nested typeof operators, arrays, and pointers do not perform array to pointer decay.

```
int main () {
      typeof(typeof(const char*)[4]) y = {
            "a",
            "b",
            "c",
            "d"
      }; // 4-element array of "pointer to const char"
      return 0;
}
```

12 EXAMPLE 7 Function, pointer, and array types may be substituted with typeof operations.

```
void f(int);

typeof(f(5)) g(double x) {         // g has type "void(double)"
      printf("value %g\n", x);
}

typeof(g)* h;                      // h has type "void(*)(double)"
typeof(true ? g : nullptr) k;      // k has type "void(*)(double)"

void j(double A[5], typeof(A)* B); // j has type "void(double*, double**)"

extern typeof(double[]) D;         // D has an incomplete type
typeof(D) C = { 0.7, 99 };         // C has type "double[2]"

typeof(D) D = { 5, 8.9, 0.1, 99 }; // D is now completed to "double[4]"
typeof(D) E;                       // E has type "double[4]" from D's completed type
```

6.7.4 Type qualifiers

6.7.4.1 General

Syntax

1 *type-qualifier:*

const
restrict
volatile
_Atomic

Constraints

2 Types other than pointer types whose referenced type is an object type and (possibly multi-dimensional) array types with such pointer types as element type shall not be restrict-qualified.

3 The **_Atomic** qualifier shall not be used if the implementation does not support atomic types (see 6.10.10.4).

4 The type modified by the **_Atomic** qualifier shall not be an array type or a function type.

Semantics

5 The properties associated with qualified types are meaningful only for expressions that are lvalues.[148)]

6 If the same qualifier appears more than once in the same specifier-qualifier list or as declaration specifiers, either directly, via one or more typeof specifiers, or via one or more **typedef**s, the behavior is the same as if it appeared only once. If other qualifiers appear along with the **_Atomic** qualifier

[148)] The implementation can place a **const** object that is not **volatile** in a read-only region of storage. Moreover, the implementation does not have to allocate storage for such an object if its address is never used.

the resulting type is the so-qualified atomic type.

7 If an attempt is made to modify an object defined with a const-qualified type through use of an lvalue with non-const-qualified type, the behavior is undefined. If an attempt is made to refer to an object defined with a volatile-qualified type through use of an lvalue with non-volatile-qualified type, the behavior is undefined.[149)]

8 An object that has volatile-qualified type may be modified in ways unknown to the implementation or have other unknown side effects. Therefore, any expression referring to such an object shall be evaluated strictly according to the rules of the abstract machine, as described in 5.1.2.4. Furthermore, at every sequence point the value last stored in the object shall agree with that prescribed by the abstract machine, except as modified by the unknown factors mentioned previously.[150)] What constitutes an access to an object that has volatile-qualified type is implementation-defined.

9 An object that is accessed through a restrict-qualified pointer has a special association with that pointer. This association, defined in 6.7.4.2, requires that all accesses to that object use, directly or indirectly, the value of that pointer.[151)] The intended use of the **restrict** qualifier (like the **register** storage class) is to promote optimization, and deleting all instances of the qualifier from all preprocessing translation units composing a conforming program does not change its meaning (i.e. observable behavior), unless **_Generic** is used to distinguish whether or not a type has that qualifier.

10 If the specification of an array type includes any type qualifiers, both the array and the element type are so-qualified. If the specification of a function type includes any type qualifiers, the behavior is undefined.[152)]

11 For two qualified types to be compatible, both shall have the identically qualified version of a compatible type; the order of type qualifiers within a list of specifiers or qualifiers does not affect the specified type.

12 EXAMPLE 1 An object declared

```
extern const volatile int real_time_clock;
```

may be modifiable by hardware, but cannot be assigned to, incremented, or decremented.

13 EXAMPLE 2 The following declarations and expressions illustrate the behavior when type qualifiers modify an aggregate type:

```
const struct s { int mem; } cs = { 1 };
struct s ncs;  // the object ncs is modifiable
typedef int A[2][3];
const A a = {{4, 5, 6}, {7, 8, 9}}; // array of array of const int
int *pi;
const int *pci;

ncs = cs;      // valid
cs = ncs;      // violates modifiable lvalue constraint for =
pi = &ncs.mem; // valid
pi = &cs.mem;  // violates type constraints for =
pci = &cs.mem; // valid
pi = a[0];     // invalid:  a[0] has type "const int *"
```

14 EXAMPLE 3 The declaration

[149)] This applies to those objects that behave as if they were defined with qualified types, even if they are never actually defined as objects in the program (such as an object at a memory-mapped input/output address).

[150)] A **volatile** declaration can be used to describe an object corresponding to a memory-mapped input/output port or an object accessed by an asynchronously interrupting function. Actions on objects so declared are not allowed to be "optimized out" by an implementation or reordered except as permitted by the rules for evaluating expressions.

[151)] For example, a statement that assigns a value returned by **malloc** to a single pointer establishes this association between the allocated object and the pointer.

[152)] This can occur with **typedef** s. Note that this rule does not apply to the **_Atomic** qualifier, and that qualifiers do not have any direct effect on the array type itself, but affect conversion rules for pointer types that reference an array type.


```
_Atomic volatile int *p;
```

specifies that `p` has the type "pointer to volatile atomic int", a pointer to a volatile-qualified atomic type.

6.7.4.2 Formal definition of `restrict`

1 Let `D` be a declaration of an ordinary identifier that provides a means of designating an object `P` as a restrict-qualified pointer to type `T`.

2 If `D` appears inside a block and does not have storage class **`extern`**, let `B` denote the block. If `D` appears in the list of parameter declarations of a function definition, let `B` denote the associated block. Otherwise, let `B` denote the block of **`main`** (or the block of whatever function is called at program startup in a freestanding environment).

3 In what follows, a pointer expression `E` is said to be *based* on object `P` if (at some sequence point in the execution of `B` prior to the evaluation of `E`) modifying `P` to point to a copy of the array object into which it formerly pointed would change the value of `E`.[153] Note that "based" is defined only for expressions with pointer types.

4 During each execution of `B`, let `L` be any lvalue that has &`L` based on `P`. If `L` is used to access the value of the object `X` that it designates, and `X` is also modified (by any means), then the following requirements apply: `T` shall not be const-qualified. Every other lvalue used to access the value of `X` shall also have its address based on `P`. Every access that modifies `X` shall be considered also to modify `P`, for the purposes of this subclause. If `P` is assigned the value of a pointer expression `E` that is based on another restricted pointer object `P2`, associated with block `B2`, then either the execution of `B2` shall begin before the execution of `B`, or the execution of `B2` shall end prior to the assignment. If these requirements are not met, then the behavior is undefined.

5 Here an execution of `B` means that portion of the execution of the program that would correspond to the lifetime of an object with scalar type and automatic storage duration associated with `B`.

6 A translator is free to ignore any or all aliasing implications of uses of **`restrict`**.

7 EXAMPLE 1 The file scope declarations

```
int * restrict a;
int * restrict b;
extern int c[];
```

assert that if an object is accessed using one of `a`, `b`, or `c`, and that object is modified anywhere in the program, then it is never accessed using either of the other two.

8 EXAMPLE 2 The function parameter declarations in the following example

```
void f(int n, int * restrict p, int * restrict q)
{
      while (n--  >  0)
            *p++ = *q++;
}
```

assert that, during each execution of the function, if an object is accessed through one of the pointer parameters, then it is not also accessed through the other. The translator can make this no-aliasing inference based on the parameter declarations alone, without analyzing the function body.

9 The benefit of the **`restrict`** qualifiers is that they enable a translator to make an effective dependence analysis of function `f` without examining any of the calls of `f` in the program. The cost is that the programmer has to examine all those calls to ensure that none give undefined behavior. For example, the second call of `f` in `g` has undefined behavior because each of `d[1]` through `d[49]` is accessed through both `p` and `q`.

```
void g(void)
{
```

[153] In other words, `E` depends on the value of `P` itself rather than on the value of an object referenced indirectly through `P`. For example, if identifier `p` has type `(int **restrict)`, then the pointer expressions `p` and `p+1` are based on the restricted pointer object designated by `p`, but the pointer expressions `*p` and `p[1]` are not.

```
        extern int d[100];
        f(50, d + 50, d); // valid
        f(50, d +  1, d); // undefined behavior
}
```

10 EXAMPLE 3 The function parameter declarations

```
void h(int n, int * restrict p, int * restrict q, int * restrict r)
{
        int i;
        for (i = 0; i  <  n; i++)
                p[i] = q[i] + r[i];
}
```

illustrate how an unmodified object can be aliased through two restricted pointers. If `a` and `b` are disjoint arrays, a call of the form `h(100, a, b, b)` has defined behavior, because array `b` is not modified within function `h`.

11 EXAMPLE 4 The rule limiting assignments between restricted pointers does not distinguish between a function call and an equivalent nested block. With one exception, only "outer-to-inner" assignments between restricted pointers declared in nested blocks have defined behavior.

```
{
        int * restrict p1;
        int * restrict q1;
        p1 = q1; // undefined behavior
        {
                int * restrict p2 = p1; // valid
                int * restrict q2 = q1; // valid
                p1 = q2;                // undefined behavior
                p2 = q2;                // undefined behavior
        }
}
```

12 The one exception allows the value of a restricted pointer to be carried out of the block in which it (or, more precisely, the ordinary identifier used to designate it) is declared when that block finishes execution. For example, this permits `new_vector` to return a `vector`.

```
typedef struct { int n; float * restrict v; } vector;
vector new_vector(int n)
{
        vector t;
        t.n = n;
        t.v = malloc(n * sizeof(float));
        return t;
}
```

13 EXAMPLE 5 Suppose that a programmer knows that references of the form `p[i]` and `q[j]` are never aliases in the body of a function:

```
void f(int n, int *p, int *q) { /* ... */ }
```

There are several ways that this information could be conveyed to a translator using the **`restrict`** qualifier. Example 2 shows the most effective way, qualifying all pointer parameters, and can be used provided that neither `p` nor `q` becomes based on the other in the function body. A potentially effective alternative is:

```
void f(int n, int * restrict p, int * const q) { /* ... */ }
```

Again, it is possible for a translator to make the no-aliasing inference based on the parameter declarations alone, though now subtler reasoning is used: that the const-qualification of `q` precludes it becoming based on `p`. There is also a requirement that `q` is not modified, so this alternative cannot be used for the function in Example 2, as written.

14 EXAMPLE 6 Another potentially effective alternative is:

```
void f(int n, int *p, int const * restrict q) { /* ... */ }
```

Again, it is possible for a translator to make the no-aliasing inference based on the parameter declarations alone, though now even subtler reasoning is used: that this combination of **restrict** and **const** means that objects referenced using q cannot be modified, and so no modified object can be referenced using both p and q.

15 EXAMPLE 7 The least effective alternative is:

```
void f(int n, int * restrict p, int *q) { /* ... */ }
```

Here the translator can make the no-aliasing inference only by analyzing the body of the function and proving that q cannot become based on p. Some translator designs may choose to exclude this analysis, given availability of the more effective alternatives described previously. Such a translator is required to assume that aliases are present because assuming that aliases are not present may result in an incorrect translation. Also, a translator that attempts the analysis may not succeed in all cases and consequently need to conservatively assume that aliases are present.

6.7.5 Function specifiers

Syntax

1 *function-specifier:*

inline
_Noreturn

Constraints

2 Function specifiers shall be used only in the declaration of an identifier for a function.

3 An inline definition of a function with external linkage shall not contain, anywhere in the tokens making up the function definition, a definition of a modifiable object with static or thread storage duration, and shall not contain, anywhere in the tokens making up the function definition, a reference to an identifier with internal linkage.

4 In a hosted environment, no function specifier(s) shall appear in a declaration of **main**.

Semantics

5 A function specifier may appear more than once; the behavior is the same as if it appeared only once.

6 A function declared with an **inline** function specifier is an *inline function*. Making a function an inline function suggests that calls to the function be as fast as possible.[154] The extent to which such suggestions are effective is implementation-defined.[155]

7 Any function with internal linkage can be an inline function. For a function with external linkage, the following restrictions apply: If a function is declared with an **inline** function specifier, then it shall also be defined in the same translation unit. If all the file scope declarations for a function in a translation unit include the **inline** function specifier without **extern**, then the definition in that translation unit is an *inline definition*. An inline definition does not provide an external definition for the function and does not forbid an external definition in another translation unit. Inline definitions provide an alternative to external definitions, which a translator may use to implement any call to the function in the same translation unit. It is unspecified whether a call to the function uses the

[154] By using, for example, an alternative to the usual function call mechanism, such as "inline substitution". Inline substitution is not textual substitution, nor does it create a new function. Therefore, for example, the expansion of a macro used within the body of the function uses the definition it had at the point the function body appears, and not where the function is called; and identifiers refer to the declarations in scope where the body occurs. Likewise, the function has a single address, regardless of the number of inline definitions that occur in addition to the external definition.

[155] For example, an implementation may never perform inline substitution, or may only perform inline substitutions to calls in the scope of an **inline** declaration.

inline definition or the external definition.[156)]

8 A function declared with a **_Noreturn** function specifier shall not return to its caller. The attribute **[[noreturn]]** provides similar semantics. The **_Noreturn** function specifier is an obsolescent feature (6.7.13.7).

Recommended practice

9 The implementation should produce a diagnostic message for a function declared with a **_Noreturn** function specifier that appears to be capable of returning to its caller.

10 EXAMPLE 1 The declaration of an inline function with external linkage can result in either an external definition, or a definition available for use only within the translation unit. A file scope declaration with **extern** creates an external definition. The following example shows an entire translation unit.

```
inline double fahr(double t)
{
      return (9.0 * t) / 5.0 + 32.0;
}

inline double cels(double t)
{
      return (5.0 * (t - 32.0)) / 9.0;
}

extern double fahr(double);   // creates an external definition

double convert(int is_fahr, double temp)
{
      /* A translator may perform inline substitutions */
      return is_fahr ? cels(temp): fahr(temp);
}
```

11 Note that the definition of `fahr` is an external definition because `fahr` is also declared with **extern**, but the definition of `cels` is an inline definition. Because `cels` has external linkage and is referenced, an external definition has to appear in another translation unit (see 6.9); the inline definition and the external definition are distinct and either can be used for the call.

12 EXAMPLE 2 The following inline definitions are invalid:

```
static int a;
typeof (a) inline f() { return 0; }
typeof ((int) { 0 }) inline g() { return 0; }
```

Forward references: function definitions (6.9.2).

6.7.6 Alignment specifier

Syntax

1 *alignment-specifier:*

alignas (*type-name* **)**
alignas (*constant-expression* **)**

Constraints

2 An alignment specifier shall appear only in the declaration specifiers of a declaration, or in the *specifier-qualifier* list of a member declaration, or in the type name of a compound literal. An alignment specifier shall not be used in conjunction with either of the storage-class specifiers **typedef** or **register**, nor in a declaration of a function or bit-field.

[156)] Since an inline definition is distinct from the corresponding external definition and from any other corresponding inline definitions in other translation units, all corresponding objects with static storage duration are also distinct in each of the definitions.

3 The constant expression shall be an integer constant expression. It shall evaluate to a valid fundamental alignment, or to a valid extended alignment supported by the implementation for an object of the storage duration (if any) being declared, or to zero.

4 An object shall not be declared with an over-aligned type with an extended alignment requirement not supported by the implementation for an object of that storage duration.

5 The combined effect of all alignment specifiers in a declaration shall not specify an alignment that is less strict than the alignment that would otherwise be required for the type of the object or member being declared.

Semantics

6 The first form is equivalent to **alignas**(**alignof**(*type-name*)).

7 The alignment requirement of the declared object or member is taken to be the specified alignment. An alignment specification of zero has no effect.[157] When multiple alignment specifiers occur in a declaration, the effective alignment requirement is the strictest specified alignment.

8 If the definition of an object has an alignment specifier, any other declaration of that object shall either specify equivalent alignment or have no alignment specifier. If the definition of an object does not have an alignment specifier, any other declaration of that object shall also have no alignment specifier. If declarations of an object in different translation units have different alignment specifiers, the behavior is undefined.

6.7.7 Declarators

6.7.7.1 General

Syntax

1 *declarator:*
pointer$_{\text{opt}}$ *direct-declarator*

direct-declarator:
identifier *attribute-specifier-sequence*$_{\text{opt}}$
(*declarator* **)**
array-declarator *attribute-specifier-sequence*$_{\text{opt}}$
function-declarator *attribute-specifier-sequence*$_{\text{opt}}$

array-declarator:
direct-declarator **[** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression*$_{\text{opt}}$ **]**
direct-declarator **[** **static** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression* **]**
direct-declarator **[** *type-qualifier-list* **static** *assignment-expression* **]**
direct-declarator **[** *type-qualifier-list*$_{\text{opt}}$ ***** **]**

function-declarator:
direct-declarator **(** *parameter-type-list*$_{opt}$ **)**

pointer:
***** *attribute-specifier-sequence*$_{\text{opt}}$ *type-qualifier-list*$_{\text{opt}}$
***** *attribute-specifier-sequence*$_{\text{opt}}$ *type-qualifier-list*$_{\text{opt}}$ *pointer*
type-qualifier-list:
type-qualifier
type-qualifier-list *type-qualifier*
parameter-type-list:
parameter-list
parameter-list **, ...**
...
parameter-list:
parameter-declaration

[157] An alignment specification of zero also does not affect other alignment specifications in the same declaration.

parameter-list **,** *parameter-declaration*

parameter-declaration:

attribute-specifier-sequence$_{\text{opt}}$ *declaration-specifiers* *declarator*

attribute-specifier-sequence$_{\text{opt}}$ *declaration-specifiers* *abstract-declarator*$_{\text{opt}}$

Semantics

2 Each declarator declares an identifier for a single object, function, or type, within a declaration. The preceding specifiers indicate the type, storage class, or other properties of the identifier or identifiers being declared. Each declarator specifies one declaration and names it and/or modifies the type of the specifiers with operators such as `*` (pointer to) and `()` (function returning).

3 A *full declarator* is a declarator that is not part of another declarator. If, in the nested sequence of declarators in a full declarator, there is a declarator specifying a variable length array type, the type specified by the full declarator is said to be *variably modified*. Furthermore, any type derived by declarator type derivation from a variably modified type is itself variably modified.

4 In the following subclauses, consider a declaration

```
T D1
```

where `T` contains the declaration specifiers that specify a type *T* (such as **`int`**) and `D1` is a declarator that contains an identifier *ident*. The type specified for the identifier *ident* in the various forms of declarator is described inductively using this notation.

5 If, in the declaration "`T D1`", `D1` has the form

identifier *attribute-specifier-sequence*$_{\text{opt}}$

then the type specified for *ident* is *T* and the optional attribute specifier sequence appertains to the entity that is declared.

6 If, in the declaration "`T D1`", `D1` has the form

```
( D )
```

then *ident* has the type specified by the declaration "`T D`". Thus, a declarator in parentheses is identical to the unparenthesized declarator, but the binding of complicated declarators may be altered by parentheses.

Implementation limits

7 As discussed in 5.2.5.2, an implementation may limit the number of pointer, array, and function declarators that modify an arithmetic, structure, union, or **`void`** type, either directly or via one or more **`typedef`**s.

Forward references: array declarators (6.7.7.3), type definitions (6.7.9).

6.7.7.2 Pointer declarators

Semantics

1 If, in the declaration "`T D1`", `D1` has the form

`*` *attribute-specifier-sequence*$_{\text{opt}}$ *type-qualifier-list*$_{\text{opt}}$ `D`

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the type specified for *ident* is "*derived-declarator-type-list type-qualifier-list* pointer to *T*". For each type qualifier in the list, *ident* is a so-qualified pointer. The optional attribute specifier sequence appertains to the pointer and not the object pointed to.

2 For two pointer types to be compatible, both shall be identically qualified and both shall be pointers to compatible types.

3 EXAMPLE The following pair of declarations demonstrates the difference between a "variable pointer to a constant value" and a "constant pointer to a variable value".

```
const int *ptr_to_constant;
int *const constant_ptr;
```


The contents of any object pointed to by `ptr_to_constant` cannot be modified through that pointer, but `ptr_to_constant` itself can be changed to point to another object. Similarly, the contents of the **`int`** pointed to by `constant_ptr` can be modified, but `constant_ptr` itself always points to the same location.

4 The declaration of the constant pointer `constant_ptr` can be clarified by including a definition for the type "pointer to **`int`**".

```
typedef int *int_ptr;
const int_ptr constant_ptr;
```

declares `constant_ptr` as an object that has type "const-qualified pointer to **`int`**".

6.7.7.3 Array declarators

Constraints

1 In addition to optional type qualifiers and the keyword **`static`**, the [and] may delimit an expression or *. If they delimit an expression (which specifies the size of an array), the expression shall have an integer type. If the expression is a constant expression, it shall have a value greater than zero. The element type shall not be an incomplete or function type. The optional type qualifiers and the keyword **`static`** shall appear only in a declaration of a function parameter with an array type, and then only in the outermost array type derivation.

2 If an identifier is declared as having a variably modified type, it shall be an ordinary identifier (as defined in 6.2.3), have no linkage, and have either block scope or function prototype scope. If an identifier is declared to be an object with static or thread storage duration, it shall not have a variable length array type.

Semantics

3 If, in the declaration "`T D1`", `D1` has one of the forms:

`D` **`[`** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression*$_{\text{opt}}$ **`]`** *attribute-specifier-sequence*$_{\text{opt}}$
`D` **`[`** **`static`** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression* **`]`** *attribute-specifier-sequence*$_{\text{opt}}$
`D` **`[`** *type-qualifier-list* **`static`** *assignment-expression* **`]`** *attribute-specifier-sequence*$_{\text{opt}}$
`D` **`[`** *type-qualifier-list*$_{\text{opt}}$ **`*`** **`]`** *attribute-specifier-sequence*$_{\text{opt}}$

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the type specified for *ident* is "*derived-declarator-type-list* array of *T*".[158)][159)] The optional attribute specifier sequence appertains to the array. (See 6.7.7.4 for the meaning of the optional type qualifiers and the keyword **`static`**.)

4 If the size is not present, the array type is an incomplete type. If the size is * instead of being an expression, the array type is a *variable length array* type of unspecified size, which can only be used as part of the nested sequence of declarators or abstract declarators for a parameter declaration, not including anything inside an array size expression in one of those declarators;[160)] such arrays are nonetheless complete types. If the size is an integer constant expression and the element type has a known constant size, the array type is not a variable length array type; otherwise, the array type is a *variable length array* type. (Variable length arrays with automatic storage duration are a conditional feature that implementations may support; see 6.10.10.4.)

5 If the size is an expression that is not an integer constant expression: if it occurs in a declaration at function prototype scope, it is treated as if it were replaced by *; otherwise, each time it is evaluated it shall have a value greater than zero. The size of each instance of a variable length array type does not change during its lifetime. Where a size expression is part of the operand of a typeof or **`sizeof`** operator and changing the value of the size expression would not affect the result of the operator, it is unspecified whether or not the size expression is evaluated. Where a size expression is part of the operand of an **`alignof`** operator, that expression is not evaluated.

[158)] When several "array of" specifications are adjacent, a multidimensional array is declared.
[159)] The array is considered identically qualified to `T` according to 6.2.5.
[160)] They can be used only in function declarations that are not definitions (see 6.7.7.4 and 6.9.2).

6 For two array types to be compatible, both shall have compatible element types, and if both size specifiers are present, and are integer constant expressions, then both size specifiers shall have the same constant value. If the two array types are used in a context which requires them to be compatible, it is undefined behavior if the two size specifiers evaluate to unequal values.

7 EXAMPLE 1

```
float fa[11], *afp[17];
```

declares an array of **float** numbers and an array of pointers to **float** numbers.

8 EXAMPLE 2 Note the distinction between the declarations

```
extern int *x;
extern int y[];
```

The first declares x to be a pointer to **int**; the second declares y to be an array of **int** of unknown size (an incomplete type), the storage for which is defined elsewhere.

9 EXAMPLE 3 The following declarations demonstrate the compatibility rules for variably modified types.

```
extern int n;
extern int m;

void fcompat(void)
{
      int a[n][6][m];
      int (*p)[4][n+1];
      int c[n][n][6][m];
      int (*r)[n][n][n+1];
      p = a;      // invalid:  not compatible because 4 != 6
      r = c;      // compatible, but defined behavior only if
                  // n == 6 and m == n+1
}
```

10 EXAMPLE 4 All valid declarations of variably modified (VM) types are either at block scope or function prototype scope. Array objects declared with the **thread_local**, **static**, or **extern** storage-class specifier cannot have a variable length array (VLA) type. However, an object declared with the **static** storage-class specifier can have a VM type (that is, a pointer to a VLA type). Finally, only ordinary identifiers can be declared with a VM type and identifiers with VM type cannot, therefore, be members of structures or unions.

```
extern int n;
int A[n];                         // invalid:  file scope VLA
extern int (*p2)[n];              // invalid:  file scope VM
int B[100];                       // valid:  file scope but not VM

void fvla(int m, int C[m][m]);    // valid:  VLA with prototype scope

void fvla(int m, int C[m][m])     // valid:  adjusted to auto pointer to VLA
{
      typedef int VLA[m][m];      // valid:  block scope typedef VLA

      struct tag {
            int (*y)[n];          // invalid:  y not ordinary identifier
            int z[n];             // invalid:  z not ordinary identifier
      };
      int D[m];                   // valid:  auto VLA
      static int E[m];            // invalid:  static block scope VLA
      extern int F[m];            // invalid:  F has linkage and is VLA
      int (*s)[m];                // valid:  auto pointer to VLA
      extern int (*r)[m];         // invalid:  r has linkage and points to VLA
      static int (*q)[m] = &B;    // valid:  q is a static block pointer to VLA
}
```


11 EXAMPLE 5 The following is invalid, because the use of `[*]` is inside an array size expression rather than directly part of the nested sequence of abstract declarators for a parameter declaration:

```
void f(int (*)[sizeof(int (*)[*])]);
```

Forward references: function declarators (6.7.7.4), function definitions (6.9.2), initialization (6.7.11).

6.7.7.4 Function declarators

Constraints

1 A function declarator shall not specify a return type that is a function type or an array type.

2 The only storage-class specifier that shall occur in a parameter declaration is **`register`**.

3 After adjustment, the parameters in a parameter type list in a function declarator that is part of a definition of that function shall not have incomplete type.

Semantics

4 If, in the declaration "`T D1`", `D1` has the form

`D (` *parameter-type-list*$_{\text{opt}}$ `)` *attribute-specifier-sequence*$_{\text{opt}}$

and the type specified for *ident* in the declaration "`T D`" is "*derived-declarator-type-list T*", then the type specified for *ident* is "*derived-declarator-type-list* function returning the unqualified, non-atomic version of *T*". The optional attribute specifier sequence appertains to the function type.

5 A parameter type list specifies the types of, and may declare identifiers for, the parameters of the function.

6 A declaration of a parameter as "array of *type*" shall be adjusted to "qualified pointer to *type*", where the type qualifiers (if any) are those specified within the `[` and `]` of the array type derivation. If the keyword **`static`** also appears within the `[` and `]` of the array type derivation, then for each call to the function, the value of the corresponding actual argument shall provide access to the first element of an array with at least as many elements as specified by the size expression.

7 A declaration of a parameter as "function returning *type*" shall be adjusted to "pointer to function returning *type*", as in 6.3.2.1.

8 If the list terminates with an ellipsis (`...`), no information about the number or types of the parameters after the comma is supplied.[161)]

9 The special case of an unnamed parameter of type **`void`** as the only item in the list specifies that the function has no parameters.

10 If, in a parameter declaration, an identifier can be treated either as a typedef name or as a parameter name, it shall be taken as a typedef name.

11 If the function declarator is not part of a definition of that function, parameters may have incomplete type and may use the `[*]` notation in their sequences of declarator specifiers to specify variable length array types.

12 The storage-class specifier in the declaration specifiers for a parameter declaration, if present, is ignored unless the declared parameter is one of the members of the parameter type list for a function definition. The optional attribute specifier sequence in a parameter declaration appertains to the parameter.

13 For a function declarator without a parameter type list: the effect is as if it were declared with a parameter type list consisting of the keyword **`void`**. A function declarator provides a prototype for the function.

14 For two function types to be compatible, both shall specify compatible return types. Moreover, the parameter type lists shall agree in the number of parameters and in use of the final ellipsis; corresponding parameters shall have compatible types. In the determination of type compatibility and of a composite type, each parameter declared with function or array type is taken as having the

[161)] The macros defined in the `<stdarg.h>` header (7.16) can be used to access arguments that correspond to the ellipsis.

adjusted type and each parameter declared with qualified type is taken as having the unqualified version of its declared type.

15 EXAMPLE 1 The declaration

```
int f(void), *fip(), (*pfi)();
```

declares a function `f` with no parameters returning an **`int`**, a function `fip` with no parameters returning a pointer to an **`int`**, and a pointer `pfi` to a function with no parameters returning an **`int`**. It is especially useful to compare the last two. The binding of `*fip()` is `*(fip())`, so that the declaration suggests, and the same construction in an expression requires, the calling of a function `fip`, and then using indirection through the pointer result to yield an **`int`**. In the declarator `(*pfi)()`, the extra parentheses are necessary to indicate that indirection through a pointer to a function yields a function designator, which is then used to call the function; it returns an **`int`**.

16 If the declaration occurs outside of any function, the identifiers have file scope and external linkage. If the declaration occurs inside a function, the identifiers of the functions `f` and `fip` have block scope and either internal or external linkage (depending on what file scope declarations for these identifiers are visible), and the identifier of the pointer `pfi` has block scope and no linkage.

17 EXAMPLE 2 The declaration

```
int (*apfi[3])(int *x, int *y);
```

declares an array `apfi` of three pointers to functions returning **`int`**. Each of these functions has two parameters that are pointers to **`int`**. The identifiers `x` and `y` are declared for descriptive purposes only and go out of scope at the end of the declaration of `apfi`.

18 EXAMPLE 3 The declaration

```
int (*fpfi(int (*)(long), int))(int, ...);
```

declares a function `fpfi` that returns a pointer to a function returning an **`int`**. The function `fpfi` has two parameters: a pointer to a function returning an **`int`** (with one parameter of type **`long int`**), and an **`int`**. The pointer returned by `fpfi` points to a function that has one **`int`** parameter and accepts zero or more additional arguments of any type.

19 EXAMPLE 4 The following prototype has a variably modified parameter.

```
void addscalar(int n, int m,
      double a[n][n*m+300], double x);

int main(void)
{
      double b[4][308];
      addscalar(4, 2, b, 2.17);
      return 0;
}

void addscalar(int n, int m,
      double a[n][n*m+300], double x)
{
      for (int i = 0; i  <  n; i++)
            for (int j = 0, k = n*m+300; j  <  k; j++)
                  // a is a pointer to a VLA with n*m+300 elements
                  a[i][j] += x;
}
```

20 EXAMPLE 5 The following are all compatible function prototype declarators.

```
double maximum(int n, int m, double a[n][m]);
double maximum(int n, int m, double a[*][*]);
double maximum(int n, int m, double a[ ][*]);
double maximum(int n, int m, double a[ ][m]);
```


as are:

```
void f(double (* restrict a)[5]);
void f(double a[restrict][5]);
void f(double a[restrict 3][5]);
void f(double a[restrict static 3][5]);
```

The last declaration also specifies that the argument corresponding to `a` in any call to `f` can be expected to be a non-null pointer to the first of at least three arrays of 5 doubles, which the others do not.

Forward references: function definitions (6.9.2), type names (6.7.8).

6.7.8 Type names

Syntax

1 *type-name:*
specifier-qualifier-list *abstract-declarator*$_{\text{opt}}$
abstract-declarator:
pointer
pointer$_{\text{opt}}$ *direct-abstract-declarator*
direct-abstract-declarator:
(*abstract-declarator* **)**
array-abstract-declarator *attribute-specifier-sequence*$_{\text{opt}}$
function-abstract-declarator *attribute-specifier-sequence*$_{\text{opt}}$
array-abstract-declarator:
direct-abstract-declarator$_{\text{opt}}$ **[** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression*$_{\text{opt}}$ **]**
direct-abstract-declarator$_{\text{opt}}$ **[static** *type-qualifier-list*$_{\text{opt}}$ *assignment-expression* **]**
direct-abstract-declarator$_{\text{opt}}$ **[** *type-qualifier-list* **static** *assignment-expression* **]**
direct-abstract-declarator$_{\text{opt}}$ **[*]**

function-abstract-declarator:
direct-abstract-declarator$_{\text{opt}}$ **(** *parameter-type-list*$_{\text{opt}}$ **)**

Semantics

2 In several contexts, it is necessary to specify a type. This is accomplished using a *type name*, which is syntactically a declaration for a function or an object of that type that omits the identifier.[162)] The optional attribute specifier sequence in a direct abstract declarator appertains to the preceding array or function type. The attribute specifier sequence affects the type only for the declaration it appears in, not other declarations involving the same type.

3 EXAMPLE The constructions

```
(a)       int
(b)       int *
(c)       int *[3]
(d)       int (*)[3]
(e)       int (*)[*]
(f)       int *()
(g)       int (*)(void)
(h)       int (*const [])(unsigned int, ...)
```

name respectively the types

(a) **int**,

(b) pointer to **int**,

(c) array of three pointers to **int**,

[162)] As indicated by the syntax, empty parentheses in a type name are interpreted as "function with no parameters", rather than redundant parentheses around the omitted identifier.

(d) pointer to an array of three **int** s,

(e) pointer to a variable length array of an unspecified number of **int** s,

(f) function with no parameters returning a pointer to **int**,

(g) pointer to function with no parameters returning an **int**, and

(h) array of an unspecified number of constant pointers to functions, each with one parameter that has type **unsigned int** and an unspecified number of other parameters, returning an **int**.

6.7.9 Type definitions

Syntax

1 *typedef-name:*
identifier

Constraints

2 If a typedef name specifies a variably modified type then it shall have block scope.

Semantics

3 In a declaration whose storage-class specifier is **typedef**, each declarator defines an identifier to be a typedef name that denotes the type specified for the identifier in the way described in 6.7.7. Any array size expressions associated with variable length array declarators and typeof operators are evaluated each time the declaration of the typedef name is reached in the order of execution. A **typedef** declaration does not introduce a new type, only a synonym for the type so specified. That is, in the following declarations:

```
typedef T type_ident;
type_ident D;
```

`type_ident` is defined as a typedef name with the type specified by the declaration specifiers in `T` (known as *T*), and the identifier in `D` has the type "*derived-declarator-type-list T*" where the *derived-declarator-type-list* is specified by the declarators of `D`. A typedef name shares the same name space as other identifiers declared in ordinary declarators. If the identifier is redeclared in an enclosed block, the type of the inner declaration shall not be inferred (6.7.10).

4 EXAMPLE 1 After

```
typedef int MILES, KLICKSP();
typedef struct { double hi, lo; } range;
```

the constructions

```
MILES distance;
extern KLICKSP *metricp;
range x;
range z, *zp;
```

are all valid declarations. The type of `distance` is **int**, that of `metricp` is "pointer to function with no parameters returning **int**", and that of `x` and `z` is the specified structure; `zp` is a pointer to such a structure. The object `distance` has a type compatible with any other **int** object.

5 EXAMPLE 2 After the declarations

```
typedef struct s1 { int x; } t1, *tp1;
typedef struct s2 { int x; } t2, *tp2;
```

type `t1` and the type pointed to by `tp1` are compatible. Type `t1` is also compatible with type **struct** `s1`, but not compatible with the types **struct** `s2`, `t2`, the type pointed to by `tp2`, or **int**.

6 EXAMPLE 3 The following obscure constructions


```
typedef signed int t;
typedef int plain;
struct tag {
      unsigned t:4;
      const t:5;
      plain r:5;
};
```

declare a typedef name `t` with type **`signed int`**, a typedef name `plain` with type **`int`**, and a structure with three bit-field members, one named `t` that contains values in the range $[0, 15]$, an unnamed const-qualified bit-field which (if it could be accessed) would contain values in the range $[-16, +15]$, and one named `r` that contains values in one of the ranges $[0, 31]$ or $[-16, +15]$. (The choice of range is implementation-defined.) The first two bit-field declarations differ in that **`unsigned`** is a type specifier (which forces `t` to be the name of a structure member), while **`const`** is a type qualifier (which modifies `t` which is still visible as a typedef name). If these declarations are followed in an inner scope by

```
t f(t (t));
long t;
```

then a function `f` is declared with type "function returning **`signed int`** with one unnamed parameter with type pointer to function returning **`signed int`** with one unnamed parameter with type **`signed int`**", and an identifier `t` with type **`long int`**.

7 EXAMPLE 4 On the other hand, typedef names can be used to improve code readability. All three of the following declarations of the **`signal`** function specify exactly the same type, the first without making use of any typedef names.

```
typedef void fv(int), (*pfv)(int);

void (*signal(int, void (*)(int)))(int);
fv *signal(int, fv *);
pfv signal(int, pfv);
```

8 EXAMPLE 5 If a typedef name denotes a variable length array type, the length of the array is fixed at the time the typedef name is defined, not each time it is used:

```
void copyt(int n)
{
      typedef int B[n];   // B is n ints, n evaluated now
      n += 1;
      B a;                // a is n ints, n without += 1
      int b[n];           // a and b are different sizes
      for (int i = 1; i  <  n; i++)
            a[i-1] = b[i];
}
```

6.7.10 Type inference

Constraints

1 A declaration for which the type is inferred shall contain the storage-class specifier **`auto`**.

Semantics

2 For such a declaration that is the definition of an object the init-declarator shall have the form

direct-declarator **`=`** *assignment-expression*

The inferred type of the declared object is the type of the assignment expression after lvalue, array to pointer or function to pointer conversion, additionally qualified by qualifiers and amended by attributes as they appear in the declaration specifiers, if any.[163)] Implementations may or may not

[163)] The scope rules as described in 6.2.1 also prohibit the use of the identifier of the declarator within the assignment expression.

accept a direct declarator that is not of the form

identifier attribute-specifier-sequence$_{\text{opt}}$

optionally enclosed in balanced pairs of parentheses; if a direct declarator of a different form is accepted, the behavior is implementation-defined.[164)]

3 NOTE A declaration that also defines a structure or union type has implementation-defined behavior. Here, the identifier `x` which is not ordinary but in the name space of the structure type is declared.

```
auto p = (struct { int x; } *)0;
```

Even a forward declaration of a structure tag

```
struct s;
auto p = (struct s { int x; } *)0;
```

would not change that situation. A direct use of the structure definition as the type specifier ensures portability of the declaration.

```
struct s { int x; } * p = 0;
```

The following also has implementation-defined behavior:

```
auto alignas (struct s *) x = 0;
```

4 EXAMPLE 1 The following file scope definitions:

```
static auto a = 3.5;
auto p = &a;
```

are interpreted as if they had been written as:

```
static double a = 3.5;
double * p = &a;
```

So effectively `a` is a **`double`** and `p` is a **`double`***. Note that the restrictions on the syntax of such declarations does not allow the declarator to be `*p`, but that the final type here nevertheless is a pointer type.

5 EXAMPLE 2 The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
{
      double a = 7;
      double b = 9;
      {
            double b = b * b;  // undefined, uses uninitialized
                               // variable without address
            printf("%g\n", a); // valid, uses "a" from outer scope, prints 7
            auto a   = a * a;  // invalid, "a" from outer scope is not
                               // visible during initialization
      }
      {
            auto b   = a * a;  // valid, uses "a" from outer scope
            auto a   = b;      // valid, "a" from outer scope not visible now
            // ...
            printf("%g\n", a); // valid, uses "a" from inner scope, prints 49
      }
```

[164)] It is recommended that implementations that accept different forms of direct declarators follow the syntax and semantics of the corresponding feature in ISO/IEC 14882.


```
        // ...
    }
```

6 EXAMPLE 3 In the following, declarations of `pA` and `qA` are valid. The type of `A` after array-to-pointer conversion is a pointer type, and `qA` is a pointer to array.

```
    double A[3] = { 0 };
    auto pA = A;
    auto qA = &A;
```

7 EXAMPLE 4 Type inference can be used to capture the type of a call to a type-generic function. It ensures that the same type as the argument `x` is used.

```
    #include <tgmath.h>
    auto y = cos(x);
```

If instead the type of `y` is explicitly specified to a different type than `x`, a diagnosis of the mismatch is not enforced.

8 EXAMPLE 5 A type-generic macro that generalizes the **`div`** functions (7.24.6.2) is defined and used as follows.

```
    #define div(X, Y) _Generic((X)+(Y),\
          int: div,\
          long: ldiv,\
          long long: lldiv)((X), (Y))
    auto z = div(x, y);
    auto q = z.quot;
    auto r = z.rem;
```

9 EXAMPLE 6 Definitions of objects with inferred type are valid in all contexts that allow the initializer syntax as described. In particular they can be used to ensure type safety of **`for`**-loop controlling expressions.

```
    for (auto i = j; i < 2*j; ++i) {
        // ...
    }
```

Here, regardless of the integer rank or signedness of the type of `j`, `i` will have the non-atomic unqualified version of `j`'s type. So, after lvalue conversion and possible promotion, the two operands of the `<` operator in the controlling expression are guaranteed to have the same type, and, in particular, the same signedness.

6.7.11 Initialization

Syntax

1 *braced-initializer:*
{ }
{ *initializer-list* **}**
{ *initializer-list* **, }**

initializer:
assignment-expression
braced-initializer

initializer-list:
$designation_{opt}$ *initializer*
initializer-list **,** $designation_{opt}$ *initializer*

designation:
designator-list **=**

designator-list:
designator
designator-list *designator*
designator:
[*constant-expression* **]**
. *identifier*

2 An empty brace pair ({}) is called an *empty initializer* and is referred to as *empty initialization*.

Constraints

3 No initializer shall attempt to provide a value for an object not contained within the entity being initialized.

4 The type of the entity to be initialized shall be an array of unknown size or a complete object type. An entity of variable length array type shall not be initialized except by an empty initializer. An array of unknown size shall not be initialized by an empty initializer.

5 All the expressions in an initializer for an object that has static or thread storage duration or is declared with the **`constexpr`** storage-class specifier shall be constant expressions or string literals.

6 If the declaration of an identifier has block scope, and the identifier has external or internal linkage, the declaration shall have no initializer for the identifier.

7 If a designator has the form

[*constant-expression* **]**

then the current object (defined subsequently in this subclause) shall have array type and the expression shall be an integer constant expression. If the array is of unknown size, any nonnegative value is valid.

8 If a designator has the form

. *identifier*

then the current object (defined subsequently in this subclause) shall have structure or union type and the identifier shall be the name of a member of that type.

Semantics

9 An initializer specifies the initial value stored in an object. For objects with atomic type additional restrictions apply, see 7.17.2 and 7.17.8.

10 Except where explicitly stated otherwise, for the purposes of this subclause unnamed members of objects of structure and union type do not participate in initialization. Unnamed members of structure objects have indeterminate representation even after initialization.

11 If an object that has automatic storage duration is not initialized explicitly, its representation is indeterminate. If an object that has static or thread storage duration is not initialized explicitly, or any object is initialized with an empty initializer, then it is subject to *default initialization*, which initializes an object as follows:

— if it has pointer type, it is initialized to a null pointer;

— if it has decimal floating type, it is initialized to positive zero, and the quantum exponent is implementation-defined;[165)]

— if it has arithmetic type, and it does not have decimal floating type, it is initialized to (positive or unsigned) zero;

— if it is an aggregate, every member is initialized (recursively) according to these rules, and any padding is initialized to zero bits;

[165)] A representation with all bits zero results in a decimal floating-point zero with the most negative exponent.

— if it is a union, the first named member is initialized (recursively) according to these rules, and any padding is initialized to zero bits.

12 The initializer for a scalar shall be a single expression, optionally enclosed in braces, or it shall be an empty initializer. If the initializer is not the empty initializer, the initial value of the object is that of the expression (after conversion); the same type constraints and conversions as for simple assignment apply, taking the type of the scalar to be the unqualified version of its declared type.

13 The rest of this subclause deals with initializers for objects that have aggregate or union type.

14 The initializer for a structure or union object shall be either an initializer list as described subsequently in this subclause, or a single expression that has compatible structure or union type. In the latter case, the initial value of the object, including unnamed members, is that of the expression.[166)]

15 An array of character type may be initialized by a character string literal or UTF-8 string literal, optionally enclosed in braces. Successive bytes of the string literal (including the terminating null character if there is room or if the array is of unknown size) initialize the elements of the array.

16 An array with element type compatible with a qualified or unqualified **`wchar_t`**, **`char16_t`**, or **`char32_t`** may be initialized by a wide string literal with the corresponding encoding prefix (**`L`**, **`u`**, or **`U`**, respectively), optionally enclosed in braces. Successive wide characters of the wide string literal (including the terminating null wide character if there is room or if the array is of unknown size) initialize the elements of the array.

17 Otherwise, the initializer for an object that has aggregate or union type shall be a brace-enclosed list of initializers for the elements or named members.

18 Each brace-enclosed initializer list has an associated *current object*. When no designations are present, subobjects of the current object are initialized in order according to the type of the current object: array elements in increasing subscript order, structure members in declaration order, and the first named member of a union.[167)] In contrast, a designation causes the following initializer to begin initialization of the subobject described by the designator. Initialization then continues forward in order, beginning with the next subobject after that described by the designator.[168)]

19 Each designator list begins its description with the current object associated with the closest surrounding brace pair. Each item in the designator list (in order) specifies a particular member of its current object and changes the current object for the next designator (if any) to be that member.[169)] The current object that results at the end of the designator list is the subobject to be initialized by the following initializer.

20 The initialization shall occur in initializer list order, each initializer provided for a particular subobject overriding any previously listed initializer for the same subobject;[170)] all subobjects that are not initialized explicitly are subject to default initialization.

21 If the aggregate or union contains elements or members that are aggregates or unions, these rules apply recursively to the subaggregates or contained unions. If the initializer of a subaggregate or contained union begins with a left brace, the initializers enclosed by that brace and its matching right brace initialize the elements or members of the subaggregate or the contained union. Otherwise, only enough initializers from the list are taken to account for the elements or members of the subaggregate or the first member of the contained union; any remaining initializers are left to initialize the next element or member of the aggregate of which the current subaggregate or contained union is a part.

22 If there are fewer initializers in a brace-enclosed list than there are elements or members of an

[166)] If the object being initialized does not have automatic storage duration, this case violates a constraint unless the expression is a named constant or compound literal constant (6.6).

[167)] If the initializer list for a subaggregate or contained union does not begin with a left brace, its subobjects are initialized as usual, but the subaggregate or contained union does not become the current object: current objects are associated only with brace-enclosed initializer lists.

[168)] After a union member is initialized, the next object is not the next member of the union; instead, it is the next subobject of an object containing the union.

[169)] Thus, a designator can only specify a strict subobject of the aggregate or union that is associated with the surrounding brace pair. Note, too, that each separate designator list is independent.

[170)] Any initializer for the subobject which is overridden and so not used to initialize that subobject may not be evaluated at all.

aggregate, or fewer characters in a string literal used to initialize an array of known size than there are elements in the array, the remainder of the aggregate is subject to default initialization.

23 If an array of unknown size is initialized, its size is determined by the largest indexed element with an explicit initializer. The array type is completed at the end of its initializer list.

24 The evaluations of the initialization list expressions are indeterminately sequenced with respect to one another and thus the order in which any side effects occur is unspecified.[171)]

25 EXAMPLE 1 Provided that `<complex.h>` has been included, the declarations

```
int i = 3.5;
double complex c = 5 + 3 * I;
```

define and initialize `i` with the value 3 and `c` with the value $5.0 + i3.0$.

26 EXAMPLE 2 The declaration

```
int x[] = { 1, 3, 5 };
```

defines and initializes `x` as a one-dimensional array object that has three elements, as no size was specified and there are three initializers.

27 EXAMPLE 3 The declaration

```
int y[4][3] = {
      { 1, 3, 5 },
      { 2, 4, 6 },
      { 3, 5, 7 },
};
```

is a definition with a fully bracketed initialization: 1, 3, and 5 initialize the first row of `y` (the array object `y[0]`), namely `y[0][0]`, `y[0][1]`, and `y[0][2]`. Likewise the next two lines initialize `y[1]` and `y[2]`. The initializer ends early, so `y[3]` is initialized with zeros. Precisely the same effect could have been achieved by

```
int y[4][3] = {
      1, 3, 5, 2, 4, 6, 3, 5, 7
};
```

The initializer for `y[0]` does not begin with a left brace, so three items from the list are used. Likewise the next three are taken successively for `y[1]` and `y[2]`.

28 EXAMPLE 4 The declaration

```
int z[4][3] = {
      { 1 }, { 2 }, { 3 }, { 4 }
};
```

initializes the first column of `z` as specified and initializes the rest with zeros.

29 EXAMPLE 5 The declaration

```
struct { int a[3], b; } w[] = { { 1 }, 2 };
```

is a definition with an inconsistently bracketed initialization. It defines an array with two element structures: `w[0].a[0]` is 1 and `w[1].a[0]` is 2; all the other elements are zero.

30 EXAMPLE 6 The declaration

```
short q[4][3][2] = {
      { 1 },
      { 2, 3 },
      { 4, 5, 6 }
```

[171)] In particular, the evaluation order can be the same or different as the order of subobject initialization.


```
};
```

contains an incompletely but consistently bracketed initialization. It defines a three-dimensional array object: `q[0][0][0]` is 1, `q[1][0][0]` is 2, `q[1][0][1]` is 3, and 4, 5, and 6 initialize `q[2][0][0]`, `q[2][0][1]`, and `q[2][1][0]`, respectively; all the rest are zero. The initializer for `q[0][0]` does not begin with a left brace, so up to six items from the current list could be used. There is only one, so the values for the remaining five elements are initialized with zero. Likewise, the initializers for `q[1][0]` and `q[2][0]` do not begin with a left brace, so each uses up to six items, initializing their respective two-dimensional subaggregates. If there had been more than six items in any of the lists, a diagnostic message would have been issued. The same initialization result could have been achieved by:

```
short q[4][3][2] = {
      1, 0, 0, 0, 0, 0,
      2, 3, 0, 0, 0, 0,
      4, 5, 6
};
```

or by:

```
short q[4][3][2] = {
      {
            { 1 },
      },
      {
            { 2, 3 },
      },
      {
            { 4, 5 },
            { 6 },
      }
};
```

in a fully bracketed form.

31 Note that the fully bracketed and minimally bracketed forms of initialization are, in general, less likely to cause confusion.

32 EXAMPLE 7 One form of initialization that completes array types involves typedef names. Given the declaration

```
typedef int A[];  // OK - declared with block scope
```

the declaration

```
A a = { 1, 2 }, b = { 3, 4, 5 };
```

is identical to

```
int a[] = { 1, 2 }, b[] = { 3, 4, 5 };
```

due to the rules for incomplete types.

33 EXAMPLE 8 The declaration

```
char s[] = "abc", t[3] = "abc";
```

defines "plain" **char** array objects `s` and `t` whose elements are initialized with character string literals. This declaration is identical to

```
char s[] = { 'a', 'b', 'c', '\0' },
     t[] = { 'a', 'b', 'c' };
```

The contents of the arrays are modifiable. On the other hand, the declaration

```
char *p = "abc";
```

defines `p` with type "pointer to **char**" and initializes it to point to an object with type "array of **char**" with length 4 whose elements are initialized with a character string literal. If an attempt is made to use `p` to modify the contents of the array, the behavior is undefined.

34 EXAMPLE 9 Arrays can be initialized to correspond to the elements of an enumeration by using designators:

```
enum { member_one, member_two };
const char *nm[] = {
      [member_two] = "member two",
      [member_one] = "member one",
};
```

35 EXAMPLE 10 Structure members can be initialized to nonzero values without depending on their order:

```
div_t answer = {.quot = 2, .rem = -1 };
```

36 EXAMPLE 11 Designators can be used to provide explicit initialization when unadorned initializer lists are difficult to understand:

```
struct { int a[3], b; } w[] =
      { [0].a = {1}, [1].a[0] = 2 };
```

37 EXAMPLE 12

```
struct T {
      int k;
      int l;
};

struct S {
      int i;
      struct T t;
};

struct T x = {.l = 43, .k = 42, };

void f(void)
{
      struct S l = { 1, .t = x, .t.l = 41, };
}
```

The value of `l.t.k` is 42, because implicit initialization does not override explicit initialization.

38 EXAMPLE 13 Space can be "allocated" from both ends of an array by using a single designator:

```
int a[A_MAX] = {
      1, 3, 5, 7, 9, [A_MAX-5] = 8, 6, 4, 2, 0
};
```

In the preceding snippet, if **A_MAX** is greater than ten, there will be some zero-valued elements in the middle; if it is less than ten, some of the values provided by the first five initializers will be overridden by the second five.

39 EXAMPLE 14 Any member of a union can be initialized:

```
union { /* ... */ } u = {.any_member = 42 };
```

Forward references: common definitions <stddef.h> (7.21).

6.7.12 Static assertions

Syntax

1 *static_assert-declaration:*

static_assert **(** *constant-expression* **,** *string-literal* **)** **;**
static_assert **(** *constant-expression* **)** **;**

Constraints

2 The constant expression shall compare unequal to 0.

Semantics

3 The constant expression shall be an integer constant expression. If the value of the constant expression compares unequal to 0, the declaration has no effect. Otherwise, the constraint is violated and the implementation shall produce a diagnostic message which should include the text of the string literal, if present.

Forward references: diagnostics (7.2).

6.7.13 Attributes

6.7.13.1 Introduction

1 Attributes specify additional information for various source constructs such as types, objects, identifiers, or blocks. They are identified by an *attribute token*, which can either be a *attribute prefixed token* (for implementation-specific attributes) or a *standard attribute* specified by an identifier (for attributes specified in this document).

2 Support for any of the standard attributes specified in this document is implementation-defined and optional. For an attribute token (including an attribute prefixed token) not specified in this document, the behavior is implementation-defined. Any attribute token that is not supported by the implementation is ignored.

3 Attributes are said to appertain to some source construct, identified by the syntactic context where they appear, and for each individual attribute, the corresponding clause constrains the syntactic context in which this appurtenance is valid. The attribute specifier sequence appertaining to some source construct shall contain only attributes that are allowed to apply to that source construct.

4 In all aspects of the language, a standard attribute specified by this document as an identifier `attr` and an identifier of the form `__attr__` shall behave the same when used as an attribute token, except for the spelling.[172)]

5 For all standard attributes specified by this document, the current value when its token sequence is given to the **__has_c_attribute** conditional inclusion expression (6.10.2) is written in the associated subclause for that attribute. A history of those values can be found in Annex M.

Recommended practice

6 It is recommended that implementations support all standard attributes as defined in this document.

6.7.13.2 General

Syntax

1 *attribute-specifier-sequence:*

attribute-specifier-sequence$_{\text{opt}}$ *attribute-specifier*

attribute-specifier:

[**[** *attribute-list* **]** **]**

attribute-list:

attribute$_{\text{opt}}$
attribute-list **,** *attribute*$_{\text{opt}}$

[172)] Thus, the attributes `[[nodiscard]]` and `[[__nodiscard__]]` can be freely interchanged. Implementations are encouraged to behave similarly for attribute tokens (including attribute prefixed tokens) they provide.

attribute:
attribute-token *attribute-argument-clause*$_{\text{opt}}$
attribute-token:
standard-attribute
attribute-prefixed-token
standard-attribute:
identifier

attribute-prefixed-token:
attribute-prefix **::** *identifier*
attribute-prefix:
identifier
attribute-argument-clause:
(*balanced-token-sequence*$_{\text{opt}}$ **)**
balanced-token-sequence:
balanced-token
balanced-token-sequence *balanced-token*
balanced-token:
(*balanced-token-sequence*$_{\text{opt}}$ **)**
[*balanced-token-sequence*$_{\text{opt}}$ **]**
{ *balanced-token-sequence*$_{\text{opt}}$ **}**
any token other than a parenthesis, a bracket, or a brace

Constraints

2 The identifier in a standard attribute shall be one of:

`deprecated`	`maybe_unused`	`noreturn`	`unsequenced`
`fallthrough`	`nodiscard`	`_Noreturn`	`reproducible`

Semantics

3 An attribute specifier that contains no attributes has no effect. The order in which attribute tokens appear in an attribute list is not significant. If a keyword (6.4.1) that satisfies the syntactic requirements of an identifier (6.4.2) is contained in an attribute token, it is considered an identifier. A strictly conforming program using a standard attribute remains strictly conforming in the absence of that attribute.[173)]

4 NOTE For each standard attribute, the form of the balanced token sequence, if any, will be specified.

Recommended practice

5 Each implementation should choose a distinctive name for the attribute prefix in an attribute prefixed token. Implementations should not define attributes without an attribute prefix unless it is a standard attribute as specified in this document.

6 EXAMPLE 1 Suppose that an implementation chooses the attribute prefix `hal` and provides specific attributes named `daisy` and `rosie`.

```
[[deprecated, hal::daisy]] double nine1000(double);
[[deprecated]] [[hal::daisy]] double nine1000(double);
[[deprecated]] double nine1000 [[hal::daisy]] (double);
```

Then all the following declarations should be equivalent aside from the spelling:

```
[[__deprecated__, __hal__::__daisy__]] double nine1000(double);
[[__deprecated__]] [[__hal__::__daisy__]] double nine1000(double);
[[__deprecated__]] double nine1000 [[__hal__::__daisy__]] (double);
```

[173)] Standard attributes specified by this document can be parsed but ignored by an implementation without changing the semantics of a correct program; the same is not true for attributes not specified by this document.

These use the alternate spelling that is required for all standard attributes and recommended for prefixed attributes. These may be better-suited for use in header files, where the use of the alternate spelling avoids naming conflicts with user-provided macros.

7 EXAMPLE 2 For the same implementation, the following two declarations are equivalent, because the ordering inside attribute lists is not important.

```
[[hal::daisy, hal::rosie]] double nine999(double);
[[hal::rosie, hal::daisy]] double nine999(double);
```

On the other hand the following two declarations are not equivalent, because the ordering of different attribute specifiers may affect the semantics.

```
[[hal::daisy]] [[hal::rosie]] double nine999(double);
[[hal::rosie]] [[hal::daisy]] double nine999(double); // may have different semantics
```

6.7.13.3 The `nodiscard` attribute

Constraints

1 The **`nodiscard`** attribute shall be applied to a function or to the definition of a structure, union, or enumerated type. If an attribute argument clause is present, it shall have the form:

(*string-literal* **)**

Semantics

2 The **`__has_c_attribute`** conditional inclusion expression (6.10.2) shall return the value 202311L when given **`nodiscard`** as the pp-tokens operand if the implementation supports the attribute.

3 A name or entity declared without the **`nodiscard`** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked after the first declaration that marks it.

Recommended practice

4 A nodiscard call is a function call expression that calls a function previously declared with attribute **`nodiscard`**, or whose return type is a structure, union, or enumerated type marked with attribute **`nodiscard`**. Evaluation of a nodiscard call as a void expression (6.8.4) is discouraged unless explicitly cast to **`void`**. Implementations are encouraged to issue a diagnostic in such cases. This is typically because immediately discarding the return value of a **`nodiscard`** call has surprising consequences.

5 The diagnostic message should include text provided by the string literal within the attribute argument clause of any **`nodiscard`** attribute applied to the name or entity.

6 EXAMPLE 1

```
struct [[nodiscard]] error_info { /*...*/ };
struct error_info enable_missile_safety_mode(void);
void launch_missiles(void);
void test_missiles(void) {
      enable_missile_safety_mode();
      launch_missiles();
}
```

A diagnostic for the call to `enable_missile_safety_mode` is encouraged.

7 EXAMPLE 2

```
[[nodiscard]] int important_func(void);
void call(void) {
      int i = important_func();
}
```

No diagnostic for the call to `important_func` is encouraged despite the value of `i` not being used.

8 EXAMPLE 3

```
[[nodiscard("armer needs to check armed state")]]
bool arm_detonator(int within);

void call(void) {
      arm_detonator(3);
      detonate();
}
```

A diagnostic for the call to `arm_detonator` using the string literal `"armer needs to check armed state"` from the attribute argument clause is encouraged.

6.7.13.4 The `maybe_unused` attribute

Constraints

1 The **`maybe_unused`** attribute shall be applied to the declaration of a structure, a union, a **`typedef`** name, an object, a structure or union member, a function, an enumeration, an enumerator, or a label. No attribute argument clause shall be present.

Semantics

2 The **`maybe_unused`** attribute indicates that a name or entity is possibly intentionally unused.

3 The **`__has_c_attribute`** conditional inclusion expression (6.10.2) shall return the value 202311L when given **`maybe_unused`** as the pp-tokens operand if the implementation supports the attribute.

A name or entity declared without the **`maybe_unused`** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked with the attribute after the first declaration that marks it.

Recommended practice

4 For an entity marked **`maybe_unused`**, implementations are encouraged not to emit a diagnostic that the entity is unused, or that the entity is used despite the presence of the attribute.

5 EXAMPLE

```
[[maybe_unused]] void f([[maybe_unused]] int i) {
      [[maybe_unused]] int j = i + 100;
      assert(j);
}
```

Implementations are encouraged not to diagnose that `j` is unused, even if **`NDEBUG`** is defined.

6.7.13.5 The `deprecated` attribute

Constraints

1 The **`deprecated`** attribute shall be applied to the declaration of a structure, a union, a **`typedef`** name, an object, a structure or union member, a function, an enumeration, or an enumerator.

2 If an attribute argument clause is present, it shall have the form:

(*string-literal* **)**

Semantics

3 The **`deprecated`** attribute can be used to mark names and entities whose use is still allowed, but is discouraged for some reason.[174)]

4 The **`__has_c_attribute`** conditional inclusion expression (6.10.2) shall return the value 202311L when given **`deprecated`** as the pp-tokens operand if the implementation supports the attribute.

5 A name or entity declared without the **`deprecated`** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked with the attribute after the first declaration that marks it.

[174)] In particular, **`deprecated`** is appropriate for names and entities that are obsolescent, insecure, unsafe, or otherwise unfit for purpose.

Recommended practice

6 Implementations should use the **deprecated** attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute, when the reference to the name or entity is not within the context of a related deprecated entity. The diagnostic message should include text provided by the string literal within the attribute argument clause of any **deprecated** attribute applied to the name or entity.

7 EXAMPLE

```
struct [[deprecated]] S {
      int a;
};

enum [[deprecated]] E1 {
      one
};

enum E2 {
      two [[deprecated("use 'three' instead")]],
      three
};

[[deprecated]] typedef int Foo;

void f1(struct S s) { // Diagnose use of S
      int i = one; // Diagnose use of E1
      int j = two; // Diagnose use of two: "use 'three' instead"
      int k = three;
      Foo f; // Diagnose use of Foo
}

[[deprecated]] void f2(struct S s) {
      int i = one;
      int j = two;
      int k = three;
      Foo f;
}

struct [[deprecated]] T {
      Foo f;
      struct S s;
};
```

Implementations are encouraged to diagnose the use of deprecated entities within a context which is not itself deprecated, as indicated for function `f1`, but not to diagnose within function `f2` and **struct** `T`, as they are themselves deprecated.

6.7.13.6 The fallthrough attribute

Constraints

1 The attribute token **fallthrough** shall only appear in an attribute declaration (6.7); such a declaration is a *fallthrough declaration*. No attribute argument clause shall be present. A fallthrough declaration may only appear within an enclosing **switch** statement (6.8.5.3). The next block item (6.8.3) that would be encountered after a fallthrough declaration shall be a **case** label or **default** label associated with the innermost enclosing **switch** statement and, if the fallthrough declaration is contained in an iteration statement, the next statement shall be part of the same execution of the secondary block of the innermost enclosing iteration statement.

Semantics

2 The **__has_c_attribute** conditional inclusion expression (6.10.2) shall return the value 202311L when given **fallthrough** as the pp-tokens operand if the implementation supports the attribute.

Recommended practice

3 The use of a fallthrough declaration is intended to suppress a diagnostic that an implementation may otherwise issue for a **case** or **default** label that is reachable from another **case** or **default** label along some path of execution. Implementations are encouraged to issue a diagnostic if a fallthrough declaration is not dynamically reachable.

4 EXAMPLE

```
void f(int n) {
      void g(void), h(void), i(void);
      switch (n) {
      case 1: /* diagnostic on fallthrough discouraged */
      case 2:
            g();
            [[fallthrough]];
      case 3: /* diagnostic on fallthrough discouraged */
            do {
                  [[fallthrough]]; /* constraint violation: next statement is not
                                      part of the same secondary block execution */
            } while(false);
      case 6:
            do {
                  [[fallthrough]]; /* constraint violation: next statement is not
                                      part of the same secondary block execution */
            } while (n--);
      case 7:
            while (false) {
                  [[fallthrough]]; /* constraint violation: next statement is not
                                      part of the same secondary block execution */
            }
      case 5:
            h();
      case 4: /* fallthrough diagnostic encouraged */
            i();
            [[fallthrough]]; /* constraint violation */
      }
}
```

6.7.13.7 The `noreturn` and `_Noreturn` attributes

Description

1 When `_Noreturn` is used as an attribute token (instead of a function specifier), the constraints and semantics are identical to that of the `noreturn` attribute token. Use of `_Noreturn` as an attribute token is an obsolescent feature.[175)]

Constraints

2 The `noreturn` attribute shall be applied to a function. No attribute argument clause shall be present.

Semantics

3 The first declaration of a function shall specify the `noreturn` attribute if any declaration of that function specifies the `noreturn` attribute. If a function is declared with the `noreturn` attribute in one translation unit and the same function is declared without the `noreturn` attribute in another translation unit, the behavior is undefined.

4 If a function `f` is called where `f` was previously declared with the `noreturn` attribute and `f` eventually returns, the behavior is undefined.

5 The `__has_c_attribute` conditional inclusion expression (6.10.2) shall return the value 202311L when given `noreturn` as the pp-tokens operand if the implementation supports the attribute.

[175)] `[[_Noreturn]]` and `[[noreturn]]` are equivalent attributes to support code that includes `<stdnoreturn.h>`, because that header defines `noreturn` as a macro that expands to `_Noreturn`.

Recommended practice

6 The implementation should produce a diagnostic message for a function declared with a **noreturn** attribute that appears to be capable of returning to its caller.

7 EXAMPLE

```
[[noreturn]] void f(void) {
      abort(); // ok
}

[[noreturn]] void g(int i) { // causes undefined behavior if i <= 0
      if (i > 0) abort();
}

[[noreturn]] int h(void);
```

Implementations are encouraged to diagnose the definition of `g()` because it is capable of returning to its caller. Implementations are similarly encouraged to diagnose the declaration of `h()` because it appears capable of returning to its caller due to the non-**void** return type.

6.7.13.8 Standard attributes for function types

6.7.13.8.1 General

Constraints

1 The identifier in a standard function type attribute shall be one of:

unsequenced **reproducible**

2 An attribute for a function type shall be applied to a function declarator[176] or to a type specifier that has a function type. The corresponding attribute is a property of the function type.[177] No attribute argument clause shall be present.

Description

3 The main purpose of the function type properties and attributes defined in this clause is to provide the translator with information about the access of objects by a function such that certain properties of function calls can be deduced; the properties distinguish read operations (stateless and independent) and write operations (effectless, idempotent and reproducible) or a combination of both (unsequenced). Although semantically attached to a function type, the attributes described are not part of the prototype of such a function, and redeclarations and conversions that drop such an attribute are valid and constitute compatible types. Conversely, if a definition that does not have the asserted property is accessed by a function declaration or a function pointer with a type that has the attribute, the behavior is undefined.[178]

4 To allow reordering of calls to functions as they are described here, possible access to objects with a lifetime that starts before or ends after a call has to be restricted; effects on all objects that are accessed during a function call are restricted to the same thread as the call and the based-on relation between pointer parameters and lvalues (6.7.4.2) models the fact that objects do not change inadvertently during the call. In the following, an operation is said to be sequenced *during* a function call if it is sequenced after the start of the function call[179] and before the call terminates. An object definition of an object X in a function f *escapes* if an access to X happens while no call to f is active. An object is *local* to a call to a function f if its lifetime starts and ends during the call or if it is defined by f

[176] That is, they appear in the attributes right after the closing parenthesis of the parameter list, independently of whether the function type is, for example, used directly to declare a function or whether it is used in a pointer to function type.

[177] If several declarations of the same function or function pointer are visible, regardless whether an attribute is present at several or just one of the declarators, it is attached to the type of the corresponding function definition, function pointer object, or function pointer value.

[178] That is, the fact that a function has one of these properties is in general not determined by the specification of the translation unit in which it is found; other translation units and specific run time conditions also condition the possible assertion of the properties.

[179] The initializations of the parameters is sequenced during the function call.

but does not escape. A function call and an object X *synchronize* if all accesses to X that are not sequenced during the call happen before or after the call. Execution state that is described in the library clause, such as the floating-point environment, conversion state, locale, input/output streams, external files or **errno** are considered as objects for the purposes of these attributes; operations that access this state, even indirectly, are considered as lvalue conversions for the purposes of these attributes, and operations that allow to change this state are considered as store operations, for the purposes of these attributes.

5 A function definition f is *stateless* if any definition of an object of static or thread storage duration in f or in a function that is called by f is **const** but not **volatile** qualified.

6 An object X is *observed* by a function call if both synchronize, if X is not local to the call, if X has a lifetime that starts before the function call and if an access of X is sequenced during the call; the last value of X, if any, that is stored before the call is said to be the value of X that is observed by the call. A function pointer value f is *independent* if for any object X that is observed by some call to f through an lvalue that is not based on a parameter of the call, then all accesses to X in all calls to f during the same program execution observe the same value; otherwise if the access is based on a pointer parameter, there shall be a unique such pointer parameter P such that any access to X shall be to an lvalue that is based on P. A function definition is independent if the derived function pointer value is independent.

7 A store operation to an object X that is sequenced during a function call such that both synchronize is said to be *observable* if X is not local to the call, if the lifetime of X ends after the call, if the stored value is different from the value observed by the call, if any, and if it is the last value written before the termination of the call. An evaluation of a function call[180)] is *effectless* if any store operation that is sequenced during the call is the modification of an object that synchronizes with the call; if additionally the operation is observable, there shall be a unique pointer parameter P of the function such that any access to X shall be to an lvalue that is based on P. A function pointer value f is effectless if any evaluation of a function call that calls f is effectless. A function definition is effectless if the derived function pointer value is effectless.

8 An evaluation E is *idempotent* if a second evaluation of E can be sequenced immediately after the original one without changing the resulting value, if any, or the observable state of the execution. A function pointer value f is idempotent if any evaluation of a function call[181)] that calls f is idempotent. A function definition is idempotent if the derived function pointer value is idempotent.

9 A function is *reproducible* if it is effectless and idempotent; it is *unsequenced* if it is stateless, effectless, idempotent and independent.[182)]

10 NOTE 1 The synchronization requirements with respect to any accessed object X for the independence of functions provide boundaries up to which a function call may safely be reordered without changing the semantics of the program. If X is **const** but not **volatile** qualified the reordering is unconstrained. If it is an object that is conditioned in an initialization phase, for a single threaded program a synchronization is provided by the sequenced before relation and the reordering may, in principle, move the call just after the initialization. For a multi-threaded program, synchronization guarantees can be given by calls to synchronizing functions of the `<threads.h>` header or by an appropriate call to **atomic_thread_fence** at the end of the initialization phase. If a function is known to be independent or effectless, adding **restrict** qualifications to the declarations of all pointer parameters does not change the semantics of any call. Similarly, changing the memory order to **memory_order_relaxed** for all atomic operations during a call to such a function preserves semantics.

11 NOTE 2 In general the functions provided by the `<math.h>` header do not have the properties that are defined previously in this subclause; many of them change the floating-point state or **errno** when they encounter an error (so they have observable side effects) and the results of most of them depend on execution-wide state such as the rounding direction mode (so they are not independent). Whether a particular C library function is reproducible or unsequenced additionally often depends on properties of the implementation, such as

[180)] This considers the evaluation of the function call itself, not the evaluation of a full function call expression. Such an evaluation is sequenced after all evaluations that determine f and the call arguments, if any, have been performed.

[181)] This considers the evaluation of the function call itself, not the evaluation of a full function call expression. Such an evaluation is sequenced after all evaluations that determine f and the call arguments, if any, have been performed.

[182)] A function call of an unsequenced function can be executed as early as the function pointer value, the values of the arguments and all objects that are accessible through them, and all values of globally accessible state have been determined, and it can be executed as late as the arguments and the objects they possibly target are unchanged and as any of its return value or modified pointed-to arguments are accessed.

implementation-defined behavior for certain error conditions.

Recommended practice

12 If possible, it is recommended that implementations diagnose if an attribute of this clause is applied to a function definition that does not have the corresponding property. It is recommended that applications that assert the independent or effectless properties for functions qualify pointer parameters with **`restrict`**.

Forward references: errors `<errno.h>` (7.5), floating-point environment `<fenv.h>` (7.6), localization `<locale.h>` (7.11), mathematics `<math.h>` (7.12), fences (7.17.4), input/output `<stdio.h>` (7.23), threads `<threads.h>` (7.28), extended multibyte and wide character utilities `<wchar.h>` (7.31).

6.7.13.8.2 The `reproducible` type attribute

Description

1 The **`reproducible`** type attribute asserts that a function or pointed-to function with that type is reproducible.

2 The **`__has_c_attribute`** conditional inclusion expression (6.10.2) shall return the value 202311L when given **`reproducible`** as the pp-tokens operand if the implementation supports the attribute.

3 EXAMPLE The attribute in the following function declaration asserts that two consecutive calls to the function will result in the same return value. Changes to the abstract state during the call are possible as long as they are not observable, but no other side effects will occur. Thus the function definition may for example use local objects of static or thread storage duration to keep track of the arguments for which the function has been called and cache their computed return values.

```
size_t hash(char const[static 32]) [[reproducible]];
```

6.7.13.8.3 The `unsequenced` type attribute

Description

1 The **`unsequenced`** type attribute asserts that a function or pointed-to function with that type is unsequenced.

2 The **`__has_c_attribute`** conditional inclusion expression (6.10.2) shall return the value 202311L when given **`unsequenced`** as the pp-tokens operand if the implementation supports the attribute.

3 NOTE 1 The unsequenced type attribute asserts strong properties for such a function, in particular that certain sequencing requirements for function calls can be relaxed without affecting the state of the abstract machine. Thereby, calls to such functions are natural candidates for optimization techniques such as common subexpression elimination, local memoization or lazy evaluation.

4 NOTE 2 A proof of validity of the annotation of a function type with the **`unsequenced`** attribute may depend on the property of whether a derived function pointer escapes the translation unit or not. For a function with internal linkage where no function pointer escapes the translation unit, all calling contexts are known and it is possible, in principle, to prove that no control flow exists such that a library function is called with arguments that trigger an exceptional condition. For a function with external linkage such a proof may not be possible and the use of such a function then has to ensure that no exceptional condition results from the provided arguments.

5 NOTE 3 The unsequenced property does not necessarily imply that the function is reentrant or that calls can be executed concurrently. This is because an unsequenced function can read from and write to objects of static storage duration, as long as no change is observable after a call terminates.

6 EXAMPLE 1 The attribute in the following function declaration asserts that it doesn't depend on any modifiable state of the abstract machine. Calls to the function can be executed out of sequence before the return value is needed and two calls to the function with the same argument value will result in the same return value.

```
bool tendency(signed char) [[unsequenced]];
```

Therefore such a call for a given argument value needs only to be executed once and the returned value can be reused when appropriate. For example, calls for all possible argument values can be executed during program startup and tabulated.

7 EXAMPLE 2 The attribute in the following function declaration asserts that it doesn't depend on any modifiable state of the abstract machine. Within the same thread, calls to the function can be executed out of sequence before the return value is needed and two calls to the function will result in the same pointer return value. Therefore such a call needs only to be executed once in a given thread and the returned pointer value can be reused when appropriate. For example, a single call can be executed during thread startup and the return value `p` and the value of the object `*p` of type `toto const` can be cached.

```
typedef struct toto toto;
toto const* toto_zero(void) [[unsequenced]];
```

8 EXAMPLE 3 The unsequenced property of a function f can be locally asserted within a function g that uses it. For example the library function **sqrt** is in general not unsequenced because a negative argument will raise a domain error and because the result may depend on the rounding mode. Nevertheless in contexts similar to the following function a user can prove that it will not be called with invalid arguments, and, that the floating-point environment has the same value for all calls.

```
#include <math.h>
#include <fenv.h>

inline double distance (double const x[static 2]) [[reproducible]] {
      #pragma STDC FP_CONTRACT OFF
      #pragma STDC FENV_ROUND  FE_TONEAREST
      // We assert that sqrt will not be called with invalid arguments
      // and the result only depends on the argument value.
      extern typeof(sqrt) [[unsequenced]] sqrt;
      return sqrt(x[0]*x[0] + x[1]*x[1]);
}
```

The function `distance` potentially has the side effect of changing the floating-point environment. Nevertheless the floating environment is thread local, thus a change to that state outside the function is sequenced with the change within and additionally the observed value is restored when the function returns. Thus this side effect is not observable for a caller. Overall the function `distance` is stateless, effectless and idempotent and in particular it is reproducible as the attribute indicates. Because the function can be called in a context where the floating-point environment has different state, `distance` is not independent and thus it is also not unsequenced. Nevertheless, adding an unsequenced attribute where this is justified may introduce optimization opportunities.

```
double g (double y[static 1], double const x[static 2]) {
      // We assert that distance will not see different states of the floating
      // point environment.
      extern double distance (double const x[static 2]) [[unsequenced]];
      y[0] = distance(x);
      ...
      return distance(x);   // replacement by y[0] is valid
}
```

6.8 Statements and blocks

6.8.1 General

Syntax

1 *statement:*
labeled-statement
unlabeled-statement
unlabeled-statement:
expression-statement
attribute-specifier-sequence$_{\text{opt}}$ *primary-block*
attribute-specifier-sequence$_{\text{opt}}$ *jump-statement*
primary-block:
compound-statement
selection-statement
iteration-statement

secondary-block:
statement

Semantics

2 A *statement* specifies an action to be performed. Except as indicated, statements are executed in sequence. The optional attribute specifier sequence appertains to the respective statement.

3 A *block* is either a primary block, a secondary block, or the block associated with a function definition; it allows a set of declarations and statements to be grouped into one syntactic unit. Whenever a block *B* appears in the syntax production as part of the definition of an enclosing block *A*, scopes of identifiers and lifetimes of objects that are associated with *B* do not extend to the parts of *A* that are outside of *B*. The initializers of objects that have automatic storage duration, and any size expressions and typeof operators in declarations of ordinary identifiers with block scope, are evaluated and the values are stored in the objects (the representation of objects without an initializer becomes indeterminate) each time the declaration is reached in the order of execution, as if it were a statement, and within each declaration in the order that declarators appear.

4 A *full expression* is an expression that is not part of another expression, nor part of a declarator or abstract declarator. There is also an implicit full expression in which the non-constant size expressions for a variably modified type are evaluated; within that full expression, the evaluation of different size expressions are unsequenced with respect to one another. There is a sequence point between the evaluation of a full expression and the evaluation of the next full expression to be evaluated.

5 NOTE Each of the following is a full expression:

— a full declarator for a variably modified type,

— an initializer that is not part of a compound literal,

— the expression in an expression statement,

— the controlling expression of a selection statement (**`if`** or **`switch`**),

— the controlling expression of a **`while`** or **`do`** statement,

— each of the (optional) expressions of a **`for`** statement,

— the (optional) expression in a **`return`** statement.

While a constant expression satisfies the definition of a full expression, evaluating it does not depend on nor produce any side effects, so the sequencing implications of being a full expression are not relevant to a constant expression.

Forward references: expression and null statements (6.8.4), selection statements (6.8.5), iteration statements (6.8.6), the **`return`** statement (6.8.7.5).

6.8.2 Labeled statements

Syntax

1 *label:*
attribute-specifier-sequence$_{\text{opt}}$ *identifier* **:**
attribute-specifier-sequence$_{\text{opt}}$ **case** *constant-expression* **:**
attribute-specifier-sequence$_{\text{opt}}$ **default :**
labeled-statement:
label statement

Constraints

2 A **case** or **default** label shall appear only in a **switch** statement. Further constraints on such labels are discussed under the **switch** statement.

3 Label names shall be unique within a function.

Semantics

4 Any statement or declaration in a compound statement may be preceded by a prefix that declares an identifier as a label name. The optional attribute specifier sequence appertains to the label. Labels in themselves do not alter the flow of control, which continues unimpeded across them.

Forward references: the **goto** statement (6.8.7.2), the **switch** statement (6.8.5.3) .

6.8.3 Compound statement

Syntax

1 *compound-statement:*
{ *block-item-list*$_{\text{opt}}$ **}**
block-item-list:
block-item
block-item-list block-item
block-item:
declaration
unlabeled-statement
label

Semantics

2 A *compound statement* that is a function body together with the parameter type list and the optional attribute specifier sequence between them forms the block associated with the function definition in which it appears. Otherwise, it is a block that is different from any other block. A label shall be translated as if it were followed by a null statement.

6.8.4 Expression and null statements

Syntax

1 *expression-statement:*
expression$_{\text{opt}}$ **;**
attribute-specifier-sequence expression **;**

Semantics

2 The attribute specifier sequence appertains to the expression. The expression in an expression statement is evaluated as a void expression for its side effects.[183)]

3 A *null statement* (consisting of just a semicolon) performs no operations.

[183)] Such as assignments, and function calls which have side effects.

4 EXAMPLE 1 If a function call is evaluated as an expression statement for its side effects only, the discarding of its value can be made explicit by converting the expression to a void expression by means of a cast:

```
int p(int);
/* ... */
(void)p(0);
```

5 EXAMPLE 2 In the program fragment

```
char *s;
/* ... */
while (*s++ != '\0')
      ;
```

a null statement is used to supply an empty loop body to the iteration statement.

Forward references: iteration statements (6.8.6).

6.8.5 Selection statements

6.8.5.1 General

Syntax

1 *selection-statement:*

if (*expression* **)** *secondary-block*
if (*expression* **)** *secondary-block* **else** *secondary-block*
switch (*expression* **)** *secondary-block*

Semantics

2 A selection statement selects among a set of secondary blocks depending on the value of a controlling expression.

6.8.5.2 The `if` statement

Constraints

1 The controlling expression of an **if** statement shall have scalar type.

Semantics

2 In both forms, the first substatement is executed if the expression compares unequal to 0. In the **else** form, the second substatement is executed if the expression compares equal to 0. If the first substatement is reached via a label, the second substatement is not executed.

3 An **else** is associated with the lexically nearest preceding **if** that is allowed by the syntax.

6.8.5.3 The `switch` statement

Constraints

1 The controlling expression of a **switch** statement shall have integer type.

2 If a **switch** statement has an associated **case** or **default** label within the scope of an identifier with a variably modified type, the entire **switch** statement shall be within the scope of that identifier.[184)]

3 The expression of each **case** label shall be an integer constant expression and no two of the **case** constant expressions associated to the same **switch** statement shall have the same value after conversion. There may be at most one **default** label associated to a **switch** statement. (Any enclosed **switch** statement may have a **default** label or **case** constant expressions with values that duplicate **case** constant expressions in the enclosing **switch** statement.)

[184)] That is, the declaration either precedes the **switch** statement, or it follows the last **case** or **default** label associated with the **switch** that is in the block containing the declaration.

Semantics

4 A **switch** statement causes control to jump to, into, or past the statement that is the *switch body*, depending on the value of a controlling expression, and on the presence of a **default** label and the values of any **case** labels on or in the switch body. A **case** or **default** label is accessible only within the closest enclosing **switch** statement.

5 The integer promotions are performed on the controlling expression. The constant expression in each **case** label is converted to the promoted type of the controlling expression. If a converted value matches that of the promoted controlling expression, control jumps to the statement or declaration following the matched **case** label. Otherwise, if there is a **default** label, control jumps to the statement or declaration following the **default** label. If no converted **case** constant expression matches and there is no **default** label, no part of the switch body is executed.

Implementation limits

6 As discussed in 5.2.5.2, the implementation may limit the number of **case** values in a **switch** statement.

7 EXAMPLE In the artificial program fragment

```
switch (expr)
{
      int i = 4;
      f(i);
case 0:
      i = 17;
      /* falls through into default code */
default:
      printf("%d\n", i);
}
```

the object whose identifier is `i` exists with automatic storage duration (within the block) but is never initialized, and thus if the controlling expression has a nonzero value, the call to the **printf** function will access an object with an indeterminate representation. Similarly, the call to the function `f` cannot be reached.

6.8.6 Iteration statements

6.8.6.1 General

Syntax

1 *iteration-statement:*

while **(** *expression* **)** *secondary-block*
do *secondary-block* **while** **(** *expression* **)** **;**
for **(** $expression_{opt}$ **;** $expression_{opt}$ **;** $expression_{opt}$ **)** *secondary-block*
for **(** *declaration* $expression_{opt}$ **;** $expression_{opt}$ **)** *secondary-block*

Constraints

2 The controlling expression of an iteration statement shall have scalar type.

Semantics

3 An iteration statement causes a secondary block called the *loop body* to be executed repeatedly until the controlling expression compares equal to 0. The repetition occurs regardless of whether the loop body is entered from the iteration statement or by a jump.[185)]

4 An iteration statement may be assumed by the implementation to terminate if its controlling expression is not a constant expression,[186)] and none of the following operations are performed in its

[185)] Code jumped over is not executed. In particular, the controlling expression of a **for** or **while** statement is not evaluated before entering the loop body, nor is *clause-1* (6.8.6.4) of a **for** statement.

[186)] An omitted controlling expression is replaced by a nonzero constant, which is a constant expression.

body, controlling expression or (in the case of a **for** statement) its *expression-3*:[187)]

— input/output operations

— accessing a volatile object

— synchronization or atomic operations.

6.8.6.2 The **while** statement

1 The evaluation of the controlling expression takes place before each execution of the loop body.

6.8.6.3 The **do** statement

1 The evaluation of the controlling expression takes place after each execution of the loop body.

6.8.6.4 The **for** statement

1 The statement

```
for (clause-1; expression-2; expression-3) statement
```

behaves as follows: The expression *expression-2* is the controlling expression that is evaluated before each execution of the loop body. The expression *expression-3* is evaluated as a void expression after each execution of the loop body. If *clause-1* is a declaration, the scope of any identifiers it declares is the remainder of the declaration and the entire loop, including the other two expressions; it is reached in the order of execution before the first evaluation of the controlling expression. If *clause-1* is an expression, it is evaluated as a void expression before the first evaluation of the controlling expression.[188)]

2 Both *clause-1* and *expression-3* can be omitted. An omitted *expression-2* is replaced by a nonzero constant.

6.8.7 Jump statements

6.8.7.1 General

Syntax

1 *jump-statement:*

goto *identifier* **;**
continue ;
break ;
return *expression*$_{\text{opt}}$ **;**

Semantics

2 A jump statement causes an unconditional jump to another place.

6.8.7.2 The **goto** statement

Constraints

1 The identifier in a **goto** statement shall name a label located somewhere in the enclosing function. A **goto** statement shall not jump from outside the scope of an identifier having a variably modified type to inside the scope of that identifier.

[187)] This is intended to allow compiler transformations such as removal of empty loops even when termination cannot be proven.

[188)] Thus, *clause-1* specifies initialization for the loop, possibly declaring one or more variables for use in the loop; the controlling expression, *expression-2*, specifies an evaluation made before each iteration, such that execution of the loop continues until the expression compares equal to 0; and *expression-3* specifies an operation (such as incrementing) that is performed after each iteration.

Semantics

2 A **goto** statement causes an unconditional jump to the statement prefixed by the named label in the enclosing function.

3 EXAMPLE 1 It is sometimes convenient to jump into the middle of a complicated set of statements. The following outline presents one possible approach to a problem based on these three assumptions:

1. The general initialization code accesses objects only visible to the current function.
2. The general initialization code is too large to warrant duplication.
3. The code to determine the next operation is at the head of the loop. (To allow it to be reached by **continue** statements, for example.)

```
/* ... */
goto first_time;
for (;;) {
      // determine next operation
      /* ... */
      if (need to reinitialize) {
            // reinitialize-only code
            /* ... */
      first_time:
            // general initialization code
            /* ... */
            continue;
      }
      // handle other operations
      /* ... */
}
```

4 EXAMPLE 2 A **goto** statement which jumps past any declarations of objects with variably modified types is not conforming. A jump within the scope, however, is valid.

```
goto lab3;              // invalid:  going INTO scope of VLA.
{
      double a[n];
      a[j] = 4.4;
lab3:
      a[j] = 3.3;
      goto lab4;        // valid:  going WITHIN scope of VLA.
      a[j] = 5.5;
lab4:
      a[j] = 6.6;
}
goto lab4;              // invalid:  going INTO scope of VLA.
```

6.8.7.3 The continue statement

Constraints

1 A **continue** statement shall appear only in or as a loop body.

Semantics

2 A **continue** statement causes a jump to the loop-continuation portion of the innermost enclosing iteration statement; that is, to the end of the loop body. More precisely, in each of the statements

```
while (/* ...  */) {
   /* ...  */
   continue;
   /* ...  */
contin:
}
```

```
do {
   /* ...  */
   continue;
   /* ...  */
contin:;
} while (/* ...  */);
```

```
for (/* ...  */) {
   /* ...  */
   continue;
   /* ...  */
contin:
}
```


unless the **`continue`** statement shown is in an enclosed iteration statement (in which case it is interpreted within that statement), it is equivalent to **`goto`** `contin;` .[189)]

6.8.7.4 The `break` statement

Constraints

1 A **`break`** statement shall appear only in or as a switch body or loop body.

Semantics

2 A **`break`** statement terminates execution of the innermost enclosing **`switch`** or iteration statement.

6.8.7.5 The `return` statement

Constraints

1 A **`return`** statement with an expression shall not appear in a function whose return type is **`void`**. A **`return`** statement without an expression shall only appear in a function whose return type is **`void`**.

Semantics

2 A **`return`** statement terminates execution of the current function and returns control to its caller. A function may have any number of **`return`** statements.

3 If a **`return`** statement with an expression is executed, the value of the expression is returned to the caller as the value of the function call expression. If the expression has a type different from the return type of the function in which it appears, the value is converted as if by assignment to an object having the return type of the function.[190)]

4 EXAMPLE In:

```
struct s { double i; } f(void);
union {
      struct {
            int f1;
            struct s f2;
      } u1;
      struct {
            struct s f3;
            int f4;
      } u2;
} g;

struct s f(void)
{
      return g.u1.f2;
}

/* ... */
g.u2.f3 = f();
```

there is no undefined behavior, although there would be if the assignment were done directly (without using a function call to fetch the value).

[189)] Following the `contin:` label in the 2nd example is a null statement. The null statement in the first and third example is implied by the label (6.8.3).

[190)] The **`return`** statement is not an assignment. The overlap restriction of 6.5.17.2 does not apply to the case of function return. The representation of floating-point values can have wider range or precision than implied by the type; a cast can be used to remove this extra range and precision.

6.9 External definitions

6.9.1 General

Syntax

1 *translation-unit:*
external-declaration
translation-unit external-declaration

external-declaration:
function-definition
declaration

Constraints

2 The storage-class specifier **`register`** shall not appear in the declaration specifiers in an external declaration. The storage-class specifier **`auto`** shall only appear in the declaration specifiers in an external declaration if the type is inferred.

3 There shall be no more than one external definition for each identifier declared with internal linkage in a translation unit. Moreover, if an identifier declared with internal linkage is used in an expression there shall be exactly one external definition for the identifier in the translation unit, unless it is:

— part of the operand of a **`sizeof`** operator whose result is an integer constant;

— part of the operand of an **`alignof`** operator whose result is an integer constant;

— part of the controlling expression of a generic selection;

— part of the expression in a generic association that is not the result expression of its generic selection;

— or, part of the operand of any typeof operator whose result is not a variably modified type.

Semantics

4 As discussed in 5.1.1.1, the unit of program text after preprocessing is a translation unit, which consists of a sequence of external declarations. These are described as "external" because they appear outside any function (and hence have file scope). As discussed in 6.7, a declaration that also causes storage to be reserved for an object or a function named by the identifier is a definition.

5 An *external definition* is an external declaration that is also a definition of a function (other than an inline definition) or an object. If an identifier declared with external linkage is used in an expression (other than as part of the operand of a typeof operator whose result is not a variably modified type, part of the controlling expression of a generic selection, part of the expression in a generic association that is not the result expression of its generic selection, or part of a **`sizeof`** or **`alignof`** operator whose result is an integer constant expression), somewhere in the entire program there shall be exactly one external definition for the identifier; otherwise, there shall be no more than one.[191)]

6.9.2 Function definitions

Syntax

1 *function-definition:*
attribute-specifier-sequence$_{\text{opt}}$ *declaration-specifiers declarator function-body*

function-body:
compound-statement

[191)] Thus, if an identifier declared with external linkage is not used in an expression, there need be no external definition for it.

Constraints

2 The identifier declared in a function definition (which is the name of the function) shall have a function type, as specified by the declarator portion of the function definition.

3 The return type of a function shall be **void** or a complete object type other than array type.

4 The storage-class specifier, if any, in the declaration specifiers shall be either **extern** or **static**.

5 If the parameter list consists of a single parameter of type **void**, the parameter declarator shall not include an identifier.

6 Variable length array types of unspecified size shall not be used as part of a parameter declaration in a function definition.

Semantics

7 The optional attribute specifier sequence in a function definition appertains to the function.

8 The declarator in a function definition specifies the name of the function being defined and the types (and optionally the names) of all the parameters; the declarator also serves as a function prototype for later calls to the same function in the same translation unit. The type of each parameter is adjusted as described in 6.7.7.4.

9 If a function that accepts a variable number of arguments is defined without a parameter type list that ends with the ellipsis notation, the behavior is undefined.

10 The parameter type list, the attribute specifier sequence of the declarator that follows the parameter type list, and the compound statement of the function body form a single block.[192)] Each parameter has automatic storage duration; its identifier, if any,[193)] is an lvalue.[194)] The layout of the storage for parameters is unspecified.

11 On entry to the function, the size expressions of each variably modified parameter and typeof operators used in declarations of parameters are evaluated and the value of each argument expression is converted to the type of the corresponding parameter as if by assignment. (Array expressions and function designators as arguments were converted to pointers before the call.)

12 After all parameters have been assigned, the compound statement of the function body is executed.

13 Unless otherwise specified, if the } that terminates the function body is reached, and the value of the function call is used by the caller, the behavior is undefined.

14 NOTE In a function definition, the return type of the function and its prototype cannot be inherited from a typedef:

```
typedef int F(void);            // type F is “function with no parameters
                                // returning int”
F f, g;                         // f and g both have type compatible with F
F f { /* ... */ }               // WRONG: syntax/constraint error
F g() { /* ... */ }             // WRONG: declares that g returns a function
int f(void) { /* ... */ }       // RIGHT: f has type compatible with F
int g() { /* ... */ }           // RIGHT: g has type compatible with F
F *e(void) { /* ... */ }        // e returns a pointer to a function
F *((e))(void) { /* ... */ }    // same:  parentheses irrelevant
int (*fp)(void);                // fp points to a function that has type F
F *Fp;                          // Fp points to a function that has type F
```

15 EXAMPLE 1 In the following:

```
extern int max(int a, int b)
{
```

[192)] The visibility scope of a parameter in a function definition starts when its declaration is completed, extends to following parameter declarations, to possible attributes that follow the parameter type list, and then to the entire function body. The lifetime of each instance of a parameter starts when the declaration is evaluated starting a call and ends when that call terminates.

[193)] A parameter that has no declared name is inaccessible within the function body.

[194)] A parameter identifier cannot be redeclared in the function body except in an enclosed block.

```
        return a  >  b ? a: b;
}
```

extern is the storage-class specifier and **int** is the type specifier; `max(int a, int b)` is the function declarator; and

```
{ return a  >  b ? a: b; }
```

is the function body.

16 EXAMPLE 2 To pass one function to another, one can say

```
int f(void);
/* ... */
g(f);
```

Then the definition of `g` can read

```
void g(int (*funcp)(void))
{
      /* ... */
      (*funcp)(); /* or funcp(); ...*/
}
```

or, equivalently,

```
void g(int func(void))
{
      /* ... */
      func(); /* or (*func)(); ...*/
}
```

6.9.3 External object definitions

Semantics

1 If the declaration of an identifier for an object has file scope and an initializer, or has file scope and storage-class specifier **thread_local**, the declaration is an external definition for the identifier.

2 A declaration of an identifier for an object that has file scope without an initializer, and without the storage-class specifier **extern** or **thread_local**, constitutes a *tentative definition*. If a translation unit contains one or more tentative definitions for an identifier, and the translation unit contains no external definition for that identifier, then the behavior is exactly as if the translation unit contains a file scope declaration of that identifier with an empty initializer and a type determined as follows:

— if the composite type as of the end of the translation unit is an array of unknown size, then an array of size one with the composite element type;

— otherwise, the composite type at the end of the translation unit.

3 If the declaration of an identifier for an object is a tentative definition and has internal linkage, the declared type shall not be an incomplete type.

4 EXAMPLE 1

```
int i1 = 1;         // definition, external linkage
static int i2 = 2;  // definition, internal linkage
extern int i3 = 3;  // definition, external linkage
int i4;             // tentative definition, external linkage
static int i5;      // tentative definition, internal linkage

int i1;             // valid tentative definition, refers to previous
int i2;             // 6.2.2 renders undefined, linkage disagreement
int i3;             // valid tentative definition, refers to previous
int i4;             // valid tentative definition, refers to previous
int i5;             // 6.2.2 renders undefined, linkage disagreement

extern int i1;      // refers to previous, whose linkage is external
extern int i2;      // refers to previous, whose linkage is internal
extern int i3;      // refers to previous, whose linkage is external
extern int i4;      // refers to previous, whose linkage is external
extern int i5;      // refers to previous, whose linkage is internal
```

5 EXAMPLE 2 If at the end of the translation unit containing

```
int i[];
```

the array `i` still has incomplete type, the implicit initializer causes it to have one element, which is set to zero on program startup.

6.10 Preprocessing directives

6.10.1 General

Syntax

1 *preprocessing-file:*
group$_{\text{opt}}$

group:
group-part
group group-part

group-part:
if-section
control-line
text-line
*non-directive*

if-section:
if-group elif-groups$_{\text{opt}}$ *else-group*$_{\text{opt}}$ *endif-line*

if-group:
 # if *constant-expression new-line group*$_{\text{opt}}$
 # ifdef *identifier new-line group*$_{\text{opt}}$
 # ifndef *identifier new-line group*$_{\text{opt}}$

elif-groups:
elif-group
elif-groups elif-group

elif-group:
 # elif *constant-expression new-line group*$_{\text{opt}}$
 # elifdef *identifier new-line group*$_{\text{opt}}$
 # elifndef *identifier new-line group*$_{\text{opt}}$

else-group:
 # else *new-line group*$_{\text{opt}}$

endif-line:
 # endif *new-line*

control-line:
 # include *pp-tokens new-line*
 # embed *pp-tokens new-line*
 # define *identifier replacement-list new-line*
 # define *identifier lparen identifier-list*$_{\text{opt}}$ **)** *replacement-list new-line*
 # define *identifier lparen* **...** **)** *replacement-list new-line*
 # define *identifier lparen identifier-list* **, ...** **)** *replacement-list new-line*
 # undef *identifier new-line*
 # line *pp-tokens new-line*
 # error *pp-tokens*$_{\text{opt}}$ *new-line*
 # warning *pp-tokens*$_{\text{opt}}$ *new-line*
 # pragma *pp-tokens*$_{\text{opt}}$ *new-line*
*new-line*

text-line:
pp-tokens$_{\text{opt}}$ *new-line*

non-directive:
pp-tokens new-line

lparen:
a **(** character not immediately preceded by white space

replacement-list:
pp-tokens$_{\text{opt}}$

pp-tokens:
preprocessing-token
pp-tokens preprocessing-token

new-line:
the new-line character

identifier-list:
identifier
identifier-list **,** *identifier*

pp-parameter:
pp-parameter-name pp-parameter-clause$_{\text{opt}}$

pp-parameter-name:
pp-standard-parameter
pp-prefixed-parameter

pp-standard-parameter:
identifier

pp-prefixed-parameter:
identifier **::** *identifier*

pp-parameter-clause:
(*pp-balanced-token-sequence*$_{\text{opt}}$ **)**

pp-balanced-token-sequence:
pp-balanced-token
pp-balanced-token-sequence pp-balanced-token

pp-balanced-token:
(*pp-balanced-token-sequence*$_{\text{opt}}$ **)**
[*pp-balanced-token-sequence*$_{\text{opt}}$ **]**
{ *pp-balanced-token-sequence*$_{\text{opt}}$ **}**
any pp-token other than a parenthesis, a bracket, or a brace

embed-parameter-sequence:
pp-parameter
embed-parameter-sequence pp-parameter

Description

2 A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: The first token in the sequence is a **#** preprocessing token that (at the start of translation phase 4) is either the first character in the source file (optionally after white space containing no new-line characters) or that follows white space containing at least one new-line character. The last

token in the sequence is the first new-line character that follows the first token in the sequence.[195] A new-line character ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro.

3 A text line shall not begin with a **#** preprocessing token. A non-directive shall not begin with any of the directive names appearing in the syntax.

4 Some preprocessing directives take additional information using preprocessor parameters. A *preprocessing parameter* (pp-parameter) shall be either a *preprocessor prefixed parameter* (identified by a pp-prefixed-parameter, for implementation-defined preprocessor parameters) or a *preprocessor standard parameter* (identified with a pp-standard-parameter, for pp-parameters specified by this document).

5 In all aspects, a preprocessor standard parameter specified by this document as an identifier **pp_param** and an identifier of the form **__pp_param__** shall behave the same when used as a preprocessor parameter, except for the spelling.

6 EXAMPLE 1 Thus, the preprocessor parameters on the two binary resource inclusion directives (6.10.4):

```
#embed "boop.h" limit(5)
#embed "boop.h" __limit__(5)
```

behave the same, and can be freely interchanged. Implementations are encouraged to behave similarly for preprocessor parameters (including preprocessor prefixed parameters) they provide.

7 When in a group that is skipped (6.10.2), the directive syntax is relaxed to allow any sequence of preprocessing tokens to occur between the directive name and the following new-line character.

Constraints

8 The only white-space characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the introducing **#** preprocessing token through just before the terminating new-line character) are space and horizontal-tab (including spaces that have replaced comments or possibly other white-space characters in translation phase 3).

9 A preprocessor parameter shall be either a preprocessor standard parameter, or an implementation-defined preprocessor prefixed parameter.[196]

Semantics

10 The implementation can process and skip sections of source files conditionally, include other source files, and replace macros. These capabilities are called *preprocessing*, because conceptually they occur before translation of the resulting translation unit.

11 The preprocessing tokens within a preprocessing directive are not subject to macro expansion unless otherwise stated.

12 EXAMPLE 2 In:

```
#define EMPTY
EMPTY # include <file.h>
```

the sequence of preprocessing tokens on the second line is *not* a preprocessing directive, because it does not begin with a **#** at the start of translation phase 4 (5.1.1.2), even though it will do so after the macro `EMPTY` has been replaced.

13 The execution of a non-directive preprocessing directive results in undefined behavior.

[195] Thus, preprocessing directives are commonly called "lines". These "lines" have no other syntactic significance, as all white space is equivalent except in certain situations during preprocessing (see the **#** character string literal creation operator in 6.10.5.2, for example).

[196] An unrecognized preprocessor prefixed parameter is a constraint violation, except within has_embed expressions (6.10.2).

6.10.2 Conditional inclusion

Syntax

1 *defined-macro-expression*:
defined *identifier*
defined (***identifier* **)
h-preprocessing-token:
any *preprocessing-token* other than **>**
h-pp-tokens:
h-preprocessing-token
h-pp-tokens h-preprocessing-token
header-name-tokens:
string-literal
< *h-pp-tokens* **>**
has-include-expression:
__has_include (*header-name* **)**
__has_include (*header-name-tokens* **)**
has-embed-expression:
__has_embed (*header-name embed-parameter-sequence*$_{\text{opt}}$ **)**
__has_embed (*header-name-tokens pp-balanced-token-sequence*$_{\text{opt}}$ **)**
has-c-attribute-express:
__has_c_attribute (*pp-tokens* **)**

2 The **#if** and **#elif** directives are collectively known as the *conditional expression inclusion preprocessing directives*. The conditional expression inclusion preprocessing directives, **#ifdef**, **#ifndef**, **#elifdef**, and **#elifndef** directives are collectively known as the *conditional inclusion preprocessing directives*.

Constraints

3 The expression that controls conditional inclusion shall be an integer constant expression except that: identifiers (including those lexically identical to keywords) are interpreted as described subsequently in this subclause[197] and it may contain zero or more defined macro expressions, has_include expressions, has_embed expressions, and/or has_c_attribute expressions as unary operator expressions.

4 A defined macro expression evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has been the subject of a **#define** preprocessing directive without an intervening **#undef** directive with the same subject identifier), 0 if it is not.

5 The second form of the has_include expression and has_embed expression is considered only if the first form does not match, in which case the preprocessing tokens are processed just as in normal text.

6 The header or source file identified by the parenthesized preprocessing token sequence in each contained has_include expression is searched for as if that preprocessing token were the pp-tokens in a **#include** directive, except that no further macro expansion is performed. Such a directive shall satisfy the syntactic requirements of a **#include** directive. The has_include expression evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

7 The resource (6.10.4) identified by the header-name preprocessing token sequence in each contained has_embed expression is searched for as if those preprocessing token were the pp-tokens in a **#embed** directive, except that no further macro expansion is performed. Such a directive shall satisfy the syntactic requirements of a **#embed** directive. The has_embed expression evaluates to the same value as the following predefined macros (6.10.10.2):

— **__STDC_EMBED_NOT_FOUND__**, if the search fails or if any of the embed parameters in the embed parameter sequence specified are not supported by the implementation for the **#embed**

[197] Because the controlling constant expression is evaluated during translation phase 4, all identifiers either are or are not macro names — there simply are no keywords, enumeration constants, etc.

directive; or,

— **__STDC_EMBED_FOUND__**, if the search for the resource succeeds and all embed parameters in the embed parameter sequence specified are supported by the implementation for the **#embed** directive and the resource is not empty; or,

— **__STDC_EMBED_EMPTY__**, if the search for the resource succeeds and all embed parameters in the embed parameter sequence specified are supported by the implementation for the **#embed** directive and the resource is empty.

8 NOTE 1 Unrecognized preprocessor prefixed parameters in has_embed expressions are not a constraint violation and instead cause the expression to be evaluated to 0, as specified previously.

9 Each has_c_attribute expression is replaced by a nonzero pp-number matching the form of an integer constant if the implementation supports an attribute with the name specified by interpreting the pp-tokens as an attribute token, and by 0 otherwise. The pp-tokens shall match the form of an attribute token.

10 Each preprocessing token that remains (in the list of preprocessing tokens that will become the controlling expression) after all macro replacements have occurred shall be in the lexical form of a token (6.4).

Semantics

11 The **#ifdef**, **#ifndef**, **#elifdef**, and **#elifndef** directives, and the **defined** conditional inclusion operator, shall treat **__has_include**, **__has_embed** and **__has_c_attribute** as if they were the name of defined macros. The identifiers **__has_include**, **__has_embed**, and **__has_c_attribute** shall not appear in any context not mentioned in this subclause.

12 Preprocessing directives of the forms

 # if *constant-expression new-line* $group_{\text{opt}}$
 # elif *constant-expression new-line* $group_{\text{opt}}$

check whether the controlling constant expression evaluates to nonzero.

13 Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the **defined** unary operator), just as in normal text. If the token **defined** is generated as a result of this replacement process or use of the **defined** unary operator does not match one of the two specified forms prior to macro replacement, the behavior is undefined. After all replacements due to macro expansion and evaluations of defined macro expressions, has_include expressions, has_embed expressions, and has_c_attribute expressions have been performed, all remaining identifiers other than **true** (including those lexically identical to keywords such as **false**) are replaced with the pp-number `0`, **true** is replaced with pp-number `1`, and then each preprocessing token is converted into a token. The resulting tokens compose the controlling constant expression which is evaluated according to the rules of 6.6. For the purposes of this token conversion and evaluation, all signed integer types and all unsigned integer types act as if they have the same representation as, respectively, the types **intmax_t** and **uintmax_t** defined in the header `<stdint.h>`. This includes interpreting character constants, which may involve converting escape sequences into execution character set members. Whether the numeric value for these character constants matches the value obtained when an identical character constant occurs in an expression (other than within a **#if** or **#elif** directive) is implementation-defined. Whether a single-character character constant may have a negative value is implementation-defined.

14 NOTE 2 On an implementation where **INT_MAX** is `0x7FFF` and **UINT_MAX** is `0xFFFF`, the constant `0x8000` is signed and positive within a **#if** expression even though it would be unsigned in translation phase 7 (5.1.1.2).

15 NOTE 3 The constant expression in the following **#if** directive and **if** statement is not guaranteed to evaluate to the same value in these two contexts.

```
#if 'z' - 'a' == 25
if ('z' - 'a' == 25)
```


16 Preprocessing directives of the forms

 # ifdef *identifier new-line* $group_{opt}$
 # ifndef *identifier new-line* $group_{opt}$
 # elifdef *identifier new-line* $group_{opt}$
 # elifndef *identifier new-line* $group_{opt}$

check whether the identifier is or is not currently defined as a macro name. Their conditions are equivalent to **#if defined** *identifier*, **#if !defined** *identifier*, **#elif defined** *identifier*, and **#elif !defined** *identifier* respectively.

17 Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the name that determines the directive to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed; any following groups are skipped and their controlling directives are processed as if they were in a group that is skipped. If none of the conditions evaluates to true, and there is a **#else** directive, the group controlled by the **#else** is processed; lacking a **#else** directive, all the groups until the **#endif** are skipped.[198)]

18 EXAMPLE 1 This demonstrates a way to include a header file only if it is available.

```
#if __has_include(<optional.h>)
#     include <optional.h>
#     define have_optional 1
#elif __has_include(<experimental/optional.h>)
#     include <experimental/optional.h>
#     define have_optional 1
#     define have_experimental_optional 1
#endif
#ifndef have_optional
#     define have_optional 0
#endif
```

19 EXAMPLE 2

```
/* Fallback for compilers not yet implementing this feature. */
#ifndef __has_c_attribute
#define __has_c_attribute(x) 0
#endif /* __has_c_attribute */

#if __has_c_attribute(fallthrough)
/* Standard attribute is available, use it. */
#define FALLTHROUGH [[fallthrough]]
#elif __has_c_attribute(vendor::fallthrough)
/* Vendor attribute is available, use it. */
#define FALLTHROUGH [[vendor::fallthrough]]
#else
/* Fallback implementation. */
#define FALLTHROUGH
#endif
```

20 EXAMPLE 3

```
#ifdef __STDC__
#define TITLE "ISO C Compilation"
#elifndef __cplusplus
#define TITLE "Non-ISO C Compilation"
#else
```

[198)] As indicated by the syntax, no preprocessing tokens are allowed to follow a **#else** or **#endif** directive before the terminating new-line character. However, comments can appear anywhere in a source file, including within a preprocessing directive.

```
/* C++ */
#define TITLE "C++ Compilation"
#endif
```

21 EXAMPLE 4 A combination of **__FILE__** (6.10.10.2) and **__has_embed** could be used to check for support of specific implementation extensions for the **#embed** (6.10.4) directive's parameters.

```
#if __has_embed(__FILE__ ext::token(0xB055))
#define DESCRIPTION "Supports extended token embed parameter"
#else
#define DESCRIPTION "Does not support extended token embed parameter"
#endif
```

22 EXAMPLE 5 The following snippet uses **__has_embed** to check for support of a specific implementation-defined embed parameter, and otherwise uses standard behavior to produce the same effect.

```
void parse_into_s(short* ptr, unsigned char* ptr_bytes, unsigned long long size);

int main () {
#if __has_embed ("bits.bin" ds9000::element_type(short))
      /* Implementation extension: create short integers from the */
      /* translation environment resource into */
      /*  a sequence of integer constants */
      short meow[] = {
#embed "bits.bin" ds9000::element_type(short)
      };
#elif __has_embed ("bits.bin")
      /* no support for implementation-specific */
      /* ds9000::element_type(short) parameter */
      const unsigned char meow_bytes[] = {
#embed "bits.bin"
      };
      short meow[sizeof(meow_bytes) / sizeof(short)] = {};
      /* parse meow_bytes into short values by-hand! */
      parse_into_s(meow, meow_bytes, sizeof(meow_bytes));
#else
#error "cannot find bits.bin resource"
#endif
      return (int)(meow[0] + meow[(sizeof(meow) / sizeof(*meow)) - 1]);
}
```

23 EXAMPLE 6 If the search for the resource is successful, this resource is always considered empty due to the **limit**(0) embed parameter, including in **__has_embed** expressions.

```
int main () {
#if __has_embed(<infinite-resource> limit(0)) == 2
      // if <infinite-resource> exists, this
      // token sequence is always taken.
      return 0;
#else
      // the 'infinite-resource' resource does not exist
      #error "The resource does not exist"
#endif
}
```

Forward references: macro replacement (6.10.5), source file inclusion (6.10.3), mandatory macros (6.10.10.2), largest integer types (7.22.1.5).

6.10.3 Source file inclusion

Constraints

1 A **#include** directive shall identify a header or source file that can be processed by the implementation.

Semantics

2 A preprocessing directive of the form

 # include < *h-char-sequence* **>** *new-line*

searches a sequence of implementation-defined places for a header identified uniquely by the specified sequence between the < and > delimiters, and causes the replacement of that directive by the entire contents of the header. How the places are specified or the header identified is implementation-defined.

3 A preprocessing directive of the form

 # include " *q-char-sequence* **"** *new-line*

causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the " delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

 # include < *h-char-sequence* **>** *new-line*

with the identical contained sequence (including > characters, if any) from the original directive.

4 A preprocessing directive of the form

 # include *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **include** in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms.[199] The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single header name preprocessing token is implementation-defined.

5 The implementation shall provide unique mappings for sequences consisting of one or more nondigits or digits (6.4.2.1) followed by a period (.) and a single nondigit. The first character shall not be a digit. The implementation may ignore distinctions of alphabetical case and restrict the mapping to eight significant characters before the period.

6 A **#include** preprocessing directive may appear in a source file that has been read because of a **#include** directive in another file, up to an implementation-defined nesting limit (see 5.2.5.2).

7 EXAMPLE 1 The most common uses of **#include** preprocessing directives are as in the following:

```
#include <stdio.h>
#include "myprog.h"
```

8 EXAMPLE 2 This illustrates macro-replaced **#include** directives:

```
#if VERSION == 1
      #define INCFILE  "vers1.h"
#elif VERSION == 2
      #define INCFILE  "vers2.h"   // and so on
#else
      #define INCFILE  "versN.h"
```

[199] Note that adjacent string literals are not concatenated into a single string literal (see the translation phases in 5.1.1.2); thus, an expansion that results in two string literals is an invalid directive.

```
#endif
#include INCFILE
```

Forward references: macro replacement (6.10.5).

6.10.4 Binary resource inclusion

6.10.4.1 `#embed` preprocessing directive

Description

1 A *resource* is a source of data accessible from the translation environment. An *embed parameter* is a single preprocessor parameter in the embed parameter sequence. It has an *implementation resource width*, which is the implementation-defined size in bits of the located resource. It also has a *resource width*, which is either:

— the number of bits as computed from the optionally-provided **`limit`** embed parameter (6.10.4.2), if present; or,

— the implementation resource width.

2 An *embed parameter sequence* is a whitespace-delimited list of preprocessor parameters which may modify the result of the replacement for the **`#embed`** preprocessing directive.

Constraints

3 An **`#embed`** directive shall identify a resource that can be processed by the implementation as a binary data sequence given the provided embed parameters.

4 Embed parameters not specified in this document shall be implementation-defined. Implementation-defined embed parameters may change the subsequently-defined semantics of the directive; otherwise, **`#embed`** directives which do not contain implementation-defined embed parameters shall behave as described in this document.

5 A resource is considered empty when its resource width is zero.

6 Let *embed element width* be either:

— an integer constant expression greater than zero determined by an implementation-defined embed parameter; or,

— **`CHAR_BIT`** (5.2.5.3.2).

The result of $(resource\ width)\ \%\ (embed\ element\ width)$ shall be zero.[200)]

Semantics

7 The expansion of a **`#embed`** directive is a token sequence formed from the list of integer constant expressions described later in this subclause. The group of tokens for each integer constant expression in the list is separated in the token sequence from the group of tokens for the previous integer constant expression in the list by a comma. The sequence neither begins nor ends in a comma. If the list of integer constant expressions is empty, the token sequence is empty. The directive is replaced by its expansion and, with the presence of certain embed parameters, additional or replacement token sequences.

8 A preprocessing directive of the form

`#` **`embed`** **`<`** *h-char-sequence* **`>`** $embed\text{-}parameter\text{-}sequence_{opt}$ *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the < and >. The search for the named resource is done in an implementation-defined manner.

[200)] This constraint helps ensure data is neither filled with padding values nor truncated in a given environment, and helps ensure the data is portable with respect to usages of **`memcpy`** (7.26.2.1) with character type arrays initialized from the data.

9 A preprocessing directive of the form

 # embed " *q-char-sequence* **"** *embed-parameter-sequence*$_{\text{opt}}$ *new-line*

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the " delimiters. The search for the named resource is done in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

 # embed < *h-char-sequence* **>** *embed-parameter-sequence*$_{\text{opt}}$ *new-line*

with the identical contained q-char-sequence (including > characters, if any) from the original directive.

10 Either form of the **#embed** directive specified previously behaves as specified later in this subclause. The values of the integer constant expressions in the expanded sequence are determined by an implementation-defined mapping of the resource's data. Each integer constant expression's value is in the range from 0 to $(2^{\textit{embed element width}}) - 1$, inclusive.[201] If:

— the list of integer constant expressions is used to initialize an array of a type compatible with **unsigned char**, or compatible with **char** if **char** cannot hold negative values; and,

— the embed element width is equal to **CHAR_BIT** (5.2.5.3.2),

then the contents of the initialized elements of the array are as-if the resource's binary data is **fread** (7.23.8.1) into the array at translation time.

11 A preprocessing directive of the form

 # embed *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **embed** in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms.[202] The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single resource name preprocessing token is implementation-defined.

12 An embed parameter with a preprocessor parameter token that is one of the following is a *standard embed parameter*:

limit **prefix** **suffix** **if_empty**

The significance of these standard embed parameters is specified later in this subclause.

Recommended practice

13 The **#embed** directive is meant to translate binary data in a resource to a sequence of integer constant expressions in a way that preserves the value of the resource's bit stream where possible.

14 A mechanism similar to, but distinct from, the implementation-defined search paths used for source file inclusion (6.10.3) is encouraged.

15 Implementations should take into account translation-time bit and byte orders as well as execution-time bit and byte orders to more appropriately represent the resource's binary data from the directive. This maximizes the chance that, if the resource referenced at translation time through the **#embed** directive is the same one accessed through execution-time means, the data that is e.g. **fread** or similar into contiguous storage will compare bit-for-bit equal to an array of character type initialized from an **#embed** directive's expanded contents.

16 EXAMPLE 1 Placing a small image resource.

[201] For example, an embed element width of 8 will yield a range of values from 0 to 255, inclusive.

[202] Note that adjacent string literals are not concatenated into a single string literal (see the translation phases in 5.1.1.2); thus, an expansion that results in two string literals is an invalid directive.

```
#include <stddef.h>

void have_you_any_wool(const unsigned char*, size_t);

int main (int, char*[]) {
      static const unsigned char baa_baa[] = {
#embed "black_sheep.ico"
      };

      have_you_any_wool(baa_baa, sizeof(baa_baa));

      return 0;
}
```

17 EXAMPLE 2 This snippet:

```
int main (int, char*[]) {
      static const unsigned char coefficients[] = {
#embed "only_8_bits.bin" // potential constraint violation
      };

      return 0;
}
```

may violate the constraint that $(resource\ width)\ \%\ (embed\ element\ width)$ is 0. The 8 bits may potentially not be evenly divisible by the embed element width (e.g. on a system where **CHAR_BIT** is 16). Issuing a diagnostic in this case may aid in portability by calling attention to potentially incompatible expectations between implementations and their resources.

18 EXAMPLE 3 Initialization of non-arrays.

```
int main () {
      /* Braces may be kept or elided as per normal initialization rules */
      int i = {
#embed "i.dat"
      }; /* valid if i.dat produces 1 value,
            i value is [0, 2^(embed element width)) */
      int i2 =
#embed "i.dat"
      ; /* valid if i.dat produces 1 value,
           i2 value is [0, 2^(embed element width)) */
      struct s {
            double a, b, c;
            struct { double e, f, g; };
            double h, i, j;
      };
      struct s x = {
      /* initializes each element in order according to initialization
      rules with comma-separated list of integer constant expressions
      inside of braces */
#embed "s.dat"
      };
      return 0;
}
```

Non-array types can still be initialized since the directive produces a comma-delimited list of integer constant expressions, a single integer constant expression, or nothing.

19 EXAMPLE 4 Equivalency of bit sequence and bit order between a translation-time read and an execution-time read of the same resource/file.

```
#include <string.h>
```



```
#include <stddef.h>
#include <stdio.h>

int main(void) {
      static const unsigned char embed_data[] = {
#embed <data.dat>
      };

      const size_t f_size = sizeof(embed_data);
      unsigned char f_data[f_size];
      FILE* f_source = fopen("data.dat", "rb");
      if (f_source == nullptr)
            return 1;
      char* f_ptr = (char*)&f_data[0];
      if (fread(f_ptr, 1, f_size, f_source) != f_size) {
            fclose(f_source);
            return 1;
      }
      fclose(f_source);

      int is_same = memcmp(&embed_data[0], f_ptr, f_size);
      // if both operations refers to the same resource/file at
      // execution time and translation time, "is_same" should be 0
      return is_same == 0 ? 0 : 1;
}
```

6.10.4.2 `limit` parameter

Constraints

1 The **`limit`** standard embed parameter may appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*constant-expression* **)**

and shall be an integer constant expression. The integer constant expression shall not evaluate to a value less than 0.

2 The token **`defined`** shall not appear within the constant expression.

Semantics

3 The embed parameter with a preprocessor parameter token limit denotes a balanced preprocessing token sequence that will be used to compute the resource width. Independently of any macro replacement done previously (e.g. when matching the form of **`#embed`**), the constant expression is evaluated after the balanced preprocessing token sequence is processed as in normal text, using the rules specified for conditional inclusion (6.10.2), with the exception that any defined macro expressions are not permitted.

4 The resource width is:

— 0, if the integer constant expression evaluates to 0; or,

— the implementation resource width if it is less than the embed element width multiplied by the integer constant expression; or,

— the embed element width multiplied by the integer constant expression, if it is less than or equal to the implementation resource width.

5 EXAMPLE 1 Checking the first 4 elements of a sound resource.

```
#include <assert.h>

int main (int, char*[]) {
```

```
      static const char sound_signature[] = {
#embed <sdk/jump.wav> limit(2+2)
      };
      static_assert((sizeof(sound_signature) / sizeof(*sound_signature)) == 4,
            "There should only be 4 elements in this array.");

      // verify PCM WAV resource
      assert(sound_signature[0] == 'R');
      assert(sound_signature[1] == 'I');
      assert(sound_signature[2] == 'F');
      assert(sound_signature[3] == 'F');
      assert(sizeof(sound_signature) == 4);

      return 0;
}
```

6 EXAMPLE 2 Similar to a previous example, except it illustrates macro expansion specifically done for the **limit**(...) parameter.

```
#include <assert.h>

#define TWO_PLUS_TWO 2+2

int main (int, char*[]) {
      const char sound_signature[] = {
      /* the token sequence within the parentheses
      for the "limit" parameter undergoes macro
      expansion, at least once, resulting in
#embed <sdk/jump.wav> limit(2+2)
      */
#embed <sdk/jump.wav> limit(TWO_PLUS_TWO)
      };
      static_assert((sizeof(sound_signature) / sizeof(*sound_signature)) == 4,
            "There should only be 4 elements in this array.");

      // verify PCM WAV resource
      assert(sound_signature[0] == 'R');
      assert(sound_signature[1] == 'I');
      assert(sound_signature[2] == 'F');
      assert(sound_signature[3] == 'F');
      assert(sizeof(sound_signature) == 4);

      return 0;
}
```

7 EXAMPLE 3 A potential constraint violation from a resource that may not have enough information in an environment that has a **CHAR_BIT** greater than 24.

```
int main (int, char*[]) {
      const unsigned char arr[] = {
#embed "24_bits.bin" limit(1) // may be a constraint violation
      };

      return 0;
}
```

8 EXAMPLE 4 Resources interfacing with certain implementations may have an infinite stream of data, such as the </owo/uwurandom> resource used in the following snippet:

```
int main (int, char*[]) {
      const unsigned char arr[] = {
```



```
      #embed </owo/uwurandom> limit(513)
            };

            return 0;
      }
```

The **limit** parameter may help process only a portion of that information and prevent exhaustion of an implementation's internal resources when processing such data.

6.10.4.3 **suffix** parameter

Constraints

1 The **suffix** standard embed parameter may appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*pp-balanced-token-sequence*$_{\text{opt}}$ **)**

Semantics

2 The embed parameter with a preprocessing parameter token **suffix** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will be placed immediately after the result of the associated **#embed** directive's expansion.

3 If the resource is empty, then **suffix** has no effect and is ignored.

4 EXAMPLE Extra elements added to array initializer.

```
      #include <string.h>

      #ifndef SHADER_TARGET
      #define SHADER_TARGET "edith-impl.glsl"
      #endif

      extern char* null_term_shader_data;

      void fill_in_data () {
            const char internal_data[] = {
      #embed SHADER_TARGET \
                  suffix(,)
                  0
            };

            strcpy(null_term_shader_data, internal_data);
      }
```

6.10.4.4 **prefix** parameter

Constraints

1 The **prefix** standard embed parameter may appear zero times or one time in the embed parameter sequence. Its preprocessor parameter clause shall be present and have the form:

(*pp-balanced-token-sequence*$_{\text{opt}}$ **)**

Semantics

2 The embed parameter with a preprocessor parameter token **prefix** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will be placed immediately before the result of the associated **#embed** directive's expansion, if any.

3 If the resource is empty, then **prefix** has no effect and is ignored.

4 EXAMPLE A null-terminated character array with prefixed and suffixed additional tokens when the resource is not empty, providing null termination and a byte order mark.

```
      #include <assert.h>
      #include <string.h>
```

```
#ifndef SHADER_TARGET
#define SHADER_TARGET "ches.glsl"
#endif

extern char* merp;

void init_data () {
      const char whl[] = {
#embed SHADER_TARGET                                    \
            prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
            suffix(,)
            0
      };
      // always null terminated,
      // contains BOM if not-empty
      int is_good = (sizeof(whl) == 1 && whl[0] == '\0')
      || (whl[0] == '\xEF' && whl[1] == '\xBB'
      && whl[2] == '\xBF' && whl[sizeof(whl) - 1] == '\0');
      assert(is_good);
      strcpy(merp, whl);
}
```

6.10.4.5 `if_empty` parameter

Constraints

1 The **`if_empty`** standard embed parameter may appear zero times or one time in the embed parameter sequence. Its preprocessor argument clause shall be present and have the form:

(*pp-balanced-token-sequence*$_{\text{opt}}$ **)**

Semantics

2 The embed parameter with a preprocessing parameter token **`if_empty`** denotes a balanced preprocessing token sequence within its preprocessor argument clause that will replace the **`#embed`** directive entirely.

If the resource is not empty, then **`if_empty`** has no effect and is ignored.

3 EXAMPLE 1 If the search for the resource is successful, this resource is always considered empty due to the **`limit`**`(0)` embed parameter. This program always returns 0, even if the resource is searched for and found successfully by the implementation and has an implementation resource width greater than 0.

```
int main () {
      return
#embed <some_resource> limit(0) prefix(1) if_empty(0)
      ;
      // becomes:
      // return 0;
}
```

4 EXAMPLE 2 An example similar to using the suffix **`embed`** parameter, but changed slightly.

```
#include <string.h>

#ifndef SHADER_TARGET
#define SHADER_TARGET "edith-impl.glsl"
#endif

extern char* null_term_shader_data;

void fill_in_data () {
      const char internal_data[] = {
```

```
      #embed SHADER_TARGET   \
                  suffix(, 0) \
                  if_empty(0)
            };

            strcpy(null_term_shader_data, internal_data);
      }
```

EXAMPLE 3 This resource is considered empty due to the **limit**(0) embed parameter, meaning an **if_empty** expression replaces the directive as specified previously. A constraint is still violated if the search for the resource is unsuccessful.

```
      int main () {
            return
                  #embed <infinite-resource> limit(0) if_empty(45540)
            ;
      }
```

becomes:

```
      int main () {
            return 45540;
      }
```

6.10.5 Macro replacement

Constraints

1 Two replacement lists are identical if and only if the preprocessing tokens in both have the same number, ordering, spelling, and white-space separation, where all white-space separations are considered identical.

2 An identifier currently defined as an object-like macro shall not be redefined by another **#define** preprocessing directive unless the second definition is an object-like macro definition and the two replacement lists are identical. Likewise, an identifier currently defined as a function-like macro shall not be redefined by another **#define** preprocessing directive unless the second definition is a function-like macro definition that has the same number and spelling of parameters, and the two replacement lists are identical.

3 There shall be white space between the identifier and the replacement list in the definition of an object-like macro.

4 If the identifier-list in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be at least as many arguments in the invocation as there are parameters in the macro definition (excluding the `...`). There shall exist a `)` preprocessing token that terminates the invocation.

5 The identifiers **__VA_ARGS__** and **__VA_OPT__** shall occur only in the replacement-list of a function-like macro that uses the ellipsis notation in the parameters.

6 A parameter identifier in a function-like macro shall be uniquely declared within its scope.

Semantics

7 The identifier immediately following the **define** is called the *macro name*. There is one name space for macro names. Any white-space characters preceding or following the replacement list of preprocessing tokens are not considered part of the replacement list for either form of macro.

8 If a **#** preprocessing token, followed by an identifier, occurs lexically at the point at which a preprocessing directive could begin, the identifier is not subject to macro replacement.

9 A preprocessing directive of the form

 # define *identifier replacement-list new-line*

defines an *object-like macro* that causes each subsequent instance of the macro name[203] to be replaced by the replacement list of preprocessing tokens that constitute the remainder of the directive. The replacement list is then rescanned for more macro names as specified later in this subclause.

10 A preprocessing directive of the form

**define** *identifier* *lparen* *identifier-list*$_{\text{opt}}$ **)** *replacement-list* *new-line*
**define** *identifier* *lparen* **...** **)** *replacement-list* *new-line*
**define** *identifier* *lparen* *identifier-list* **,** **...** **)** *replacement-list* *new-line*

defines a *function-like macro* with parameters, whose use is similar syntactically to a function call. The parameters are specified by the optional list of identifiers, whose scope extends from their declaration in the identifier list until the new-line character that terminates the **#define** preprocessing directive. Each subsequent instance of the function-like macro name followed by a (as the next preprocessing token introduces the sequence of preprocessing tokens that is replaced by the replacement list in the definition (an invocation of the macro). The replaced sequence of preprocessing tokens is terminated by the matching) preprocessing token, skipping intervening matched pairs of left and right parenthesis preprocessing tokens. Within the sequence of preprocessing tokens making up an invocation of a function-like macro, new-line is considered a normal white-space character.

11 The sequence of preprocessing tokens bounded by the outside-most matching parentheses forms the list of arguments for the function-like macro. The individual arguments within the list are separated by comma preprocessing tokens, but comma preprocessing tokens between matching inner parentheses do not separate arguments. If there are sequences of preprocessing tokens within the list of arguments that would otherwise act as preprocessing directives,[204] the behavior is undefined.

12 If there is a ... in the identifier-list in the macro definition, then the trailing arguments (if any), including any separating comma preprocessing tokens, are merged to form a single item: the *variable arguments*. The number of arguments so combined is such that, following merger, the number of arguments is one more than the number of parameters in the macro definition (excluding the ...), except that if there are as many arguments as named parameters, the macro invocation behaves as if a comma token has been appended to the argument list such that variable arguments are formed that contain no pp-tokens.

6.10.5.1 Argument substitution

Syntax

1 *va-opt-replacement*:

__VA_OPT__ **(** *pp-tokens*$_{\text{opt}}$ **)**

Description

2 Argument substitution is a process during macro expansion in which identifiers corresponding to the parameters of the macro definition and the special constructs **__VA_ARGS__** and **__VA_OPT__** are replaced with token sequences from the arguments of the macro invocation and possibly of the argument of the feature **__VA_OPT__**. The latter process allows to control a substitute token sequence that is only expanded if the argument list that corresponds to a trailing ... of the parameter list is present and has a non-empty substitution.

Constraints

3 The identifier **__VA_OPT__** shall always occur as part of the preprocessing token sequence va-opt-replacement; its closing) is determined by skipping intervening pairs of matching left and right parentheses in its pp-tokens. The pp-tokens of a va-opt-replacement shall not contain **__VA_OPT__**. The pp-tokens shall form a valid replacement list for the current function-like macro.

[203] Since, by macro-replacement time, all character constants and string literals are preprocessing tokens, not sequences possibly containing identifier-like subsequences (see 5.1.1.2, translation phases), they are never scanned for macro names or parameters.
[204] Despite the name, a non-directive is a preprocessing directive.

Semantics

4 After the arguments for the invocation of a function-like macro have been identified, argument substitution takes place. A va-opt-replacement is treated as if it were a parameter. For each parameter in the replacement list that is neither preceded by a **#** or **##** preprocessing token nor followed by a **##** preprocessing token, the preprocessing tokens naming the parameter are replaced by a token sequence determined as follows:

— If the parameter is of the form va-opt-replacement, the replacement preprocessing tokens are the preprocessing token sequence for the corresponding argument, as specified later in this subclause.

— Otherwise, the replacement preprocessing tokens are the preprocessing tokens of the corresponding argument after all macros contained therein have been expanded. The argument's preprocessing tokens are completely macro replaced before being substituted as if they formed the rest of the preprocessing file with no other preprocessing tokens being available.

5 EXAMPLE 1

```
#define LPAREN() (
#define G(Q) 42
#define F(R, X, ...)  __VA_OPT__(G R X) )
int x = F(LPAREN(), 0, <:-);     // replaced by int x = 42;
```

6 An identifier **__VA_ARGS__** that occurs in the replacement list is treated as if it were a parameter, and the variable arguments form the preprocessing tokens used to replace it.

7 The preprocessing token sequence for the corresponding argument of a va-opt-replacement is defined as follows. If a (hypothetical) substitution of **__VA_ARGS__** as neither an operand of **#** nor **##** consists of no preprocessing tokens, the argument consists of a single placemarker preprocessing token (6.10.5.3, 6.10.5.4). Otherwise, the argument consists of the results of the expansion of the contained pp-tokens as the replacement list of the current function-like macro before removal of placemarker tokens, rescanning, and further replacement.

8 NOTE The placemarker tokens are removed before stringization (6.10.5.2), and can be removed by rescanning and further replacement (6.10.5.4).

9 EXAMPLE 2

```
#define F(...)           f(0 __VA_OPT__(,) __VA_ARGS__)
#define G(X, ...)        f(0, X __VA_OPT__(,) __VA_ARGS__)
#define SDEF(sname, ...) S sname __VA_OPT__(= { __VA_ARGS__ })
#define EMP

F(a, b, c)        // replaced by f(0, a, b, c)
F()               // replaced by f(0)
F(EMP)            // replaced by f(0)

G(a, b, c)        // replaced by f(0, a, b, c)
G(a, )            // replaced by f(0, a)
G(a)              // replaced by f(0, a)

SDEF(foo);        // replaced by S foo;
SDEF(bar, 1, 2);  // replaced by S bar = { 1, 2 };

#define H1(X, ...)    X __VA_OPT__(##) __VA_ARGS__
                  // error: ## on line above
                  // may not appear at the beginning of a replacement
                  // list (6.10.5.3)

#define H2(X, Y, ...) __VA_OPT__(X ## Y,) __VA_ARGS__
```

```
H2(a, b, c, d)    // replaced by ab, c, d

#define H3(X, ...)    #__VA_OPT__(X##X X##X)
H3(, 0)           // replaced by ""

#define H4(X, ...)    __VA_OPT__(a X ## X) ## b
H4(, 1)           // replaced by a b

#define H5A(...)      __VA_OPT__()/**/__VA_OPT__()
#define H5B(X)        a ## X ## b
#define H5C(X)        H5B(X)
H5C(H5A())        // replaced by ab
```

6.10.5.2 The # operator

Constraints

1 Each **#** preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.

Semantics

2 If, in the replacement list, a parameter is immediately preceded by a **#** preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument (excluding placemarker tokens).

3 Let the *stringizing argument* be the preprocessing token sequence for the corresponding argument with placemarker tokens removed. Each occurrence of white space between the stringizing argument's preprocessing tokens becomes a single space character in the character string literal. White space before the first preprocessing token and after the last preprocessing token composing the stringizing argument is deleted. Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of string literals and character constants: a \ character is inserted before each " and \ character of a character constant or string literal (including the delimiting " characters), except that it is implementation-defined whether a \ character is inserted before the \ character beginning a universal character name.

4 If the replacement that results is not a valid character string literal, the behavior is undefined. The character string literal corresponding to an empty stringizing argument is "". The order of evaluation of **#** and **##** operators is unspecified.

6.10.5.3 The ## operator

Constraints

1 A **##** preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.

Semantics

2 If, in the replacement list of a function-like macro, a parameter is immediately preceded or followed by a **##** preprocessing token, the parameter is replaced by the corresponding argument's preprocessing token sequence; however, if an argument consists of no preprocessing tokens, the parameter is replaced by a *placemarker* preprocessing token instead.[205)]

3 For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a **##** preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemarker preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemarker preprocessing token, and concatenation of a placemarker with a non-placemarker preprocessing token results in the non-placemarker preprocessing token. If the result is not a valid preprocessing token, the behavior is undefined. The

[205)] Placemarker preprocessing tokens do not appear in the syntax because they are temporary entities that exist only within translation phase 4 (5.1.1.2).

resulting token is available for further macro replacement. The order of evaluation of **##** operators is unspecified.

4 EXAMPLE In the following fragment:

```
#define hash_hash # ## #
#define mkstr(a) # a
#define in_between(a) mkstr(a)
#define join(c, d) in_between(c hash_hash d)

char p[] = join(x, y); // equivalent to
                       // char p[] = "x ## y";
```

The expansion produces, at various stages:

```
join(x, y)

in_between(x hash_hash y)

in_between(x ## y)

mkstr(x ## y)

"x ## y"
```

In other words, expanding `hash_hash` produces a new token, consisting of two adjacent sharp signs, but this new token is not the **##** operator.

6.10.5.4 Rescanning and further replacement

1 After all parameters in the replacement list have been substituted and **#** and **##** processing has taken place, all placemarker preprocessing tokens are removed. The resulting preprocessing token sequence is then rescanned, along with all subsequent preprocessing tokens of the source file, for more macro names to replace.

2 If the name of the macro being replaced is found during this scan of the replacement list (not including the rest of the source file's preprocessing tokens), it is not replaced. Furthermore, if any nested replacements encounter the name of the macro being replaced, it is not replaced. These nonreplaced macro name preprocessing tokens are no longer available for further replacement even if they are later (re)examined in contexts in which that macro name preprocessing token would otherwise have been replaced.

3 The resulting completely macro-replaced preprocessing token sequence is not processed as a preprocessing directive even if it resembles one, but all pragma unary operator expressions within it are then processed as specified in 6.10.11.

4 EXAMPLE There are cases where it is not clear whether a replacement is nested or not. For example, given the following macro definitions:

```
#define f(a) a*g
#define g(a) f(a)
```

the invocation

```
f(2)(9)
```

could expand to either

```
2*f(9)
```

or

```
2*9*g
```

Strictly conforming programs are not permitted to depend on such unspecified behavior.

6.10.5.5 Scope of macro definitions

1 A macro definition lasts (independent of block structure) until a corresponding **#undef** directive is encountered or (if none is encountered) until the end of the preprocessing translation unit. Macro definitions have no significance after translation phase 4.

2 A preprocessing directive of the form

 # undef *identifier new-line*

causes the specified identifier no longer to be defined as a macro name. It is ignored if the specified identifier is not currently defined as a macro name.

3 EXAMPLE 1 The simplest use of this facility is to define a "manifest constant", as in

```
#define TABSIZE 100

int table[TABSIZE];
```

4 EXAMPLE 2 The following defines a function-like macro whose value is the maximum of its arguments. It has the advantages of working for any compatible types of the arguments and of generating in-line code without the overhead of function calling. It has the disadvantages of evaluating one or the other of its arguments a second time (including side effects) and generating more code than a function if invoked several times. It also cannot have its address taken, as it has none.

```
#define max(a, b) ((a)  >  (b) ? (a): (b))
```

The parentheses ensure that the arguments and the resulting expression are bound properly.

5 EXAMPLE 3 To illustrate the rules for redefinition and reexamination, the sequence

```
#define x      3
#define f(a)   f(x * (a))
#undef  x
#define x      2
#define g      f
#define z      z[0]
#define h      g(~
#define m(a)   a(w)
#define w      0,1
#define t(a)   a
#define p()    int
#define q(x)   x
#define r(x,y) x ## y
#define str(x) # x

f(y+1) + f(f(z)) % t(t(g)(0) + t)(1);
g(x+(3,4)-w) | h 5) & m
      (f)^m(m);
p() i[q()] = { q(1), r(2,3), r(4,), r(,5), r(,) };
char c[2][6] = { str(hello), str() };
```

results in

```
f(2 * (y+1)) + f(2 * (f(2 * (z[0])))) % f(2 * (0)) + t(1);
f(2 * (2+(3,4)-0,1)) | f(2 * (~ 5)) & f(2 * (0,1))^m(0,1);
int i[] = { 1, 23, 4, 5,  };
char c[2][6] = { "hello", "" };
```

6 EXAMPLE 4 To illustrate the rules for creating character string literals and concatenating tokens, the sequence

```
#define str(s)      # s
```



```
#define xstr(s)     str(s)
#define debug(s, t) printf("x" # s "= %d, x" # t "= %s", \
                        x ## s, x ## t)
#define INCFILE(n)  vers ## n
#define glue(a, b)  a ## b
#define xglue(a, b) glue(a, b)
#define HIGHLOW     "hello"
#define LOW         LOW ", world"

debug(1, 2);
fputs(str(strncmp("abc\0d", "abc", '\4') // this goes away
      == 0) str(: @\n), s);
#include xstr(INCFILE(2).h)
glue(HIGH, LOW);
xglue(HIGH, LOW)
```

results in

```
printf("x" "1" "= %d, x" "2" "= %s", x1, x2);
fputs(
  "strncmp(\"abc\\0d\", \"abc\", '\\4') == 0" ": @\n",
  s);
#include "vers2.h"    (after macro replacement, before file access)
"hello";
"hello" ", world"
```

or, after concatenation of the character string literals,

```
printf("x1= %d, x2= %s", x1, x2);
fputs(
  "strncmp(\"abc\\0d\", \"abc\", '\\4') == 0: @\n",
  s);
#include "vers2.h"    (after macro replacement, before file access)
"hello";
"hello, world"
```

Space around the **#** and **##** tokens in the macro definition is optional.

7 EXAMPLE 5 To illustrate the rules for placemarker preprocessing tokens, the sequence

```
#define t(x,y,z) x ## y ## z
int j[] = { t(1,2,3), t(,4,5), t(6,,7), t(8,9,),
            t(10,,), t(,11,), t(,,12), t(,,) };
```

results in

```
int j[] = { 123, 45, 67, 89,
            10, 11, 12,  };
```

8 EXAMPLE 6 To demonstrate the redefinition rules, the following sequence is valid.

```
#define OBJ_LIKE      (1-1)
#define OBJ_LIKE      /* white space */ (1-1) /* other */
#define FUNC_LIKE(a)   ( a )
#define FUNC_LIKE( a ) (      /* note the white space */ \
                        a /* other stuff on this line
                           */)
```

But the following redefinitions of the preceding definitions are invalid:

```
#define OBJ_LIKE    (0)     // different token sequence
```

```
#define OBJ_LIKE    (1 - 1) // different white space
#define FUNC_LIKE(b) ( a )    // different parameter usage
#define FUNC_LIKE(b) ( b )    // different parameter spelling
```

9 EXAMPLE 7 Finally, to show the variable argument list macro facilities:

```
#define debug(...)      fprintf(stderr, __VA_ARGS__)
#define showlist(...)   puts(#__VA_ARGS__)
#define report(test, ...) ((test)?puts(#test):\
            printf(__VA_ARGS__))
debug("Flag");
debug("X = %d\n", x);
showlist(The first, second, and third items.);
report(x>y, "x is %d but y is %d", x, y);
```

results in

```
fprintf(stderr,  "Flag");
fprintf(stderr,  "X = %d\n", x);
puts("The first, second, and third items.");
((x>y)?puts("x>y"):
            printf("x is %d but y is %d", x, y));
```

6.10.6 Line control

Constraints

1 The string literal of a **#line** directive, if present, shall be a character string literal.

Semantics

2 The *line number* of the current source line is one greater than the number of new-line characters read or introduced in translation phase 1 (5.1.1.2) while processing the source file to the current token.

3 If a preprocessing token (in particular **__LINE__**) spans two or more physical lines, it is unspecified which of those line numbers is associated with that token. If a preprocessing directive spans two or more physical lines, it is unspecified which of those line numbers is associated with the preprocessing directive. If a macro invocation spans multiple physical lines, it is unspecified which of those line numbers is associated with that invocation. The line number of a preprocessing token is independent of the context (in particular, as a macro argument or in a preprocessing directive). The line number of a **__LINE__** in a macro body is the line number of the macro invocation.

4 A preprocessing directive of the form

 # line *digit-sequence new-line*

causes the implementation to behave as if the following sequence of source lines begins with a source line that has a line number as specified by the digit sequence (interpreted as a decimal integer, ignoring any optional digit separators (6.4.4.2) in the digit sequence). The digit sequence shall not specify zero, nor a number greater than 2147483647.

5 A preprocessing directive of the form

 # line *digit-sequence* **"** *s-char-sequence*$_{\text{opt}}$ **"** new-line

sets the presumed line number similarly and changes the presumed name of the source file to be the contents of the character string literal.

6 A preprocessing directive of the form

 # line *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **line** on the directive are processed just as in normal text (each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). The directive resulting after

all replacements shall match one of the two previous forms and is then processed as appropriate.[206)]

Recommended practice

7 The line number associated with a *pp-token* should be the line number of the first character of the *pp-token*. The line number associated with a preprocessing directive should be the line number of the line with the first **#** token. The line number associated with a macro invocation should be the line number of the first character of the macro name in the invocation.

6.10.7 Diagnostic directives

Semantics

1 A preprocessing directive of either form

 # error *pp-tokens*$_{\text{opt}}$ *new-line*
 # warning *pp-tokens*$_{\text{opt}}$ *new-line*

causes the implementation to produce a diagnostic message that includes the specified sequence of preprocessing tokens.

6.10.8 Pragma directive

Semantics

1 A preprocessing directive of the form

 # pragma *pp-tokens*$_{\text{opt}}$ *new-line*

where the preprocessing token **STDC** does not immediately follow **pragma** in the directive (prior to any macro replacement)[207)] causes the implementation to behave in an implementation-defined manner. The behavior may cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any such **pragma** that is not recognized by the implementation is ignored.

2 If the preprocessing token **STDC** does immediately follow **pragma** in the directive (prior to any macro replacement), then no macro replacement is performed on the directive, and the directive shall have one of the following forms[208)] whose meanings are described elsewhere:

standard-pragma:

 # pragma STDC FP_CONTRACT *on-off-switch*
 # pragma STDC FENV_ACCESS *on-off-switch*
 # pragma STDC FENV_DEC_ROUND *dec-direction*
 # pragma STDC FENV_ROUND *direction*
 # pragma STDC CX_LIMITED_RANGE *on-off-switch*

on-off-switch: one of

ON **OFF** **DEFAULT**

direction: one of

FE_DOWNWARD **FE_TONEAREST** **FE_TONEARESTFROMZERO**
FE_TOWARDZERO **FE_UPWARD** **FE_DYNAMIC**

[206)] Because a new-line is explicitly included as part of the **#line** directive, the number of new-line characters read while processing to the first *pp-token* can be different depending on whether the implementation uses a one-pass preprocessor. Therefore, there are two possible values for the line number following a directive of the form **#line __LINE__** *new-line*.

[207)] An implementation is not required to perform macro replacement in pragmas, but it is permitted except for in standard pragmas (where **STDC** immediately follows **pragma**). If the result of macro replacement in a non-standard pragma has the same form as a standard pragma, the behavior is still implementation-defined; an implementation is permitted to behave as if it were the standard pragma, but is not required to.

[208)] See "future language directions" (6.11.6).

dec-direction: one of
FE_DEC_DOWNWARD **FE_DEC_TONEAREST** **FE_DEC_TONEARESTFROMZERO**
FE_DEC_TOWARDZERO **FE_DEC_UPWARD** **FE_DEC_DYNAMIC**

Recommended practice

3 Implementations are encouraged to diagnose unrecognized pragmas.

Forward references: the **FP_CONTRACT** pragma (7.12.2), the **FENV_ACCESS** pragma (7.6.1), the **FENV_DEC_ROUND** pragma (7.6.3), the **FENV_ROUND** pragma (7.6.2), the **CX_LIMITED_RANGE** pragma (7.3.4).

6.10.9 Null directive

Semantics

1 A preprocessing directive of the form

*new-line*

has no effect.

6.10.10 Predefined macro names

6.10.10.1 General

1 The values of the predefined macros listed in the following subclauses[209] (except for **__FILE__** and **__LINE__**) remain constant throughout the translation unit.

2 None of the following macro names in this subclause nor the identifiers **defined**, **__has_c_attribute**, **__has_include**, or **__has_embed** shall be the subject of a **#define** or a **#undef** preprocessing directive. Any other predefined macro names: shall begin with a leading underscore followed by an uppercase letter; or, a second underscore; or, shall be any of the identifiers **alignas**, **alignof**, **bool**, **false**, **static_assert**, **thread_local**, or **true**.

3 The implementation shall not predefine the macro **__cplusplus**, nor shall it define it in any standard header.

Forward references: standard headers (7.1.2).

6.10.10.2 Mandatory macros

1 The following macro names shall be defined by the implementation:

__DATE__ The date of translation of the preprocessing translation unit: a character string literal of the form `"Mmm dd yyyy"`, where the names of the months are the same as those generated by the **asctime** function, and the first character of `dd` is a space character if the value is less than 10. If the date of translation is not available, an implementation-defined valid date shall be supplied.

__FILE__ The presumed name of the current source file (a character string literal).[210]

__LINE__ The presumed line number (within the current source file) of the current source line (an integer constant).[210]

__STDC__ The integer constant `1`, intended to indicate a conforming implementation.

__STDC_EMBED_NOT_FOUND__, **__STDC_EMBED_FOUND__**, **__STDC_EMBED_EMPTY__** The integer constants `0`, `1`, and `2`, respectively.

__STDC_HOSTED__ The integer constant `1` if the implementation is a hosted implementation or the integer constant `0` if it is not.

[209] See "future language directions" (6.11.7).
[210] The presumed source file name and line number can be changed by the **#line** directive.

__STDC_UTF_16__ The integer constant `1`, intended to indicate that values of type **`char16_t`** are UTF-16 encoded.

__STDC_UTF_32__ The integer constant `1`, intended to indicate that values of type **`char32_t`** are UTF-32 encoded.

__STDC_VERSION__ The integer constant `202311L`.[211)]

__TIME__ The time of translation of the preprocessing translation unit: a character string literal of the form `"hh:mm:ss"` as in the time generated by the **`asctime`** function. If the time of translation is not available, an implementation-defined valid time shall be supplied.

Forward references: the **`asctime`** function (7.29.3.1).

6.10.10.3 Environment macros

1 The following macro names are conditionally defined by the implementation:

__STDC_ISO_10646__ An integer constant of the form `yyyymmL` (for example, `202012L`). If this symbol is defined, then every character in the Unicode required set, when stored in an object of type **`wchar_t`**, has the same value as the short identifier of that character. The *Unicode required set* consists of all the characters that are defined by ISO/IEC 10646, along with all amendments and technical corrigenda, as of the specified year and month. If some other encoding is used, the macro shall not be defined and the actual encoding used is implementation-defined.

__STDC_MB_MIGHT_NEQ_WC__ The integer constant `1`, intended to indicate that, in the encoding for **`wchar_t`**, a member of the basic character set is not required to have a code value equal to its value when used as the lone character in an integer character constant.

Forward references: common definitions (7.21), Unicode utilities (7.30).

6.10.10.4 Conditional feature macros

1 The following macro names are conditionally defined by the implementation:

__STDC_ANALYZABLE__ The integer constant `1`, if the implementation conforms to the specifications in Annex L (Analyzability).

__STDC_IEC_60559_BFP__ The integer constant `202311L`, intended to indicate conformance to Annex F (ISO/IEC 60559 floating-point arithmetic) for binary floating-point arithmetic.

__STDC_IEC_559__ The integer constant `1`, intended to indicate conformance to the specifications in Annex F (ISO/IEC 60559 floating-point arithmetic) for binary floating-point arithmetic. Use of this macro is an obsolescent feature.

__STDC_IEC_60559_DFP__ The integer constant `202311L`, intended to indicate support of decimal floating types and conformance to Annex F (ISO/IEC 60559 floating-point arithmetic) for decimal floating-point arithmetic.

__STDC_IEC_60559_COMPLEX__ The integer constant `202311L`, intended to indicate conformance to the specifications in Annex G (ISO/IEC 60559 compatible complex arithmetic).

__STDC_IEC_60559_TYPES__ The integer constant `202311L`, intended to indicate conformance to the specification in Annex H (ISO/IEC 60559 interchange and extended types).

__STDC_IEC_559_COMPLEX__ The integer constant `1`, intended to indicate adherence to the specifications in Annex G (ISO/IEC 60559 compatible complex arithmetic). Use of this macro is an obsolescent feature.

[211)] See Annex M for the values in previous editions of this document. The intention is that this will remain an integer constant of type **`long int`** that is increased with each revision of this document.

__STDC_LIB_EXT1__ The integer constant `202311L`, intended to indicate support for the extensions defined in Annex K (Bounds-checking interfaces).

__STDC_NO_ATOMICS__ The integer constant `1`, intended to indicate that the implementation does not support atomic types (including the **_Atomic** type qualifier) and the `<stdatomic.h>` header.

__STDC_NO_COMPLEX__ The integer constant `1`, intended to indicate that the implementation does not support complex types or the `<complex.h>` header.

__STDC_NO_THREADS__ The integer constant `1`, intended to indicate that the implementation does not support the `<threads.h>` header.

__STDC_NO_VLA__ The integer constant `1`, intended to indicate that the implementation does not support variable length arrays with automatic storage duration. Parameters declared with variable length array types are adjusted and then define objects of automatic storage duration with pointer types. Thus, support for such declarations is mandatory.

2 NOTE The intention for the macros **__STDC_LIB_EXT1__**, **__STDC_IEC_60559_BFP__**, **__STDC_IEC_60559_DFP__**, **__STDC_IEC_60559_COMPLEX__**, and **__STDC_IEC_60559_TYPES__**, with the value `202311L`, is that this will remain an integer constant of type **long int** that is increased with each revision of this document.

3 An implementation that defines **__STDC_NO_COMPLEX__** shall not define **__STDC_IEC_60559_COMPLEX__** or **__STDC_IEC_559_COMPLEX__**.

6.10.11 Pragma operator

Semantics

1 A unary operator expression of the form:

_Pragma (*string-literal* **)**

is processed as follows: The string literal is *destringized* by deleting any encoding prefix, deleting the leading and trailing double-quotes, replacing each escape sequence `\"` by a double-quote, and replacing each escape sequence `\\` by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a pragma directive. The original four preprocessing tokens in the unary operator expression are removed.

2 EXAMPLE A directive of the form:

```
#pragma listing on "..\listing.dir"
```

can also be expressed as:

```
_Pragma ("listing on \"..\\listing.dir\"")
```

The latter form is processed in the same way whether it appears literally as shown, or results from macro replacement, as in:

```
#define LISTING(x) PRAGMA(listing on #x)
#define PRAGMA(x)  _Pragma(#x)

LISTING (..\listing.dir)
```


6.11 Future language directions

6.11.1 Floating types

1 Future standardization may include additional floating types, including those with greater range, precision, or both than **`long double`**.

6.11.2 Linkages of identifiers

1 Declaring an identifier with internal linkage at file scope without the **`static`** or **`constexpr`** storage-class specifier is an obsolescent feature.

6.11.3 External names

1 Restriction of the significance of an external name to fewer than 255 characters (considering each universal character name or extended source character as a single character) is an obsolescent feature that is a concession to existing implementations.

6.11.4 Character escape sequences

1 Lowercase letters as escape sequences are reserved for future standardization. Other characters may be used in extensions.

6.11.5 Storage-class specifiers

1 The placement of a storage-class specifier other than at the beginning of the declaration specifiers in a declaration is an obsolescent feature.

2 Future standardization may change the **`auto`** storage-class specifier to a type specifier.

6.11.6 Pragma directives

1 Pragmas whose first preprocessing token is **`STDC`** are reserved for future standardization.

6.11.7 Predefined macro names

1 Macro names beginning with **`__STDC_`** are reserved for future standardization.

2 Uses of the **`__STDC_IEC_559__`** and **`__STDC_IEC_559_COMPLEX__`** macros are obsolescent features.